

Studentai DB

Generated by Doxygen 1.13.2

1 OOP	1
1.1 Projekto aprašymas su abstracia klase	1
1.2 Funkcionalumas su abstrakčia klase	1
1.3 Testavimo rezultatai	2
1.4 Abstraktumo įrodymas	2
1.5 Perdengtų metodų aprašymas	2
1.5.1 Duomenų įvestis	2
1.5.2 Papildoma informacija	2
1.5.3 <i>The Rule of five</i> Testai atlikti su <i>doctest</i>	3
1.6 Visų testų rezultatai	3
1.7 Executable su O1 failo dydis 221KB	3
1.8 Executable su O2 failo dydis 205KB	3
1.9 Executable su O3 failo dydis 238KB	3
1.10 FAILŲ TYRIMŲ REZULTATAI SU O3 vėliavėlę:	3
1.11 FAILŲ TYRIMŲ REZULTATAI SU O2 vėliavėlę:	3
1.12 FAILŲ TYRIMŲ REZULTATAI SU O1 vėliavėlę:	3
1.13 Testavimo aplinka	4
1.14 Naudojimosi instrukcija	4
1.15 Įdiegimo instrukcija	4
2 Hierarchical Index	5
2.1 Class Hierarchy	5
3 Class Index	7
3.1 Class List	7
4 File Index	9
4.1 File List	9
5 Class Documentation	11
5.1 doctest::Approx Struct Reference	11
5.2 doctest::AssertData Struct Reference	12
5.3 std::basic_istream< charT, traits > Class Template Reference	12
5.4 std::basic_ostream< charT, traits > Class Template Reference	13
5.5 std::char_traits< charT > Struct Template Reference	13
5.6 doctest::Contains Class Reference	13
5.7 doctest::Context Class Reference	13
5.8 doctest::ContextOptions Struct Reference	14
5.8.1 Detailed Description	14
5.9 doctest::detail::ContextScope< L > Class Template Reference	15
5.9.1 Member Function Documentation	15
5.9.1.1 stringify()	15
5.10 doctest::detail::ContextScopeBase Struct Reference	16
5.11 doctest::CurrentTestCaseStats Struct Reference	16

5.12 doctest::detail::deferred_false< T > Struct Template Reference	17
5.13 doctest::detail::types::enable_if< COND, T > Struct Template Reference	17
5.14 doctest::detail::types::enable_if< true, T > Struct Template Reference	17
5.15 doctest::detail::ExceptionTranslator< T > Class Template Reference	18
5.15.1 Detailed Description	18
5.15.2 Member Function Documentation	18
5.15.2.1 translate()	18
5.16 doctest::detail::Expression_lhs< L > Struct Template Reference	18
5.16.1 Member Function Documentation	19
5.16.1.1 operator Result()	19
5.17 doctest::detail::ExpressionDecomposer Struct Reference	19
5.18 doctest::detail::types::false_type Struct Reference	19
5.19 doctest::detail::filldata< T > Struct Template Reference	20
5.20 doctest::detail::filldata< const char[N]> Struct Template Reference	20
5.21 doctest::detail::filldata< const void * > Struct Reference	20
5.22 doctest::detail::filldata< T * > Struct Template Reference	21
5.23 doctest::detail::filldata< T[N]> Struct Template Reference	21
5.24 doctest::detail::has_insertion_operator< T, typename > Struct Template Reference	21
5.25 doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >()), declval< const T & >()), void())> Struct Template Reference	22
5.26 doctest::IContextScope Struct Reference	22
5.27 doctest::detail::ExceptionTranslator Struct Reference	23
5.28 doctest::IReporter Struct Reference	23
5.29 doctest::detail::types::is_array< T > Struct Template Reference	24
5.30 doctest::detail::types::is_array< T[SIZE]> Struct Template Reference	24
5.31 doctest::detail::types::is_enum< T > Struct Template Reference	25
5.32 doctest::detail::types::is_pointer< T > Struct Template Reference	25
5.33 doctest::detail::types::is_pointer< T * > Struct Template Reference	25
5.34 doctest::detail::types::is_rvalue_reference< T > Struct Template Reference	26
5.35 doctest::detail::types::is_rvalue_reference< T && > Struct Template Reference	26
5.36 doctest::IsNaN< F > Struct Template Reference	27
5.37 doctest::detail::MessageBuilder Struct Reference	27
5.38 doctest::MessageData Struct Reference	28
5.39 doctest::QueryData Struct Reference	28
5.40 doctest::detail::RelationalComparator< int, L, R > Struct Template Reference	29
5.41 doctest::detail::types::remove_const< T > Struct Template Reference	29
5.42 doctest::detail::types::remove_const< const T > Struct Template Reference	29
5.43 doctest::detail::types::remove_reference< T > Struct Template Reference	29
5.44 doctest::detail::types::remove_reference< T & > Struct Template Reference	30
5.45 doctest::detail::types::remove_reference< T && > Struct Template Reference	30
5.46 doctest::detail::Result Struct Reference	30
5.47 doctest::detail::ResultBuilder Struct Reference	31

5.47.1 Member Function Documentation	32
5.47.1.1 unary_assert()	32
5.48 doctest::detail::should_stringify_as_underlying_type< T > Struct Template Reference	32
5.49 doctest::String Class Reference	32
5.50 doctest::AssertData::StringContains Class Reference	33
5.51 doctest::StringMaker< T > Struct Template Reference	33
5.52 doctest::detail::StringMakerBase< C > Struct Template Reference	34
5.53 doctest::detail::StringMakerBase< true > Struct Reference	34
5.54 Student Class Reference	34
5.54.1 Member Function Documentation	35
5.54.1.1 getPavarde()	35
5.54.1.2 getVardas()	35
5.54.1.3 setPavarde()	35
5.54.1.4 setVardas()	35
5.55 doctest::detail::Subcase Struct Reference	36
5.56 doctest::SubcaseSignature Struct Reference	36
5.57 doctest::detail::TestCase Struct Reference	36
5.58 doctest::TestCaseData Struct Reference	37
5.59 doctest::TestCaseException Struct Reference	38
5.60 doctest::detail::TestFailureException Struct Reference	38
5.61 doctest::TestRunStats Struct Reference	38
5.62 doctest::detail::TestSuite Struct Reference	39
5.63 doctest::detail::types::true_type Struct Reference	39
5.64 std::tuple< Types > Class Template Reference	39
5.65 doctest::detail::types::underlying_type< T > Struct Template Reference	40
5.66 Zmogus Class Reference	40
6 File Documentation	41
6.1 DequeO.h	41
6.2 doctest.h	41
6.3 failo_oper.h	125
6.4 generavimasf.h	126
6.5 ListO.h	126
6.6 papild.h	126
6.7 studentas.h	126
6.8 testai.h	127
6.9 zmogus.h	127
Index	129

Chapter 1

OOP

1.1 Projekto aprašymas su abstrakčia klase

Šis projektas yra skirtas objektinio programavimo principų demonstravimui, naudojant abstrakčią bazinę klasę `Zmogus` ir iš jos išvestinę klasę `Studentas`. Projektas apima įvairias funkcijas, tokias kaip duomenų įvedimas, apdorojimas, rūšiavimas ir testavimas.

1.2 Funkcionalumas su abstrakčia klase

1. Abstrakti bazinė klasė `Zmogus`:

- Aprašo bendras žmogaus savybes: vardą ir pavardę.
- Turi grynai virtualią funkciją `spausdintiInformacija`, kurią įgyvendina išvestinės klasės.

2. Išvestinė klasė `Studentas`:

- Paveldi `Zmogus` klasę ir prideda papildomas savybes: namų darbų pažymius ir egzamino rezultatą.
- Implementuoja funkcijas:
 - Vidurkio ir medianos skaičiavimas.
 - Galutinio pažymio skaičiavimas.
 - Duomenų išvedimas naudojant perdengtą operatorį `<<`.

3. Duomenų įvedimas:

- Rankinė įvestis per konsolę.
- Automatinė įvestis naudojant atsitiktinių duomenų generavimą.
- Įvestis iš failo, kurio formatas turi būti tinkamai struktūruotas.

4. Testavimas:

- Naudojama `doctest` biblioteka, kuri tikrina:
 - Konstruktorius ir destruktorius.
 - Kopijavimo ir perkėlimo operatorius (Rule of Five).
 - Vidurkio, medianos ir galutinio pažymio skaičiavimą.
 - Abstrakčios klasės funkcionalumą.

5. Failų apdorojimas:

- Studentai rūšiuojami į „vargšiukus“ ir „kietiokus“ pagal jų galutinį pažymį.
- Rezultatai išsaugomi atskiruose failuose.

1.3 Testavimo rezultatai

- Testai atlikti naudojant `doctest` biblioteką.
- Tikrinamos šios funkcijos:
 - Konstruktoriai ir destruktoriai.
 - Kopijavimo ir perkėlimo operatoriai.
 - Vidurkio, medianos ir galutinio pažymio skaičiavimas.
 - Abstrakčios klasės funkcionalumas.

1.4 Abstraktumo įrodymas

1.5 Perdengtų metodų aprašymas

Šiame projekte naudojami perdengti metodai, kurie leidžia patogiai dirbti su studentų duomenimis.

1.5.1 Duomenų įvestis

1. Rankinė įvestis:

- Naudojamas perdengtas `operator>>` metodas, kuris leidžia patogiai įvesti studento duomenis iš standartinio įvesties srauto.
- Įvedimo metu tikrinama, ar vardas ir pavardė sudaryti tik iš raidžių, o pažymiai ir egzamino rezultatas yra tinkamo intervalo (0–10).
- Pavyzdys:

```
Iveskite studento vardą: Jonas
Iveskite studento pavardę: Jonaitis
Iveskite namų darbų pažymius (baigti - ne skaičius): 8 9 10
Iveskite egzamino pažymį: 9
```

2. Automatinė įvestis:

- Naudojama funkcija `generuotiStudentus`, kuri automatiškai sugeneruoja nurodytą kiekį studentų su atsitiktiniais vardais, pavardėmis, pažymiais ir egzamino rezultatais.
- Pavyzdys:

```
std::vector<Student> studentai = generuotiStudentus(100);
```

3. Įvestis iš failo:

- Naudojama funkcija `nuskaitytiStudentus`, kuri skaito studentų duomenis iš failo. Failo formatas turi atitikti numatytą struktūrą (vardas, pavardė, pažymiai, egzaminas).
- Pavyzdys:

```
Jonas Jonaitis 8 9 10 9
Petras Petraitis 7 8 6 8
```

1.5.2 Papildoma informacija

- **Destruktorius:**
 - `Student::~~Student` užtikrina, kad visi dinaminiai resursai (pvz., `nd_balai`) būtų tinkamai atlaisvinti.
- **Konstruktoriai ir operatoriai:**
 - Implementuota „penkių taisyklė“ (Rule of Five), įskaitant kopijavimo ir perkėlimo konstruktorius bei priskyrimo operatorius.

1.5.3 The Rule of five Testai atlikti su doctest

Testo nuotrauka	Aprašymas
	Kopijavimo testai: tikrina kopijavimo konstruktorių ir priskyrimo operatorių, kurie sukuria tikslias objektų kopijas išsaugant visus duomenis
	Perkėlimo testai: tikrina move konstruktorių ir priskyrimo operatorių, kurie efektyviai perkėlia resursus iš vieno objekto į kitą

1.6 Visų testų rezultatai

1.7 Executable su O1 failo dydis 221KB

1.8 Executable su O2 failo dydis 205KB

1.9 Executable su O3 failo dydis 238KB

1.10 FAILŲ TYRIMŲ REZULTATAI SU O3 vėliavėle:

FAILAS	VECTOR(STRUCT) #3ST
Gstudentai100000.txt	0.197912
Gstudentai1000000.txt	1.80717

FAILAS	VECTOR(CLASS) #3ST
Gstudentai100000.txt	0.132619
Gstudentai1000000.txt	1.28624

1.11 FAILŲ TYRIMŲ REZULTATAI SU O2 vėliavėle:

FAILAS	VECTOR(STRUCT) #3ST
Gstudentai100000.txt	0.193232
Gstudentai1000000.txt	1.71543

FAILAS	VECTOR(CLASS) #3ST
Gstudentai100000.txt	0.138643
Gstudentai1000000.txt	2.40901

1.12 FAILŲ TYRIMŲ REZULTATAI SU O1 vėliavėle:

FAILAS	VECTOR(STRUCT) #3ST
Gstudentai100000.txt	0.194721
Gstudentai1000000.txt	1.68131

FAILAS	VECTOR(CLASS) #3ST
Gstudentai100000.txt	0.133387
Gstudentai1000000.txt	1.28121

1.13 Testavimo aplinka

- **Procesorius:** Apple M3
- **RAM:** 8GB
- **SSD:** 256GB

1.14 Naudojimosi instrukcija

1. Failų generavimas:

- Paleiskite programą ir pasirinkite failų generavimo funkciją.
- Po failų generavimo reikės pasirinkti testai su failais.

2. Failų darbai:

- Pasirinkite norimą failą iš pateikto sąrašo.
- Pasirinkite norimą rušiavimą
- Pasirinkite norimą skirstymo strategiją:
- Programa išrūšiuos studentus į „vargšiukus“ ir „kietiokus“ bei išsaugos juos atskiruose failuose.

3. Rezultatų analizė:

- Peržiūrėkite sugeneruotus failus ir programos išvestį, kurioje pateikiamas veikimo laikas.

1.15 Įdiegimo instrukcija

1. Reikalavimai:

- GCC arba kitas C++ kompiliatorius su C++17 ar naujesne versija.
- `make` įrankis (Unix sistemose įdiegtas pagal nutylėjimą).

2. Įdiegimas:

- Atsisiųskite projektą: `git clone https://github.com/frogg-kek/OOP-2`
- Paleiskite `make` komandą
- Sukurtas vykdomasis failas bus pavadintas `kursiokai`.

3. Paleidimas:

- Paleiskite programą: `./kursiokai`

4. Valymas: `make clean`

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

doctest::Approx	11
doctest::AssertData	12
doctest::detail::ResultBuilder	31
std::basic_istream< charT, traits >	12
std::basic_ostream< charT, traits >	13
std::char_traits< charT >	13
doctest::Contains	13
doctest::Context	13
doctest::ContextOptions	14
doctest::CurrentTestCaseStats	16
doctest::detail::types::enable_if< COND, T >	17
doctest::detail::types::enable_if< true, T >	17
doctest::detail::Expression_lhs< L >	18
doctest::detail::ExpressionDecomposer	19
doctest::detail::types::false_type	19
doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >()), declval< const T & >()), void())>	22
doctest::detail::types::is_array< T[SIZE]>	24
doctest::detail::types::is_pointer< T * >	25
doctest::detail::types::is_rvalue_reference< T && >	26
doctest::detail::deferred_false< T >	17
doctest::detail::has_insertion_operator< T, typename >	21
doctest::detail::types::is_array< T >	24
doctest::detail::types::is_pointer< T >	25
doctest::detail::types::is_rvalue_reference< T >	26
doctest::detail::filldata< T >	20
doctest::detail::filldata< const char[N]>	20
doctest::detail::filldata< const void * >	20
doctest::detail::filldata< T * >	21
doctest::detail::filldata< T[N]>	21
doctest::IContextScope	22
doctest::detail::ContextScopeBase	16
doctest::detail::ContextScope< L >	15
doctest::detail::IExceptionTranslator	23

doctest::detail::ExceptionTranslator< T >	18
doctest::IReporter	23
doctest::detail::types::is_enum< T >	25
doctest::IsNaN< F >	27
doctest::MessageData	28
doctest::detail::MessageBuilder	27
doctest::QueryData	28
doctest::detail::RelationalComparator< int, L, R >	29
doctest::detail::types::remove_const< T >	29
doctest::detail::types::remove_const< const T >	29
doctest::detail::types::remove_reference< T >	29
doctest::detail::types::remove_reference< T & >	30
doctest::detail::types::remove_reference< T && >	30
doctest::detail::Result	30
doctest::detail::should_stringify_as_underlying_type< T >	32
doctest::String	32
doctest::AssertData::StringContains	33
doctest::detail::StringMakerBase< C >	34
doctest::detail::StringMakerBase< detail::has_insertion_operator< T >::value detail::types::is_pointer< T >::value detail::types::is_array< T >::value >	34
doctest::StringMaker< T >	33
doctest::detail::StringMakerBase< true >	34
doctest::detail::Subcase	36
doctest::SubcaseSignature	36
doctest::TestCaseData	37
doctest::detail::TestCase	36
doctest::TestCaseException	38
doctest::detail::TestFailureException	38
doctest::TestRunStats	38
doctest::detail::TestSuite	39
doctest::detail::types::true_type	39
doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >(), declval< const T & >()), void())>	22
doctest::detail::types::is_array< T[SIZE]>	24
doctest::detail::types::is_pointer< T * >	25
doctest::detail::types::is_rvalue_reference< T && >	26
std::tuple< Types >	39
doctest::detail::types::underlying_type< T >	40
Zmogus	40
Student	34

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

doctest::Approx	11
doctest::AssertData	12
std::basic_istream< charT, traits >	12
std::basic_ostream< charT, traits >	13
std::char_traits< charT >	13
doctest::Contains	13
doctest::Context	13
doctest::ContextOptions	
OCLINT too many fields	14
doctest::detail::ContextScope< L >	15
doctest::detail::ContextScopeBase	16
doctest::CurrentTestCaseStats	16
doctest::detail::deferred_false< T >	17
doctest::detail::types::enable_if< COND, T >	17
doctest::detail::types::enable_if< true, T >	17
doctest::detail::ExceptionTranslator< T >	
OCLINT destructor of virtual class	18
doctest::detail::Expression_lhs< L >	18
doctest::detail::ExpressionDecomposer	19
doctest::detail::types::false_type	19
doctest::detail::filldata< T >	20
doctest::detail::filldata< const char[N]>	20
doctest::detail::filldata< const void * >	20
doctest::detail::filldata< T * >	21
doctest::detail::filldata< T[N]>	21
doctest::detail::has_insertion_operator< T, typename >	21
doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >(), declval< const T & >()), void())>	
22	
doctest::!ContextScope	22
doctest::detail::!ExceptionTranslator	23
doctest::!Reporter	23
doctest::detail::types::is_array< T >	24
doctest::detail::types::is_array< T[SIZE]>	24
doctest::detail::types::is_enum< T >	25
doctest::detail::types::is_pointer< T >	25

doctest::detail::types::is_pointer< T * >	25
doctest::detail::types::is_rvalue_reference< T >	26
doctest::detail::types::is_rvalue_reference< T && >	26
doctest::IsNaN< F >	27
doctest::detail::MessageBuilder	27
doctest::MessageData	28
doctest::QueryData	28
doctest::detail::RelationalComparator< int, L, R >	29
doctest::detail::types::remove_const< T >	29
doctest::detail::types::remove_const< const T >	29
doctest::detail::types::remove_reference< T >	29
doctest::detail::types::remove_reference< T & >	30
doctest::detail::types::remove_reference< T && >	30
doctest::detail::Result	30
doctest::detail::ResultBuilder	31
doctest::detail::should_stringify_as_underlying_type< T >	32
doctest::String	32
doctest::AssertData::StringContains	33
doctest::StringMaker< T >	33
doctest::detail::StringMakerBase< C >	34
doctest::detail::StringMakerBase< true >	34
Student	34
doctest::detail::Subcase	36
doctest::SubcaseSignature	36
doctest::detail::TestCase	36
doctest::TestCaseData	37
doctest::TestCaseException	38
doctest::detail::TestFailureException	38
doctest::TestRunStats	38
doctest::detail::TestSuite	39
doctest::detail::types::true_type	39
std::tuple< Types >	39
doctest::detail::types::underlying_type< T >	40
Zmogus	40

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

DequeO.h	41
doctest.h	41
failo_oper.h	125
generavimasf.h	126
ListO.h	126
papild.h	126
studentas.h	126
testai.h	127
zmogus.h	127

Chapter 5

Class Documentation

5.1 doctest::Approx Struct Reference

Public Member Functions

- **Approx** (double value)
- [Approx](#) **operator()** (double value) const
- [Approx](#) & **epsilon** (double newEpsilon)
- [Approx](#) & **scale** (double newScale)

Public Attributes

- double **m_epsilon**
- double **m_scale**
- double **m_value**

Friends

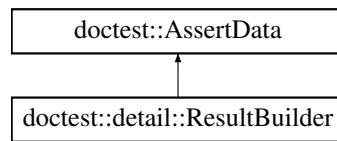
- DOctest_INTERFACE friend bool **operator==** (double lhs, const [Approx](#) &rhs)
- DOctest_INTERFACE friend bool **operator==** (const [Approx](#) &lhs, double rhs)
- DOctest_INTERFACE friend bool **operator!=** (double lhs, const [Approx](#) &rhs)
- DOctest_INTERFACE friend bool **operator!=** (const [Approx](#) &lhs, double rhs)
- DOctest_INTERFACE friend bool **operator<=** (double lhs, const [Approx](#) &rhs)
- DOctest_INTERFACE friend bool **operator<=** (const [Approx](#) &lhs, double rhs)
- DOctest_INTERFACE friend bool **operator>=** (double lhs, const [Approx](#) &rhs)
- DOctest_INTERFACE friend bool **operator>=** (const [Approx](#) &lhs, double rhs)
- DOctest_INTERFACE friend bool **operator<** (double lhs, const [Approx](#) &rhs)
- DOctest_INTERFACE friend bool **operator<** (const [Approx](#) &lhs, double rhs)
- DOctest_INTERFACE friend bool **operator>** (double lhs, const [Approx](#) &rhs)
- DOctest_INTERFACE friend bool **operator>** (const [Approx](#) &lhs, double rhs)

The documentation for this struct was generated from the following file:

- doctest.h

5.2 doctest::AssertData Struct Reference

Inheritance diagram for doctest::AssertData:



Classes

- class [StringContains](#)

Public Member Functions

- **AssertData** (assertType::Enum at, const char *file, int line, const char *expr, const char *exception_type, const [StringContains](#) &exception_string)

Public Attributes

- const [TestCaseData](#) * **m_test_case**
- assertType::Enum **m_at**
- const char * **m_file**
- int **m_line**
- const char * **m_expr**
- bool **m_failed**
- bool **m_threw**
- [String](#) **m_exception**
- [String](#) **m_decomp**
- bool **m_threw_as**
- const char * **m_exception_type**
- class DOCTEST_INTERFACE [doctest::AssertData::StringContains](#) **m_exception_string**

The documentation for this struct was generated from the following file:

- doctest.h

5.3 std::basic_istream< charT, traits > Class Template Reference

The documentation for this class was generated from the following file:

- doctest.h

5.4 `std::basic_ostream< charT, traits >` Class Template Reference

The documentation for this class was generated from the following file:

- `doctest.h`

5.5 `std::char_traits< charT >` Struct Template Reference

The documentation for this struct was generated from the following file:

- `doctest.h`

5.6 `doctest::Contains` Class Reference

Public Member Functions

- **Contains** (const [String](#) &string)
- bool **checkWith** (const [String](#) &other) const

Public Attributes

- [String](#) **string**

The documentation for this class was generated from the following file:

- `doctest.h`

5.7 `doctest::Context` Class Reference

Public Member Functions

- **Context** (int argc=0, const char *const *argv=nullptr)
- **Context** (const [Context](#) &)=delete
- **Context** ([Context](#) &&)=delete
- [Context](#) & **operator=** (const [Context](#) &)=delete
- [Context](#) & **operator=** ([Context](#) &&)=delete
- void **applyCommandLine** (int argc, const char *const *argv)
- void **addFilter** (const char *filter, const char *value)
- void **clearFilters** ()
- void **setOption** (const char *option, bool value)
- void **setOption** (const char *option, int value)
- void **setOption** (const char *option, const char *value)
- bool **shouldExit** ()
- void **setAsDefaultForAssertsOutOfTestCases** ()
- void **setAssertHandler** (detail::assert_handler ah)
- void **setCout** ([std::ostream](#) *out)
- int **run** ()

The documentation for this class was generated from the following file:

- `doctest.h`

5.8 doctest::ContextOptions Struct Reference

OCLINT too many fields.

```
#include <doctest.h>
```

Public Attributes

- `std::ostream * cout` = nullptr
- `String binary_name`
- `const detail::TestCase * currentTest` = nullptr
- `String out`
- `String order_by`
- `unsigned rand_seed`
- `unsigned first`
- `unsigned last`
- `int abort_after`
- `int subcase_filter_levels`
- `bool success`
- `bool case_sensitive`
- `bool exit`
- `bool duration`
- `bool minimal`
- `bool quiet`
- `bool no_throw`
- `bool no_exitcode`
- `bool no_run`
- `bool no_intro`
- `bool no_version`
- `bool no_colors`
- `bool force_colors`
- `bool no_breaks`
- `bool no_skip`
- `bool gnu_file_line`
- `bool no_path_in_filenames`
- `bool no_line_numbers`
- `bool no_debug_output`
- `bool no_skipped_summary`
- `bool no_time_in_output`
- `bool help`
- `bool version`
- `bool count`
- `bool list_test_cases`
- `bool list_test_suites`
- `bool list_reporters`

5.8.1 Detailed Description

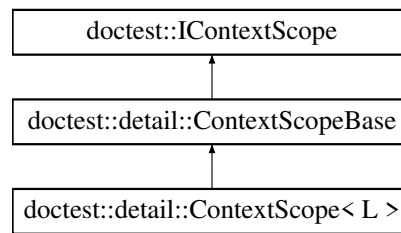
OCLINT too many fields.

The documentation for this struct was generated from the following file:

- `doctest.h`

5.9 doctest::detail::ContextScope< L > Class Template Reference

Inheritance diagram for doctest::detail::ContextScope< L >:



Public Member Functions

- **ContextScope** (const L &lambda)
- **ContextScope** (L &&lambda)
- **ContextScope** (const [ContextScope](#) &)=delete
- **ContextScope** ([ContextScope](#) &&) noexcept=default
- [ContextScope](#) & **operator=** (const [ContextScope](#) &)=delete
- [ContextScope](#) & **operator=** ([ContextScope](#) &&)=delete
- void [stringify](#) ([std::ostream](#) *s) const override

Public Member Functions inherited from [doctest::detail::ContextScopeBase](#)

- **ContextScopeBase** (const [ContextScopeBase](#) &)=delete
- [ContextScopeBase](#) & **operator=** (const [ContextScopeBase](#) &)=delete
- [ContextScopeBase](#) & **operator=** ([ContextScopeBase](#) &&)=delete

Additional Inherited Members

Protected Member Functions inherited from [doctest::detail::ContextScopeBase](#)

- **ContextScopeBase** ([ContextScopeBase](#) &&other) noexcept
- void **destroy** ()

Protected Attributes inherited from [doctest::detail::ContextScopeBase](#)

- bool **need_to_destroy** {true}

5.9.1 Member Function Documentation

5.9.1.1 stringify()

```

template<typename L>
void doctest::detail::ContextScope< L >::stringify (
    std::ostream * s) const [inline], [override], [virtual]
  
```

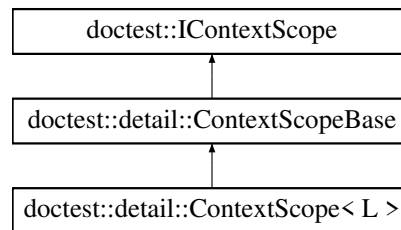
Implements [doctest::IContextScope](#).

The documentation for this class was generated from the following file:

- doctest.h

5.10 doctest::detail::ContextScopeBase Struct Reference

Inheritance diagram for doctest::detail::ContextScopeBase:



Public Member Functions

- **ContextScopeBase** (const [ContextScopeBase](#) &)=delete
- [ContextScopeBase](#) & **operator=** (const [ContextScopeBase](#) &)=delete
- [ContextScopeBase](#) & **operator=** ([ContextScopeBase](#) &&)=delete

Public Member Functions inherited from [doctest::IContextScope](#)

- virtual void **stringify** ([std::ostream](#) *) const =0

Protected Member Functions

- **ContextScopeBase** ([ContextScopeBase](#) &&other) noexcept
- void **destroy** ()

Protected Attributes

- bool **need_to_destroy** {true}

The documentation for this struct was generated from the following file:

- doctest.h

5.11 doctest::CurrentTestCaseStats Struct Reference

Public Attributes

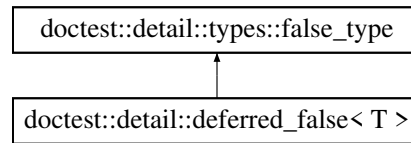
- int **numAssertsCurrentTest**
- int **numAssertsFailedCurrentTest**
- double **seconds**
- int **failure_flags**
- bool **testCaseSuccess**

The documentation for this struct was generated from the following file:

- doctest.h

5.12 doctest::detail::deferred_false< T > Struct Template Reference

Inheritance diagram for doctest::detail::deferred_false< T >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static DOCTEST_CONSTEXPR bool **value** = false

The documentation for this struct was generated from the following file:

- doctest.h

5.13 doctest::detail::types::enable_if< COND, T > Struct Template Reference

The documentation for this struct was generated from the following file:

- doctest.h

5.14 doctest::detail::types::enable_if< true, T > Struct Template Reference

Public Types

- using **type** = T

The documentation for this struct was generated from the following file:

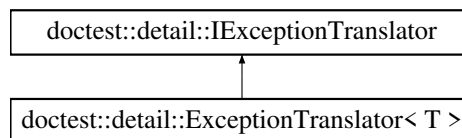
- doctest.h

5.15 doctest::detail::ExceptionTranslator< T > Class Template Reference

OCLINT destructor of virtual class.

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::ExceptionTranslator< T >:



Public Member Functions

- **ExceptionTranslator** ([String](#)(*translateFunction)(T))
- bool [translate](#) ([String](#) &res) const override

5.15.1 Detailed Description

```
template<typename T>
class doctest::detail::ExceptionTranslator< T >
```

OCLINT destructor of virtual class.

5.15.2 Member Function Documentation

5.15.2.1 translate()

```
template<typename T>
bool doctest::detail::ExceptionTranslator< T >::translate (
    String & res) const [inline], [override], [virtual]
```

Implements [doctest::detail::IExceptionTranslator](#).

The documentation for this class was generated from the following file:

- doctest.h

5.16 doctest::detail::Expression_Ihs< L > Struct Template Reference

Public Member Functions

- **Expression_Ihs** (L &&in, assertType::Enum at)
- DOCTEST_NOINLINE [operator Result](#) ()
- **operator L** () const

Public Attributes

- `L lhs`
- `assertType::Enum m_at`

5.16.1 Member Function Documentation

5.16.1.1 operator Result()

```
template<typename L>
DOCTEST_NOINLINE doctest::detail::Expression_lhs< L >::operator Result () [inline]
```

OCLINT bitwise operator in conditional

The documentation for this struct was generated from the following file:

- `doctest.h`

5.17 doctest::detail::ExpressionDecomposer Struct Reference

Public Member Functions

- **ExpressionDecomposer** (`assertType::Enum at`)
- `template<typename L>`
`Expression_lhs< L > operator<< (L &&operand)`
- `template<typename L, typename types::enable_if<!doctest::detail::types::is_rvalue_reference< L >::value, void >::type * = nullptr>`
`Expression_lhs< const L & > operator<< (const L &operand)`

Public Attributes

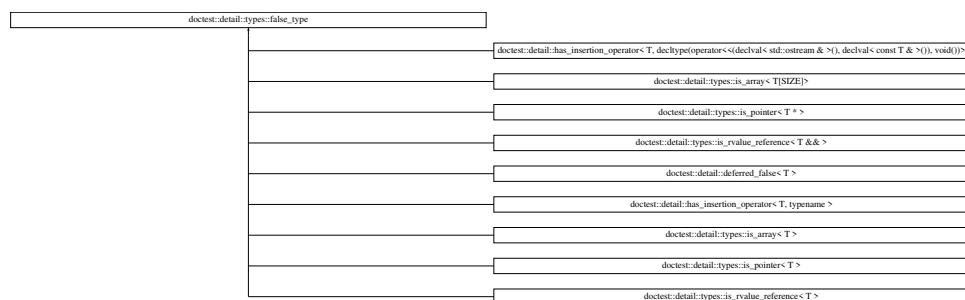
- `assertType::Enum m_at`

The documentation for this struct was generated from the following file:

- `doctest.h`

5.18 doctest::detail::types::false_type Struct Reference

Inheritance diagram for `doctest::detail::types::false_type`:



Static Public Attributes

- static DOCTEST_CONSTEXPR bool **value** = false

The documentation for this struct was generated from the following file:

- doctest.h

5.19 doctest::detail::filldata< T > Struct Template Reference

Static Public Member Functions

- static void **fill** ([std::ostream](#) *stream, const T &in)

The documentation for this struct was generated from the following file:

- doctest.h

5.20 doctest::detail::filldata< const char[N]> Struct Template Reference

Static Public Member Functions

- static void **fill** ([std::ostream](#) *stream, const char(&in)[N])
- static void **fill** ([std::ostream](#) *stream, const const char &in)

The documentation for this struct was generated from the following file:

- doctest.h

5.21 doctest::detail::filldata< const void * > Struct Reference

Static Public Member Functions

- static void **fill** ([std::ostream](#) *stream, const void *in)
- static void **fill** ([std::ostream](#) *stream, const const void *&in)

The documentation for this struct was generated from the following file:

- doctest.h

5.22 doctest::detail::filldata< T * > Struct Template Reference

Static Public Member Functions

- static void **fill** ([std::ostream](#) *stream, const T *in)
- static void **fill** ([std::ostream](#) *stream, const T &in)

The documentation for this struct was generated from the following file:

- doctest.h

5.23 doctest::detail::filldata< T[N]> Struct Template Reference

Static Public Member Functions

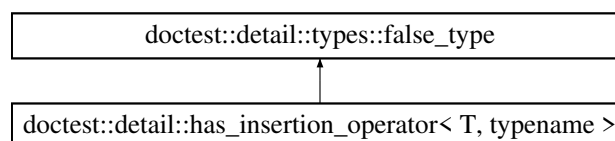
- static void **fill** ([std::ostream](#) *stream, const T(&in)[N])
- static void **fill** ([std::ostream](#) *stream, const T &in)

The documentation for this struct was generated from the following file:

- doctest.h

5.24 doctest::detail::has_insertion_operator< T, typename > Struct Template Reference

Inheritance diagram for doctest::detail::has_insertion_operator< T, typename >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

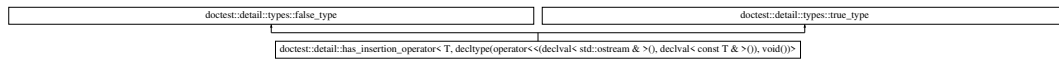
- static DOCTEST_CONSTEXPR bool **value** = false

The documentation for this struct was generated from the following file:

- doctest.h

5.25 doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >()), declval< const T & >()), void())> Struct Template Reference

Inheritance diagram for doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >()), declval< const T & >()), void())>:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static DOCTEST_CONSTEXPR bool **value** = false

Static Public Attributes inherited from [doctest::detail::types::true_type](#)

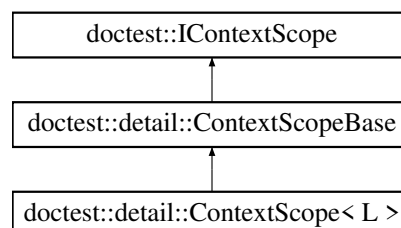
- static DOCTEST_CONSTEXPR bool **value** = true

The documentation for this struct was generated from the following file:

- doctest.h

5.26 doctest::IContextScope Struct Reference

Inheritance diagram for doctest::IContextScope:



Public Member Functions

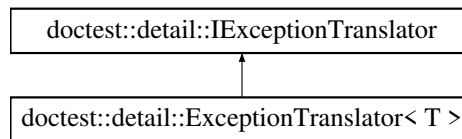
- virtual void **stringify** ([std::ostream](#) *) const =0

The documentation for this struct was generated from the following file:

- doctest.h

5.27 doctest::detail::IExceptionTranslator Struct Reference

Inheritance diagram for doctest::detail::IExceptionTranslator:



Public Member Functions

- virtual bool **translate** ([String](#) &) const =0

The documentation for this struct was generated from the following file:

- doctest.h

5.28 doctest::IReporter Struct Reference

Public Member Functions

- virtual void **report_query** (const [QueryData](#) &)=0
- virtual void **test_run_start** ()=0
- virtual void **test_run_end** (const [TestRunStats](#) &)=0
- virtual void **test_case_start** (const [TestCaseData](#) &)=0
- virtual void **test_case_reenter** (const [TestCaseData](#) &)=0
- virtual void **test_case_end** (const [CurrentTestCaseStats](#) &)=0
- virtual void **test_case_exception** (const [TestCaseException](#) &)=0
- virtual void **subcase_start** (const [SubcaseSignature](#) &)=0
- virtual void **subcase_end** ()=0
- virtual void **log_assert** (const [AssertData](#) &)=0
- virtual void **log_message** (const [MessageData](#) &)=0
- virtual void **test_case_skipped** (const [TestCaseData](#) &)=0

Static Public Member Functions

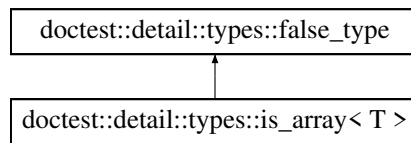
- static int **get_num_active_contexts** ()
- static const [IContextScope](#) *const * **get_active_contexts** ()
- static int **get_num_stringified_contexts** ()
- static const [String](#) * **get_stringified_contexts** ()

The documentation for this struct was generated from the following file:

- doctest.h

5.29 doctest::detail::types::is_array< T > Struct Template Reference

Inheritance diagram for doctest::detail::types::is_array< T >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

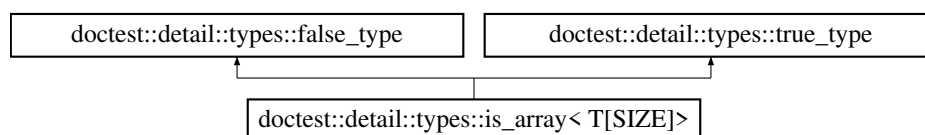
- static DOCTEST_CONSTEXPR bool **value** = false

The documentation for this struct was generated from the following file:

- doctest.h

5.30 doctest::detail::types::is_array< T[SIZE]> Struct Template Reference

Inheritance diagram for doctest::detail::types::is_array< T[SIZE]>:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static DOCTEST_CONSTEXPR bool **value** = false

Static Public Attributes inherited from [doctest::detail::types::true_type](#)

- static DOCTEST_CONSTEXPR bool **value** = true

The documentation for this struct was generated from the following file:

- doctest.h

5.31 doctest::detail::types::is_enum< T > Struct Template Reference

Static Public Attributes

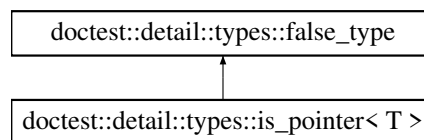
- static DOCTEST_CONSTEXPR bool **value** = __is_enum(T)

The documentation for this struct was generated from the following file:

- doctest.h

5.32 doctest::detail::types::is_pointer< T > Struct Template Reference

Inheritance diagram for doctest::detail::types::is_pointer< T >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

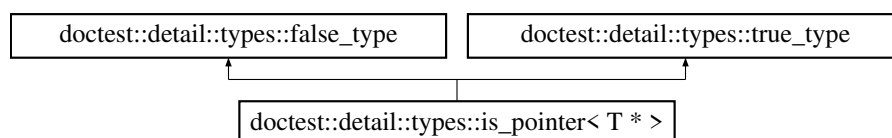
- static DOCTEST_CONSTEXPR bool **value** = false

The documentation for this struct was generated from the following file:

- doctest.h

5.33 doctest::detail::types::is_pointer< T * > Struct Template Reference

Inheritance diagram for doctest::detail::types::is_pointer< T * >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static DOCTEST_CONSTEXPR bool **value** = false

Static Public Attributes inherited from [doctest::detail::types::true_type](#)

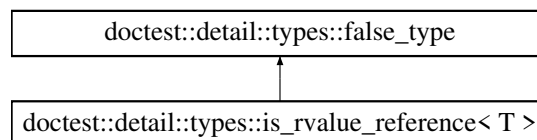
- static DOCTEST_CONSTEXPR bool **value** = true

The documentation for this struct was generated from the following file:

- doctest.h

5.34 doctest::detail::types::is_rvalue_reference< T > Struct Template Reference

Inheritance diagram for doctest::detail::types::is_rvalue_reference< T >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

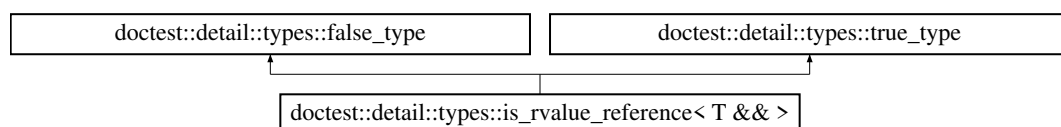
- static DOCTEST_CONSTEXPR bool **value** = false

The documentation for this struct was generated from the following file:

- doctest.h

5.35 doctest::detail::types::is_rvalue_reference< T && > Struct Template Reference

Inheritance diagram for doctest::detail::types::is_rvalue_reference< T && >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static DOCTEST_CONSTEXPR bool **value** = false

Static Public Attributes inherited from [doctest::detail::types::true_type](#)

- static DOCTEST_CONSTEXPR bool **value** = true

The documentation for this struct was generated from the following file:

- doctest.h

5.36 doctest::IsNaN< F > Struct Template Reference

Public Member Functions

- **IsNaN** (F f, bool flip=false)
- [IsNaN](#)< F > **operator!** () const
- **operator bool** () const

Public Attributes

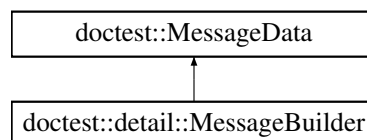
- F **value**
- bool **flipped**

The documentation for this struct was generated from the following file:

- doctest.h

5.37 doctest::detail::MessageBuilder Struct Reference

Inheritance diagram for doctest::detail::MessageBuilder:



Public Member Functions

- **MessageBuilder** (const char *file, int line, assertType::Enum severity)
- **MessageBuilder** (const [MessageBuilder](#) &)=delete
- **MessageBuilder** ([MessageBuilder](#) &&)=delete
- [MessageBuilder](#) & **operator=** (const [MessageBuilder](#) &)=delete
- [MessageBuilder](#) & **operator=** ([MessageBuilder](#) &&)=delete
- template<typename T>
[MessageBuilder](#) & **operator,** (const T &in)
- template<typename T>
DOCTEST_MSVC_SUPPRESS_WARNING_POP [MessageBuilder](#) & **operator**<< (const T &in)
- template<typename T>
[MessageBuilder](#) & **operator*** (const T &in)
- bool **log** ()
- void **react** ()

Public Attributes

- `std::ostream * m_stream`
- `bool logged = false`

Public Attributes inherited from `doctest::MessageData`

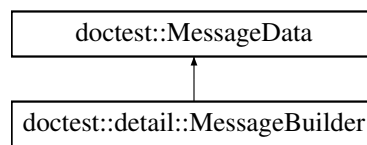
- `String m_string`
- `const char * m_file`
- `int m_line`
- `assertType::Enum m_severity`

The documentation for this struct was generated from the following file:

- `doctest.h`

5.38 `doctest::MessageData` Struct Reference

Inheritance diagram for `doctest::MessageData`:

**Public Attributes**

- `String m_string`
- `const char * m_file`
- `int m_line`
- `assertType::Enum m_severity`

The documentation for this struct was generated from the following file:

- `doctest.h`

5.39 `doctest::QueryData` Struct Reference**Public Attributes**

- `const TestRunStats * run_stats = nullptr`
- `const TestCaseData ** data = nullptr`
- `unsigned num_data = 0`

The documentation for this struct was generated from the following file:

- `doctest.h`

5.40 doctest::detail::RelationalComparator< int, L, R > Struct Template Reference

Public Member Functions

- bool **operator()** (const DOCTEST_REF_WRAP(L), const DOCTEST_REF_WRAP(R)) const

The documentation for this struct was generated from the following file:

- doctest.h

5.41 doctest::detail::types::remove_const< T > Struct Template Reference

Public Types

- using **type** = T

The documentation for this struct was generated from the following file:

- doctest.h

5.42 doctest::detail::types::remove_const< const T > Struct Template Reference

Public Types

- using **type** = T
- using **type**

The documentation for this struct was generated from the following file:

- doctest.h

5.43 doctest::detail::types::remove_reference< T > Struct Template Reference

Public Types

- using **type** = T

The documentation for this struct was generated from the following file:

- doctest.h

5.44 doctest::detail::types::remove_reference< T & > Struct Template Reference

Public Types

- using **type** = T
- using **type**

The documentation for this struct was generated from the following file:

- doctest.h

5.45 doctest::detail::types::remove_reference< T && > Struct Template Reference

Public Types

- using **type** = T
- using **type**

The documentation for this struct was generated from the following file:

- doctest.h

5.46 doctest::detail::Result Struct Reference

Public Member Functions

- **Result** (bool passed, const [String](#) &decomposition=[String](#)())

Public Attributes

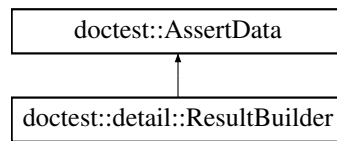
- bool **m_passed**
- [String](#) **m_decomp**

The documentation for this struct was generated from the following file:

- doctest.h

5.47 doctest::detail::ResultBuilder Struct Reference

Inheritance diagram for doctest::detail::ResultBuilder:



Public Member Functions

- **ResultBuilder** (assertType::Enum at, const char *file, int line, const char *expr, const char *exception_type="", const [String](#) &exception_string="")
- **ResultBuilder** (assertType::Enum at, const char *file, int line, const char *expr, const char *exception_type, const [StringContains](#) &exception_string)
- void **setResult** (const [Result](#) &res)
- template<int comparison, typename L, typename R>
DOCTEST_NOINLINE bool **binary_assert** (const DOCTEST_REF_WRAP(L) lhs, const DOCTEST_REF_WRAP(R) rhs)
- template<typename L>
DOCTEST_NOINLINE bool **unary_assert** (const DOCTEST_REF_WRAP(L) val)
- void **translateException** ()
- bool **log** ()
- void **react** () const

Public Member Functions inherited from [doctest::AssertData](#)

- **AssertData** (assertType::Enum at, const char *file, int line, const char *expr, const char *exception_type, const [StringContains](#) &exception_string)

Additional Inherited Members

Public Attributes inherited from [doctest::AssertData](#)

- const [TestCaseData](#) * **m_test_case**
- assertType::Enum **m_at**
- const char * **m_file**
- int **m_line**
- const char * **m_expr**
- bool **m_failed**
- bool **m_threw**
- [String](#) **m_exception**
- [String](#) **m_decomp**
- bool **m_threw_as**
- const char * **m_exception_type**
- class DOCTEST_INTERFACE [doctest::AssertData::StringContains](#) **m_exception_string**

5.47.1 Member Function Documentation

5.47.1.1 unary_assert()

```
template<typename L>
DOCTEST_NOINLINE bool doctest::detail::ResultBuilder::unary_assert (
    const DOCTEST_REF_WRAP(L) val) [inline]
```

OCLINT bitwise operator in conditional

The documentation for this struct was generated from the following file:

- doctest.h

5.48 doctest::detail::should_stringify_as_underlying_type< T > Struct Template Reference

Static Public Attributes

- static DOCTEST_CONSTEXPR bool **value** = [detail::types::is_enum<T>::value](#) && ![doctest::detail::has_insertion_operator<T>::value](#)

The documentation for this struct was generated from the following file:

- doctest.h

5.49 doctest::String Class Reference

Public Types

- using **size_type** = DOCTEST_CONFIG_STRING_SIZE_TYPE

Public Member Functions

- **String** (const char *in)
- **String** (const char *in, size_type in_size)
- **String** ([std::istream](#) &in, size_type in_size)
- **String** (const [String](#) &other)
- [String](#) & **operator=** (const [String](#) &other)
- [String](#) & **operator+=** (const [String](#) &other)
- **String** ([String](#) &&other) noexcept
- [String](#) & **operator=** ([String](#) &&other) noexcept
- char **operator[]** (size_type i) const
- char & **operator[]** (size_type i)
- const char * **c_str** () const
- char * **c_str** ()
- size_type **size** () const
- size_type **capacity** () const
- [String](#) **substr** (size_type pos, size_type cnt=npos) &&
- [String](#) **substr** (size_type pos, size_type cnt=npos) const &
- size_type **find** (char ch, size_type pos=0) const
- size_type **rfind** (char ch, size_type pos=npos) const
- int **compare** (const char *other, bool no_case=false) const
- int **compare** (const [String](#) &other, bool no_case=false) const

Static Public Attributes

- static DOctest_CONSTEXPR size_type **npos** = static_cast<size_type>(-1)

Friends

- DOctest_INTERFACE [std::ostream](#) & **operator**<< ([std::ostream](#) &s, const [String](#) &in)

The documentation for this class was generated from the following file:

- doctest.h

5.50 doctest::AssertData::StringContains Class Reference

Public Member Functions

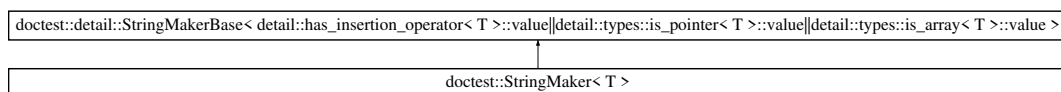
- **StringContains** (const [String](#) &str)
- **StringContains** ([Contains](#) cntn)
- bool **check** (const [String](#) &str)
- **operator const String &** () const
- const char * **c_str** () const

The documentation for this class was generated from the following file:

- doctest.h

5.51 doctest::StringMaker< T > Struct Template Reference

Inheritance diagram for doctest::StringMaker< T >:



Additional Inherited Members

Static Public Member Functions inherited from

[doctest::detail::StringMakerBase< \[detail::has_insertion_operator< T >::value||detail::types::is_pointer\]\(#\)](#)

- static [String](#) **convert** (const DOctest_REF_WRAP(T))

The documentation for this struct was generated from the following file:

- doctest.h

5.52 doctest::detail::StringMakerBase< C > Struct Template Reference

Static Public Member Functions

- `template<typename T>`
static [String convert](#) (const DOCTEST_REF_WRAP(T))

The documentation for this struct was generated from the following file:

- `doctest.h`

5.53 doctest::detail::StringMakerBase< true > Struct Reference

Static Public Member Functions

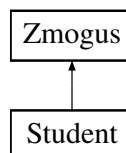
- `template<typename T>`
static [String convert](#) (const DOCTEST_REF_WRAP(T) in)
- static [String convert](#) (const DOCTEST_REF_WRAP(T))

The documentation for this struct was generated from the following file:

- `doctest.h`

5.54 Student Class Reference

Inheritance diagram for Student:



Public Member Functions

- **Student** (const [Student](#) &kitas)
- [Student](#) & **operator=** (const [Student](#) &kitas)
- **Student** ([Student](#) &&kitas)
- [Student](#) & **operator=** ([Student](#) &&kitas)
- `std::string` [getVardas](#) () const
- `std::string` [getPavarde](#) () const
- `std::vector< int >` [getNdBalai](#) () const
- `int` [getEgzaminas](#) () const
- `void` [setVardas](#) (const `std::string` &vardas)
- `void` [setPavarde](#) (const `std::string` &pavarde)
- `void` [setNdBalai](#) (const `std::vector< int >` &nd)
- `void` [setEgzaminas](#) (`int` egzaminas)
- `double` [skaiciuotiVidurki](#) () const
- `double` [skaiciuotiMediana](#) () const
- `double` [galutinisPazymys](#) (`bool` naudotiVidurki=true) const

Friends

- `std::istream & operator>> (std::istream &ivedimas, Student &s)`
- `std::ostream & operator<< (std::ostream &isvedimas, const Student &s)`

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- `std::string vardas`
- `std::string pavarde`

5.54.1 Member Function Documentation

5.54.1.1 `getPavarde()`

```
std::string Student::getPavarde () const [virtual]
```

Implements [Zmogus](#).

5.54.1.2 `getVardas()`

```
std::string Student::getVardas () const [virtual]
```

Implements [Zmogus](#).

5.54.1.3 `setPavarde()`

```
void Student::setPavarde (  
    const std::string & pavarde) [virtual]
```

Implements [Zmogus](#).

5.54.1.4 `setVardas()`

```
void Student::setVardas (  
    const std::string & vardas) [virtual]
```

Implements [Zmogus](#).

The documentation for this class was generated from the following files:

- `studentas.h`
- `studentas.cpp`

5.55 doctest::detail::Subcase Struct Reference

Public Member Functions

- **Subcase** (const [String](#) &name, const char *file, int line)
- **Subcase** (const [Subcase](#) &)=delete
- **Subcase** ([Subcase](#) &&)=delete
- [Subcase](#) & **operator=** (const [Subcase](#) &)=delete
- [Subcase](#) & **operator=** ([Subcase](#) &&)=delete
- **operator bool** () const

Public Attributes

- [SubcaseSignature](#) **m_signature**
- bool **m_entered** = false

The documentation for this struct was generated from the following file:

- doctest.h

5.56 doctest::SubcaseSignature Struct Reference

Public Member Functions

- bool **operator==** (const [SubcaseSignature](#) &other) const
- bool **operator<** (const [SubcaseSignature](#) &other) const

Public Attributes

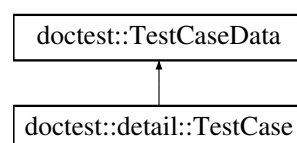
- [String](#) **m_name**
- const char * **m_file**
- int **m_line**

The documentation for this struct was generated from the following file:

- doctest.h

5.57 doctest::detail::TestCase Struct Reference

Inheritance diagram for doctest::detail::TestCase:



Public Member Functions

- **TestCase** (funcType test, const char *file, unsigned line, const [TestSuite](#) &test_suite, const [String](#) &type=[String](#)(), int template_id=-1)
- **TestCase** (const [TestCase](#) &other)
- **TestCase** ([TestCase](#) &&)=delete
- [TestCase](#) & **operator=** (const [TestCase](#) &other)
- DOCTEST_MSVC_SUPPRESS_WARNING_POP [TestCase](#) & **operator=** ([TestCase](#) &&)=delete
- [TestCase](#) & **operator*** (const char *in)
- template<typename T>
[TestCase](#) & **operator*** (const T &in)
- bool **operator<** (const [TestCase](#) &other) const

Public Attributes

- funcType **m_test**
- [String](#) **m_type**
- int **m_template_id**
- [String](#) **m_full_name**

Public Attributes inherited from [doctest::TestCaseData](#)

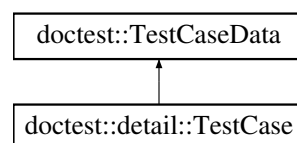
- [String](#) **m_file**
- unsigned **m_line**
- const char * **m_name**
- const char * **m_test_suite**
- const char * **m_description**
- bool **m_skip**
- bool **m_no_breaks**
- bool **m_no_output**
- bool **m_may_fail**
- bool **m_should_fail**
- int **m_expected_failures**
- double **m_timeout**

The documentation for this struct was generated from the following file:

- doctest.h

5.58 doctest::TestCaseData Struct Reference

Inheritance diagram for doctest::TestCaseData:



Public Attributes

- [String](#) **m_file**
- unsigned **m_line**
- const char * **m_name**
- const char * **m_test_suite**
- const char * **m_description**
- bool **m_skip**
- bool **m_no_breaks**
- bool **m_no_output**
- bool **m_may_fail**
- bool **m_should_fail**
- int **m_expected_failures**
- double **m_timeout**

The documentation for this struct was generated from the following file:

- doctest.h

5.59 doctest::TestCaseException Struct Reference

Public Attributes

- [String](#) **error_string**
- bool **is_crash**

The documentation for this struct was generated from the following file:

- doctest.h

5.60 doctest::detail::TestFailureException Struct Reference

The documentation for this struct was generated from the following file:

- doctest.h

5.61 doctest::TestRunStats Struct Reference

Public Attributes

- unsigned **numTestCases**
- unsigned **numTestCasesPassingFilters**
- unsigned **numTestSuitesPassingFilters**
- unsigned **numTestCasesFailed**
- int **numAsserts**
- int **numAssertsFailed**

The documentation for this struct was generated from the following file:

- doctest.h

5.62 doctest::detail::TestSuite Struct Reference

Public Member Functions

- [TestSuite](#) & **operator*** (const char *in)
- template<typename T>
[TestSuite](#) & **operator*** (const T &in)

Public Attributes

- const char * **m_test_suite** = nullptr
- const char * **m_description** = nullptr
- bool **m_skip** = false
- bool **m_no_breaks** = false
- bool **m_no_output** = false
- bool **m_may_fail** = false
- bool **m_should_fail** = false
- int **m_expected_failures** = 0
- double **m_timeout** = 0

The documentation for this struct was generated from the following file:

- doctest.h

5.63 doctest::detail::types::true_type Struct Reference

Inheritance diagram for doctest::detail::types::true_type:



Static Public Attributes

- static DOCTEST_CONSTEXPR bool **value** = true

The documentation for this struct was generated from the following file:

- doctest.h

5.64 std::tuple< Types > Class Template Reference

The documentation for this class was generated from the following file:

- doctest.h

5.65 doctest::detail::types::underlying_type< T > Struct Template Reference

Public Types

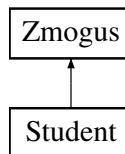
- using **type** = __underlying_type(T)

The documentation for this struct was generated from the following file:

- doctest.h

5.66 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- virtual std::string **getVardas** () const =0
- virtual std::string **getPavarde** () const =0
- virtual void **setVardas** (const std::string &v)=0
- virtual void **setPavarde** (const std::string &p)=0

Protected Attributes

- std::string **vardas**
- std::string **pavarde**

The documentation for this class was generated from the following file:

- zmogus.h

Chapter 6

File Documentation

6.1 DequeO.h

```
00001 #ifndef DEQUEO_H
00002 #define DEQUEO_H
00003
00004 #include <deque>
00005 #include "studentas.h"
00006
00007 void rusiotiStudentusISFailusDeque(const std::string& failas, std::deque<std::string>& failugenDeque);
00008 void rikiuotiStudentusPriesSkirstymaDeq(std::deque<Student>& studentai);
00009
00010 #endif
```

6.2 doctest.h

```
00001 // ===== lgtm
00002 // == DO NOT MODIFY THIS FILE BY HAND - IT IS AUTO GENERATED BY CMAKE! ==
00003 // =====
00004 //
00005 // doctest.h - the lightest feature-rich C++ single-header testing framework for unit tests and TDD
00006 //
00007 // Copyright (c) 2016-2023 Viktor Kirilov
00008 //
00009 // Distributed under the MIT Software License
00010 // See accompanying file LICENSE.txt or copy at
00011 // https://opensource.org/licenses/MIT
00012 //
00013 // The documentation can be found at the library's page:
00014 // https://github.com/doctest/doctest/blob/master/doc/markdown/readme.md
00015 //
00016 // =====
00017 // =====
00018 // =====
00019 //
00020 // The library is heavily influenced by Catch - https://github.com/catchorg/Catch2
00021 // which uses the Boost Software License - Version 1.0
00022 // see here - https://github.com/catchorg/Catch2/blob/master/LICENSE.txt
00023 //
00024 // The concept of subcases (sections in Catch) and expression decomposition are from there.
00025 // Some parts of the code are taken directly:
00026 // - stringification - the detection of "ostream& operator<<(ostream&, const T&)" and StringMaker<>
00027 // - the Approx() helper class for floating point comparison
00028 // - colors in the console
00029 // - breaking into a debugger
00030 // - signal / SEH handling
00031 // - timer
00032 // - XmlWriter class - thanks to Phil Nash for allowing the direct reuse (AKA copy/paste)
00033 //
00034 // The expression decomposing templates are taken from lest - https://github.com/martinmoene/lest
00035 // which uses the Boost Software License - Version 1.0
00036 // see here - https://github.com/martinmoene/lest/blob/master/LICENSE.txt
00037 //
00038 // =====
00039 // =====
```

```

00040 // =====
00041
00042 #ifndef DOCTEST_LIBRARY_INCLUDED
00043 #define DOCTEST_LIBRARY_INCLUDED
00044
00045 // =====
00046 // == VERSION =====
00047 // =====
00048
00049 #define DOCTEST_VERSION_MAJOR 2
00050 #define DOCTEST_VERSION_MINOR 4
00051 #define DOCTEST_VERSION_PATCH 11
00052
00053 // util we need here
00054 #define DOCTEST_TOSTR_IMPL(x) #x
00055 #define DOCTEST_TOSTR(x) DOCTEST_TOSTR_IMPL(x)
00056
00057 #define DOCTEST_VERSION_STR                                     \
00058     DOCTEST_TOSTR(DOCTEST_VERSION_MAJOR) ". "              \
00059     DOCTEST_TOSTR(DOCTEST_VERSION_MINOR) ". "              \
00060     DOCTEST_TOSTR(DOCTEST_VERSION_PATCH)                    \
00061
00062 #define DOCTEST_VERSION                                       \
00063     (DOCTEST_VERSION_MAJOR * 10000 + DOCTEST_VERSION_MINOR * 100 + DOCTEST_VERSION_PATCH) \
00064
00065 // =====
00066 // == COMPILER VERSION =====
00067 // =====
00068
00069 // ideas for the version stuff are taken from here: https://github.com/cxxstuff/cxx\_detect
00070
00071 #ifdef _MSC_VER
00072 #define DOCTEST_CPLUSPLUS _MSC_VER
00073 #else
00074 #define DOCTEST_CPLUSPLUS __cplusplus
00075 #endif
00076
00077 #define DOCTEST_COMPILER(MAJOR, MINOR, PATCH) ((MAJOR)*1000000 + (MINOR)*100000 + (PATCH))
00078
00079 // GCC/Clang and GCC/MSVC are mutually exclusive, but Clang/MSVC are not because of clang-cl...
00080 #if defined(_MSC_VER) && defined(_MSC_FULL_VER)
00081 #if _MSC_VER == _MSC_FULL_VER / 10000
00082 #define DOCTEST_MSVC DOCTEST_COMPILER(_MSC_VER / 100, _MSC_VER % 100, _MSC_FULL_VER % 10000)
00083 #else // MSVC
00084 #define DOCTEST_MSVC                                     \
00085     DOCTEST_COMPILER(_MSC_VER / 100, (_MSC_FULL_VER / 100000) % 100, _MSC_FULL_VER % 100000) \
00086 #endif // MSVC
00087 #endif // MSVC
00088 #if defined(__clang__) && defined(__clang_minor__) && defined(__clang_patchlevel__)
00089 #define DOCTEST_CLANG DOCTEST_COMPILER(__clang_major__, __clang_minor__, __clang_patchlevel__)
00090 #elif defined(__GNUC__) && defined(__GNUC_MINOR__) && defined(__GNUC_PATCHLEVEL__) &&
00091     !defined(__INTEL_COMPILER)
00092 #define DOCTEST_GCC DOCTEST_COMPILER(__GNUC__, __GNUC_MINOR__, __GNUC_PATCHLEVEL__)
00093 #endif // GCC
00094 #if defined(__INTEL_COMPILER)
00095 #define DOCTEST_ICC DOCTEST_COMPILER(__INTEL_COMPILER / 100, __INTEL_COMPILER % 100, 0)
00096 #endif // ICC
00097
00098 #ifndef DOCTEST_MSVC
00099 #define DOCTEST_MSVC 0
00100 #endif // DOCTEST_MSVC
00101 #ifndef DOCTEST_CLANG
00102 #define DOCTEST_CLANG 0
00103 #endif // DOCTEST_CLANG
00104 #ifndef DOCTEST_GCC
00105 #define DOCTEST_GCC 0
00106 #endif // DOCTEST_GCC
00107 #ifndef DOCTEST_ICC
00108 #define DOCTEST_ICC 0
00109 #endif // DOCTEST_ICC
00110
00111 // =====
00112 // == COMPILER WARNINGS HELPERS =====
00113 // =====
00114
00115 #if DOCTEST_CLANG && !DOCTEST_ICC
00116 #define DOCTEST_PRAGMA_TO_STR(x) _Pragma(#x)
00117 #define DOCTEST_CLANG_SUPPRESS_WARNING_PUSH _Pragma("clang diagnostic push")
00118 #define DOCTEST_CLANG_SUPPRESS_WARNING(w) DOCTEST_PRAGMA_TO_STR(clang diagnostic ignored w)
00119 #define DOCTEST_CLANG_SUPPRESS_WARNING_POP _Pragma("clang diagnostic pop")
00120 #define DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH(w) \
00121     DOCTEST_CLANG_SUPPRESS_WARNING_PUSH DOCTEST_CLANG_SUPPRESS_WARNING(w) \
00122 #else // DOCTEST_CLANG
00123 #define DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
00124 #define DOCTEST_CLANG_SUPPRESS_WARNING(w)
00125 #define DOCTEST_CLANG_SUPPRESS_WARNING_POP
00126 #define DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH(w)

```



```

00127 #endif // DOCTEST_CLANG
00128
00129 #if DOCTEST_GCC
00130 #define DOCTEST_PRAGMA_TO_STR(x) _Pragma(#x)
00131 #define DOCTEST_GCC_SUPPRESS_WARNING_PUSH _Pragma("GCC diagnostic push")
00132 #define DOCTEST_GCC_SUPPRESS_WARNING(w) DOCTEST_PRAGMA_TO_STR(GCC diagnostic ignored w)
00133 #define DOCTEST_GCC_SUPPRESS_WARNING_POP _Pragma("GCC diagnostic pop")
00134 #define DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH(w) \
00135     DOCTEST_GCC_SUPPRESS_WARNING_PUSH DOCTEST_GCC_SUPPRESS_WARNING(w)
00136 #else // DOCTEST_GCC
00137 #define DOCTEST_GCC_SUPPRESS_WARNING_PUSH
00138 #define DOCTEST_GCC_SUPPRESS_WARNING(w)
00139 #define DOCTEST_GCC_SUPPRESS_WARNING_POP
00140 #define DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH(w)
00141 #endif // DOCTEST_GCC
00142
00143 #if DOCTEST_MSVC
00144 #define DOCTEST_MSVC_SUPPRESS_WARNING_PUSH __pragma(warning(push))
00145 #define DOCTEST_MSVC_SUPPRESS_WARNING(w) __pragma(warning(disable : w))
00146 #define DOCTEST_MSVC_SUPPRESS_WARNING_POP __pragma(warning(pop))
00147 #define DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(w) \
00148     DOCTEST_MSVC_SUPPRESS_WARNING_PUSH DOCTEST_MSVC_SUPPRESS_WARNING(w)
00149 #else // DOCTEST_MSVC
00150 #define DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
00151 #define DOCTEST_MSVC_SUPPRESS_WARNING(w)
00152 #define DOCTEST_MSVC_SUPPRESS_WARNING_POP
00153 #define DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(w)
00154 #endif // DOCTEST_MSVC
00155
00156 // =====
00157 // == COMPILER WARNINGS =====
00158 // =====
00159
00160 // both the header and the implementation suppress all of these,
00161 // so it only makes sense to aggregate them like so
00162 #define DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH \
00163     DOCTEST_CLANG_SUPPRESS_WARNING_PUSH \
00164     DOCTEST_CLANG_SUPPRESS_WARNING("-Wunknown-pragmas") \
00165     DOCTEST_CLANG_SUPPRESS_WARNING("-Wweak-vtables") \
00166     DOCTEST_CLANG_SUPPRESS_WARNING("-Wpadded") \
00167     DOCTEST_CLANG_SUPPRESS_WARNING("-Wmissing-prototypes") \
00168     DOCTEST_CLANG_SUPPRESS_WARNING("-Wc++98-compat") \
00169     DOCTEST_CLANG_SUPPRESS_WARNING("-Wc++98-compat-pedantic") \
00170 \
00171     DOCTEST_GCC_SUPPRESS_WARNING_PUSH \
00172     DOCTEST_GCC_SUPPRESS_WARNING("-Wunknown-pragmas") \
00173     DOCTEST_GCC_SUPPRESS_WARNING("-Wpragmas") \
00174     DOCTEST_GCC_SUPPRESS_WARNING("-Weffc++") \
00175     DOCTEST_GCC_SUPPRESS_WARNING("-Wstrict-overflow") \
00176     DOCTEST_GCC_SUPPRESS_WARNING("-Wstrict-aliasing") \
00177     DOCTEST_GCC_SUPPRESS_WARNING("-Wmissing-declarations") \
00178     DOCTEST_GCC_SUPPRESS_WARNING("-Wuseless-cast") \
00179     DOCTEST_GCC_SUPPRESS_WARNING("-Wnoexcept") \
00180 \
00181     DOCTEST_MSVC_SUPPRESS_WARNING_PUSH \
00182     /* these 4 also disabled globally via cmake: */ \
00183     DOCTEST_MSVC_SUPPRESS_WARNING(4514) /* unreferenced inline function has been removed */ \
00184     DOCTEST_MSVC_SUPPRESS_WARNING(4571) /* SEH related */ \
00185     DOCTEST_MSVC_SUPPRESS_WARNING(4710) /* function not inlined */ \
00186     DOCTEST_MSVC_SUPPRESS_WARNING(4711) /* function selected for inline expansion */ \
00187     /* common ones */ \
00188     DOCTEST_MSVC_SUPPRESS_WARNING(4616) /* invalid compiler warning */ \
00189     DOCTEST_MSVC_SUPPRESS_WARNING(4619) /* invalid compiler warning */ \
00190     DOCTEST_MSVC_SUPPRESS_WARNING(4996) /* The compiler encountered a deprecated declaration */ \
00191     DOCTEST_MSVC_SUPPRESS_WARNING(4706) /* assignment within conditional expression */ \
00192     DOCTEST_MSVC_SUPPRESS_WARNING(4512) /* 'class' : assignment operator could not be generated */ \
00193     DOCTEST_MSVC_SUPPRESS_WARNING(4127) /* conditional expression is constant */ \
00194     DOCTEST_MSVC_SUPPRESS_WARNING(4820) /* padding */ \
00195     DOCTEST_MSVC_SUPPRESS_WARNING(4625) /* copy constructor was implicitly deleted */ \
00196     DOCTEST_MSVC_SUPPRESS_WARNING(4626) /* assignment operator was implicitly deleted */ \
00197     DOCTEST_MSVC_SUPPRESS_WARNING(5027) /* move assignment operator was implicitly deleted */ \
00198     DOCTEST_MSVC_SUPPRESS_WARNING(5026) /* move constructor was implicitly deleted */ \
00199     DOCTEST_MSVC_SUPPRESS_WARNING(4640) /* construction of local static object not thread-safe */ \
00200     DOCTEST_MSVC_SUPPRESS_WARNING(5045) /* Spectre mitigation for memory load */ \
00201     DOCTEST_MSVC_SUPPRESS_WARNING(5264) /* 'variable-name': 'const' variable is not used */ \
00202     /* static analysis */ \
00203     DOCTEST_MSVC_SUPPRESS_WARNING(26439) /* Function may not throw. Declare it 'noexcept' */ \
00204     DOCTEST_MSVC_SUPPRESS_WARNING(26495) /* Always initialize a member variable */ \
00205     DOCTEST_MSVC_SUPPRESS_WARNING(26451) /* Arithmetic overflow ... */ \
00206     DOCTEST_MSVC_SUPPRESS_WARNING(26444) /* Avoid unnamed objects with custom ctor and dtor... */ \
00207     DOCTEST_MSVC_SUPPRESS_WARNING(26812) /* Prefer 'enum class' over 'enum' */ \
00208 \
00209 #define DOCTEST_SUPPRESS_COMMON_WARNINGS_POP \
00210     DOCTEST_CLANG_SUPPRESS_WARNING_POP \
00211     DOCTEST_GCC_SUPPRESS_WARNING_POP \
00212     DOCTEST_MSVC_SUPPRESS_WARNING_POP \
00213

```

```

00214 DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH
00215
00216 DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
00217 DOCTEST_CLANG_SUPPRESS_WARNING("-Wnon-virtual-dtor")
00218 DOCTEST_CLANG_SUPPRESS_WARNING("-Wdeprecated")
00219
00220 DOCTEST_GCC_SUPPRESS_WARNING_PUSH
00221 DOCTEST_GCC_SUPPRESS_WARNING("-Wctor-dtor-privacy")
00222 DOCTEST_GCC_SUPPRESS_WARNING("-Wnon-virtual-dtor")
00223 DOCTEST_GCC_SUPPRESS_WARNING("-Wsign-promo")
00224
00225 DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
00226 DOCTEST_MSVC_SUPPRESS_WARNING(4623) // default constructor was implicitly defined as deleted
00227
00228 #define DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN                                     \
00229     DOCTEST_MSVC_SUPPRESS_WARNING_PUSH                                                                \
00230     DOCTEST_MSVC_SUPPRESS_WARNING(4548) /* before comma no effect; expected side - effect */           \
00231     DOCTEST_MSVC_SUPPRESS_WARNING(4265) /* virtual functions, but destructor is not virtual */         \
00232     DOCTEST_MSVC_SUPPRESS_WARNING(4986) /* exception specification does not match previous */         \
00233     DOCTEST_MSVC_SUPPRESS_WARNING(4350) /* 'member1' called instead of 'member2' */                 \
00234     DOCTEST_MSVC_SUPPRESS_WARNING(4668) /* not defined as a preprocessor macro */                   \
00235     DOCTEST_MSVC_SUPPRESS_WARNING(4365) /* signed/unsigned mismatch */                               \
00236     DOCTEST_MSVC_SUPPRESS_WARNING(4774) /* format string not a string literal */                   \
00237     DOCTEST_MSVC_SUPPRESS_WARNING(4820) /* padding */                                                \
00238     DOCTEST_MSVC_SUPPRESS_WARNING(4625) /* copy constructor was implicitly deleted */               \
00239     DOCTEST_MSVC_SUPPRESS_WARNING(4626) /* assignment operator was implicitly deleted */           \
00240     DOCTEST_MSVC_SUPPRESS_WARNING(5027) /* move assignment operator implicitly deleted */           \
00241     DOCTEST_MSVC_SUPPRESS_WARNING(5026) /* move constructor was implicitly deleted */               \
00242     DOCTEST_MSVC_SUPPRESS_WARNING(4623) /* default constructor was implicitly deleted */           \
00243     DOCTEST_MSVC_SUPPRESS_WARNING(5039) /* pointer to pot. throwing function passed to extern C */ \
00244     DOCTEST_MSVC_SUPPRESS_WARNING(5045) /* Spectre mitigation for memory load */                   \
00245     DOCTEST_MSVC_SUPPRESS_WARNING(5105) /* macro producing 'defined' has undefined behavior */     \
00246     DOCTEST_MSVC_SUPPRESS_WARNING(4738) /* storing float result in memory, loss of performance */ \
00247     DOCTEST_MSVC_SUPPRESS_WARNING(5262) /* implicit fall-through */
00248
00249 #define DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END DOCTEST_MSVC_SUPPRESS_WARNING_POP
00250
00251 // =====
00252 // == FEATURE DETECTION =====
00253 // =====
00254
00255 // general compiler feature support table: https://en.cppreference.com/w/cpp/compiler_support
00256 // MSVC C++11 feature support table: https://msdn.microsoft.com/en-us/library/hh567368.aspx
00257 // GCC C++11 feature support table: https://gcc.gnu.org/projects/cxx-status.html
00258 // MSVC version table:
00259 // https://en.wikipedia.org/wiki/Microsoft_Visual_C%2B%2B#Internal_version_numbering
00260 // MSVC++ 14.3 (17) _MSC_VER == 1930 (Visual Studio 2022)
00261 // MSVC++ 14.2 (16) _MSC_VER == 1920 (Visual Studio 2019)
00262 // MSVC++ 14.1 (15) _MSC_VER == 1910 (Visual Studio 2017)
00263 // MSVC++ 14.0      _MSC_VER == 1900 (Visual Studio 2015)
00264 // MSVC++ 12.0      _MSC_VER == 1800 (Visual Studio 2013)
00265 // MSVC++ 11.0      _MSC_VER == 1700 (Visual Studio 2012)
00266 // MSVC++ 10.0      _MSC_VER == 1600 (Visual Studio 2010)
00267 // MSVC++ 9.0       _MSC_VER == 1500 (Visual Studio 2008)
00268 // MSVC++ 8.0       _MSC_VER == 1400 (Visual Studio 2005)
00269
00270 // Universal Windows Platform support
00271 #if defined(WINAPI_FAMILY) && (WINAPI_FAMILY == WINAPI_FAMILY_APP)
00272 #define DOCTEST_CONFIG_NO_WINDOWS_SEH
00273 #endif // WINAPI_FAMILY
00274 #if DOCTEST_MSVC && !defined(DOCTEST_CONFIG_WINDOWS_SEH)
00275 #define DOCTEST_CONFIG_WINDOWS_SEH
00276 #endif // MSVC
00277 #if defined(DOCTEST_CONFIG_NO_WINDOWS_SEH) && defined(DOCTEST_CONFIG_WINDOWS_SEH)
00278 #undef DOCTEST_CONFIG_WINDOWS_SEH
00279 #endif // DOCTEST_CONFIG_NO_WINDOWS_SEH
00280
00281 #if !defined(_WIN32) && !defined(__QNX__) && !defined(DOCTEST_CONFIG_POSIX_SIGNALS) && \
00282     !defined(__EMSCRIPTEN__) && !defined(__wasi__) \
00283 #define DOCTEST_CONFIG_POSIX_SIGNALS
00284 #endif // _WIN32
00285 #if defined(DOCTEST_CONFIG_NO_POSIX_SIGNALS) && defined(DOCTEST_CONFIG_POSIX_SIGNALS)
00286 #undef DOCTEST_CONFIG_POSIX_SIGNALS
00287 #endif // DOCTEST_CONFIG_NO_POSIX_SIGNALS
00288
00289 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
00290 #if !defined(__cpp_exceptions) && !defined(__EXCEPTIONS) && !defined(_CPPUNWIND) \
00291     || defined(__wasi__) \
00292 #define DOCTEST_CONFIG_NO_EXCEPTIONS
00293 #endif // no exceptions
00294 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
00295
00296 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS
00297 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
00298 #define DOCTEST_CONFIG_NO_EXCEPTIONS
00299 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
00300 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS

```

```

00301
00302 #if defined(DOCTEST_CONFIG_NO_EXCEPTIONS) && !defined(DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS)
00303 #define DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
00304 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS && !DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
00305
00306 #ifdef __wasi__
00307 #define DOCTEST_CONFIG_NO_MULTITHREADING
00308 #endif
00309
00310 #if defined(DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN) && !defined(DOCTEST_CONFIG_IMPLEMENT)
00311 #define DOCTEST_CONFIG_IMPLEMENT
00312 #endif // DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
00313
00314 #if defined(_WIN32) || defined(__CYGWIN__)
00315 #if DOCTEST_MSVC
00316 #define DOCTEST_SYMBOL_EXPORT __declspec(dllexport)
00317 #define DOCTEST_SYMBOL_IMPORT __declspec(dllimport)
00318 #else // MSVC
00319 #define DOCTEST_SYMBOL_EXPORT __attribute__((dllexport))
00320 #define DOCTEST_SYMBOL_IMPORT __attribute__((dllimport))
00321 #endif // MSVC
00322 #else // _WIN32
00323 #define DOCTEST_SYMBOL_EXPORT __attribute__((visibility("default")))
00324 #define DOCTEST_SYMBOL_IMPORT
00325 #endif // _WIN32
00326
00327 #ifdef DOCTEST_CONFIG_IMPLEMENTATION_IN_DLL
00328 #if DOCTEST_CONFIG_IMPLEMENT
00329 #define DOCTEST_INTERFACE DOCTEST_SYMBOL_EXPORT
00330 #else // DOCTEST_CONFIG_IMPLEMENT
00331 #define DOCTEST_INTERFACE DOCTEST_SYMBOL_IMPORT
00332 #endif // DOCTEST_CONFIG_IMPLEMENT
00333 #else // DOCTEST_CONFIG_IMPLEMENTATION_IN_DLL
00334 #define DOCTEST_INTERFACE
00335 #endif // DOCTEST_CONFIG_IMPLEMENTATION_IN_DLL
00336
00337 // needed for extern template instantiations
00338 // see https://github.com/fmtlib/fmt/issues/2228
00339 #if DOCTEST_MSVC
00340 #define DOCTEST_INTERFACE_DECL
00341 #define DOCTEST_INTERFACE_DEF DOCTEST_INTERFACE
00342 #else // DOCTEST_MSVC
00343 #define DOCTEST_INTERFACE_DECL DOCTEST_INTERFACE
00344 #define DOCTEST_INTERFACE_DEF
00345 #endif // DOCTEST_MSVC
00346
00347 #define DOCTEST_EMPTY
00348
00349 #if DOCTEST_MSVC
00350 #define DOCTEST_NOINLINE __declspec(noinline)
00351 #define DOCTEST_UNUSED
00352 #define DOCTEST_ALIGNMENT(x)
00353 #elif DOCTEST_CLANG && DOCTEST_CLANG < DOCTEST_COMPILER(3, 5, 0)
00354 #define DOCTEST_NOINLINE
00355 #define DOCTEST_UNUSED
00356 #define DOCTEST_ALIGNMENT(x)
00357 #else
00358 #define DOCTEST_NOINLINE __attribute__((noinline))
00359 #define DOCTEST_UNUSED __attribute__((unused))
00360 #define DOCTEST_ALIGNMENT(x) __attribute__((aligned(x)))
00361 #endif
00362
00363 #ifdef DOCTEST_CONFIG_NO_CONTRADICTING_INLINE
00364 #define DOCTEST_INLINE_NOINLINE inline
00365 #else
00366 #define DOCTEST_INLINE_NOINLINE inline DOCTEST_NOINLINE
00367 #endif
00368
00369 #ifndef DOCTEST_NORETURN
00370 #if DOCTEST_MSVC && (DOCTEST_MSVC < DOCTEST_COMPILER(19, 0, 0))
00371 #define DOCTEST_NORETURN
00372 #else // DOCTEST_MSVC
00373 #define DOCTEST_NORETURN [[noreturn]]
00374 #endif // DOCTEST_MSVC
00375 #endif // DOCTEST_NORETURN
00376
00377 #ifndef DOCTEST_NOEXCEPT
00378 #if DOCTEST_MSVC && (DOCTEST_MSVC < DOCTEST_COMPILER(19, 0, 0))
00379 #define DOCTEST_NOEXCEPT
00380 #else // DOCTEST_MSVC
00381 #define DOCTEST_NOEXCEPT noexcept
00382 #endif // DOCTEST_MSVC
00383 #endif // DOCTEST_NOEXCEPT
00384
00385 #ifndef DOCTEST_CONSTEXPR
00386 #if DOCTEST_MSVC && (DOCTEST_MSVC < DOCTEST_COMPILER(19, 0, 0))
00387 #define DOCTEST_CONSTEXPR const

```

```

00388 #define DOCTEST_CONSTEXPR_FUNC inline
00389 #else // DOCTEST_MSVC
00390 #define DOCTEST_CONSTEXPR constexpr
00391 #define DOCTEST_CONSTEXPR_FUNC constexpr
00392 #endif // DOCTEST_MSVC
00393 #endif // DOCTEST_CONSTEXPR
00394
00395 #ifndef DOCTEST_NO_SANITIZE_INTEGER
00396 #if DOCTEST_CLANG >= DOCTEST_COMPILER(3, 7, 0)
00397 #define DOCTEST_NO_SANITIZE_INTEGER __attribute__((no_sanitize("integer")))
00398 #else
00399 #define DOCTEST_NO_SANITIZE_INTEGER
00400 #endif
00401 #endif // DOCTEST_NO_SANITIZE_INTEGER
00402
00403 // =====
00404 // == FEATURE DETECTION END ==
00405 // =====
00406
00407 #define DOCTEST_DECLARE_INTERFACE(name) \
00408     virtual ~name(); \
00409     name() = default; \
00410     name(const name&) = delete; \
00411     name(name&&) = delete; \
00412     name& operator=(const name&) = delete; \
00413     name& operator=(name&&) = delete; \
00414
00415 #define DOCTEST_DEFINE_INTERFACE(name) \
00416     name::~name() = default; \
00417
00418 // internal macros for string concatenation and anonymous variable name generation
00419 #define DOCTEST_CAT_IMPL(s1, s2) s1##s2
00420 #define DOCTEST_CAT(s1, s2) DOCTEST_CAT_IMPL(s1, s2)
00421 #ifdef __COUNTER__ // not standard and may be missing for some compilers
00422 #define DOCTEST_ANONYMOUS(x) DOCTEST_CAT(x, __COUNTER__)
00423 #else // __COUNTER__
00424 #define DOCTEST_ANONYMOUS(x) DOCTEST_CAT(x, __LINE__)
00425 #endif // __COUNTER__
00426
00427 #ifndef DOCTEST_CONFIG_ASSERTION_PARAMETERS_BY_VALUE
00428 #define DOCTEST_REF_WRAP(x) x&
00429 #else // DOCTEST_CONFIG_ASSERTION_PARAMETERS_BY_VALUE
00430 #define DOCTEST_REF_WRAP(x) x
00431 #endif // DOCTEST_CONFIG_ASSERTION_PARAMETERS_BY_VALUE
00432
00433 // not using __APPLE__ because... this is how Catch does it
00434 #ifdef __MAC_OS_X_VERSION_MIN_REQUIRED
00435 #define DOCTEST_PLATFORM_MAC
00436 #elif defined(__IPHONE_OS_VERSION_MIN_REQUIRED)
00437 #define DOCTEST_PLATFORM_IPHONE
00438 #elif defined(__WIN32)
00439 #define DOCTEST_PLATFORM_WINDOWS
00440 #elif defined(__wasi__)
00441 #define DOCTEST_PLATFORM_WASI
00442 #else // DOCTEST_PLATFORM
00443 #define DOCTEST_PLATFORM_LINUX
00444 #endif // DOCTEST_PLATFORM
00445
00446 namespace doctest { namespace detail {
00447     static DOCTEST_CONSTEXPR int consume(const int*, int) noexcept { return 0; }
00448 }}
00449
00450 #define DOCTEST_GLOBAL_NO_WARNINGS(var, ...) \
00451     DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wglobal-constructors") \
00452     static const int var = doctest::detail::consume(&var, __VA_ARGS__); \
00453     DOCTEST_CLANG_SUPPRESS_WARNING_POP
00454
00455 #ifndef DOCTEST_BREAK_INTO_DEBUGGER
00456 // should probably take a look at https://github.com/scottt/debugbreak
00457 #ifdef DOCTEST_PLATFORM_LINUX
00458 #if defined(__GNUC__) && (defined(__i386) || defined(__x86_64))
00459 // Break at the location of the failing check if possible
00460 #define DOCTEST_BREAK_INTO_DEBUGGER() __asm__("int $3\n" : :) // NOLINT(hicpp-no-assembler)
00461 #else
00462 #include <signal.h>
00463 #define DOCTEST_BREAK_INTO_DEBUGGER() raise(SIGTRAP)
00464 #endif
00465 #elif defined(DOCTEST_PLATFORM_MAC)
00466 #if defined(__x86_64) || defined(__x86_64__) || defined(__amd64__) || defined(__i386)
00467 #define DOCTEST_BREAK_INTO_DEBUGGER() __asm__("int $3\n" : :) // NOLINT(hicpp-no-assembler)
00468 #elif defined(__ppc__) || defined(__ppc64__)
00469 // https://www.cocoawithlove.com/2008/03/break-into-debugger.html
00470 #define DOCTEST_BREAK_INTO_DEBUGGER() __asm__("li r0, 20\nsc\nnop\nli r0, 37\nli r4, 2\nsc\nnop\n": : "memory", "r0", "r3", "r4") // NOLINT(hicpp-no-assembler)
00471 #else
00472 #define DOCTEST_BREAK_INTO_DEBUGGER() __asm__("brk #0"); // NOLINT(hicpp-no-assembler)
00473 #endif

```

```

00474 #elif DOCTEST_MSVC
00475 #define DOCTEST_BREAK_INTTO_DEBUGGER() __debugbreak()
00476 #elif defined(__MINGW32__)
00477 DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wredundant-decls")
00478 extern "C" __declspec(dllimport) void __stdcall DebugBreak();
00479 DOCTEST_GCC_SUPPRESS_WARNING_POP
00480 #define DOCTEST_BREAK_INTTO_DEBUGGER() ::DebugBreak()
00481 #else // linux
00482 #define DOCTEST_BREAK_INTTO_DEBUGGER() (static_cast<void>(0))
00483 #endif // linux
00484 #endif // DOCTEST_BREAK_INTTO_DEBUGGER
00485
00486 // this is kept here for backwards compatibility since the config option was changed
00487 #ifdef DOCTEST_CONFIG_USE_IOSFWD
00488 #ifndef DOCTEST_CONFIG_USE_STD_HEADERS
00489 #define DOCTEST_CONFIG_USE_STD_HEADERS
00490 #endif
00491 #endif // DOCTEST_CONFIG_USE_IOSFWD
00492
00493 // for clang - always include ciso646 (which drags some std stuff) because
00494 // we want to check if we are using libc++ with the _LIBCPP_VERSION macro in
00495 // which case we don't want to forward declare stuff from std - for reference:
00496 // https://github.com/doctest/doctest/issues/126
00497 // https://github.com/doctest/doctest/issues/356
00498 #if DOCTEST_CLANG
00499 #include <ciso646>
00500 #endif // clang
00501
00502 #ifdef _LIBCPP_VERSION
00503 #ifndef DOCTEST_CONFIG_USE_STD_HEADERS
00504 #define DOCTEST_CONFIG_USE_STD_HEADERS
00505 #endif
00506 #endif // _LIBCPP_VERSION
00507
00508 #ifdef DOCTEST_CONFIG_USE_STD_HEADERS
00509 #ifndef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
00510 #define DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
00511 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
00512 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
00513 #include <cstdint>
00514 #include <ostream>
00515 #include <istream>
00516 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END
00517 #else // DOCTEST_CONFIG_USE_STD_HEADERS
00518
00519 // Forward declaring 'X' in namespace std is not permitted by the C++ Standard.
00520 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4643)
00521
00522 namespace std { // NOLINT(cert-dcl58-cpp)
00523 typedef decltype(nullptr) nullptr_t; // NOLINT(modernize-use-using)
00524 typedef decltype(sizeof(void*)) size_t; // NOLINT(modernize-use-using)
00525 template <class charT>
00526 struct char_traits;
00527 template <>
00528 struct char_traits<char>;
00529 template <class charT, class traits>
00530 class basic_ostream; // NOLINT(fuchsia-virtual-inheritance)
00531 typedef basic_ostream<char, char_traits<char>> ostream; // NOLINT(modernize-use-using)
00532 template <class traits>
00533 // NOLINTNEXTLINE
00534 basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, const char*);
00535 template <class charT, class traits>
00536 class basic_istream;
00537 typedef basic_istream<char, char_traits<char>> istream; // NOLINT(modernize-use-using)
00538 template <class... Types>
00539 class tuple;
00540 #if DOCTEST_MSVC >= DOCTEST_COMPILER(19, 20, 0)
00541 // see this issue on why this is needed: https://github.com/doctest/doctest/issues/183
00542 template <class Ty>
00543 class allocator;
00544 template <class Elem, class Traits, class Alloc>
00545 class basic_string;
00546 using string = basic_string<char, char_traits<char>, allocator<char>;
00547 #endif // VS 2019
00548 } // namespace std
00549
00550 DOCTEST_MSVC_SUPPRESS_WARNING_POP
00551
00552 #endif // DOCTEST_CONFIG_USE_STD_HEADERS
00553
00554 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
00555 #include <type_traits>
00556 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
00557
00558 namespace doctest {
00559
00560 using std::size_t;

```

```

00561
00562 DOCTEST_INTERFACE extern bool is_running_in_test;
00563
00564 #ifndef DOCTEST_CONFIG_STRING_SIZE_TYPE
00565 #define DOCTEST_CONFIG_STRING_SIZE_TYPE unsigned
00566 #endif
00567
00568 // A 24 byte string class (can be as small as 17 for x64 and 13 for x86) that can hold strings with
length
00569 // of up to 23 chars on the stack before going on the heap - the last byte of the buffer is used for:
00570 // - "is small" bit - the highest bit - if "0" then it is small - otherwise its "1" (128)
00571 // - if small - capacity left before going on the heap - using the lowest 5 bits
00572 // - if small - 2 bits are left unused - the second and third highest ones
00573 // - if small - acts as a null terminator if strlen() is 23 (24 including the null terminator)
00574 // and the "is small" bit remains "0" ("as well as the capacity left") so its OK
00575 // Idea taken from this lecture about the string implementation of facebook/folly - fbstring
00576 // https://www.youtube.com/watch?v=kPR8h4-qZdk
00577 // TODO:
00578 // - optimizations - like not deleting memory unnecessarily in operator= and etc.
00579 // - resize/reserve/clear
00580 // - replace
00581 // - back/front
00582 // - iterator stuff
00583 // - find & friends
00584 // - push_back/pop_back
00585 // - assign/insert/erase
00586 // - relational operators as free functions - taking const char* as one of the params
00587 class DOCTEST_INTERFACE String
00588 {
00589 public:
00590     using size_type = DOCTEST_CONFIG_STRING_SIZE_TYPE;
00591
00592 private:
00593     static DOCTEST_CONSTEXPR size_type len = 24;
00594     static DOCTEST_CONSTEXPR size_type last = len - 1;
00595
00596     struct view // len should be more than sizeof(view) - because of the final byte for flags
00597     {
00598         char* ptr;
00599         size_type size;
00600         size_type capacity;
00601     };
00602
00603     union
00604     {
00605         char buf[len]; // NOLINT(*-avoid-c-arrays)
00606         view data;
00607     };
00608
00609     char* allocate(size_type sz);
00610
00611     bool isOnStack() const noexcept { return (buf[last] & 128) == 0; }
00612     void setOnHeap() noexcept;
00613     void setLast(size_type in = last) noexcept;
00614     void setSize(size_type sz) noexcept;
00615
00616     void copy(const String& other);
00617
00618 public:
00619     static DOCTEST_CONSTEXPR size_type npos = static_cast<size_type>(-1);
00620
00621     String() noexcept;
00622     ~String();
00623
00624     // cppcheck-suppress noExplicitConstructor
00625     String(const char* in);
00626     String(const char* in, size_type in_size);
00627
00628     String(std::istream& in, size_type in_size);
00629
00630     String(const String& other);
00631     String& operator=(const String& other);
00632
00633     String& operator+=(const String& other);
00634
00635     String(String&& other) noexcept;
00636     String& operator=(String&& other) noexcept;
00637
00638     char operator[](size_type i) const;
00639     char& operator[](size_type i);
00640
00641     // the only functions I'm willing to leave in the interface - available for inlining
00642     const char* c_str() const { return const_cast<String*>(this)->c_str(); } // NOLINT
00643     char* c_str() {
00644         if (isOnStack()) {
00645             return reinterpret_cast<char*>(buf);
00646         }

```

```

00647         return data.ptr;
00648     }
00649
00650     size_type size() const;
00651     size_type capacity() const;
00652
00653     String substr(size_type pos, size_type cnt = npos) &&;
00654     String substr(size_type pos, size_type cnt = npos) const &;
00655
00656     size_type find(char ch, size_type pos = 0) const;
00657     size_type rfind(char ch, size_type pos = npos) const;
00658
00659     int compare(const char* other, bool no_case = false) const;
00660     int compare(const String& other, bool no_case = false) const;
00661
00662 friend DOCTEST_INTERFACE std::ostream& operator<<(std::ostream& s, const String& in);
00663 };
00664
00665 DOCTEST_INTERFACE String operator+(const String& lhs, const String& rhs);
00666
00667 DOCTEST_INTERFACE bool operator==(const String& lhs, const String& rhs);
00668 DOCTEST_INTERFACE bool operator!=(const String& lhs, const String& rhs);
00669 DOCTEST_INTERFACE bool operator<(const String& lhs, const String& rhs);
00670 DOCTEST_INTERFACE bool operator>(const String& lhs, const String& rhs);
00671 DOCTEST_INTERFACE bool operator<=(const String& lhs, const String& rhs);
00672 DOCTEST_INTERFACE bool operator>=(const String& lhs, const String& rhs);
00673
00674 class DOCTEST_INTERFACE Contains {
00675 public:
00676     explicit Contains(const String& string);
00677
00678     bool checkWith(const String& other) const;
00679
00680     String string;
00681 };
00682
00683 DOCTEST_INTERFACE String toString(const Contains& in);
00684
00685 DOCTEST_INTERFACE bool operator==(const String& lhs, const Contains& rhs);
00686 DOCTEST_INTERFACE bool operator==(const Contains& lhs, const String& rhs);
00687 DOCTEST_INTERFACE bool operator!=(const String& lhs, const Contains& rhs);
00688 DOCTEST_INTERFACE bool operator!=(const Contains& lhs, const String& rhs);
00689
00690 namespace Color {
00691     enum Enum
00692     {
00693         None = 0,
00694         White,
00695         Red,
00696         Green,
00697         Blue,
00698         Cyan,
00699         Yellow,
00700         Grey,
00701
00702         Bright = 0x10,
00703
00704         BrightRed   = Bright | Red,
00705         BrightGreen = Bright | Green,
00706         LightGrey   = Bright | Grey,
00707         BrightWhite = Bright | White
00708     };
00709
00710     DOCTEST_INTERFACE std::ostream& operator<<(std::ostream& s, Color::Enum code);
00711 } // namespace Color
00712
00713 namespace assertType {
00714     enum Enum
00715     {
00716         // macro traits
00717
00718         is_warn      = 1,
00719         is_check     = 2 * is_warn,
00720         is_require   = 2 * is_check,
00721
00722         is_normal    = 2 * is_require,
00723         is_throws    = 2 * is_normal,
00724         is_throws_as = 2 * is_throws,
00725         is_throws_with = 2 * is_throws_as,
00726         is_nothrow   = 2 * is_throws_with,
00727
00728         is_false     = 2 * is_nothrow,
00729         is_unary     = 2 * is_false, // not checked anywhere - used just to distinguish the types
00730
00731         is_eq        = 2 * is_unary,
00732         is_ne        = 2 * is_eq,
00733     }

```



```

00734         is_lt = 2 * is_ne,
00735         is_gt = 2 * is_lt,
00736
00737         is_ge = 2 * is_gt,
00738         is_le = 2 * is_ge,
00739
00740         // macro types
00741
00742         DT_WARN      = is_normal | is_warn,
00743         DT_CHECK     = is_normal | is_check,
00744         DT_REQUIRE   = is_normal | is_require,
00745
00746         DT_WARN_FALSE = is_normal | is_false | is_warn,
00747         DT_CHECK_FALSE = is_normal | is_false | is_check,
00748         DT_REQUIRE_FALSE = is_normal | is_false | is_require,
00749
00750         DT_WARN_THROWS = is_throws | is_warn,
00751         DT_CHECK_THROWS = is_throws | is_check,
00752         DT_REQUIRE_THROWS = is_throws | is_require,
00753
00754         DT_WARN_THROWS_AS = is_throws_as | is_warn,
00755         DT_CHECK_THROWS_AS = is_throws_as | is_check,
00756         DT_REQUIRE_THROWS_AS = is_throws_as | is_require,
00757
00758         DT_WARN_THROWS_WITH = is_throws_with | is_warn,
00759         DT_CHECK_THROWS_WITH = is_throws_with | is_check,
00760         DT_REQUIRE_THROWS_WITH = is_throws_with | is_require,
00761
00762         DT_WARN_THROWS_WITH_AS = is_throws_with | is_throws_as | is_warn,
00763         DT_CHECK_THROWS_WITH_AS = is_throws_with | is_throws_as | is_check,
00764         DT_REQUIRE_THROWS_WITH_AS = is_throws_with | is_throws_as | is_require,
00765
00766         DT_WARN_NOTHROW = is_nothrow | is_warn,
00767         DT_CHECK_NOTHROW = is_nothrow | is_check,
00768         DT_REQUIRE_NOTHROW = is_nothrow | is_require,
00769
00770         DT_WARN_EQ = is_normal | is_eq | is_warn,
00771         DT_CHECK_EQ = is_normal | is_eq | is_check,
00772         DT_REQUIRE_EQ = is_normal | is_eq | is_require,
00773
00774         DT_WARN_NE = is_normal | is_ne | is_warn,
00775         DT_CHECK_NE = is_normal | is_ne | is_check,
00776         DT_REQUIRE_NE = is_normal | is_ne | is_require,
00777
00778         DT_WARN_GT = is_normal | is_gt | is_warn,
00779         DT_CHECK_GT = is_normal | is_gt | is_check,
00780         DT_REQUIRE_GT = is_normal | is_gt | is_require,
00781
00782         DT_WARN_LT = is_normal | is_lt | is_warn,
00783         DT_CHECK_LT = is_normal | is_lt | is_check,
00784         DT_REQUIRE_LT = is_normal | is_lt | is_require,
00785
00786         DT_WARN_GE = is_normal | is_ge | is_warn,
00787         DT_CHECK_GE = is_normal | is_ge | is_check,
00788         DT_REQUIRE_GE = is_normal | is_ge | is_require,
00789
00790         DT_WARN_LE = is_normal | is_le | is_warn,
00791         DT_CHECK_LE = is_normal | is_le | is_check,
00792         DT_REQUIRE_LE = is_normal | is_le | is_require,
00793
00794         DT_WARN_UNARY = is_normal | is_unary | is_warn,
00795         DT_CHECK_UNARY = is_normal | is_unary | is_check,
00796         DT_REQUIRE_UNARY = is_normal | is_unary | is_require,
00797
00798         DT_WARN_UNARY_FALSE = is_normal | is_false | is_unary | is_warn,
00799         DT_CHECK_UNARY_FALSE = is_normal | is_false | is_unary | is_check,
00800         DT_REQUIRE_UNARY_FALSE = is_normal | is_false | is_unary | is_require,
00801     };
00802 } // namespace assertType
00803
00804 DOCTEST_INTERFACE const char* assertString(assertType::Enum at);
00805 DOCTEST_INTERFACE const char* failureString(assertType::Enum at);
00806 DOCTEST_INTERFACE const char* skipPathFromFilename(const char* file);
00807
00808 struct DOCTEST_INTERFACE TestCaseData
00809 {
00810     String      m_file;           // the file in which the test was registered (using String - see #350)
00811     unsigned    m_line;          // the line where the test was registered
00812     const char* m_name;          // name of the test case
00813     const char* m_test_suite;    // the test suite in which the test was added
00814     const char* m_description;
00815     bool        m_skip;
00816     bool        m_no_breaks;
00817     bool        m_no_output;
00818     bool        m_may_fail;
00819     bool        m_should_fail;
00820     int         m_expected_failures;

```



```

00821     double         m_timeout;
00822 };
00823
00824 struct DOCTEST_INTERFACE AssertData
00825 {
00826     // common - for all asserts
00827     const TestCaseData* m_test_case;
00828     assertType::Enum    m_at;
00829     const char*         m_file;
00830     int                 m_line;
00831     const char*         m_expr;
00832     bool                m_failed;
00833
00834     // exception-related - for all asserts
00835     bool m_threw;
00836     String m_exception;
00837
00838     // for normal asserts
00839     String m_decomp;
00840
00841     // for specific exception-related asserts
00842     bool m_threw_as;
00843     const char* m_exception_type;
00844
00845     class DOCTEST_INTERFACE StringContains {
00846     private:
00847         Contains content;
00848         bool isContains;
00849
00850     public:
00851         StringContains(const String& str) : content(str), isContains(false) { }
00852         StringContains(Contains cntn) : content(static_cast<Contains&&>(cntn)), isContains(true) { }
00853     }
00854
00855     bool check(const String& str) { return isContains ? (content == str) : (content.string ==
str); }
00856
00857     operator const String&() const { return content.string; }
00858
00859     const char* c_str() const { return content.string.c_str(); }
00860 } m_exception_string;
00861
00862 AssertData(assertType::Enum at, const char* file, int line, const char* expr,
const char* exception_type, const StringContains& exception_string);
00863 };
00864
00865 struct DOCTEST_INTERFACE MessageData
00866 {
00867     String m_string;
00868     const char* m_file;
00869     int m_line;
00870     assertType::Enum m_severity;
00871 };
00872
00873 struct DOCTEST_INTERFACE SubcaseSignature
00874 {
00875     String m_name;
00876     const char* m_file;
00877     int m_line;
00878
00879     bool operator==(const SubcaseSignature& other) const;
00880     bool operator<(const SubcaseSignature& other) const;
00881 };
00882
00883 struct DOCTEST_INTERFACE IContextScope
00884 {
00885     DOCTEST_DECLARE_INTERFACE(IContextScope)
00886     virtual void stringify(std::ostream*) const = 0;
00887 };
00888
00889 namespace detail {
00890     struct DOCTEST_INTERFACE TestCase;
00891 } // namespace detail
00892
00893 struct ContextOptions
00894 {
00895     std::ostream* cout = nullptr; // stdout stream
00896     String binary_name; // the test binary name
00897
00898     const detail::TestCase* currentTest = nullptr;
00899
00900     // == parameters from the command line
00901     String out; // output filename
00902     String order_by; // how tests should be ordered
00903     unsigned rand_seed; // the seed for rand ordering
00904
00905     unsigned first; // the first (matching) test to be executed

```

```

00906     unsigned last; // the last (matching) test to be executed
00907
00908     int abort_after; // stop tests after this many failed assertions
00909     int subcase_filter_levels; // apply the subcase filters for the first N levels
00910
00911     bool success; // include successful assertions in output
00912     bool case_sensitive; // if filtering should be case sensitive
00913     bool exit; // if the program should be exited after the tests are ran/whatever
00914     bool duration; // print the time duration of each test case
00915     bool minimal; // minimal console output (only test failures)
00916     bool quiet; // no console output
00917     bool no_throw; // to skip exceptions-related assertion macros
00918     bool no_exitcode; // if the framework should return 0 as the exitcode
00919     bool no_run; // to not run the tests at all (can be done with an "*" exclude)
00920     bool no_intro; // to not print the intro of the framework
00921     bool no_version; // to not print the version of the framework
00922     bool no_colors; // if output to the console should be colorized
00923     bool force_colors; // forces the use of colors even when a tty cannot be detected
00924     bool no_breaks; // to not break into the debugger
00925     bool no_skip; // don't skip test cases which are marked to be skipped
00926     bool gnu_file_line; // if line numbers should be surrounded with :x: and not (x):
00927     bool no_path_in_filenames; // if the path to files should be removed from the output
00928     bool no_line_numbers; // if source code line numbers should be omitted from the output
00929     bool no_debug_output; // no output in the debug console when a debugger is attached
00930     bool no_skipped_summary; // don't print "skipped" in the summary !!! UNDOCUMENTED !!!
00931     bool no_time_in_output; // omit any time/timestamps from output !!! UNDOCUMENTED !!!
00932
00933     bool help; // to print the help
00934     bool version; // to print the version
00935     bool count; // if only the count of matching tests is to be retrieved
00936     bool list_test_cases; // to list all tests matching the filters
00937     bool list_test_suites; // to list all suites matching the filters
00938     bool list_reporters; // lists all registered reporters
00939 };
00940
00941 namespace detail {
00942     namespace types {
00943         #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
00944             using namespace std;
00945         #else
00946             template <bool COND, typename T = void>
00947             struct enable_if { };
00948
00949             template <typename T>
00950             struct enable_if<true, T> { using type = T; };
00951
00952             struct true_type { static DOCTEST_CONSTEXPR bool value = true; };
00953             struct false_type { static DOCTEST_CONSTEXPR bool value = false; };
00954
00955             template <typename T> struct remove_reference { using type = T; };
00956             template <typename T> struct remove_reference<T&> { using type = T; };
00957             template <typename T> struct remove_reference<T&&> { using type = T; };
00958
00959             template <typename T> struct is_rvalue_reference : false_type { };
00960             template <typename T> struct is_rvalue_reference<T&&> : true_type { };
00961
00962             template <typename T> struct remove_const { using type = T; };
00963             template <typename T> struct remove_const<const T> { using type = T; };
00964
00965             // Compiler intrinsics
00966             template <typename T> struct is_enum { static DOCTEST_CONSTEXPR bool value = __is_enum(T); };
00967             template <typename T> struct underlying_type { using type = __underlying_type(T); };
00968
00969             template <typename T> struct is_pointer : false_type { };
00970             template <typename T> struct is_pointer<T*> : true_type { };
00971
00972             template <typename T> struct is_array : false_type { };
00973             // NOLINTNEXTLINE(*-avoid-c-arrays)
00974             template <typename T, size_t SIZE> struct is_array<T[SIZE]> : true_type { };
00975         #endif
00976     }
00977
00978     // <utility>
00979     template <typename T>
00980     T&& declval();
00981
00982     template <class T>
00983     DOCTEST_CONSTEXPR_FUNC T&& forward(typename types::remove_reference<T>::type& t) DOCTEST_NOEXCEPT
00984     {
00985         return static_cast<T&&>(t);
00986     }
00987
00988     template <class T>
00989     DOCTEST_CONSTEXPR_FUNC T&& forward(typename types::remove_reference<T>::type&& t) DOCTEST_NOEXCEPT
00990     {
00991         return static_cast<T&&>(t);
00992     }

```

```

00991
00992     template <typename T>
00993     struct deferred_false : types::false_type { };
00994
00995 // MSVS 2015 :(
00996 #if !DOCTEST_CLANG && defined(_MSC_VER) && _MSC_VER <= 1900
00997     template <typename T, typename = void>
00998     struct has_global_inserter_operator : types::false_type { };
00999
01000     template <typename T>
01001     struct has_global_inserter_operator<T, decltype(::operator<<(declval<std::ostream&>()),
declval<const T&>()), void())> : types::true_type { };
01002
01003     template <typename T, typename = void>
01004     struct has_inserter_operator { static DOCTEST_CONSTEXPR bool value =
has_global_inserter_operator<T>::value; };
01005
01006     template <typename T, bool global>
01007     struct insert_hack;
01008
01009     template <typename T>
01010     struct insert_hack<T, true> {
01011         static void insert(std::ostream& os, const T& t) { ::operator<<(os, t); }
01012     };
01013
01014     template <typename T>
01015     struct insert_hack<T, false> {
01016         static void insert(std::ostream& os, const T& t) { operator<<(os, t); }
01017     };
01018
01019     template <typename T>
01020     using insert_hack_t = insert_hack<T, has_global_inserter_operator<T>::value>;
01021 #else
01022     template <typename T, typename = void>
01023     struct has_inserter_operator : types::false_type { };
01024 #endif
01025
01026     template <typename T>
01027     struct has_inserter_operator<T, decltype(operator<<(declval<std::ostream&>()), declval<const
T&>()), void())> : types::true_type { };
01028
01029     template <typename T>
01030     struct should_stringify_as_underlying_type {
01031         static DOCTEST_CONSTEXPR bool value = detail::types::is_enum<T>::value &&
!doctest::detail::has_inserter_operator<T>::value;
01032     };
01033
01034     DOCTEST_INTERFACE std::ostream* tlssPush();
01035     DOCTEST_INTERFACE String tlssPop();
01036
01037     template <bool C>
01038     struct StringMakerBase {
01039         template <typename T>
01040         static String convert(const DOCTEST_REF_WRAP(T)) {
01041 #ifndef DOCTEST_CONFIG_REQUIRE_STRINGIFICATION_FOR_ALL_USED_TYPES
01042             static_assert(deferred_false<T>::value, "No stringification detected for type T. See
string conversion manual");
01043 #endif
01044             return "{?}";
01045         }
01046     };
01047
01048     template <typename T>
01049     struct filldata;
01050
01051     template <typename T>
01052     void fillloss(std::ostream* stream, const T& in) {
01053         filldata<T>::fill(stream, in);
01054     }
01055
01056     template <typename T, size_t N>
01057     void fillloss(std::ostream* stream, const T (&in)[N]) { // NOLINT(*-avoid-c-arrays)
01058         // T[N], T(&)[N], T(&&)[N] have same behaviour.
01059         // Hence remove reference.
01060         fillloss<typename types::remove_reference<decltype(in)>::type>(stream, in);
01061     }
01062
01063     template <typename T>
01064     String toStream(const T& in) {
01065         std::ostream* stream = tlssPush();
01066         fillloss(stream, in);
01067         return tlssPop();
01068     }
01069
01070     template <>
01071     struct StringMakerBase<true> {
01072         template <typename T>

```

```

01073         static String convert(const DOCTEST_REF_WRAP(T) in) {
01074             return toString(in);
01075         }
01076     };
01077 } // namespace detail
01078
01079 template <typename T>
01080 struct StringMaker : public detail::StringMakerBase<
01081     detail::has_insertion_operator<T>::value || detail::types::is_pointer<T>::value ||
01082     detail::types::is_array<T>::value>
01083 {};
01084
01084 #ifndef DOCTEST_STRINGIFY
01085 #ifdef DOCTEST_CONFIG_DOUBLE_STRINGIFY
01086 #define DOCTEST_STRINGIFY(...) toString(toString(__VA_ARGS__))
01087 #else
01088 #define DOCTEST_STRINGIFY(...) toString(__VA_ARGS__)
01089 #endif
01090 #endif
01091
01092 template <typename T>
01093 String toString() {
01094     #if DOCTEST_CLANG == 0 && DOCTEST_GCC == 0 && DOCTEST_ICC == 0
01095         String ret = __FUNCSIG__; // class doctest::String __cdecl doctest::toString<TYPE>(void)
01096         String::size_type beginPos = ret.find('<');
01097         return ret.substr(beginPos + 1, ret.size() - beginPos -
01098             static_cast<String::size_type>(sizeof(">(void)")));
01099     #else
01100         String ret = __PRETTY_FUNCTION__; // doctest::String toString() [with T = TYPE]
01101         String::size_type begin = ret.find('=') + 2;
01102         return ret.substr(begin, ret.size() - begin - 1);
01103     #endif
01104 }
01105
01105 template <typename T, typename
01106     detail::types::enable_if<!detail::should_stringify_as_underlying_type<T>::value, bool>::type = true>
01107 String toString(const DOCTEST_REF_WRAP(T) value) {
01108     return StringMaker<T>::convert(value);
01109 }
01110
01110 #ifdef DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01111 DOCTEST_INTERFACE String toString(const char* in);
01112 #endif // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01113
01114 #if DOCTEST_MSVC >= DOCTEST_COMPILER(19, 20, 0)
01115 // see this issue on why this is needed: https://github.com/doctest/doctest/issues/183
01116 DOCTEST_INTERFACE String toString(const std::string& in);
01117 #endif // VS 2019
01118
01119 DOCTEST_INTERFACE String toString(String in);
01120
01121 DOCTEST_INTERFACE String toString(std::nullptr_t);
01122
01123 DOCTEST_INTERFACE String toString(bool in);
01124
01125 DOCTEST_INTERFACE String toString(float in);
01126 DOCTEST_INTERFACE String toString(double in);
01127 DOCTEST_INTERFACE String toString(double long in);
01128
01129 DOCTEST_INTERFACE String toString(char in);
01130 DOCTEST_INTERFACE String toString(char signed in);
01131 DOCTEST_INTERFACE String toString(char unsigned in);
01132 DOCTEST_INTERFACE String toString(short in);
01133 DOCTEST_INTERFACE String toString(short unsigned in);
01134 DOCTEST_INTERFACE String toString(signed in);
01135 DOCTEST_INTERFACE String toString(unsigned in);
01136 DOCTEST_INTERFACE String toString(long in);
01137 DOCTEST_INTERFACE String toString(long unsigned in);
01138 DOCTEST_INTERFACE String toString(long long in);
01139 DOCTEST_INTERFACE String toString(long long unsigned in);
01140
01141 template <typename T, typename
01142     detail::types::enable_if<detail::should_stringify_as_underlying_type<T>::value, bool>::type = true>
01143 String toString(const DOCTEST_REF_WRAP(T) value) {
01144     using UT = typename detail::types::underlying_type<T>::type;
01145     return (DOCTEST_STRINGIFY(static_cast<UT>(value)));
01146 }
01147
01147 namespace detail {
01148     template <typename T>
01149     struct filldata
01150     {
01151         static void fill(std::ostream* stream, const T& in) {
01152             #if defined(_MSC_VER) && _MSC_VER <= 1900
01153                 insert_hack_t<T>::insert(*stream, in);
01154             #else
01155                 operator<<(*stream, in);

```

```

01156 #endif
01157     }
01158 };
01159
01160 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4866)
01161 // NOLINTBEGIN(*-avoid-c-arrays)
01162     template <typename T, size_t N>
01163     struct filldata<T[N]> {
01164         static void fill(std::ostream* stream, const T(&in)[N]) {
01165             *stream << "[";
01166             for (size_t i = 0; i < N; i++) {
01167                 if (i != 0) { *stream << ", "; }
01168                 *stream << (DOCTEST_STRINGIFY(in[i]));
01169             }
01170             *stream << "]";
01171         }
01172     };
01173 // NOLINTEND(*-avoid-c-arrays)
01174 DOCTEST_MSVC_SUPPRESS_WARNING_POP
01175
01176 // Specialized since we don't want the terminating null byte!
01177 // NOLINTBEGIN(*-avoid-c-arrays)
01178     template <size_t N>
01179     struct filldata<const char[N]> {
01180         static void fill(std::ostream* stream, const char (&in)[N]) {
01181             *stream << String(in, in[N - 1] ? N : N - 1);
01182         } // NOLINT(clang-analyzer-cplusplus.NewDeleteLeaks)
01183     };
01184 // NOLINTEND(*-avoid-c-arrays)
01185
01186     template <>
01187     struct filldata<const void*> {
01188         static void fill(std::ostream* stream, const void* in);
01189     };
01190
01191     template <typename T>
01192     struct filldata<T*> {
01193 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4180)
01194         static void fill(std::ostream* stream, const T* in) {
01195 DOCTEST_MSVC_SUPPRESS_WARNING_POP
01196 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wmicrosoft-cast")
01197             filldata<const void*>::fill(stream,
01198             reinterpret_cast<const void*>(in)
01199             #else
01200                 *reinterpret_cast<const void* const*>(&in)
01201             #endif
01202         );
01203 DOCTEST_CLANG_SUPPRESS_WARNING_POP
01204     }
01205 };
01206
01207 }
01208
01209 struct DOCTEST_INTERFACE Approx
01210 {
01211     Approx(double value);
01212
01213     Approx operator()(double value) const;
01214
01215 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01216     template <typename T>
01217     explicit Approx(const T& value,
01218         typename detail::types::enable_if<std::is_constructible<double, T::value>::type*
01219         =
01220             static_cast<T*>(nullptr)) {
01221         *this = static_cast<double>(value);
01222     }
01223 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01224
01225     Approx& epsilon(double newEpsilon);
01226
01227 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01228     template <typename T>
01229     typename std::enable_if<std::is_constructible<double, T::value, Approx&>::type epsilon(
01230         const T& newEpsilon) {
01231         m_epsilon = static_cast<double>(newEpsilon);
01232         return *this;
01233     }
01234 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01235
01236     Approx& scale(double newScale);
01237
01238 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01239     template <typename T>
01240     typename std::enable_if<std::is_constructible<double, T::value, Approx&>::type scale(
01241         const T& newScale) {
01242         m_scale = static_cast<double>(newScale);

```

```

01242         return *this;
01243     }
01244 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01245
01246     // clang-format off
01247     DOCTEST_INTERFACE friend bool operator==(double lhs, const Approx & rhs);
01248     DOCTEST_INTERFACE friend bool operator==(const Approx & lhs, double rhs);
01249     DOCTEST_INTERFACE friend bool operator!=(double lhs, const Approx & rhs);
01250     DOCTEST_INTERFACE friend bool operator!=(const Approx & lhs, double rhs);
01251     DOCTEST_INTERFACE friend bool operator<=(double lhs, const Approx & rhs);
01252     DOCTEST_INTERFACE friend bool operator<=(const Approx & lhs, double rhs);
01253     DOCTEST_INTERFACE friend bool operator>=(double lhs, const Approx & rhs);
01254     DOCTEST_INTERFACE friend bool operator>=(const Approx & lhs, double rhs);
01255     DOCTEST_INTERFACE friend bool operator< (double lhs, const Approx & rhs);
01256     DOCTEST_INTERFACE friend bool operator< (const Approx & lhs, double rhs);
01257     DOCTEST_INTERFACE friend bool operator> (double lhs, const Approx & rhs);
01258     DOCTEST_INTERFACE friend bool operator> (const Approx & lhs, double rhs);
01259
01260 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01261 #define DOCTEST_APPROX_PREFIX \
01262     template <typename T> friend typename std::enable_if<std::is_constructible<double, T>::value,
01263     bool>::type
01264     DOCTEST_APPROX_PREFIX operator==(const T& lhs, const Approx& rhs) { return
01265     operator==(static_cast<double>(lhs), rhs); }
01266     DOCTEST_APPROX_PREFIX operator==(const Approx& lhs, const T& rhs) { return operator==(rhs, lhs); }
01267     DOCTEST_APPROX_PREFIX operator!=(const T& lhs, const Approx& rhs) { return !operator==(lhs, rhs); }
01268     DOCTEST_APPROX_PREFIX operator!=(const Approx& lhs, const T& rhs) { return !operator==(rhs, lhs); }
01269     DOCTEST_APPROX_PREFIX operator<=(const T& lhs, const Approx& rhs) { return
01270     static_cast<double>(lhs) < rhs.m_value || lhs == rhs; }
01271     DOCTEST_APPROX_PREFIX operator<=(const Approx& lhs, const T& rhs) { return lhs.m_value <
01272     static_cast<double>(rhs) || lhs == rhs; }
01273     DOCTEST_APPROX_PREFIX operator>=(const T& lhs, const Approx& rhs) { return
01274     static_cast<double>(lhs) > rhs.m_value || lhs == rhs; }
01275     DOCTEST_APPROX_PREFIX operator>=(const Approx& lhs, const T& rhs) { return lhs.m_value >
01276     static_cast<double>(rhs) || lhs == rhs; }
01277     DOCTEST_APPROX_PREFIX operator< (const T& lhs, const Approx& rhs) { return
01278     static_cast<double>(lhs) < rhs.m_value && lhs != rhs; }
01279     DOCTEST_APPROX_PREFIX operator< (const Approx& lhs, const T& rhs) { return lhs.m_value <
01280     static_cast<double>(rhs) && lhs != rhs; }
01281     DOCTEST_APPROX_PREFIX operator> (const T& lhs, const Approx& rhs) { return
01282     static_cast<double>(lhs) > rhs.m_value && lhs != rhs; }
01283     DOCTEST_APPROX_PREFIX operator> (const Approx& lhs, const T& rhs) { return lhs.m_value >
01284     static_cast<double>(rhs) && lhs != rhs; }
01285 #undef DOCTEST_APPROX_PREFIX
01286 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01287
01288     // clang-format on
01289
01290     double m_epsilon;
01291     double m_scale;
01292     double m_value;
01293 };
01294
01295 DOCTEST_INTERFACE String toString(const Approx& in);
01296
01297 DOCTEST_INTERFACE const ContextOptions* getContextOptions();
01298
01299 template <typename F>
01300 struct DOCTEST_INTERFACE_DECL IsNaN
01301 {
01302     F value; bool flipped;
01303     IsNaN(F f, bool flip = false) : value(f), flipped(flip) { }
01304     IsNaN<F> operator!() const { return { value, !flipped }; }
01305     operator bool() const;
01306 };
01307
01308 #ifndef __MINGW32__
01309 extern template struct DOCTEST_INTERFACE_DECL IsNaN<float>;
01310 extern template struct DOCTEST_INTERFACE_DECL IsNaN<double>;
01311 extern template struct DOCTEST_INTERFACE_DECL IsNaN<long double>;
01312 #endif
01313
01314 DOCTEST_INTERFACE String toString(IsNaN<float> in);
01315 DOCTEST_INTERFACE String toString(IsNaN<double> in);
01316 DOCTEST_INTERFACE String toString(IsNaN<double long> in);
01317
01318 #ifndef DOCTEST_CONFIG_DISABLE
01319 namespace detail {
01320     // clang-format off
01321     #ifndef DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01322     template<class T> struct decay_array { using type = T; };
01323     template<class T, unsigned N> struct decay_array<T[N]> { using type = T*; };
01324     template<class T> struct decay_array<T[]> { using type = T*; };
01325
01326     template<class T> struct not_char_pointer { static DOCTEST_CONSTEXPR int value = 1;

```

```

};
01317     template<>         struct not_char_pointer<char*>         { static DOCTEST_CONSTEXPR int value = 0;
};
01318     template<>         struct not_char_pointer<const char*> { static DOCTEST_CONSTEXPR int value = 0;
};
01319
01320     template<class T> struct can_use_op : public not_char_pointer<typename decay_array<T>::type> {};
01321 #endif // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01322 // clang-format on
01323
01324     struct DOCTEST_INTERFACE TestFailureException
01325     {
01326     };
01327
01328     DOCTEST_INTERFACE bool checkIfShouldThrow(assertType::Enum at);
01329
01330 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
01331     DOCTEST_NORETURN
01332 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
01333     DOCTEST_INTERFACE void throwException();
01334
01335     struct DOCTEST_INTERFACE Subcase
01336     {
01337         SubcaseSignature m_signature;
01338         bool m_entered = false;
01339
01340         Subcase(const String& name, const char* file, int line);
01341         Subcase(const Subcase&) = delete;
01342         Subcase(Subcase&&) = delete;
01343         Subcase& operator=(const Subcase&) = delete;
01344         Subcase& operator=(Subcase&&) = delete;
01345         ~Subcase();
01346
01347         operator bool() const;
01348
01349     private:
01350         bool checkFilters();
01351     };
01352
01353     template <typename L, typename R>
01354     String stringifyBinaryExpr(const DOCTEST_REF_WRAP(L) lhs, const char* op,
01355                               const DOCTEST_REF_WRAP(R) rhs) {
01356         return (DOCTEST_STRINGIFY(lhs)) + op + (DOCTEST_STRINGIFY(rhs));
01357     }
01358
01359 #if DOCTEST_CLANG && DOCTEST_CLANG < DOCTEST_COMPILER(3, 6, 0)
01360 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wunused-comparison")
01361 #endif
01362
01363 // This will check if there is any way it could find a operator like member or friend and uses it.
01364 // If not it doesn't find the operator or if the operator at global scope is defined after
01365 // this template, the template won't be instantiated due to SFINAE. Once the template is not
01366 // instantiated it can look for global operator using normal conversions.
01367 #ifdef __NVCC__
01368 #define SFINAE_OP(ret,op) ret
01369 #else
01370 #define SFINAE_OP(ret,op) decltype((void) (doctest::detail::declval<L>() op
doctest::detail::declval<R>()),ret{})
01371 #endif
01372
01373 #define DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(op, op_str, op_macro) \
01374     template <typename R> \
01375     DOCTEST_NOINLINE SFINAE_OP(Result,op) operator op(R&& rhs) { \
01376     bool res = op_macro(doctest::detail::forward<const L>(lhs), doctest::detail::forward<R>(rhs)); \
01377         if(m_at & assertType::is_false) \
01378             res = !res; \
01379         if(!res || doctest::getContextOptions()->success) \
01380             return Result(res, stringifyBinaryExpr(lhs, op_str, rhs)); \
01381         return Result(res); \
01382     }
01383
01384 // more checks could be added - like in Catch:
01385 // https://github.com/catchorg/Catch2/pull/1480/files
01386 // https://github.com/catchorg/Catch2/pull/1481/files
01387 #define DOCTEST_FORBIT_EXPRESSION(rt, op) \
01388     template <typename R> \
01389     rt& operator op(const R&) { \
01390         static_assert(deferred_false<R>::value, \
01391             "Expression Too Complex Please Rewrite As Binary Comparison!"); \
01392         return *this; \
01393     }
01394
01395     struct DOCTEST_INTERFACE Result // NOLINT(*-member-init)
01396     {
01397         bool m_passed;
01398         String m_decomp;
01399     }

```

```

01400     Result() = default; // TODO: Why do we need this? (To remove NOLINT)
01401     Result(bool passed, const String& decomposition = String());
01402
01403     // forbidding some expressions based on this table:
01404     https://en.cppreference.com/w/cpp/language/operator\_precedence
01405     DOCTEST_FORBIT_EXPRESSION(Result, &)
01406     DOCTEST_FORBIT_EXPRESSION(Result, ^)
01407     DOCTEST_FORBIT_EXPRESSION(Result, |)
01408     DOCTEST_FORBIT_EXPRESSION(Result, &&)
01409     DOCTEST_FORBIT_EXPRESSION(Result, ||)
01410     DOCTEST_FORBIT_EXPRESSION(Result, ==)
01411     DOCTEST_FORBIT_EXPRESSION(Result, !=)
01412     DOCTEST_FORBIT_EXPRESSION(Result, <)
01413     DOCTEST_FORBIT_EXPRESSION(Result, >)
01414     DOCTEST_FORBIT_EXPRESSION(Result, <=)
01415     DOCTEST_FORBIT_EXPRESSION(Result, >=)
01416     DOCTEST_FORBIT_EXPRESSION(Result, =)
01417     DOCTEST_FORBIT_EXPRESSION(Result, +=)
01418     DOCTEST_FORBIT_EXPRESSION(Result, -=)
01419     DOCTEST_FORBIT_EXPRESSION(Result, *=)
01420     DOCTEST_FORBIT_EXPRESSION(Result, /=)
01421     DOCTEST_FORBIT_EXPRESSION(Result, %=)
01422     DOCTEST_FORBIT_EXPRESSION(Result, <<=)
01423     DOCTEST_FORBIT_EXPRESSION(Result, >>=)
01424     DOCTEST_FORBIT_EXPRESSION(Result, &=)
01425     DOCTEST_FORBIT_EXPRESSION(Result, ^=)
01426     DOCTEST_FORBIT_EXPRESSION(Result, |=)
01427 };
01428 #ifndef DOCTEST_CONFIG_NO_COMPARISON_WARNING_SUPPRESSION
01429
01430     DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
01431     DOCTEST_CLANG_SUPPRESS_WARNING("-Wsign-conversion")
01432     DOCTEST_CLANG_SUPPRESS_WARNING("-Wsign-compare")
01433     //DOCTEST_CLANG_SUPPRESS_WARNING("-Wdouble-promotion")
01434     //DOCTEST_CLANG_SUPPRESS_WARNING("-Wconversion")
01435     //DOCTEST_CLANG_SUPPRESS_WARNING("-Wfloat-equal")
01436
01437     DOCTEST_GCC_SUPPRESS_WARNING_PUSH
01438     DOCTEST_GCC_SUPPRESS_WARNING("-Wsign-conversion")
01439     DOCTEST_GCC_SUPPRESS_WARNING("-Wsign-compare")
01440     //DOCTEST_GCC_SUPPRESS_WARNING("-Wdouble-promotion")
01441     //DOCTEST_GCC_SUPPRESS_WARNING("-Wconversion")
01442     //DOCTEST_GCC_SUPPRESS_WARNING("-Wfloat-equal")
01443
01444     DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
01445     // https://stackoverflow.com/questions/39479163/whats-the-difference-between-4018-and-4389
01446     DOCTEST_MSVC_SUPPRESS_WARNING(4388) // signed/unsigned mismatch
01447     DOCTEST_MSVC_SUPPRESS_WARNING(4389) // 'operator' : signed/unsigned mismatch
01448     DOCTEST_MSVC_SUPPRESS_WARNING(4018) // 'expression' : signed/unsigned mismatch
01449     //DOCTEST_MSVC_SUPPRESS_WARNING(4805) // 'operation' : unsafe mix of type 'type' and type 'type'
01450     in operation
01451 #endif // DOCTEST_CONFIG_NO_COMPARISON_WARNING_SUPPRESSION
01452
01453     // clang-format off
01454 #ifndef DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01455 #define DOCTEST_COMPARISON_RETURN_TYPE bool
01456 #else // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01457 #define DOCTEST_COMPARISON_RETURN_TYPE typename types::enable_if<can_use_op<L>::value ||
01458     can_use_op<R>::value, bool>::type
01459     inline bool eq(const char* lhs, const char* rhs) { return String(lhs) == String(rhs); }
01460     inline bool ne(const char* lhs, const char* rhs) { return String(lhs) != String(rhs); }
01461     inline bool lt(const char* lhs, const char* rhs) { return String(lhs) < String(rhs); }
01462     inline bool gt(const char* lhs, const char* rhs) { return String(lhs) > String(rhs); }
01463     inline bool le(const char* lhs, const char* rhs) { return String(lhs) <= String(rhs); }
01464     inline bool ge(const char* lhs, const char* rhs) { return String(lhs) >= String(rhs); }
01465 #endif // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01466 // clang-format on
01467
01468 #define DOCTEST_RELATIONAL_OP(name, op)
01469     template <typename L, typename R>
01470     DOCTEST_COMPARISON_RETURN_TYPE name(const DOCTEST_REF_WRAP(L) lhs,
01471     const DOCTEST_REF_WRAP(R) rhs) {
01472         return lhs op rhs;
01473     }
01474
01475     DOCTEST_RELATIONAL_OP(eq, ==)
01476     DOCTEST_RELATIONAL_OP(ne, !=)
01477     DOCTEST_RELATIONAL_OP(lt, <)
01478     DOCTEST_RELATIONAL_OP(gt, >)
01479     DOCTEST_RELATIONAL_OP(le, <=)
01480     DOCTEST_RELATIONAL_OP(ge, >=)
01481
01482 #ifndef DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01483 #define DOCTEST_CMP_EQ(l, r) l == r
01484 #define DOCTEST_CMP_NE(l, r) l != r

```



```

01484 #define DOCTEST_CMP_GT(l, r) l > r
01485 #define DOCTEST_CMP_LT(l, r) l < r
01486 #define DOCTEST_CMP_GE(l, r) l >= r
01487 #define DOCTEST_CMP_LE(l, r) l <= r
01488 #else // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01489 #define DOCTEST_CMP_EQ(l, r) eq(l, r)
01490 #define DOCTEST_CMP_NE(l, r) ne(l, r)
01491 #define DOCTEST_CMP_GT(l, r) gt(l, r)
01492 #define DOCTEST_CMP_LT(l, r) lt(l, r)
01493 #define DOCTEST_CMP_GE(l, r) ge(l, r)
01494 #define DOCTEST_CMP_LE(l, r) le(l, r)
01495 #endif // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01496
01497     template <typename L>
01498     // cppcheck-suppress copyCtorAndEqOperator
01499     struct Expression_lhs
01500     {
01501         L lhs;
01502         assertType::Enum m_at;
01503
01504         explicit Expression_lhs(L&& in, assertType::Enum at)
01505             : lhs(static_cast<L&&>(in))
01506             , m_at(at) {}
01507
01508         DOCTEST_NOINLINE operator Result() {
01509             // this is needed only for MSVC 2015
01510             DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4800) // 'int': forcing value to bool
01511             bool res = static_cast<bool>(lhs);
01512             DOCTEST_MSVC_SUPPRESS_WARNING_POP
01513             if(m_at & assertType::is_false) {
01514                 res = !res;
01515             }
01516
01517             if(!res || getContextOptions()->success) {
01518                 return { res, (DOCTEST_STRINGIFY(lhs)) };
01519             }
01520             return { res };
01521         }
01522
01523         /* This is required for user-defined conversions from Expression_lhs to L */
01524         operator L() const { return lhs; }
01525
01526         // clang-format off
01527         DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(==, " == ", DOCTEST_CMP_EQ)
01528         DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(!=, " != ", DOCTEST_CMP_NE)
01529         DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(>, " > ", DOCTEST_CMP_GT)
01530         DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(<, " < ", DOCTEST_CMP_LT)
01531         DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(>=, " >= ", DOCTEST_CMP_GE)
01532         DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(<=, " <= ", DOCTEST_CMP_LE)
01533         // clang-format on
01534
01535         // forbidding some expressions based on this table:
01536         // https://en.cppreference.com/w/cpp/language/operator_precedence
01537         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, &)
01538         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, ^)
01539         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, |)
01540         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, &&)
01541         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, ||)
01542         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, =)
01543         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, +=)
01544         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, -=)
01545         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, *=)
01546         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, /=)
01547         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, %=)
01548         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, >>)
01549         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, &=)
01550         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, ^=)
01551         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, |=)
01552         // these 2 are unfortunate because they should be allowed - they have higher precedence over
01553         // the comparisons, but the
01554         // ExpressionDecomposer class uses the left shift operator to capture the left operand of the
01555         // binary expression...
01556         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, <<)
01557         DOCTEST_FORBIT_EXPRESSION(Expression_lhs, >>)
01558     };
01559
01560 #ifndef DOCTEST_CONFIG_NO_COMPARISON_WARNING_SUPPRESSION
01561     DOCTEST_CLANG_SUPPRESS_WARNING_POP
01562     DOCTEST_MSVC_SUPPRESS_WARNING_POP
01563     DOCTEST_GCC_SUPPRESS_WARNING_POP
01564 #endif // DOCTEST_CONFIG_NO_COMPARISON_WARNING_SUPPRESSION
01565
01566 #if DOCTEST_CLANG && DOCTEST_CLANG < DOCTEST_COMPILER(3, 6, 0)
01567     DOCTEST_CLANG_SUPPRESS_WARNING_POP

```

```

01568 #endif
01569
01570 struct DOCTEST_INTERFACE ExpressionDecomposer
01571 {
01572     assertType::Enum m_at;
01573
01574     ExpressionDecomposer(assertType::Enum at);
01575
01576     // The right operator for capturing expressions is "<=" instead of "<" (based on the operator
precedence table)
01577     // but then there will be warnings from GCC about "-Wparentheses" and since "_Pragma()" is
problematic this will stay for now...
01578     // https://github.com/catchorg/Catch2/issues/870
01579     // https://github.com/catchorg/Catch2/issues/565
01580     template <typename L>
01581     Expression_lhs<L> operator<<(L&& operand) {
01582         return Expression_lhs<L>(static_cast<L&&>(operand), m_at);
01583     }
01584
01585     template <typename L,typename
types::enable_if<!doctest::detail::types::is_rvalue_reference<L>::value,void >::type* = nullptr>
01586     Expression_lhs<const L&> operator<<(const L &operand) {
01587         return Expression_lhs<const L&>(operand, m_at);
01588     }
01589 };
01590
01591 struct DOCTEST_INTERFACE TestSuite
01592 {
01593     const char* m_test_suite = nullptr;
01594     const char* m_description = nullptr;
01595     bool m_skip = false;
01596     bool m_no_breaks = false;
01597     bool m_no_output = false;
01598     bool m_may_fail = false;
01599     bool m_should_fail = false;
01600     int m_expected_failures = 0;
01601     double m_timeout = 0;
01602
01603     TestSuite& operator*(const char* in);
01604
01605     template <typename T>
01606     TestSuite& operator*(const T& in) {
01607         in.fill(*this);
01608         return *this;
01609     }
01610 };
01611
01612 using funcType = void (*)();
01613
01614 struct DOCTEST_INTERFACE TestCase : public TestCaseData
01615 {
01616     funcType m_test; // a function pointer to the test case
01617
01618     String m_type; // for templated test cases - gets appended to the real name
01619     int m_template_id; // an ID used to distinguish between the different versions of a templated
test case
01620     String m_full_name; // contains the name (only for templated test cases!) + the template type
01621
01622     TestCase(funcType test, const char* file, unsigned line, const TestSuite& test_suite,
const String& type = String(), int template_id = -1);
01623
01624     TestCase(const TestCase& other);
01625     TestCase(TestCase&&) = delete;
01626
01627     DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(26434) // hides a non-virtual function
01628     TestCase& operator=(const TestCase& other);
01629     DOCTEST_MSVC_SUPPRESS_WARNING_POP
01630
01631     TestCase& operator=(TestCase&&) = delete;
01632
01633     TestCase& operator*(const char* in);
01634
01635     template <typename T>
01636     TestCase& operator*(const T& in) {
01637         in.fill(*this);
01638         return *this;
01639     }
01640
01641     bool operator<(const TestCase& other) const;
01642
01643     ~TestCase() = default;
01644 };
01645
01646 // forward declarations of functions used by the macros
01647 DOCTEST_INTERFACE int regTest(const TestCase& tc);
01648 DOCTEST_INTERFACE int setTestSuite(const TestSuite& ts);
01649 DOCTEST_INTERFACE bool isDebuggerActive();

```

```

01651
01652     template<typename T>
01653     int instantiationHelper(const T&) { return 0; }
01654
01655     namespace binaryAssertComparison {
01656         enum Enum
01657         {
01658             eq = 0,
01659             ne,
01660             gt,
01661             lt,
01662             ge,
01663             le
01664         };
01665     } // namespace binaryAssertComparison
01666
01667     // clang-format off
01668     template <int, class L, class R> struct RelationalComparator { bool operator()(const
DOCTEST_REF_WRAP(L),      const DOCTEST_REF_WRAP(R)      ) const { return false; } };
01669
01670     #define DOCTEST_BINARY_RELATIONAL_OP(n, op) \
01671     template <class L, class R> struct RelationalComparator<n, L, R> { bool operator()(const
DOCTEST_REF_WRAP(L) lhs, const DOCTEST_REF_WRAP(R) rhs) const { return op(lhs, rhs); } };
01672     // clang-format on
01673
01674     DOCTEST_BINARY_RELATIONAL_OP(0, doctest::detail::eq)
01675     DOCTEST_BINARY_RELATIONAL_OP(1, doctest::detail::ne)
01676     DOCTEST_BINARY_RELATIONAL_OP(2, doctest::detail::gt)
01677     DOCTEST_BINARY_RELATIONAL_OP(3, doctest::detail::lt)
01678     DOCTEST_BINARY_RELATIONAL_OP(4, doctest::detail::ge)
01679     DOCTEST_BINARY_RELATIONAL_OP(5, doctest::detail::le)
01680
01681     struct DOCTEST_INTERFACE ResultBuilder : public AssertData
01682     {
01683         ResultBuilder(assertType::Enum at, const char* file, int line, const char* expr,
const char* exception_type = "", const String& exception_string = "");
01684
01685         ResultBuilder(assertType::Enum at, const char* file, int line, const char* expr,
const char* exception_type, const Contains& exception_string);
01686
01687         void setResult(const Result& res);
01688
01689         template <int comparison, typename L, typename R>
01690         DOCTEST_NOINLINE bool binary_assert(const DOCTEST_REF_WRAP(L) lhs,
const DOCTEST_REF_WRAP(R) rhs) {
01691             m_failed = !RelationalComparator<comparison, L, R>()(lhs, rhs);
01692             if (m_failed || getContextOptions()->success) {
01693                 m_decomp = stringifyBinaryExpr(lhs, " ", rhs);
01694             }
01695             return !m_failed;
01696         }
01697
01698         template <typename L>
01699         DOCTEST_NOINLINE bool unary_assert(const DOCTEST_REF_WRAP(L) val) {
01700             m_failed = !val;
01701
01702             if (m_at & assertType::is_false) {
01703                 m_failed = !m_failed;
01704             }
01705
01706             if (m_failed || getContextOptions()->success) {
01707                 m_decomp = (DOCTEST_STRINGIFY(val));
01708             }
01709
01710             return !m_failed;
01711         }
01712
01713         void translateException();
01714
01715         bool log();
01716         void react() const;
01717     };
01718
01719     namespace assertAction {
01720         enum Enum
01721         {
01722             nothing      = 0,
01723             dbgbreak     = 1,
01724             shouldthrow  = 2
01725         };
01726     } // namespace assertAction
01727
01728     DOCTEST_INTERFACE void failed_out_of_a_testing_context(const AssertData& ad);
01729
01730     DOCTEST_INTERFACE bool decomp_assert(assertType::Enum at, const char* file, int line,
const char* expr, const Result& result);
01731
01732
01733
01734
01735

```

```

01736 #define DOCTEST_ASSERT_OUT_OF_TESTS(decomp)
01737     do {
01738         if(!is_running_in_test) {
01739             if(failed) {
01740                 ResultBuilder rb(at, file, line, expr);
01741                 rb.m_failed = failed;
01742                 rb.m_decomp = decomp;
01743                 failed_out_of_a_testing_context(rb);
01744                 if(isDebuggerActive() && !getContextOptions()->no_breaks)
01745                     DOCTEST_BREAK_INTO_DEBUGGER();
01746                 if(checkIfShouldThrow(at))
01747                     throwException();
01748             }
01749             return !failed;
01750         }
01751     } while(false)
01752
01753 #define DOCTEST_ASSERT_IN_TESTS(decomp)
01754     ResultBuilder rb(at, file, line, expr);
01755     rb.m_failed = failed;
01756     if(rb.m_failed || getContextOptions()->success)
01757         rb.m_decomp = decomp;
01758     if(rb.log())
01759         DOCTEST_BREAK_INTO_DEBUGGER();
01760     if(rb.m_failed && checkIfShouldThrow(at))
01761         throwException()
01762
01763 template <int comparison, typename L, typename R>
01764 DOCTEST_NOINLINE bool binary_assert(assertType::Enum at, const char* file, int line,
01765                                     const char* expr, const DOCTEST_REF_WRAP(L) lhs,
01766                                     const DOCTEST_REF_WRAP(R) rhs) {
01767     bool failed = !RelationalComparator<comparison, L, R>()(lhs, rhs);
01768
01769     // #####
01770     // IF THE DEBUGGER BREAKS HERE - GO 1 LEVEL UP IN THE CALLSTACK FOR THE FAILING ASSERT
01771     // THIS IS THE EFFECT OF HAVING 'DOCTEST_CONFIG_SUPER_FAST_ASSERTS' DEFINED
01772     // #####
01773     DOCTEST_ASSERT_OUT_OF_TESTS(stringifyBinaryExpr(lhs, " ", rhs));
01774     DOCTEST_ASSERT_IN_TESTS(stringifyBinaryExpr(lhs, " ", rhs));
01775     return !failed;
01776 }
01777
01778 template <typename L>
01779 DOCTEST_NOINLINE bool unary_assert(assertType::Enum at, const char* file, int line,
01780                                     const char* expr, const DOCTEST_REF_WRAP(L) val) {
01781     bool failed = !val;
01782
01783     if(at & assertType::is_false)
01784         failed = !failed;
01785
01786     // #####
01787     // IF THE DEBUGGER BREAKS HERE - GO 1 LEVEL UP IN THE CALLSTACK FOR THE FAILING ASSERT
01788     // THIS IS THE EFFECT OF HAVING 'DOCTEST_CONFIG_SUPER_FAST_ASSERTS' DEFINED
01789     // #####
01790     DOCTEST_ASSERT_OUT_OF_TESTS((DOCTEST_STRINGIFY(val)));
01791     DOCTEST_ASSERT_IN_TESTS((DOCTEST_STRINGIFY(val)));
01792     return !failed;
01793 }
01794
01795 struct DOCTEST_INTERFACE IExceptionTranslator
01796 {
01797     DOCTEST_DECLARE_INTERFACE(IExceptionTranslator)
01798     virtual bool translate(String&) const = 0;
01799 };
01800
01801 template <typename T>
01802 class ExceptionTranslator : public IExceptionTranslator
01803 {
01804 public:
01805     explicit ExceptionTranslator(String (*translateFunction)(T)) (T)
01806         : m_translateFunction(translateFunction) {}
01807
01808     bool translate(String& res) const override {
01809 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
01810         try {
01811             throw; // lgtm [cpp/rethrow-no-exception]
01812             // cppcheck-suppress catchExceptionByValue
01813         } catch(const T& ex) {
01814             res = m_translateFunction(ex);
01815             return true;
01816         } catch(...) {}
01817 #endif
01818         // DOCTEST_CONFIG_NO_EXCEPTIONS
01819         static_cast<void>(res); // to silence -Wunused-parameter
01820         return false;
01821     }
01822 private:

```

```

01823     String (*m_translateFunction)(T);
01824 };
01825
01826 DOCTEST_INTERFACE void registerExceptionTranslatorImpl(const IExceptionTranslator* et);
01827
01828 // ContextScope base class used to allow implementing methods of ContextScope
01829 // that don't depend on the template parameter in doctest.cpp.
01830 struct DOCTEST_INTERFACE ContextScopeBase : public IContextScope {
01831     ContextScopeBase(const ContextScopeBase&) = delete;
01832
01833     ContextScopeBase& operator=(const ContextScopeBase&) = delete;
01834     ContextScopeBase& operator=(ContextScopeBase&&) = delete;
01835
01836     ~ContextScopeBase() override = default;
01837
01838 protected:
01839     ContextScopeBase();
01840     ContextScopeBase(ContextScopeBase&& other) noexcept;
01841
01842     void destroy();
01843     bool need_to_destroy{true};
01844 };
01845
01846 template <typename L> class ContextScope : public ContextScopeBase
01847 {
01848     L lambda_;
01849
01850 public:
01851     explicit ContextScope(const L &lambda) : lambda_(lambda) {}
01852     explicit ContextScope(L&& lambda) : lambda_(static_cast<L&&>(lambda)) { }
01853
01854     ContextScope(const ContextScope&) = delete;
01855     ContextScope(ContextScope&&) noexcept = default;
01856
01857     ContextScope& operator=(const ContextScope&) = delete;
01858     ContextScope& operator=(ContextScope&&) = delete;
01859
01860     void stringify(std::ostream* s) const override { lambda_(s); }
01861
01862     ~ContextScope() override {
01863         if (need_to_destroy) {
01864             destroy();
01865         }
01866     }
01867 };
01868
01869 struct DOCTEST_INTERFACE MessageBuilder : public MessageData
01870 {
01871     std::ostream* m_stream;
01872     bool logged = false;
01873
01874     MessageBuilder(const char* file, int line, assertType::Enum severity);
01875
01876     MessageBuilder(const MessageBuilder&) = delete;
01877     MessageBuilder(MessageBuilder&&) = delete;
01878
01879     MessageBuilder& operator=(const MessageBuilder&) = delete;
01880     MessageBuilder& operator=(MessageBuilder&&) = delete;
01881
01882     ~MessageBuilder();
01883
01884     // the preferred way of chaining parameters for stringification
01885 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4866)
01886     template <typename T>
01887     MessageBuilder& operator,(const T& in) {
01888         *m_stream << (DOCTEST_STRINGIFY(in));
01889         return *this;
01890     }
01891 DOCTEST_MSVC_SUPPRESS_WARNING_POP
01892
01893     // kept here just for backwards-compatibility - the comma operator should be preferred now
01894     template <typename T>
01895     MessageBuilder& operator<(const T& in) { return this->operator,(in); }
01896
01897     // the `,' operator has the lowest operator precedence - if `«' is used by the user then
01898     // the `,' operator will be called last which is not what we want and thus the `*' operator
01899     // is used first (has higher operator precedence compared to `«') so that we guarantee that
01900     // an operator of the MessageBuilder class is called first before the rest of the parameters
01901     template <typename T>
01902     MessageBuilder& operator*(const T& in) { return this->operator,(in); }
01903
01904     bool log();
01905     void react();
01906 };
01907
01908 template <typename L>
01909 ContextScope<L> MakeContextScope(const L &lambda) {

```

```

01910         return ContextScope<L>(lambda);
01911     }
01912 } // namespace detail
01913
01914 #define DOCTEST_DEFINE_DECORATOR(name, type, def)
01915     struct name
01916     {
01917         type data;
01918         name(type in = def)
01919             : data(in) {}
01920         void fill(detail::TestCase& state) const { state.DOCTEST_CAT(m_, name) = data; }
01921         void fill(detail::TestSuite& state) const { state.DOCTEST_CAT(m_, name) = data; }
01922     }
01923
01924 DOCTEST_DEFINE_DECORATOR(test_suite, const char*, "");
01925 DOCTEST_DEFINE_DECORATOR(description, const char*, "");
01926 DOCTEST_DEFINE_DECORATOR(skip, bool, true);
01927 DOCTEST_DEFINE_DECORATOR(no_breaks, bool, true);
01928 DOCTEST_DEFINE_DECORATOR(no_output, bool, true);
01929 DOCTEST_DEFINE_DECORATOR(timeout, double, 0);
01930 DOCTEST_DEFINE_DECORATOR(may_fail, bool, true);
01931 DOCTEST_DEFINE_DECORATOR(should_fail, bool, true);
01932 DOCTEST_DEFINE_DECORATOR(expected_failures, int, 0);
01933
01934 template <typename T>
01935 int registerExceptionTranslator(String (*translateFunction)(T)) {
01936     DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wexit-time-destructors")
01937     static detail::ExceptionTranslator<T> exceptionTranslator(translateFunction);
01938     DOCTEST_CLANG_SUPPRESS_WARNING_POP
01939     detail::registerExceptionTranslatorImpl(&exceptionTranslator);
01940     return 0;
01941 }
01942
01943 } // namespace doctest
01944
01945 // in a separate namespace outside of doctest because the DOCTEST_TEST_SUITE macro
01946 // introduces an anonymous namespace in which getCurrentTestSuite gets overridden
01947 namespace doctest_detail_test_suite_ns {
01948     DOCTEST_INTERFACE doctest::detail::TestSuite& getCurrentTestSuite();
01949 } // namespace doctest_detail_test_suite_ns
01950
01951 namespace doctest {
01952 #else // DOCTEST_CONFIG_DISABLE
01953     template <typename T>
01954     int registerExceptionTranslator(String (*)(T)) {
01955         return 0;
01956     }
01957 #endif // DOCTEST_CONFIG_DISABLE
01958
01959 namespace detail {
01960     using assert_handler = void (*)(const AssertData&);
01961     struct ContextState;
01962 } // namespace detail
01963
01964 class DOCTEST_INTERFACE Context
01965 {
01966     detail::ContextState* p;
01967
01968     void parseArgs(int argc, const char* const* argv, bool withDefaults = false);
01969
01970 public:
01971     explicit Context(int argc = 0, const char* const* argv = nullptr);
01972
01973     Context(const Context&) = delete;
01974     Context(Context&&) = delete;
01975
01976     Context& operator=(const Context&) = delete;
01977     Context& operator=(Context&&) = delete;
01978
01979     ~Context(); // NOLINT(performance-trivially-destructible)
01980
01981     void applyCommandLine(int argc, const char* const* argv);
01982
01983     void addFilter(const char* filter, const char* value);
01984     void clearFilters();
01985     void setOption(const char* option, bool value);
01986     void setOption(const char* option, int value);
01987     void setOption(const char* option, const char* value);
01988
01989     bool shouldExit();
01990
01991     void setAsDefaultForAssertsOutOfTestCases();
01992
01993     void setAssertHandler(detail::assert_handler ah);
01994
01995     void setCout(std::ostream* out);
01996

```

```

01997     int run();
01998 };
01999
02000 namespace TestCaseFailureReason {
02001     enum Enum
02002     {
02003         None = 0,
02004         AssertFailure = 1, // an assertion has failed in the test case
02005         Exception = 2, // test case threw an exception
02006         Crash = 4, // a crash...
02007         TooManyFailedAsserts = 8, // the abort-after option
02008         Timeout = 16, // see the timeout decorator
02009         ShouldHaveFailedButDidnt = 32, // see the should_fail decorator
02010         ShouldHaveFailedAndDid = 64, // see the should_fail decorator
02011         DidntFailExactlyNumTimes = 128, // see the expected_failures decorator
02012         FailedExactlyNumTimes = 256, // see the expected_failures decorator
02013         CouldHaveFailedAndDid = 512 // see the may_fail decorator
02014     };
02015 } // namespace TestCaseFailureReason
02016
02017 struct DOCTEST_INTERFACE CurrentTestCaseStats
02018 {
02019     int numAssertsCurrentTest;
02020     int numAssertsFailedCurrentTest;
02021     double seconds;
02022     int failure_flags; // use TestCaseFailureReason::Enum
02023     bool testCaseSuccess;
02024 };
02025
02026 struct DOCTEST_INTERFACE TestCaseException
02027 {
02028     String error_string;
02029     bool is_crash;
02030 };
02031
02032 struct DOCTEST_INTERFACE TestRunStats
02033 {
02034     unsigned numTestCases;
02035     unsigned numTestCasesPassingFilters;
02036     unsigned numTestSuitesPassingFilters;
02037     unsigned numTestCasesFailed;
02038     int numAsserts;
02039     int numAssertsFailed;
02040 };
02041
02042 struct QueryData
02043 {
02044     const TestRunStats* run_stats = nullptr;
02045     const TestCaseData** data = nullptr;
02046     unsigned num_data = 0;
02047 };
02048
02049 struct DOCTEST_INTERFACE IReporter
02050 {
02051     // The constructor has to accept "const ContextOptions&" as a single argument
02052     // which has most of the options for the run + a pointer to the stdout stream
02053     // Reporter(const ContextOptions& in)
02054
02055     // called when a query should be reported (listing test cases, printing the version, etc.)
02056     virtual void report_query(const QueryData&) = 0;
02057
02058     // called when the whole test run starts
02059     virtual void test_run_start() = 0;
02060     // called when the whole test run ends (caching a pointer to the input doesn't make sense here)
02061     virtual void test_run_end(const TestRunStats&) = 0;
02062
02063     // called when a test case is started (safe to cache a pointer to the input)
02064     virtual void test_case_start(const TestCaseData&) = 0;
02065     // called when a test case is reentered because of unfinished subcases (safe to cache a pointer to
the input)
02066     virtual void test_case_reenter(const TestCaseData&) = 0;
02067     // called when a test case has ended
02068     virtual void test_case_end(const CurrentTestCaseStats&) = 0;
02069
02070     // called when an exception is thrown from the test case (or it crashes)
02071     virtual void test_case_exception(const TestCaseException&) = 0;
02072
02073     // called whenever a subcase is entered (don't cache pointers to the input)
02074     virtual void subcase_start(const SubcaseSignature&) = 0;
02075     // called whenever a subcase is exited (don't cache pointers to the input)
02076     virtual void subcase_end() = 0;
02077
02078     // called for each assert (don't cache pointers to the input)
02079     virtual void log_assert(const AssertData&) = 0;
02080     // called for each message (don't cache pointers to the input)
02081     virtual void log_message(const MessageData&) = 0;
02082

```

```

02083 // called when a test case is skipped either because it doesn't pass the filters, has a skip
decorator
02084 // or isn't in the execution range (between first and last) (safe to cache a pointer to the input)
02085 virtual void test_case_skipped(const TestCaseData&) = 0;
02086
02087 DOCTEST_DECLARE_INTERFACE(IReporter)
02088
02089 // can obtain all currently active contexts and stringify them if one wishes to do so
02090 static int get_num_active_contexts();
02091 static const IContextScope* const* get_active_contexts();
02092
02093 // can iterate through contexts which have been stringified automatically in their destructors
when an exception has been thrown
02094 static int get_num_stringified_contexts();
02095 static const String* get_stringified_contexts();
02096 };
02097
02098 namespace detail {
02099     using reporterCreatorFunc = IReporter* (*)(const ContextOptions&);
02100
02101     DOCTEST_INTERFACE void registerReporterImpl(const char* name, int prio, reporterCreatorFunc c,
bool isReporter);
02102
02103     template <typename Reporter>
02104     IReporter* reporterCreator(const ContextOptions& o) {
02105         return new Reporter(o);
02106     }
02107 } // namespace detail
02108
02109 template <typename Reporter>
02110 int registerReporter(const char* name, int priority, bool isReporter) {
02111     detail::registerReporterImpl(name, priority, detail::reporterCreator<Reporter>, isReporter);
02112     return 0;
02113 }
02114 } // namespace doctest
02115
02116 #ifdef DOCTEST_CONFIG_ASSERTS_RETURN_VALUES
02117 #define DOCTEST_FUNC_EMPTY [] { return false; }()
02118 #else
02119 #define DOCTEST_FUNC_EMPTY (void)0
02120 #endif
02121
02122 // if registering is not disabled
02123 #ifndef DOCTEST_CONFIG_DISABLE
02124
02125 #ifdef DOCTEST_CONFIG_ASSERTS_RETURN_VALUES
02126 #define DOCTEST_FUNC_SCOPE_BEGIN [&]
02127 #define DOCTEST_FUNC_SCOPE_END ()
02128 #define DOCTEST_FUNC_SCOPE_RET(v) return v
02129 #else
02130 #define DOCTEST_FUNC_SCOPE_BEGIN do
02131 #define DOCTEST_FUNC_SCOPE_END while(false)
02132 #define DOCTEST_FUNC_SCOPE_RET(v) (void)0
02133 #endif
02134
02135 // common code in asserts - for convenience
02136 #define DOCTEST_ASSERT_LOG_REACT_RETURN(b) //
02137     if(b.log()) DOCTEST_BREAK_INTO_DEBUGGER(); //
02138     b.react(); //
02139     DOCTEST_FUNC_SCOPE_RET(!b.m_failed)
02140
02141 #ifdef DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
02142 #define DOCTEST_WRAP_IN_TRY(x) x;
02143 #else // DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
02144 #define DOCTEST_WRAP_IN_TRY(x) //
02145     try { //
02146         x; //
02147     } catch(...) { DOCTEST_RB.translateException(); }
02148 #endif // DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
02149
02150 #ifdef DOCTEST_CONFIG_VOID_CAST_EXPRESSIONS
02151 #define DOCTEST_CAST_TO_VOID(...) //
02152     DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wuseless-cast")
02153     static_cast<void>(__VA_ARGS__);
02154     DOCTEST_GCC_SUPPRESS_WARNING_POP
02155 #else // DOCTEST_CONFIG_VOID_CAST_EXPRESSIONS
02156 #define DOCTEST_CAST_TO_VOID(...) __VA_ARGS__;
02157 #endif // DOCTEST_CONFIG_VOID_CAST_EXPRESSIONS
02158
02159 // registers the test by initializing a dummy var with a function
02160 #define DOCTEST_REGISTER_FUNCTION(global_prefix, f, decorators) //
02161     global_prefix DOCTEST_GLOBAL_NO_WARNINGS(DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_), /* NOLINT */ //
02162     doctest::detail::regTest( //
02163         doctest::detail::TestCase( //
02164             f, __FILE__, __LINE__, //
02165             doctest_detail_test_suite_ns::getCurrentTestSuite()) * //
02166         decorators)

```



```

02167
02168 #define DOCTEST_IMPLEMENT_FIXTURE(der, base, func, decorators)
02169     namespace { /* NOLINT */
02170         struct der : public base
02171         {
02172             void f();
02173         };
02174         static DOCTEST_INLINE_NOINLINE void func() {
02175             der v;
02176             v.f();
02177         }
02178         DOCTEST_REGISTER_FUNCTION(DOCTEST_EMPTY, func, decorators)
02179     }
02180     DOCTEST_INLINE_NOINLINE void der::f() // NOLINT(misc-definitions-in-headers)
02181
02182 #define DOCTEST_CREATE_AND_REGISTER_FUNCTION(f, decorators)
02183     static void f();
02184     DOCTEST_REGISTER_FUNCTION(DOCTEST_EMPTY, f, decorators)
02185     static void f()
02186
02187 #define DOCTEST_CREATE_AND_REGISTER_FUNCTION_IN_CLASS(f, proxy, decorators)
02188     static doctest::detail::funcType proxy() { return f; }
02189     DOCTEST_REGISTER_FUNCTION(inline, proxy(), decorators)
02190     static void f()
02191
02192 // for registering tests
02193 #define DOCTEST_TEST_CASE(decorators)
02194     DOCTEST_CREATE_AND_REGISTER_FUNCTION(DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), decorators)
02195
02196 // for registering tests in classes - requires C++17 for inline variables!
02197 #if DOCTEST_CPLUSPLUS >= 201703L
02198 #define DOCTEST_TEST_CASE_CLASS(decorators)
02199     DOCTEST_CREATE_AND_REGISTER_FUNCTION_IN_CLASS(DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_),
02200                                                    DOCTEST_ANONYMOUS(DOCTEST_ANON_PROXY_),
02201                                                    decorators)
02202 #else // DOCTEST_TEST_CASE_CLASS
02203 #define DOCTEST_TEST_CASE_CLASS(...)
02204     TEST_CASES_CAN_BE_REGISTERED_IN_CLASSES_ONLY_IN_CPPI7_MODE_OR_WITH_VS_2017_OR_NEWER
02205 #endif // DOCTEST_TEST_CASE_CLASS
02206
02207 // for registering tests with a fixture
02208 #define DOCTEST_TEST_CASE_FIXTURE(c, decorators)
02209     DOCTEST_IMPLEMENT_FIXTURE(DOCTEST_ANONYMOUS(DOCTEST_ANON_CLASS_), c,
02210                               DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), decorators)
02211
02212 // for converting types to strings without the <typeinfo> header and demangling
02213 #define DOCTEST_TYPE_TO_STRING_AS(str, ...)
02214     namespace doctest {
02215         template <>
02216         inline String toString<__VA_ARGS__>() {
02217             return str;
02218         }
02219     }
02220     static_assert(true, "")
02221
02222 #define DOCTEST_TYPE_TO_STRING(...) DOCTEST_TYPE_TO_STRING_AS(#__VA_ARGS__, __VA_ARGS__)
02223
02224 #define DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(dec, T, iter, func)
02225     template <typename T>
02226     static void func();
02227     namespace { /* NOLINT */
02228         template <typename Tuple>
02229         struct iter;
02230         template <typename Type, typename... Rest>
02231         struct iter<std::tuple<Type, Rest...>>
02232         {
02233             iter(const char* file, unsigned line, int index) {
02234                 doctest::detail::regTest(doctest::detail::TestCase(func<Type>, file, line,
02235                                                                    doctest_detail_test_suite_ns::getCurrentTestSuite(),
02236                                                                    doctest::toString<Type>(),
02237                                                                    int(line) * 1000 + index),
02238                                           * dec);
02239                 iter<std::tuple<Rest...>>(file, line, index + 1);
02240             }
02241         };
02242         template <>
02243         struct iter<std::tuple<>>
02244         {
02245             iter(const char*, unsigned, int) {}
02246         };
02247     }
02248     template <typename T>
02249     static void func()
02250
02251 #define DOCTEST_TEST_CASE_TEMPLATE_DEFINE(dec, T, id)
02252     DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(dec, T, DOCTEST_CAT(id, ITERATOR),
02253                                           DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_))

```

```

02254
02255 #define DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(id, anon, ...) \
02256     DOCTEST_GLOBAL_NO_WARNINGS(DOCTEST_CAT(anon, DUMMY), /* NOLINT(cert-err58-cpp, \
fuchsia-statically-constructed-objects) */ \
02257     doctest::detail::instantiationHelper( \
02258         DOCTEST_CAT(id, ITERATOR)<__VA_ARGS__>(__FILE__, __LINE__, 0))) \
02259
02260 #define DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id, ...) \
02261     DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(id, DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_), \
std::tuple<__VA_ARGS__>) \
02262     static_assert(true, "")
02263
02264 #define DOCTEST_TEST_CASE_TEMPLATE_APPLY(id, ...) \
02265     DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(id, DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_), __VA_ARGS__) \
\
02266     static_assert(true, "")
02267
02268 #define DOCTEST_TEST_CASE_TEMPLATE_IMPL(dec, T, anon, ...) \
02269     DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(dec, T, DOCTEST_CAT(anon, ITERATOR), anon); \
02270     DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(anon, anon, std::tuple<__VA_ARGS__>) \
02271     template <typename T> \
02272     static void anon() \
02273
02274 #define DOCTEST_TEST_CASE_TEMPLATE(dec, T, ...) \
02275     DOCTEST_TEST_CASE_TEMPLATE_IMPL(dec, T, DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_), __VA_ARGS__) \
02276
02277 // for subcases
02278 #define DOCTEST_SUBCASE(name) \
02279     if(const doctest::detail::Subcase & DOCTEST_ANONYMOUS(DOCTEST_ANON_SUBCASE_) DOCTEST_UNUSED = \
02280         doctest::detail::Subcase(name, __FILE__, __LINE__)) \
02281
02282 // for grouping tests in test suites by using code blocks
02283 #define DOCTEST_TEST_SUITE_IMPL(decorators, ns_name) \
02284     namespace ns_name { namespace doctest_detail_test_suite_ns { \
02285         static DOCTEST_NOINLINE doctest::detail::TestSuite& getCurrentTestSuite() noexcept { \
02286             DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4640) \
02287             DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wexit-time-destructors") \
02288             DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wmissing-field-initializers") \
02289             static doctest::detail::TestSuite data{}; \
02290             static bool inited = false; \
02291             DOCTEST_MSVC_SUPPRESS_WARNING_POP \
02292             DOCTEST_CLANG_SUPPRESS_WARNING_POP \
02293             DOCTEST_GCC_SUPPRESS_WARNING_POP \
02294             if(!inited) { \
02295                 data* decorators; \
02296                 inited = true; \
02297             } \
02298             return data; \
02299         } \
02300     } \
02301 } \
02302 namespace ns_name
02303
02304 #define DOCTEST_TEST_SUITE(decorators) \
02305     DOCTEST_TEST_SUITE_IMPL(decorators, DOCTEST_ANONYMOUS(DOCTEST_ANON_SUITE_)) \
02306
02307 // for starting a testsuite block
02308 #define DOCTEST_TEST_SUITE_BEGIN(decorators) \
02309     DOCTEST_GLOBAL_NO_WARNINGS(DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_), /* NOLINT(cert-err58-cpp) */ \
02310     doctest::detail::setTestSuite(doctest::detail::TestSuite() * decorators)) \
02311     static_assert(true, "")
02312
02313 // for ending a testsuite block
02314 #define DOCTEST_TEST_SUITE_END \
02315     DOCTEST_GLOBAL_NO_WARNINGS(DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_), /* NOLINT(cert-err58-cpp) */ \
02316     doctest::detail::setTestSuite(doctest::detail::TestSuite() * "")) \
02317     using DOCTEST_ANONYMOUS(DOCTEST_ANON_FOR_SEMICOLON_) = int
02318
02319 // for registering exception translators
02320 #define DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL(translatorName, signature) \
02321     inline doctest::String translatorName(signature); \
02322     DOCTEST_GLOBAL_NO_WARNINGS(DOCTEST_ANONYMOUS(DOCTEST_ANON_TRANSLATOR_), /* NOLINT(cert-err58-cpp) \
02323     */ \
02324     doctest::registerExceptionTranslator(translatorName)) \
02325     doctest::String translatorName(signature)
02326
02327 #define DOCTEST_REGISTER_EXCEPTION_TRANSLATOR(signature) \
02328     DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL(DOCTEST_ANONYMOUS(DOCTEST_ANON_TRANSLATOR_), \
signature)
02329
02330 // for registering reporters
02331 #define DOCTEST_REGISTER_REPORTER(name, priority, reporter) \
02332     DOCTEST_GLOBAL_NO_WARNINGS(DOCTEST_ANONYMOUS(DOCTEST_ANON_REPORTER_), /* NOLINT(cert-err58-cpp) */ \
02333     doctest::registerReporter<reporter>(name, priority, true)) \
02334     static_assert(true, "")
02335

```

```

02336 // for registering listeners
02337 #define DOCTEST_REGISTER_LISTENER(name, priority, reporter) \
02338     DOCTEST_GLOBAL_NO_WARNINGS(DOCTEST_ANONYMOUS(DOCTEST_ANON_REPORTER_), /* NOLINT(cert-err58-cpp) */ \
    \
02339         doctest::registerReporter<reporter>(name, priority, false)) \
02340     static_assert(true, "")
02341
02342 // clang-format off
02343 // for logging - disabling formatting because it's important to have these on 2 separate lines - see
PR #557
02344 #define DOCTEST_INFO(...) \
02345     DOCTEST_INFO_IMPL(DOCTEST_ANONYMOUS(DOCTEST_CAPTURE_), \
02346         DOCTEST_ANONYMOUS(DOCTEST_CAPTURE_OTHER_), \
02347         __VA_ARGS__)
02348 // clang-format on
02349
02350 #define DOCTEST_INFO_IMPL(mb_name, s_name, ...) \
02351     auto DOCTEST_ANONYMOUS(DOCTEST_CAPTURE_) = doctest::detail::MakeContextScope( \
02352         [&](std::ostream* s_name) { \
02353             doctest::detail::MessageBuilder mb_name(__FILE__, __LINE__, doctest::assertType::is_warn); \
02354             mb_name.m_stream = s_name; \
02355             mb_name * __VA_ARGS__; \
02356         })
02357
02358 #define DOCTEST_CAPTURE(x) DOCTEST_INFO(#x " := ", x)
02359
02360 #define DOCTEST_ADD_AT_IMPL(type, file, line, mb, ...) \
02361     DOCTEST_FUNC_SCOPE_BEGIN { \
02362         doctest::detail::MessageBuilder mb(file, line, doctest::assertType::type); \
02363         mb * __VA_ARGS__; \
02364         if(mb.log()) \
02365             DOCTEST_BREAK_INTO_DEBUGGER(); \
02366         mb.react(); \
02367     } DOCTEST_FUNC_SCOPE_END
02368
02369 // clang-format off
02370 #define DOCTEST_ADD_MESSAGE_AT(file, line, ...) DOCTEST_ADD_AT_IMPL(is_warn, file, line, \
DOCTEST_ANONYMOUS(DOCTEST_MESSAGE_), __VA_ARGS__)
02371 #define DOCTEST_ADD_FAIL_CHECK_AT(file, line, ...) DOCTEST_ADD_AT_IMPL(is_check, file, line, \
DOCTEST_ANONYMOUS(DOCTEST_MESSAGE_), __VA_ARGS__)
02372 #define DOCTEST_ADD_FAIL_AT(file, line, ...) DOCTEST_ADD_AT_IMPL(is_require, file, line, \
DOCTEST_ANONYMOUS(DOCTEST_MESSAGE_), __VA_ARGS__)
02373 // clang-format on
02374
02375 #define DOCTEST_MESSAGE(...) DOCTEST_ADD_MESSAGE_AT(__FILE__, __LINE__, __VA_ARGS__)
02376 #define DOCTEST_FAIL_CHECK(...) DOCTEST_ADD_FAIL_CHECK_AT(__FILE__, __LINE__, __VA_ARGS__)
02377 #define DOCTEST_FAIL(...) DOCTEST_ADD_FAIL_AT(__FILE__, __LINE__, __VA_ARGS__)
02378
02379 #define DOCTEST_TO_LVALUE(...) __VA_ARGS__ // Not removed to keep backwards compatibility.
02380
02381 #ifndef DOCTEST_CONFIG_SUPER_FAST_ASSERTS
02382
02383 #define DOCTEST_ASSERT_IMPLEMENT_2(assert_type, ...) \
02384     DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Woverloaded-shift-op-parentheses") \
02385     /* NOLINTNEXTLINE(clang-analyzer-cplusplus.NewDeleteLeaks) */ \
02386     doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__, \
02387         __LINE__, #__VA_ARGS__); \
02388     DOCTEST_WRAP_IN_TRY(DOCTEST_RB.setResult( \
02389         doctest::detail::ExpressionDecomposer(doctest::assertType::assert_type) \
02390         < __VA_ARGS__ */) /* NOLINTNEXTLINE(clang-analyzer-cplusplus.NewDeleteLeaks) */ \
02391     DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB) \
02392     DOCTEST_CLANG_SUPPRESS_WARNING_POP
02393
02394 #define DOCTEST_ASSERT_IMPLEMENT_1(assert_type, ...) \
02395     DOCTEST_FUNC_SCOPE_BEGIN { \
02396         DOCTEST_ASSERT_IMPLEMENT_2(assert_type, __VA_ARGS__); \
02397     } DOCTEST_FUNC_SCOPE_END // NOLINT(clang-analyzer-cplusplus.NewDeleteLeaks)
02398
02399 #define DOCTEST_BINARY_ASSERT(assert_type, comp, ...) \
02400     DOCTEST_FUNC_SCOPE_BEGIN { \
02401         doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__, \
02402             __LINE__, #__VA_ARGS__); \
02403         DOCTEST_WRAP_IN_TRY( \
02404             DOCTEST_RB.binary_assert<doctest::detail::binaryAssertComparison::comp>( \
02405                 __VA_ARGS__) \
02406             DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB); \
02407         } DOCTEST_FUNC_SCOPE_END
02408
02409 #define DOCTEST_UNARY_ASSERT(assert_type, ...) \
02410     DOCTEST_FUNC_SCOPE_BEGIN { \
02411         doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__, \
02412             __LINE__, #__VA_ARGS__); \
02413         DOCTEST_WRAP_IN_TRY(DOCTEST_RB.unary_assert(__VA_ARGS__)) \
02414         DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB); \
02415     } DOCTEST_FUNC_SCOPE_END
02416
02417 #else // DOCTEST_CONFIG_SUPER_FAST_ASSERTS

```

```

02418
02419 // necessary for <ASSERT>_MESSAGE
02420 #define DOCTEST_ASSERT_IMPLEMENT_2 DOCTEST_ASSERT_IMPLEMENT_1
02421
02422 #define DOCTEST_ASSERT_IMPLEMENT_1(assert_type, ...) \
02423     DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Woverloaded-shift-op-parentheses") \
02424     doctest::detail::decomp_assert( \
02425         doctest::assertType::assert_type, __FILE__, __LINE__, #__VA_ARGS__, \
02426         doctest::detail::ExpressionComposer(doctest::assertType::assert_type) \
02427         « __VA_ARGS__ DOCTEST_CLANG_SUPPRESS_WARNING_POP \
02428
02429 #define DOCTEST_BINARY_ASSERT(assert_type, comparison, ...) \
02430     doctest::detail::binary_assert<doctest::detail::binaryAssertComparison::comparison>( \
02431         doctest::assertType::assert_type, __FILE__, __LINE__, #__VA_ARGS__, __VA_ARGS__) \
02432
02433 #define DOCTEST_UNARY_ASSERT(assert_type, ...) \
02434     doctest::detail::unary_assert(doctest::assertType::assert_type, __FILE__, __LINE__, \
02435         #__VA_ARGS__, __VA_ARGS__) \
02436
02437 #endif // DOCTEST_CONFIG_SUPER_FAST_ASSERTS
02438
02439 #define DOCTEST_WARN(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_WARN, __VA_ARGS__)
02440 #define DOCTEST_CHECK(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_CHECK, __VA_ARGS__)
02441 #define DOCTEST_REQUIRE(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_REQUIRE, __VA_ARGS__)
02442 #define DOCTEST_WARN_FALSE(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_WARN_FALSE, __VA_ARGS__)
02443 #define DOCTEST_CHECK_FALSE(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_CHECK_FALSE, __VA_ARGS__)
02444 #define DOCTEST_REQUIRE_FALSE(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_REQUIRE_FALSE, __VA_ARGS__)
02445
02446 // clang-format off
02447 #define DOCTEST_WARN_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__); \
DOCTEST_ASSERT_IMPLEMENT_2(DT_WARN, cond); } DOCTEST_FUNC_SCOPE_END
02448 #define DOCTEST_CHECK_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__); \
DOCTEST_ASSERT_IMPLEMENT_2(DT_CHECK, cond); } DOCTEST_FUNC_SCOPE_END
02449 #define DOCTEST_REQUIRE_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__); \
DOCTEST_ASSERT_IMPLEMENT_2(DT_REQUIRE, cond); } DOCTEST_FUNC_SCOPE_END
02450 #define DOCTEST_WARN_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__); \
DOCTEST_ASSERT_IMPLEMENT_2(DT_WARN_FALSE, cond); } DOCTEST_FUNC_SCOPE_END
02451 #define DOCTEST_CHECK_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__); \
DOCTEST_ASSERT_IMPLEMENT_2(DT_CHECK_FALSE, cond); } DOCTEST_FUNC_SCOPE_END
02452 #define DOCTEST_REQUIRE_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__); \
DOCTEST_ASSERT_IMPLEMENT_2(DT_REQUIRE_FALSE, cond); } DOCTEST_FUNC_SCOPE_END
02453 // clang-format on
02454
02455 #define DOCTEST_WARN_EQ(...) DOCTEST_BINARY_ASSERT(DT_WARN_EQ, eq, __VA_ARGS__)
02456 #define DOCTEST_CHECK_EQ(...) DOCTEST_BINARY_ASSERT(DT_CHECK_EQ, eq, __VA_ARGS__)
02457 #define DOCTEST_REQUIRE_EQ(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_EQ, eq, __VA_ARGS__)
02458 #define DOCTEST_WARN_NE(...) DOCTEST_BINARY_ASSERT(DT_WARN_NE, ne, __VA_ARGS__)
02459 #define DOCTEST_CHECK_NE(...) DOCTEST_BINARY_ASSERT(DT_CHECK_NE, ne, __VA_ARGS__)
02460 #define DOCTEST_REQUIRE_NE(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_NE, ne, __VA_ARGS__)
02461 #define DOCTEST_WARN_GT(...) DOCTEST_BINARY_ASSERT(DT_WARN_GT, gt, __VA_ARGS__)
02462 #define DOCTEST_CHECK_GT(...) DOCTEST_BINARY_ASSERT(DT_CHECK_GT, gt, __VA_ARGS__)
02463 #define DOCTEST_REQUIRE_GT(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_GT, gt, __VA_ARGS__)
02464 #define DOCTEST_WARN_LT(...) DOCTEST_BINARY_ASSERT(DT_WARN_LT, lt, __VA_ARGS__)
02465 #define DOCTEST_CHECK_LT(...) DOCTEST_BINARY_ASSERT(DT_CHECK_LT, lt, __VA_ARGS__)
02466 #define DOCTEST_REQUIRE_LT(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_LT, lt, __VA_ARGS__)
02467 #define DOCTEST_WARN_GE(...) DOCTEST_BINARY_ASSERT(DT_WARN_GE, ge, __VA_ARGS__)
02468 #define DOCTEST_CHECK_GE(...) DOCTEST_BINARY_ASSERT(DT_CHECK_GE, ge, __VA_ARGS__)
02469 #define DOCTEST_REQUIRE_GE(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_GE, ge, __VA_ARGS__)
02470 #define DOCTEST_WARN_LE(...) DOCTEST_BINARY_ASSERT(DT_WARN_LE, le, __VA_ARGS__)
02471 #define DOCTEST_CHECK_LE(...) DOCTEST_BINARY_ASSERT(DT_CHECK_LE, le, __VA_ARGS__)
02472 #define DOCTEST_REQUIRE_LE(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_LE, le, __VA_ARGS__)
02473
02474 #define DOCTEST_WARN_UNARY(...) DOCTEST_UNARY_ASSERT(DT_WARN_UNARY, __VA_ARGS__)
02475 #define DOCTEST_CHECK_UNARY(...) DOCTEST_UNARY_ASSERT(DT_CHECK_UNARY, __VA_ARGS__)
02476 #define DOCTEST_REQUIRE_UNARY(...) DOCTEST_UNARY_ASSERT(DT_REQUIRE_UNARY, __VA_ARGS__)
02477 #define DOCTEST_WARN_UNARY_FALSE(...) DOCTEST_UNARY_ASSERT(DT_WARN_UNARY_FALSE, __VA_ARGS__)
02478 #define DOCTEST_CHECK_UNARY_FALSE(...) DOCTEST_UNARY_ASSERT(DT_CHECK_UNARY_FALSE, __VA_ARGS__)
02479 #define DOCTEST_REQUIRE_UNARY_FALSE(...) DOCTEST_UNARY_ASSERT(DT_REQUIRE_UNARY_FALSE, __VA_ARGS__)
02480
02481 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
02482
02483 #define DOCTEST_ASSERT_THROWS_AS(expr, assert_type, message, ...) \
02484     DOCTEST_FUNC_SCOPE_BEGIN { \
02485         if(!doctest::getContextOptions()->no_throw) { \
02486             doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__, \
02487                 __LINE__, #expr, #__VA_ARGS__, message); \
02488             try { \
02489                 DOCTEST_CAST_TO_VOID(expr) \
02490             } catch(const typename doctest::detail::types::remove_const< \
02491                 typename doctest::detail::types::remove_reference<__VA_ARGS__>::type>::type&) { \
02492                 DOCTEST_RB.translateException(); \
02493                 DOCTEST_RB.m_threw_as = true; \
02494             } catch(...) { DOCTEST_RB.translateException(); } \
02495             DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB); \
02496         } else { /* NOLINT(*-else-after-return) */ \
02497             DOCTEST_FUNC_SCOPE_RET(false); \
02498         } \
02499     }

```

```

02499     } DOCTEST_FUNC_SCOPE_END
02500
02501 #define DOCTEST_ASSERT_THROWS_WITH(expr, expr_str, assert_type, ...)
02502     DOCTEST_FUNC_SCOPE_BEGIN {
02503         if(!doctest::getContextOptions()->no_throw) {
02504             doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__,
02505                 __LINE__, expr_str, "", __VA_ARGS__);
02506             try {
02507                 DOCTEST_CAST_TO_VOID(expr)
02508             } catch(...) { DOCTEST_RB.translateException(); }
02509             DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB);
02510         } else { /* NOLINT(*-else-after-return) */
02511             DOCTEST_FUNC_SCOPE_RET(false);
02512         }
02513     } DOCTEST_FUNC_SCOPE_END
02514
02515 #define DOCTEST_ASSERT_NOTHROW(assert_type, ...)
02516     DOCTEST_FUNC_SCOPE_BEGIN {
02517         doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__,
02518             __LINE__, #__VA_ARGS__);
02519         try {
02520             DOCTEST_CAST_TO_VOID(__VA_ARGS__)
02521         } catch(...) { DOCTEST_RB.translateException(); }
02522         DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB);
02523     } DOCTEST_FUNC_SCOPE_END
02524
02525 // clang-format off
02526 #define DOCTEST_WARN_THROWS(...) DOCTEST_ASSERT_THROWS_WITH((__VA_ARGS__), #__VA_ARGS__,
02527     DT_WARN_THROWS, "")
02528 #define DOCTEST_CHECK_THROWS(...) DOCTEST_ASSERT_THROWS_WITH((__VA_ARGS__), #__VA_ARGS__,
02529     DT_CHECK_THROWS, "")
02530 #define DOCTEST_REQUIRE_THROWS(...) DOCTEST_ASSERT_THROWS_WITH((__VA_ARGS__), #__VA_ARGS__,
02531     DT_REQUIRE_THROWS, "")
02532
02533 #define DOCTEST_WARN_THROWS_AS(expr, ...) DOCTEST_ASSERT_THROWS_AS(expr, DT_WARN_THROWS_AS, "",
02534     __VA_ARGS__)
02535 #define DOCTEST_CHECK_THROWS_AS(expr, ...) DOCTEST_ASSERT_THROWS_AS(expr, DT_CHECK_THROWS_AS, "",
02536     __VA_ARGS__)
02537 #define DOCTEST_REQUIRE_THROWS_AS(expr, ...) DOCTEST_ASSERT_THROWS_AS(expr, DT_REQUIRE_THROWS_AS, "",
02538     __VA_ARGS__)
02539
02540 #define DOCTEST_WARN_THROWS_WITH(expr, ...) DOCTEST_ASSERT_THROWS_WITH(expr, #expr,
02541     DT_WARN_THROWS_WITH, __VA_ARGS__)
02542 #define DOCTEST_CHECK_THROWS_WITH(expr, ...) DOCTEST_ASSERT_THROWS_WITH(expr, #expr,
02543     DT_CHECK_THROWS_WITH, __VA_ARGS__)
02544 #define DOCTEST_REQUIRE_THROWS_WITH(expr, ...) DOCTEST_ASSERT_THROWS_WITH(expr, #expr,
02545     DT_REQUIRE_THROWS_WITH, __VA_ARGS__)
02546
02547 #define DOCTEST_WARN_THROWS_WITH_AS(expr, message, ...) DOCTEST_ASSERT_THROWS_AS(expr,
02548     DT_WARN_THROWS_WITH_AS, message, __VA_ARGS__)
02549 #define DOCTEST_CHECK_THROWS_WITH_AS(expr, message, ...) DOCTEST_ASSERT_THROWS_AS(expr,
02550     DT_CHECK_THROWS_WITH_AS, message, __VA_ARGS__)
02551 #define DOCTEST_REQUIRE_THROWS_WITH_AS(expr, message, ...) DOCTEST_ASSERT_THROWS_AS(expr,
02552     DT_REQUIRE_THROWS_WITH_AS, message, __VA_ARGS__)
02553
02554 #define DOCTEST_WARN_NOTHROW(...) DOCTEST_ASSERT_NOTHROW(DT_WARN_NOTHROW, __VA_ARGS__)
02555 #define DOCTEST_CHECK_NOTHROW(...) DOCTEST_ASSERT_NOTHROW(DT_CHECK_NOTHROW, __VA_ARGS__)
02556 #define DOCTEST_REQUIRE_NOTHROW(...) DOCTEST_ASSERT_NOTHROW(DT_REQUIRE_NOTHROW, __VA_ARGS__)
02557
02558 #define DOCTEST_WARN_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
02559     DOCTEST_WARN_THROWS(expr); } DOCTEST_FUNC_SCOPE_END
02560 #define DOCTEST_CHECK_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
02561     DOCTEST_CHECK_THROWS(expr); } DOCTEST_FUNC_SCOPE_END
02562 #define DOCTEST_REQUIRE_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN {
02563     DOCTEST_INFO(__VA_ARGS__); DOCTEST_REQUIRE_THROWS(expr); } DOCTEST_FUNC_SCOPE_END
02564 #define DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
02565     DOCTEST_INFO(__VA_ARGS__); DOCTEST_WARN_THROWS_AS(expr, ex); } DOCTEST_FUNC_SCOPE_END
02566 #define DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
02567     DOCTEST_INFO(__VA_ARGS__); DOCTEST_CHECK_THROWS_AS(expr, ex); } DOCTEST_FUNC_SCOPE_END
02568 #define DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
02569     DOCTEST_INFO(__VA_ARGS__); DOCTEST_REQUIRE_THROWS_AS(expr, ex); } DOCTEST_FUNC_SCOPE_END
02570 #define DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_SCOPE_BEGIN {
02571     DOCTEST_INFO(__VA_ARGS__); DOCTEST_WARN_THROWS_WITH(expr, with); } DOCTEST_FUNC_SCOPE_END
02572 #define DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_SCOPE_BEGIN {
02573     DOCTEST_INFO(__VA_ARGS__); DOCTEST_CHECK_THROWS_WITH(expr, with); } DOCTEST_FUNC_SCOPE_END
02574 #define DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_SCOPE_BEGIN {
02575     DOCTEST_INFO(__VA_ARGS__); DOCTEST_REQUIRE_THROWS_WITH(expr, with); } DOCTEST_FUNC_SCOPE_END
02576 #define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
02577     DOCTEST_INFO(__VA_ARGS__); DOCTEST_WARN_THROWS_WITH_AS(expr, with, ex); } DOCTEST_FUNC_SCOPE_END
02578 #define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
02579     DOCTEST_INFO(__VA_ARGS__); DOCTEST_CHECK_THROWS_WITH_AS(expr, with, ex); } DOCTEST_FUNC_SCOPE_END
02580 #define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
02581     DOCTEST_INFO(__VA_ARGS__); DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with, ex); } DOCTEST_FUNC_SCOPE_END
02582 #define DOCTEST_WARN_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
02583     DOCTEST_WARN_NOTHROW(expr); } DOCTEST_FUNC_SCOPE_END
02584 #define DOCTEST_CHECK_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
02585     DOCTEST_CHECK_NOTHROW(expr); } DOCTEST_FUNC_SCOPE_END

```

```

02560 #define DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN {
      DOCTEST_INFO(__VA_ARGS__); DOCTEST_REQUIRE_NOTHROW(expr); } DOCTEST_FUNC_SCOPE_END
02561 // clang-format on
02562
02563 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
02564
02565 // =====
02566 // == WHAT FOLLOWS IS VERSIONS OF THE MACROS THAT DO NOT DO ANY REGISTERING! ==
02567 // == THIS CAN BE ENABLED BY DEFINING DOCTEST_CONFIG_DISABLE GLOBALLY! ==
02568 // =====
02569 #else // DOCTEST_CONFIG_DISABLE
02570
02571 #define DOCTEST_IMPLEMENT_FIXTURE(der, base, func, name) \
02572     namespace /* NOLINT */ { \
02573         template <typename DOCTEST_UNUSED_TEMPLATE_TYPE> \
02574             struct der : public base \
02575             { void f(); }; \
02576     } \
02577     template <typename DOCTEST_UNUSED_TEMPLATE_TYPE> \
02578     inline void der<DOCTEST_UNUSED_TEMPLATE_TYPE>::f() \
02579
02580 #define DOCTEST_CREATE_AND_REGISTER_FUNCTION(f, name) \
02581     template <typename DOCTEST_UNUSED_TEMPLATE_TYPE> \
02582     static inline void f() \
02583
02584 // for registering tests
02585 #define DOCTEST_TEST_CASE(name) \
02586     DOCTEST_CREATE_AND_REGISTER_FUNCTION(DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), name) \
02587
02588 // for registering tests in classes
02589 #define DOCTEST_TEST_CASE_CLASS(name) \
02590     DOCTEST_CREATE_AND_REGISTER_FUNCTION(DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), name) \
02591
02592 // for registering tests with a fixture
02593 #define DOCTEST_TEST_CASE_FIXTURE(x, name) \
02594     DOCTEST_IMPLEMENT_FIXTURE(DOCTEST_ANONYMOUS(DOCTEST_ANON_CLASS_), x, \
02595                               DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), name) \
02596
02597 // for converting types to strings without the <typeinfo> header and demangling
02598 #define DOCTEST_TYPE_TO_STRING_AS(str, ...) static_assert(true, "")
02599 #define DOCTEST_TYPE_TO_STRING(...) static_assert(true, "")
02600
02601 // for typed tests
02602 #define DOCTEST_TEST_CASE_TEMPLATE(name, type, ...) \
02603     template <typename type> \
02604     inline void DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_>() \
02605
02606 #define DOCTEST_TEST_CASE_TEMPLATE_DEFINE(name, type, id) \
02607     template <typename type> \
02608     inline void DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_>() \
02609
02610 #define DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id, ...) static_assert(true, "")
02611 #define DOCTEST_TEST_CASE_TEMPLATE_APPLY(id, ...) static_assert(true, "")
02612
02613 // for subcases
02614 #define DOCTEST_SUBCASE(name)
02615
02616 // for a testsuite block
02617 #define DOCTEST_TEST_SUITE(name) namespace // NOLINT
02618
02619 // for starting a testsuite block
02620 #define DOCTEST_TEST_SUITE_BEGIN(name) static_assert(true, "")
02621
02622 // for ending a testsuite block
02623 #define DOCTEST_TEST_SUITE_END using DOCTEST_ANONYMOUS(DOCTEST_ANON_FOR_SEMICOLON_) = int
02624
02625 #define DOCTEST_REGISTER_EXCEPTION_TRANSLATOR(signature) \
02626     template <typename DOCTEST_UNUSED_TEMPLATE_TYPE> \
02627     static inline doctest::String DOCTEST_ANONYMOUS(DOCTEST_ANON_TRANSLATOR_)(signature) \
02628
02629 #define DOCTEST_REGISTER_REPORTER(name, priority, reporter)
02630 #define DOCTEST_REGISTER_LISTENER(name, priority, reporter)
02631
02632 #define DOCTEST_INFO(...) (static_cast<void>(0))
02633 #define DOCTEST_CAPTURE(x) (static_cast<void>(0))
02634 #define DOCTEST_ADD_MESSAGE_AT(file, line, ...) (static_cast<void>(0))
02635 #define DOCTEST_ADD_FAIL_CHECK_AT(file, line, ...) (static_cast<void>(0))
02636 #define DOCTEST_ADD_FAIL_AT(file, line, ...) (static_cast<void>(0))
02637 #define DOCTEST_MESSAGE(...) (static_cast<void>(0))
02638 #define DOCTEST_FAIL_CHECK(...) (static_cast<void>(0))
02639 #define DOCTEST_FAIL(...) (static_cast<void>(0))
02640
02641 #if defined(DOCTEST_CONFIG_EVALUATE_ASSERTS_EVEN_WHEN_DISABLED) \
02642     && defined(DOCTEST_CONFIG_ASSERTS_RETURN_VALUES) \
02643
02644 #define DOCTEST_WARN(...) [&] { return __VA_ARGS__; }()
02645 #define DOCTEST_CHECK(...) [&] { return __VA_ARGS__; }()

```



```

02646 #define DOCTEST_REQUIRE(...) [&] { return __VA_ARGS__; }()
02647 #define DOCTEST_WARN_FALSE(...) [&] { return !(__VA_ARGS__); }()
02648 #define DOCTEST_CHECK_FALSE(...) [&] { return !(__VA_ARGS__); }()
02649 #define DOCTEST_REQUIRE_FALSE(...) [&] { return !(__VA_ARGS__); }()
02650
02651 #define DOCTEST_WARN_MESSAGE(cond, ...) [&] { return cond; }()
02652 #define DOCTEST_CHECK_MESSAGE(cond, ...) [&] { return cond; }()
02653 #define DOCTEST_REQUIRE_MESSAGE(cond, ...) [&] { return cond; }()
02654 #define DOCTEST_WARN_FALSE_MESSAGE(cond, ...) [&] { return !(cond); }()
02655 #define DOCTEST_CHECK_FALSE_MESSAGE(cond, ...) [&] { return !(cond); }()
02656 #define DOCTEST_REQUIRE_FALSE_MESSAGE(cond, ...) [&] { return !(cond); }()
02657
02658 namespace doctest {
02659 namespace detail {
02660 #define DOCTEST_RELATIONAL_OP(name, op)
02661     template <typename L, typename R>
02662     bool name(const DOCTEST_REF_WRAP(L) lhs, const DOCTEST_REF_WRAP(R) rhs) { return lhs op rhs; }
02663
02664     DOCTEST_RELATIONAL_OP(eq, ==)
02665     DOCTEST_RELATIONAL_OP(ne, !=)
02666     DOCTEST_RELATIONAL_OP(lt, <)
02667     DOCTEST_RELATIONAL_OP(gt, >)
02668     DOCTEST_RELATIONAL_OP(le, <=)
02669     DOCTEST_RELATIONAL_OP(ge, >=)
02670 } // namespace detail
02671 } // namespace doctest
02672
02673 #define DOCTEST_WARN_EQ(...) [&] { return doctest::detail::eq(__VA_ARGS__); }()
02674 #define DOCTEST_CHECK_EQ(...) [&] { return doctest::detail::eq(__VA_ARGS__); }()
02675 #define DOCTEST_REQUIRE_EQ(...) [&] { return doctest::detail::eq(__VA_ARGS__); }()
02676 #define DOCTEST_WARN_NE(...) [&] { return doctest::detail::ne(__VA_ARGS__); }()
02677 #define DOCTEST_CHECK_NE(...) [&] { return doctest::detail::ne(__VA_ARGS__); }()
02678 #define DOCTEST_REQUIRE_NE(...) [&] { return doctest::detail::ne(__VA_ARGS__); }()
02679 #define DOCTEST_WARN_LT(...) [&] { return doctest::detail::lt(__VA_ARGS__); }()
02680 #define DOCTEST_CHECK_LT(...) [&] { return doctest::detail::lt(__VA_ARGS__); }()
02681 #define DOCTEST_REQUIRE_LT(...) [&] { return doctest::detail::lt(__VA_ARGS__); }()
02682 #define DOCTEST_WARN_GT(...) [&] { return doctest::detail::gt(__VA_ARGS__); }()
02683 #define DOCTEST_CHECK_GT(...) [&] { return doctest::detail::gt(__VA_ARGS__); }()
02684 #define DOCTEST_REQUIRE_GT(...) [&] { return doctest::detail::gt(__VA_ARGS__); }()
02685 #define DOCTEST_WARN_LE(...) [&] { return doctest::detail::le(__VA_ARGS__); }()
02686 #define DOCTEST_CHECK_LE(...) [&] { return doctest::detail::le(__VA_ARGS__); }()
02687 #define DOCTEST_REQUIRE_LE(...) [&] { return doctest::detail::le(__VA_ARGS__); }()
02688 #define DOCTEST_WARN_GE(...) [&] { return doctest::detail::ge(__VA_ARGS__); }()
02689 #define DOCTEST_CHECK_GE(...) [&] { return doctest::detail::ge(__VA_ARGS__); }()
02690 #define DOCTEST_REQUIRE_GE(...) [&] { return doctest::detail::ge(__VA_ARGS__); }()
02691 #define DOCTEST_WARN_UNARY(...) [&] { return __VA_ARGS__; }()
02692 #define DOCTEST_CHECK_UNARY(...) [&] { return __VA_ARGS__; }()
02693 #define DOCTEST_REQUIRE_UNARY(...) [&] { return __VA_ARGS__; }()
02694 #define DOCTEST_WARN_UNARY_FALSE(...) [&] { return !(__VA_ARGS__); }()
02695 #define DOCTEST_CHECK_UNARY_FALSE(...) [&] { return !(__VA_ARGS__); }()
02696 #define DOCTEST_REQUIRE_UNARY_FALSE(...) [&] { return !(__VA_ARGS__); }()
02697
02698 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
02699
02700 #define DOCTEST_WARN_THROWS_WITH(expr, with, ...) [] { static_assert(false, "Exception translation is not available when doctest is disabled."); return false; }()
02701 #define DOCTEST_CHECK_THROWS_WITH(expr, with, ...) DOCTEST_WARN_THROWS_WITH(„)
02702 #define DOCTEST_REQUIRE_THROWS_WITH(expr, with, ...) DOCTEST_WARN_THROWS_WITH(„)
02703 #define DOCTEST_WARN_THROWS_WITH_AS(expr, with, ex, ...) DOCTEST_WARN_THROWS_WITH(„)
02704 #define DOCTEST_CHECK_THROWS_WITH_AS(expr, with, ex, ...) DOCTEST_WARN_THROWS_WITH(„)
02705 #define DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with, ex, ...) DOCTEST_WARN_THROWS_WITH(„)
02706
02707 #define DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_WARN_THROWS_WITH(„)
02708 #define DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_WARN_THROWS_WITH(„)
02709 #define DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_WARN_THROWS_WITH(„)
02710 #define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_WARN_THROWS_WITH(„)
02711 #define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_WARN_THROWS_WITH(„)
02712 #define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_WARN_THROWS_WITH(„)
02713
02714 #define DOCTEST_WARN_THROWS(...) [&] { try { __VA_ARGS__; return false; } catch (...) { return true; } }()
02715 #define DOCTEST_CHECK_THROWS(...) [&] { try { __VA_ARGS__; return false; } catch (...) { return true; } }()
02716 #define DOCTEST_REQUIRE_THROWS(...) [&] { try { __VA_ARGS__; return false; } catch (...) { return true; } }()
02717 #define DOCTEST_WARN_THROWS_AS(expr, ...) [&] { try { expr; } catch (__VA_ARGS__) { return true; } catch (...) { } return false; }()
02718 #define DOCTEST_CHECK_THROWS_AS(expr, ...) [&] { try { expr; } catch (__VA_ARGS__) { return true; } catch (...) { } return false; }()
02719 #define DOCTEST_REQUIRE_THROWS_AS(expr, ...) [&] { try { expr; } catch (__VA_ARGS__) { return true; } catch (...) { } return false; }()
02720 #define DOCTEST_WARN_NOTHROW(...) [&] { try { __VA_ARGS__; return true; } catch (...) { return false; } }()
02721 #define DOCTEST_CHECK_NOTHROW(...) [&] { try { __VA_ARGS__; return true; } catch (...) { return false; } }()
02722 #define DOCTEST_REQUIRE_NOTHROW(...) [&] { try { __VA_ARGS__; return true; } catch (...) { return false; } }()

```

```

02723
02724 #define DOCTEST_WARN_THROWS_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return false; } catch (...) {
02725     return true; } }()
02726 #define DOCTEST_CHECK_THROWS_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return false; } catch (...) {
02727     return true; } }()
02728 #define DOCTEST_REQUIRE_THROWS_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return false; } catch (...)
02729     { return true; } }()
02730 #define DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, ...) [&] { try { expr; } catch (__VA_ARGS__) { return
02731     true; } catch (...) { } return false; }()
02732 #define DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, ...) [&] { try { expr; } catch (__VA_ARGS__) {
02733     return true; } catch (...) { } return false; }()
02734 #define DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...) [&] { try { expr; } catch (__VA_ARGS__) {
02735     return true; } catch (...) { } return false; }()
02736 #define DOCTEST_WARN_NOTHROW_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return true; } catch (...) {
02737     return false; } }()
02738 #define DOCTEST_CHECK_NOTHROW_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return true; } catch (...) {
02739     return false; } }()
02740 #define DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return true; } catch (...)
02741     { return false; } }()
02742
02743 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
02744
02745 #else // DOCTEST_CONFIG_EVALUATE_ASSERTS_EVEN_WHEN_DISABLED
02746
02747 #define DOCTEST_WARN(...) DOCTEST_FUNC_EMPTY
02748 #define DOCTEST_CHECK(...) DOCTEST_FUNC_EMPTY
02749 #define DOCTEST_REQUIRE(...) DOCTEST_FUNC_EMPTY
02750 #define DOCTEST_WARN_FALSE(...) DOCTEST_FUNC_EMPTY
02751 #define DOCTEST_CHECK_FALSE(...) DOCTEST_FUNC_EMPTY
02752 #define DOCTEST_REQUIRE_FALSE(...) DOCTEST_FUNC_EMPTY
02753
02754 #define DOCTEST_WARN_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
02755 #define DOCTEST_CHECK_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
02756 #define DOCTEST_REQUIRE_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
02757 #define DOCTEST_WARN_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
02758 #define DOCTEST_CHECK_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
02759 #define DOCTEST_REQUIRE_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
02760
02761 #define DOCTEST_WARN_EQ(...) DOCTEST_FUNC_EMPTY
02762 #define DOCTEST_CHECK_EQ(...) DOCTEST_FUNC_EMPTY
02763 #define DOCTEST_REQUIRE_EQ(...) DOCTEST_FUNC_EMPTY
02764 #define DOCTEST_WARN_NE(...) DOCTEST_FUNC_EMPTY
02765 #define DOCTEST_CHECK_NE(...) DOCTEST_FUNC_EMPTY
02766 #define DOCTEST_REQUIRE_NE(...) DOCTEST_FUNC_EMPTY
02767 #define DOCTEST_WARN_GT(...) DOCTEST_FUNC_EMPTY
02768 #define DOCTEST_CHECK_GT(...) DOCTEST_FUNC_EMPTY
02769 #define DOCTEST_REQUIRE_GT(...) DOCTEST_FUNC_EMPTY
02770 #define DOCTEST_WARN_LT(...) DOCTEST_FUNC_EMPTY
02771 #define DOCTEST_CHECK_LT(...) DOCTEST_FUNC_EMPTY
02772 #define DOCTEST_REQUIRE_LT(...) DOCTEST_FUNC_EMPTY
02773 #define DOCTEST_WARN_GE(...) DOCTEST_FUNC_EMPTY
02774 #define DOCTEST_CHECK_GE(...) DOCTEST_FUNC_EMPTY
02775 #define DOCTEST_REQUIRE_GE(...) DOCTEST_FUNC_EMPTY
02776 #define DOCTEST_WARN_LE(...) DOCTEST_FUNC_EMPTY
02777 #define DOCTEST_CHECK_LE(...) DOCTEST_FUNC_EMPTY
02778 #define DOCTEST_REQUIRE_LE(...) DOCTEST_FUNC_EMPTY
02779
02780 #define DOCTEST_WARN_UNARY(...) DOCTEST_FUNC_EMPTY
02781 #define DOCTEST_CHECK_UNARY(...) DOCTEST_FUNC_EMPTY
02782 #define DOCTEST_REQUIRE_UNARY(...) DOCTEST_FUNC_EMPTY
02783 #define DOCTEST_WARN_UNARY_FALSE(...) DOCTEST_FUNC_EMPTY
02784 #define DOCTEST_CHECK_UNARY_FALSE(...) DOCTEST_FUNC_EMPTY
02785 #define DOCTEST_REQUIRE_UNARY_FALSE(...) DOCTEST_FUNC_EMPTY
02786
02787 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
02788
02789 #define DOCTEST_WARN_THROWS(...) DOCTEST_FUNC_EMPTY
02790 #define DOCTEST_CHECK_THROWS(...) DOCTEST_FUNC_EMPTY
02791 #define DOCTEST_REQUIRE_THROWS(...) DOCTEST_FUNC_EMPTY
02792 #define DOCTEST_WARN_THROWS_AS(expr, ...) DOCTEST_FUNC_EMPTY
02793 #define DOCTEST_CHECK_THROWS_AS(expr, ...) DOCTEST_FUNC_EMPTY
02794 #define DOCTEST_REQUIRE_THROWS_AS(expr, ...) DOCTEST_FUNC_EMPTY
02795 #define DOCTEST_WARN_THROWS_WITH(expr, ...) DOCTEST_FUNC_EMPTY
02796 #define DOCTEST_CHECK_THROWS_WITH(expr, ...) DOCTEST_FUNC_EMPTY
02797 #define DOCTEST_REQUIRE_THROWS_WITH(expr, ...) DOCTEST_FUNC_EMPTY
02798 #define DOCTEST_WARN_THROWS_WITH_AS(expr, with, ...) DOCTEST_FUNC_EMPTY
02799 #define DOCTEST_CHECK_THROWS_WITH_AS(expr, with, ...) DOCTEST_FUNC_EMPTY
02800 #define DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with, ...) DOCTEST_FUNC_EMPTY
02801 #define DOCTEST_WARN_NOTHROW(...) DOCTEST_FUNC_EMPTY
02802 #define DOCTEST_CHECK_NOTHROW(...) DOCTEST_FUNC_EMPTY
02803 #define DOCTEST_REQUIRE_NOTHROW(...) DOCTEST_FUNC_EMPTY
02804
02805 #define DOCTEST_WARN_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
02806 #define DOCTEST_CHECK_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
02807 #define DOCTEST_REQUIRE_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
02808 #define DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_EMPTY
02809 #define DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_EMPTY

```



```

02801 #define DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_EMPTY
02802 #define DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_EMPTY
02803 #define DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_EMPTY
02804 #define DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_EMPTY
02805 #define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_EMPTY
02806 #define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_EMPTY
02807 #define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_EMPTY
02808 #define DOCTEST_WARN_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
02809 #define DOCTEST_CHECK_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
02810 #define DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
02811
02812 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
02813
02814 #endif // DOCTEST_CONFIG_EVALUATE_ASSERTS_EVEN_WHEN_DISABLED
02815
02816 #endif // DOCTEST_CONFIG_DISABLE
02817
02818 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
02819
02820 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS
02821 #define DOCTEST_EXCEPTION_EMPTY_FUNC DOCTEST_FUNC_EMPTY
02822 #else // DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS
02823 #define DOCTEST_EXCEPTION_EMPTY_FUNC [] { static_assert(false, "Exceptions are disabled! " \
02824     "Use DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS if you want to compile with exceptions
    disabled."); return false; }()
02825
02826 #undef DOCTEST_REQUIRE
02827 #undef DOCTEST_REQUIRE_FALSE
02828 #undef DOCTEST_REQUIRE_MESSAGE
02829 #undef DOCTEST_REQUIRE_FALSE_MESSAGE
02830 #undef DOCTEST_REQUIRE_EQ
02831 #undef DOCTEST_REQUIRE_NE
02832 #undef DOCTEST_REQUIRE_GT
02833 #undef DOCTEST_REQUIRE_LT
02834 #undef DOCTEST_REQUIRE_GE
02835 #undef DOCTEST_REQUIRE_LE
02836 #undef DOCTEST_REQUIRE_UNARY
02837 #undef DOCTEST_REQUIRE_UNARY_FALSE
02838
02839 #define DOCTEST_REQUIRE DOCTEST_EXCEPTION_EMPTY_FUNC
02840 #define DOCTEST_REQUIRE_FALSE DOCTEST_EXCEPTION_EMPTY_FUNC
02841 #define DOCTEST_REQUIRE_MESSAGE DOCTEST_EXCEPTION_EMPTY_FUNC
02842 #define DOCTEST_REQUIRE_FALSE_MESSAGE DOCTEST_EXCEPTION_EMPTY_FUNC
02843 #define DOCTEST_REQUIRE_EQ DOCTEST_EXCEPTION_EMPTY_FUNC
02844 #define DOCTEST_REQUIRE_NE DOCTEST_EXCEPTION_EMPTY_FUNC
02845 #define DOCTEST_REQUIRE_GT DOCTEST_EXCEPTION_EMPTY_FUNC
02846 #define DOCTEST_REQUIRE_LT DOCTEST_EXCEPTION_EMPTY_FUNC
02847 #define DOCTEST_REQUIRE_GE DOCTEST_EXCEPTION_EMPTY_FUNC
02848 #define DOCTEST_REQUIRE_LE DOCTEST_EXCEPTION_EMPTY_FUNC
02849 #define DOCTEST_REQUIRE_UNARY DOCTEST_EXCEPTION_EMPTY_FUNC
02850 #define DOCTEST_REQUIRE_UNARY_FALSE DOCTEST_EXCEPTION_EMPTY_FUNC
02851
02852 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS
02853
02854 #define DOCTEST_WARN_THROWS(...) DOCTEST_EXCEPTION_EMPTY_FUNC
02855 #define DOCTEST_CHECK_THROWS(...) DOCTEST_EXCEPTION_EMPTY_FUNC
02856 #define DOCTEST_REQUIRE_THROWS(...) DOCTEST_EXCEPTION_EMPTY_FUNC
02857 #define DOCTEST_WARN_THROWS_AS(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02858 #define DOCTEST_CHECK_THROWS_AS(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02859 #define DOCTEST_REQUIRE_THROWS_AS(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02860 #define DOCTEST_WARN_THROWS_WITH(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02861 #define DOCTEST_CHECK_THROWS_WITH(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02862 #define DOCTEST_REQUIRE_THROWS_WITH(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02863 #define DOCTEST_WARN_THROWS_WITH_AS(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02864 #define DOCTEST_CHECK_THROWS_WITH_AS(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02865 #define DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02866 #define DOCTEST_WARN_NOTHROW(...) DOCTEST_EXCEPTION_EMPTY_FUNC
02867 #define DOCTEST_CHECK_NOTHROW(...) DOCTEST_EXCEPTION_EMPTY_FUNC
02868 #define DOCTEST_REQUIRE_NOTHROW(...) DOCTEST_EXCEPTION_EMPTY_FUNC
02869
02870 #define DOCTEST_WARN_THROWS_MESSAGE(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02871 #define DOCTEST_CHECK_THROWS_MESSAGE(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02872 #define DOCTEST_REQUIRE_THROWS_MESSAGE(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02873 #define DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02874 #define DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02875 #define DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02876 #define DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02877 #define DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02878 #define DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02879 #define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02880 #define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02881 #define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02882 #define DOCTEST_WARN_NOTHROW_MESSAGE(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02883 #define DOCTEST_CHECK_NOTHROW_MESSAGE(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02884 #define DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
02885
02886 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS

```

```

02887
02888 // clang-format off
02889 // KEPT FOR BACKWARDS COMPATIBILITY - FORWARDING TO THE RIGHT MACROS
02890 #define DOCTEST_FAST_WARN_EQ DOCTEST_WARN_EQ
02891 #define DOCTEST_FAST_CHECK_EQ DOCTEST_CHECK_EQ
02892 #define DOCTEST_FAST_REQUIRE_EQ DOCTEST_REQUIRE_EQ
02893 #define DOCTEST_FAST_WARN_NE DOCTEST_WARN_NE
02894 #define DOCTEST_FAST_CHECK_NE DOCTEST_CHECK_NE
02895 #define DOCTEST_FAST_REQUIRE_NE DOCTEST_REQUIRE_NE
02896 #define DOCTEST_FAST_WARN_GT DOCTEST_WARN_GT
02897 #define DOCTEST_FAST_CHECK_GT DOCTEST_CHECK_GT
02898 #define DOCTEST_FAST_REQUIRE_GT DOCTEST_REQUIRE_GT
02899 #define DOCTEST_FAST_WARN_LT DOCTEST_WARN_LT
02900 #define DOCTEST_FAST_CHECK_LT DOCTEST_CHECK_LT
02901 #define DOCTEST_FAST_REQUIRE_LT DOCTEST_REQUIRE_LT
02902 #define DOCTEST_FAST_WARN_GE DOCTEST_WARN_GE
02903 #define DOCTEST_FAST_CHECK_GE DOCTEST_CHECK_GE
02904 #define DOCTEST_FAST_REQUIRE_GE DOCTEST_REQUIRE_GE
02905 #define DOCTEST_FAST_WARN_LE DOCTEST_WARN_LE
02906 #define DOCTEST_FAST_CHECK_LE DOCTEST_CHECK_LE
02907 #define DOCTEST_FAST_REQUIRE_LE DOCTEST_REQUIRE_LE
02908
02909 #define DOCTEST_FAST_WARN_UNARY DOCTEST_WARN_UNARY
02910 #define DOCTEST_FAST_CHECK_UNARY DOCTEST_CHECK_UNARY
02911 #define DOCTEST_FAST_REQUIRE_UNARY DOCTEST_REQUIRE_UNARY
02912 #define DOCTEST_FAST_WARN_UNARY_FALSE DOCTEST_WARN_UNARY_FALSE
02913 #define DOCTEST_FAST_CHECK_UNARY_FALSE DOCTEST_CHECK_UNARY_FALSE
02914 #define DOCTEST_FAST_REQUIRE_UNARY_FALSE DOCTEST_REQUIRE_UNARY_FALSE
02915
02916 #define DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE(id, ...)
DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id, __VA_ARGS__)
02917 // clang-format on
02918
02919 // BDD style macros
02920 // clang-format off
02921 #define DOCTEST_SCENARIO(name) DOCTEST_TEST_CASE(" Scenario: " name)
02922 #define DOCTEST_SCENARIO_CLASS(name) DOCTEST_TEST_CASE_CLASS(" Scenario: " name)
02923 #define DOCTEST_SCENARIO_TEMPLATE(name, T, ...) DOCTEST_TEST_CASE_TEMPLATE(" Scenario: " name, T,
__VA_ARGS__)
02924 #define DOCTEST_SCENARIO_TEMPLATE_DEFINE(name, T, id) DOCTEST_TEST_CASE_TEMPLATE_DEFINE(" Scenario: "
name, T, id)
02925
02926 #define DOCTEST_GIVEN(name) DOCTEST_SUBCASE(" Given: " name)
02927 #define DOCTEST_WHEN(name) DOCTEST_SUBCASE(" When: " name)
02928 #define DOCTEST_AND_WHEN(name) DOCTEST_SUBCASE("And when: " name)
02929 #define DOCTEST_THEN(name) DOCTEST_SUBCASE(" Then: " name)
02930 #define DOCTEST_AND_THEN(name) DOCTEST_SUBCASE(" And: " name)
02931 // clang-format on
02932
02933 // == SHORT VERSIONS OF THE MACROS
02934 #ifndef DOCTEST_CONFIG_NO_SHORT_MACRO_NAMES
02935
02936 #define TEST_CASE(name) DOCTEST_TEST_CASE(name)
02937 #define TEST_CASE_CLASS(name) DOCTEST_TEST_CASE_CLASS(name)
02938 #define TEST_CASE_FIXTURE(x, name) DOCTEST_TEST_CASE_FIXTURE(x, name)
02939 #define TYPE_TO_STRING_AS(str, ...) DOCTEST_TYPE_TO_STRING_AS(str, __VA_ARGS__)
02940 #define TYPE_TO_STRING(...) DOCTEST_TYPE_TO_STRING(__VA_ARGS__)
02941 #define TEST_CASE_TEMPLATE(name, T, ...) DOCTEST_TEST_CASE_TEMPLATE(name, T, __VA_ARGS__)
02942 #define TEST_CASE_TEMPLATE_DEFINE(name, T, id) DOCTEST_TEST_CASE_TEMPLATE_DEFINE(name, T, id)
02943 #define TEST_CASE_TEMPLATE_INVOKE(id, ...) DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id, __VA_ARGS__)
02944 #define TEST_CASE_TEMPLATE_APPLY(id, ...) DOCTEST_TEST_CASE_TEMPLATE_APPLY(id, __VA_ARGS__)
02945 #define SUBCASE(name) DOCTEST_SUBCASE(name)
02946 #define TEST_SUITE(decorators) DOCTEST_TEST_SUITE(decorators)
02947 #define TEST_SUITE_BEGIN(name) DOCTEST_TEST_SUITE_BEGIN(name)
02948 #define TEST_SUITE_END DOCTEST_TEST_SUITE_END
02949 #define REGISTER_EXCEPTION_TRANSLATOR(signature) DOCTEST_REGISTER_EXCEPTION_TRANSLATOR(signature)
02950 #define REGISTER_REPORTER(name, priority, reporter) DOCTEST_REGISTER_REPORTER(name, priority,
reporter)
02951 #define REGISTER_LISTENER(name, priority, reporter) DOCTEST_REGISTER_LISTENER(name, priority,
reporter)
02952 #define INFO(...) DOCTEST_INFO(__VA_ARGS__)
02953 #define CAPTURE(x) DOCTEST_CAPTURE(x)
02954 #define ADD_MESSAGE_AT(file, line, ...) DOCTEST_ADD_MESSAGE_AT(file, line, __VA_ARGS__)
02955 #define ADD_FAIL_CHECK_AT(file, line, ...) DOCTEST_ADD_FAIL_CHECK_AT(file, line, __VA_ARGS__)
02956 #define ADD_FAIL_AT(file, line, ...) DOCTEST_ADD_FAIL_AT(file, line, __VA_ARGS__)
02957 #define MESSAGE(...) DOCTEST_MESSAGE(__VA_ARGS__)
02958 #define FAIL_CHECK(...) DOCTEST_FAIL_CHECK(__VA_ARGS__)
02959 #define FAIL(...) DOCTEST_FAIL(__VA_ARGS__)
02960 #define TO_LVALUE(...) DOCTEST_TO_LVALUE(__VA_ARGS__)
02961
02962 #define WARN(...) DOCTEST_WARN(__VA_ARGS__)
02963 #define WARN_FALSE(...) DOCTEST_WARN_FALSE(__VA_ARGS__)
02964 #define WARN_THROWS(...) DOCTEST_WARN_THROWS(__VA_ARGS__)
02965 #define WARN_THROWS_AS(expr, ...) DOCTEST_WARN_THROWS_AS(expr, __VA_ARGS__)
02966 #define WARN_THROWS_WITH(expr, ...) DOCTEST_WARN_THROWS_WITH(expr, __VA_ARGS__)
02967 #define WARN_THROWS_WITH_AS(expr, with, ...) DOCTEST_WARN_THROWS_WITH_AS(expr, with, __VA_ARGS__)
02968 #define WARN_NOTHROW(...) DOCTEST_WARN_NOTHROW(__VA_ARGS__)

```

```

02969 #define CHECK(...) DOCTEST_CHECK(__VA_ARGS__)
02970 #define CHECK_FALSE(...) DOCTEST_CHECK_FALSE(__VA_ARGS__)
02971 #define CHECK_THROWS(...) DOCTEST_CHECK_THROWS(__VA_ARGS__)
02972 #define CHECK_THROWS_AS(expr, ...) DOCTEST_CHECK_THROWS_AS(expr, __VA_ARGS__)
02973 #define CHECK_THROWS_WITH(expr, ...) DOCTEST_CHECK_THROWS_WITH(expr, __VA_ARGS__)
02974 #define CHECK_THROWS_WITH_AS(expr, with, ...) DOCTEST_CHECK_THROWS_WITH_AS(expr, with, __VA_ARGS__)
02975 #define CHECK_NO_THROW(...) DOCTEST_CHECK_NO_THROW(__VA_ARGS__)
02976 #define REQUIRE(...) DOCTEST_REQUIRE(__VA_ARGS__)
02977 #define REQUIRE_FALSE(...) DOCTEST_REQUIRE_FALSE(__VA_ARGS__)
02978 #define REQUIRE_THROWS(...) DOCTEST_REQUIRE_THROWS(__VA_ARGS__)
02979 #define REQUIRE_THROWS_AS(expr, ...) DOCTEST_REQUIRE_THROWS_AS(expr, __VA_ARGS__)
02980 #define REQUIRE_THROWS_WITH(expr, ...) DOCTEST_REQUIRE_THROWS_WITH(expr, __VA_ARGS__)
02981 #define REQUIRE_THROWS_WITH_AS(expr, with, ...) DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with,
__VA_ARGS__)
02982 #define REQUIRE_NO_THROW(...) DOCTEST_REQUIRE_NO_THROW(__VA_ARGS__)
02983
02984 #define WARN_MESSAGE(cond, ...) DOCTEST_WARN_MESSAGE(cond, __VA_ARGS__)
02985 #define WARN_FALSE_MESSAGE(cond, ...) DOCTEST_WARN_FALSE_MESSAGE(cond, __VA_ARGS__)
02986 #define WARN_THROWS_MESSAGE(expr, ...) DOCTEST_WARN_THROWS_MESSAGE(expr, __VA_ARGS__)
02987 #define WARN_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, __VA_ARGS__)
02988 #define WARN_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with,
__VA_ARGS__)
02989 #define WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr,
with, ex, __VA_ARGS__)
02990 #define WARN_NO_THROW_MESSAGE(expr, ...) DOCTEST_WARN_NO_THROW_MESSAGE(expr, __VA_ARGS__)
02991 #define CHECK_MESSAGE(cond, ...) DOCTEST_CHECK_MESSAGE(cond, __VA_ARGS__)
02992 #define CHECK_FALSE_MESSAGE(cond, ...) DOCTEST_CHECK_FALSE_MESSAGE(cond, __VA_ARGS__)
02993 #define CHECK_THROWS_MESSAGE(expr, ...) DOCTEST_CHECK_THROWS_MESSAGE(expr, __VA_ARGS__)
02994 #define CHECK_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, __VA_ARGS__)
02995 #define CHECK_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with,
__VA_ARGS__)
02996 #define CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr,
with, ex, __VA_ARGS__)
02997 #define CHECK_NO_THROW_MESSAGE(expr, ...) DOCTEST_CHECK_NO_THROW_MESSAGE(expr, __VA_ARGS__)
02998 #define REQUIRE_MESSAGE(cond, ...) DOCTEST_REQUIRE_MESSAGE(cond, __VA_ARGS__)
02999 #define REQUIRE_FALSE_MESSAGE(cond, ...) DOCTEST_REQUIRE_FALSE_MESSAGE(cond, __VA_ARGS__)
03000 #define REQUIRE_THROWS_MESSAGE(expr, ...) DOCTEST_REQUIRE_THROWS_MESSAGE(expr, __VA_ARGS__)
03001 #define REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex,
__VA_ARGS__)
03002 #define REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with,
__VA_ARGS__)
03003 #define REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...)
DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, __VA_ARGS__)
03004 #define REQUIRE_NO_THROW_MESSAGE(expr, ...) DOCTEST_REQUIRE_NO_THROW_MESSAGE(expr, __VA_ARGS__)
03005
03006 #define SCENARIO(name) DOCTEST_SCENARIO(name)
03007 #define SCENARIO_CLASS(name) DOCTEST_SCENARIO_CLASS(name)
03008 #define SCENARIO_TEMPLATE(name, T, ...) DOCTEST_SCENARIO_TEMPLATE(name, T, __VA_ARGS__)
03009 #define SCENARIO_TEMPLATE_DEFINE(name, T, id) DOCTEST_SCENARIO_TEMPLATE_DEFINE(name, T, id)
03010 #define GIVEN(name) DOCTEST_GIVEN(name)
03011 #define WHEN(name) DOCTEST_WHEN(name)
03012 #define AND_WHEN(name) DOCTEST_AND_WHEN(name)
03013 #define THEN(name) DOCTEST_THEN(name)
03014 #define AND_THEN(name) DOCTEST_AND_THEN(name)
03015
03016 #define WARN_EQ(...) DOCTEST_WARN_EQ(__VA_ARGS__)
03017 #define CHECK_EQ(...) DOCTEST_CHECK_EQ(__VA_ARGS__)
03018 #define REQUIRE_EQ(...) DOCTEST_REQUIRE_EQ(__VA_ARGS__)
03019 #define WARN_NE(...) DOCTEST_WARN_NE(__VA_ARGS__)
03020 #define CHECK_NE(...) DOCTEST_CHECK_NE(__VA_ARGS__)
03021 #define REQUIRE_NE(...) DOCTEST_REQUIRE_NE(__VA_ARGS__)
03022 #define WARN_GT(...) DOCTEST_WARN_GT(__VA_ARGS__)
03023 #define CHECK_GT(...) DOCTEST_CHECK_GT(__VA_ARGS__)
03024 #define REQUIRE_GT(...) DOCTEST_REQUIRE_GT(__VA_ARGS__)
03025 #define WARN_LT(...) DOCTEST_WARN_LT(__VA_ARGS__)
03026 #define CHECK_LT(...) DOCTEST_CHECK_LT(__VA_ARGS__)
03027 #define REQUIRE_LT(...) DOCTEST_REQUIRE_LT(__VA_ARGS__)
03028 #define WARN_GE(...) DOCTEST_WARN_GE(__VA_ARGS__)
03029 #define CHECK_GE(...) DOCTEST_CHECK_GE(__VA_ARGS__)
03030 #define REQUIRE_GE(...) DOCTEST_REQUIRE_GE(__VA_ARGS__)
03031 #define WARN_LE(...) DOCTEST_WARN_LE(__VA_ARGS__)
03032 #define CHECK_LE(...) DOCTEST_CHECK_LE(__VA_ARGS__)
03033 #define REQUIRE_LE(...) DOCTEST_REQUIRE_LE(__VA_ARGS__)
03034 #define WARN_UNARY(...) DOCTEST_WARN_UNARY(__VA_ARGS__)
03035 #define CHECK_UNARY(...) DOCTEST_CHECK_UNARY(__VA_ARGS__)
03036 #define REQUIRE_UNARY(...) DOCTEST_REQUIRE_UNARY(__VA_ARGS__)
03037 #define WARN_UNARY_FALSE(...) DOCTEST_WARN_UNARY_FALSE(__VA_ARGS__)
03038 #define CHECK_UNARY_FALSE(...) DOCTEST_CHECK_UNARY_FALSE(__VA_ARGS__)
03039 #define REQUIRE_UNARY_FALSE(...) DOCTEST_REQUIRE_UNARY_FALSE(__VA_ARGS__)
03040
03041 // KEPT FOR BACKWARDS COMPATIBILITY
03042 #define FAST_WARN_EQ(...) DOCTEST_FAST_WARN_EQ(__VA_ARGS__)
03043 #define FAST_CHECK_EQ(...) DOCTEST_FAST_CHECK_EQ(__VA_ARGS__)
03044 #define FAST_REQUIRE_EQ(...) DOCTEST_FAST_REQUIRE_EQ(__VA_ARGS__)
03045 #define FAST_WARN_NE(...) DOCTEST_FAST_WARN_NE(__VA_ARGS__)
03046 #define FAST_CHECK_NE(...) DOCTEST_FAST_CHECK_NE(__VA_ARGS__)
03047 #define FAST_REQUIRE_NE(...) DOCTEST_FAST_REQUIRE_NE(__VA_ARGS__)

```

```

03048 #define FAST_WARN_GT(...) DOCTEST_FAST_WARN_GT(__VA_ARGS__)
03049 #define FAST_CHECK_GT(...) DOCTEST_FAST_CHECK_GT(__VA_ARGS__)
03050 #define FAST_REQUIRE_GT(...) DOCTEST_FAST_REQUIRE_GT(__VA_ARGS__)
03051 #define FAST_WARN_LT(...) DOCTEST_FAST_WARN_LT(__VA_ARGS__)
03052 #define FAST_CHECK_LT(...) DOCTEST_FAST_CHECK_LT(__VA_ARGS__)
03053 #define FAST_REQUIRE_LT(...) DOCTEST_FAST_REQUIRE_LT(__VA_ARGS__)
03054 #define FAST_WARN_GE(...) DOCTEST_FAST_WARN_GE(__VA_ARGS__)
03055 #define FAST_CHECK_GE(...) DOCTEST_FAST_CHECK_GE(__VA_ARGS__)
03056 #define FAST_REQUIRE_GE(...) DOCTEST_FAST_REQUIRE_GE(__VA_ARGS__)
03057 #define FAST_WARN_LE(...) DOCTEST_FAST_WARN_LE(__VA_ARGS__)
03058 #define FAST_CHECK_LE(...) DOCTEST_FAST_CHECK_LE(__VA_ARGS__)
03059 #define FAST_REQUIRE_LE(...) DOCTEST_FAST_REQUIRE_LE(__VA_ARGS__)
03060
03061 #define FAST_WARN_UNARY(...) DOCTEST_FAST_WARN_UNARY(__VA_ARGS__)
03062 #define FAST_CHECK_UNARY(...) DOCTEST_FAST_CHECK_UNARY(__VA_ARGS__)
03063 #define FAST_REQUIRE_UNARY(...) DOCTEST_FAST_REQUIRE_UNARY(__VA_ARGS__)
03064 #define FAST_WARN_UNARY_FALSE(...) DOCTEST_FAST_WARN_UNARY_FALSE(__VA_ARGS__)
03065 #define FAST_CHECK_UNARY_FALSE(...) DOCTEST_FAST_CHECK_UNARY_FALSE(__VA_ARGS__)
03066 #define FAST_REQUIRE_UNARY_FALSE(...) DOCTEST_FAST_REQUIRE_UNARY_FALSE(__VA_ARGS__)
03067
03068 #define TEST_CASE_TEMPLATE_INSTANTIATE(id, ...) DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE(id,
__VA_ARGS__)
03069
03070 #endif // DOCTEST_CONFIG_NO_SHORT_MACRO_NAMES
03071
03072 #ifndef DOCTEST_CONFIG_DISABLE
03073
03074 // this is here to clear the 'current test suite' for the current translation unit - at the top
03075 DOCTEST_TEST_SUITE_END();
03076
03077 #endif // DOCTEST_CONFIG_DISABLE
03078
03079 DOCTEST_CLANG_SUPPRESS_WARNING_POP
03080 DOCTEST_MSVC_SUPPRESS_WARNING_POP
03081 DOCTEST_GCC_SUPPRESS_WARNING_POP
03082
03083 DOCTEST_SUPPRESS_COMMON_WARNINGS_POP
03084
03085 #endif // DOCTEST_LIBRARY_INCLUDED
03086
03087 #ifndef DOCTEST_SINGLE_HEADER
03088 #define DOCTEST_SINGLE_HEADER
03089 #endif // DOCTEST_SINGLE_HEADER
03090
03091 #if defined(DOCTEST_CONFIG_IMPLEMENT) || !defined(DOCTEST_SINGLE_HEADER)
03092
03093 #ifndef DOCTEST_SINGLE_HEADER
03094 #include "doctest_fwd.h"
03095 #endif // DOCTEST_SINGLE_HEADER
03096
03097 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wunused-macros")
03098
03099 #ifndef DOCTEST_LIBRARY_IMPLEMENTATION
03100 #define DOCTEST_LIBRARY_IMPLEMENTATION
03101
03102 DOCTEST_CLANG_SUPPRESS_WARNING_POP
03103
03104 DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH
03105
03106 DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
03107 DOCTEST_CLANG_SUPPRESS_WARNING("-Wglobal-constructors")
03108 DOCTEST_CLANG_SUPPRESS_WARNING("-Wexit-time-destructors")
03109 DOCTEST_CLANG_SUPPRESS_WARNING("-Wsign-conversion")
03110 DOCTEST_CLANG_SUPPRESS_WARNING("-Wshorten-64-to-32")
03111 DOCTEST_CLANG_SUPPRESS_WARNING("-Wmissing-variable-declarations")
03112 DOCTEST_CLANG_SUPPRESS_WARNING("-Wswitch")
03113 DOCTEST_CLANG_SUPPRESS_WARNING("-Wswitch-enum")
03114 DOCTEST_CLANG_SUPPRESS_WARNING("-Wcovered-switch-default")
03115 DOCTEST_CLANG_SUPPRESS_WARNING("-Wmissing-noreturn")
03116 DOCTEST_CLANG_SUPPRESS_WARNING("-Wdisabled-macro-expansion")
03117 DOCTEST_CLANG_SUPPRESS_WARNING("-Wmissing-braces")
03118 DOCTEST_CLANG_SUPPRESS_WARNING("-Wmissing-field-initializers")
03119 DOCTEST_CLANG_SUPPRESS_WARNING("-Wunused-member-function")
03120 DOCTEST_CLANG_SUPPRESS_WARNING("-Wnonportable-system-include-path")
03121
03122 DOCTEST_GCC_SUPPRESS_WARNING_PUSH
03123 DOCTEST_GCC_SUPPRESS_WARNING("-Wconversion")
03124 DOCTEST_GCC_SUPPRESS_WARNING("-Wsign-conversion")
03125 DOCTEST_GCC_SUPPRESS_WARNING("-Wmissing-field-initializers")
03126 DOCTEST_GCC_SUPPRESS_WARNING("-Wmissing-braces")
03127 DOCTEST_GCC_SUPPRESS_WARNING("-Wswitch")
03128 DOCTEST_GCC_SUPPRESS_WARNING("-Wswitch-enum")
03129 DOCTEST_GCC_SUPPRESS_WARNING("-Wswitch-default")
03130 DOCTEST_GCC_SUPPRESS_WARNING("-Wunsafe-loop-optimizations")
03131 DOCTEST_GCC_SUPPRESS_WARNING("-Wold-style-cast")
03132 DOCTEST_GCC_SUPPRESS_WARNING("-Wunused-function")
03133 DOCTEST_GCC_SUPPRESS_WARNING("-Wmultiple-inheritance")

```

```

03134 DOCTEST_GCC_SUPPRESS_WARNING("-Wsuggest-attribute")
03135
03136 DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
03137 DOCTEST_MSVC_SUPPRESS_WARNING(4267) // 'var' : conversion from 'x' to 'y', possible loss of data
03138 DOCTEST_MSVC_SUPPRESS_WARNING(4530) // C++ exception handler used, but unwind semantics not enabled
03139 DOCTEST_MSVC_SUPPRESS_WARNING(4577) // 'noexcept' used with no exception handling mode specified
03140 DOCTEST_MSVC_SUPPRESS_WARNING(4774) // format string expected in argument is not a string literal
03141 DOCTEST_MSVC_SUPPRESS_WARNING(4365) // conversion from 'int' to 'unsigned', signed/unsigned mismatch
03142 DOCTEST_MSVC_SUPPRESS_WARNING(5039) // pointer to potentially throwing function passed to extern C
03143 DOCTEST_MSVC_SUPPRESS_WARNING(4800) // forcing value to bool 'true' or 'false' (performance warning)
03144 DOCTEST_MSVC_SUPPRESS_WARNING(5245) // unreferenced function with internal linkage has been removed
03145
03146 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
03147
03148 // required includes - will go only in one translation unit!
03149 #include <ctime>
03150 #include <cmath>
03151 #include <climits>
03152 // borland (Embarcadero) compiler requires math.h and not cmath -
03153 // https://github.com/doctest/doctest/pull/37
03154 #ifdef __BORLANDC__
03155 #include <math.h>
03156 #endif // __BORLANDC__
03157 #include <new>
03158 #include <cstdio>
03159 #include <cstdlib>
03160 #include <cstring>
03161 #include <limits>
03162 #include <utility>
03163 #include <fstream>
03164 #include <sstream>
03165 #ifndef DOCTEST_CONFIG_NO_INCLUDE_Iostream
03166 #include <iostream>
03167 #endif // DOCTEST_CONFIG_NO_INCLUDE_Iostream
03168 #include <algorithm>
03169 #include <iomanip>
03170 #include <vector>
03171 #ifndef DOCTEST_CONFIG_NO_MULTITHREADING
03172 #include <atomic>
03173 #include <mutex>
03174 #define DOCTEST_DECLARE_MUTEX(name) std::mutex name;
03175 #define DOCTEST_DECLARE_STATIC_MUTEX(name) static DOCTEST_DECLARE_MUTEX(name)
03176 #define DOCTEST_LOCK_MUTEX(name) std::lock_guard<std::mutex>
03177 #define DOCTEST_UNLOCK_MUTEX(name)
03178 #define DOCTEST_ANONYMOUS(DOCTEST_ANON_LOCK_) (name);
03179 #else // DOCTEST_CONFIG_NO_MULTITHREADING
03180 #define DOCTEST_DECLARE_MUTEX(name)
03181 #define DOCTEST_DECLARE_STATIC_MUTEX(name)
03182 #define DOCTEST_LOCK_MUTEX(name)
03183 #define DOCTEST_UNLOCK_MUTEX(name)
03184 #define DOCTEST_ANONYMOUS(DOCTEST_ANON_LOCK_) (name);
03185 #endif // DOCTEST_CONFIG_NO_MULTITHREADING
03186 #include <set>
03187 #include <map>
03188 #include <unordered_set>
03189 #include <exception>
03190 #include <stdexcept>
03191 #include <csignal>
03192 #include <cmath>
03193 #include <cctype>
03194 #include <cstdint>
03195 #include <string>
03196
03197 #ifdef DOCTEST_PLATFORM_MAC
03198 #include <sys/types.h>
03199 #include <unistd.h>
03200 #include <sys/sysctl.h>
03201 #endif // DOCTEST_PLATFORM_MAC
03202
03203 #ifdef DOCTEST_PLATFORM_WINDOWS
03204 // defines for a leaner windows.h
03205 #ifndef WIN32_LEAN_AND_MEAN
03206 #define WIN32_LEAN_AND_MEAN
03207 #define DOCTEST_UNDEF_WIN32_LEAN_AND_MEAN
03208 #endif // WIN32_LEAN_AND_MEAN
03209 #ifndef NOMINMAX
03210 #define NOMINMAX
03211 #define DOCTEST_UNDEF_NOMINMAX
03212 #endif // NOMINMAX
03213
03214 // not sure what AfxWin.h is for - here I do what Catch does
03215 #ifndef __AFXDLL
03216 #include <AfxWin.h>
03217 #else
03218 #include <windows.h>
03219 #endif
03220 #include <io.h>
03221 #else // DOCTEST_PLATFORM_WINDOWS

```

```

03219
03220 #include <sys/time.h>
03221 #include <unistd.h>
03222
03223 #endif // DOCTEST_PLATFORM_WINDOWS
03224
03225 // this is a fix for https://github.com/doctest/doctest/issues/348
03226 // https://mail.gnome.org/archives/xml/2012-January/msg00000.html
03227 #if !defined(HAVE_UNISTD_H) && !defined(STDOUT_FILENO)
03228 #define STDOUT_FILENO fileno(stdout)
03229 #endif // HAVE_UNISTD_H
03230
03231 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END
03232
03233 // counts the number of elements in a C array
03234 #define DOCTEST_COUNTOF(x) (sizeof(x) / sizeof(x[0]))
03235
03236 #ifdef DOCTEST_CONFIG_DISABLE
03237 #define DOCTEST_BRANCH_ON_DISABLED(if_disabled, if_not_disabled) if_disabled
03238 #else // DOCTEST_CONFIG_DISABLE
03239 #define DOCTEST_BRANCH_ON_DISABLED(if_disabled, if_not_disabled) if_not_disabled
03240 #endif // DOCTEST_CONFIG_DISABLE
03241
03242 #ifndef DOCTEST_CONFIG_OPTIONS_PREFIX
03243 #define DOCTEST_CONFIG_OPTIONS_PREFIX "dt-"
03244 #endif
03245
03246 #ifndef DOCTEST_THREAD_LOCAL
03247 #if defined(DOCTEST_CONFIG_NO_MULTITHREADING) || DOCTEST_MSVC && (DOCTEST_MSVC < DOCTEST_COMPILER(19,
0, 0))
03248 #define DOCTEST_THREAD_LOCAL
03249 #else // DOCTEST_MSVC
03250 #define DOCTEST_THREAD_LOCAL thread_local
03251 #endif // DOCTEST_MSVC
03252 #endif // DOCTEST_THREAD_LOCAL
03253
03254 #ifndef DOCTEST_MULTI_LANE_ATOMICS_THREAD_LANES
03255 #define DOCTEST_MULTI_LANE_ATOMICS_THREAD_LANES 32
03256 #endif
03257
03258 #ifndef DOCTEST_MULTI_LANE_ATOMICS_CACHE_LINE_SIZE
03259 #define DOCTEST_MULTI_LANE_ATOMICS_CACHE_LINE_SIZE 64
03260 #endif
03261
03262 #ifdef DOCTEST_CONFIG_NO_UNPREFIXED_OPTIONS
03263 #define DOCTEST_OPTIONS_PREFIX_DISPLAY DOCTEST_CONFIG_OPTIONS_PREFIX
03264 #else
03265 #define DOCTEST_OPTIONS_PREFIX_DISPLAY ""
03266 #endif
03267
03268 #if defined(WINAPI_FAMILY) && (WINAPI_FAMILY == WINAPI_FAMILY_APP)
03269 #define DOCTEST_CONFIG_NO_MULTI_LANE_ATOMICS
03270 #endif
03271
03272 #ifndef DOCTEST_CDECL
03273 #define DOCTEST_CDECL __cdecl
03274 #endif
03275
03276 namespace doctest {
03277
03278 bool is_running_in_test = false;
03279
03280 namespace {
03281     using namespace detail;
03282
03283     template <typename Ex>
03284     DOCTEST_NORETURN void throw_exception(Ex const& e) {
03285         #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
03286             throw e;
03287         #else // DOCTEST_CONFIG_NO_EXCEPTIONS
03288             #ifdef DOCTEST_CONFIG_HANDLE_EXCEPTION
03289                 DOCTEST_CONFIG_HANDLE_EXCEPTION(e);
03290             #else // DOCTEST_CONFIG_HANDLE_EXCEPTION
03291                 #ifndef DOCTEST_CONFIG_NO_INCLUDE_Iostream
03292                 std::cerr << "doctest will terminate because it needed to throw an exception.\n"
03293                     << "The message was: " << e.what() << '\n';
03294                 #endif // DOCTEST_CONFIG_NO_INCLUDE_Iostream
03295             #endif // DOCTEST_CONFIG_HANDLE_EXCEPTION
03296                 std::terminate();
03297             #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
03298         }
03299
03300         #ifndef DOCTEST_INTERNAL_ERROR
03301         #define DOCTEST_INTERNAL_ERROR(msg)
03302             throw_exception(std::logic_error(
03303                 __FILE__ " " DOCTEST_TOSTR(__LINE__) ": Internal doctest error: " msg))
03304         #endif // DOCTEST_INTERNAL_ERROR

```



```

03305
03306 // case insensitive strcmp
03307 int stricmp(const char* a, const char* b) {
03308     for(;; a++, b++) {
03309         const int d = tolower(*a) - tolower(*b);
03310         if(d != 0 || !*a)
03311             return d;
03312     }
03313 }
03314
03315 struct Endianness
03316 {
03317     enum Arch
03318     {
03319         Big,
03320         Little
03321     };
03322
03323     static Arch which() {
03324         int x = 1;
03325         // casting any data pointer to char* is allowed
03326         auto ptr = reinterpret_cast<char*>(&x);
03327         if(*ptr)
03328             return Little;
03329         return Big;
03330     }
03331 };
03332 } // namespace
03333
03334 namespace detail {
03335     DOCTEST_THREAD_LOCAL class
03336     {
03337     public:
03338         std::vector<std::streampos> stack;
03339         std::stringstream ss;
03340
03341         std::ostream* push() {
03342             stack.push_back(ss.tellp());
03343             return &ss;
03344         }
03345
03346         String pop() {
03347             if (stack.empty())
03348                 DOCTEST_INTERNAL_ERROR("TLSS was empty when trying to pop!");
03349
03350             std::streampos pos = stack.back();
03351             stack.pop_back();
03352             unsigned sz = static_cast<unsigned>(ss.tellp() - pos);
03353             ss.rdbuf()->pubseekpos(pos, std::ios::in | std::ios::out);
03354             return String(ss, sz);
03355         }
03356     } g_oss;
03357
03358     std::ostream* tlssPush() {
03359         return g_oss.push();
03360     }
03361
03362     String tlssPop() {
03363         return g_oss.pop();
03364     }
03365
03366 #ifndef DOCTEST_CONFIG_DISABLE
03367 namespace timer_large_integer
03368 {
03369     #if defined(DOCTEST_PLATFORM_WINDOWS)
03370         using type = ULONGLONG;
03371     #else // DOCTEST_PLATFORM_WINDOWS
03372         using type = std::uint64_t;
03373     #endif // DOCTEST_PLATFORM_WINDOWS
03374 }
03375
03376 using ticks_t = timer_large_integer::type;
03377
03378 #ifndef DOCTEST_CONFIG_GETCURRENTTICKS
03379     ticks_t getCurrentTicks() { return DOCTEST_CONFIG_GETCURRENTTICKS(); }
03380 #elif defined(DOCTEST_PLATFORM_WINDOWS)
03381     ticks_t getCurrentTicks() {
03382         static LARGE_INTEGER hz = { 0 }, hzo = { 0 };
03383         if(!hz.QuadPart) {
03384             QueryPerformanceFrequency(&hz);
03385             QueryPerformanceCounter(&hzo);
03386         }
03387         LARGE_INTEGER t;
03388         QueryPerformanceCounter(&t);
03389         return ((t.QuadPart - hzo.QuadPart) * LONGLONG(1000000)) / hz.QuadPart;
03390     }
03391 
```

```

03392     }
03393 #else // DOCTEST_PLATFORM_WINDOWS
03394     ticks_t getCurrentTicks() {
03395         timeval t;
03396         gettimeofday(&t, nullptr);
03397         return static_cast<ticks_t>(t.tv_sec) * 1000000 + static_cast<ticks_t>(t.tv_usec);
03398     }
03399 #endif // DOCTEST_PLATFORM_WINDOWS
03400
03401     struct Timer
03402     {
03403         void start() { m_ticks = getCurrentTicks(); }
03404         unsigned int getElapsedMicroseconds() const {
03405             return static_cast<unsigned int>(getCurrentTicks() - m_ticks);
03406         }
03407         //unsigned int getElapsedMilliseconds() const {
03408         //    return static_cast<unsigned int>(getElapsedMicroseconds() / 1000);
03409         //}
03410         double getElapsedSeconds() const { return static_cast<double>(getCurrentTicks() - m_ticks) /
1000000.0; }
03411
03412     private:
03413         ticks_t m_ticks = 0;
03414     };
03415
03416 #ifndef DOCTEST_CONFIG_NO_MULTITHREADING
03417     template <typename T>
03418     using Atomic = T;
03419 #else // DOCTEST_CONFIG_NO_MULTITHREADING
03420     template <typename T>
03421     using Atomic = std::atomic<T>;
03422 #endif // DOCTEST_CONFIG_NO_MULTITHREADING
03423
03424 #if defined(DOCTEST_CONFIG_NO_MULTI_LANE_ATOMICS) || defined(DOCTEST_CONFIG_NO_MULTITHREADING)
03425     template <typename T>
03426     using MultiLaneAtomic = Atomic<T>;
03427 #else // DOCTEST_CONFIG_NO_MULTI_LANE_ATOMICS
03428     // Provides a multilane implementation of an atomic variable that supports add, sub, load,
03429     // store. Instead of using a single atomic variable, this splits up into multiple ones,
03430     // each sitting on a separate cache line. The goal is to provide a speedup when most
03431     // operations are modifying. It achieves this with two properties:
03432     //
03433     // * Multiple atomics are used, so chance of congestion from the same atomic is reduced.
03434     // * Each atomic sits on a separate cache line, so false sharing is reduced.
03435     //
03436     // The disadvantage is that there is a small overhead due to the use of TLS, and load/store
03437     // is slower because all atomics have to be accessed.
03438     template <typename T>
03439     class MultiLaneAtomic
03440     {
03441     public:
03442         struct CacheLineAlignedAtomic
03443         {
03444             Atomic<T> atomic{};
03445             char padding[DOCTEST_MULTI_LANE_ATOMICS_CACHE_LINE_SIZE - sizeof(Atomic<T>)];
03446         };
03447         CacheLineAlignedAtomic m_atomics[DOCTEST_MULTI_LANE_ATOMICS_THREAD_LANES];
03448
03449         static_assert(sizeof(CacheLineAlignedAtomic) == DOCTEST_MULTI_LANE_ATOMICS_CACHE_LINE_SIZE,
03450             "guarantee one atomic takes exactly one cache line");
03451
03452         T operator++() DOCTEST_NOEXCEPT { return fetch_add(1) + 1; }
03453
03454         T operator++(int) DOCTEST_NOEXCEPT { return fetch_add(1); }
03455
03456         T fetch_add(T arg, std::memory_order order = std::memory_order_seq_cst) DOCTEST_NOEXCEPT {
03457             return myAtomic().fetch_add(arg, order);
03458         }
03459
03460         T fetch_sub(T arg, std::memory_order order = std::memory_order_seq_cst) DOCTEST_NOEXCEPT {
03461             return myAtomic().fetch_sub(arg, order);
03462         }
03463
03464         operator T() const DOCTEST_NOEXCEPT { return load(); }
03465
03466         T load(std::memory_order order = std::memory_order_seq_cst) const DOCTEST_NOEXCEPT {
03467             auto result = T();
03468             for(auto const& c : m_atomics) {
03469                 result += c.atomic.load(order);
03470             }
03471             return result;
03472         }
03473
03474         T operator=(T desired) DOCTEST_NOEXCEPT { // lgTM [cpp/assignment-does-not-return-this]
03475             store(desired);
03476             return desired;
03477         }

```



```

03478
03479     void store(T desired, std::memory_order order = std::memory_order_seq_cst) DOCTEST_NOEXCEPT {
03480         // first value becomes desired", all others become 0.
03481         for(auto& c : m_atomics) {
03482             c.atomic.store(desired, order);
03483             desired = {};
03484         }
03485     }
03486
03487 private:
03488     // Each thread has a different atomic that it operates on. If more than NumLanes threads
03489     // use this, some will use the same atomic. So performance will degrade a bit, but still
03490     // everything will work.
03491     //
03492     // The logic here is a bit tricky. The call should be as fast as possible, so that there
03493     // is minimal to no overhead in determining the correct atomic for the current thread.
03494     //
03495     // 1. A global static counter laneCounter counts continuously up.
03496     // 2. Each successive thread will use modulo operation of that counter so it gets an atomic
03497     //    assigned in a round-robin fashion.
03498     // 3. This tlsLaneIdx is stored in the thread local data, so it is directly available with
03499     //    little overhead.
03500     Atomic<T>& myAtomic() DOCTEST_NOEXCEPT {
03501         static Atomic<size_t> laneCounter;
03502         DOCTEST_THREAD_LOCAL size_t tlsLaneIdx =
03503             laneCounter++ % DOCTEST_MULTI_LANE_ATOMICS_THREAD_LANES;
03504
03505         return m_atomics[tlsLaneIdx].atomic;
03506     }
03507 };
03508 #endif // DOCTEST_CONFIG_NO_MULTI_LANE_ATOMICS
03509
03510 // this holds both parameters from the command line and runtime data for tests
03511 struct ContextState : ContextOptions, TestRunStats, CurrentTestCaseStats
03512 {
03513     MultiLaneAtomic<int> numAssertsCurrentTest_atomic;
03514     MultiLaneAtomic<int> numAssertsFailedCurrentTest_atomic;
03515
03516     std::vector<std::vector<String>> filters = decltype(filters)(9); // 9 different filters
03517
03518     std::vector<IReporter*> reporters_currently_used;
03519
03520     assert_handler ah = nullptr;
03521
03522     Timer timer;
03523
03524     std::vector<String> stringifiedContexts; // logging from INFO() due to an exception
03525
03526     // stuff for subcases
03527     bool reachedLeaf;
03528     std::vector<SubcaseSignature> subcaseStack;
03529     std::vector<SubcaseSignature> nextSubcaseStack;
03530     std::unordered_set<unsigned long long> fullyTraversedSubcases;
03531     size_t currentSubcaseDepth;
03532     Atomic<bool> shouldLogCurrentException;
03533
03534     void resetRunData() {
03535         numTestCases = 0;
03536         numTestCasesPassingFilters = 0;
03537         numTestSuitesPassingFilters = 0;
03538         numTestCasesFailed = 0;
03539         numAsserts = 0;
03540         numAssertsFailed = 0;
03541         numAssertsCurrentTest = 0;
03542         numAssertsFailedCurrentTest = 0;
03543     }
03544
03545     void finalizeTestCaseData() {
03546         seconds = timer.getElapsedSeconds();
03547
03548         // update the non-atomic counters
03549         numAsserts += numAssertsCurrentTest_atomic;
03550         numAssertsFailed += numAssertsFailedCurrentTest_atomic;
03551         numAssertsCurrentTest = numAssertsCurrentTest_atomic;
03552         numAssertsFailedCurrentTest = numAssertsFailedCurrentTest_atomic;
03553
03554         if(numAssertsFailedCurrentTest)
03555             failure_flags |= TestCaseFailureReason::AssertFailure;
03556
03557         if(Approx(currentTest->m_timeout).epsilon(DBL_EPSILON) != 0 &&
03558            Approx(seconds).epsilon(DBL_EPSILON) > currentTest->m_timeout)
03559             failure_flags |= TestCaseFailureReason::Timeout;
03560
03561         if(currentTest->m_should_fail) {
03562             if(failure_flags) {
03563                 failure_flags |= TestCaseFailureReason::ShouldHaveFailedAndDid;
03564             } else {

```

```

03565         failure_flags |= TestCaseFailureReason::ShouldHaveFailedButDidnt;
03566     }
03567     } else if(failure_flags && currentTest->m_may_fail) {
03568         failure_flags |= TestCaseFailureReason::CouldHaveFailedAndDid;
03569     } else if(currentTest->m_expected_failures > 0) {
03570         if(numAssertsFailedCurrentTest == currentTest->m_expected_failures) {
03571             failure_flags |= TestCaseFailureReason::FailedExactlyNumTimes;
03572         } else {
03573             failure_flags |= TestCaseFailureReason::DidntFailExactlyNumTimes;
03574         }
03575     }
03576
03577     bool ok_to_fail = (TestCaseFailureReason::ShouldHaveFailedAndDid & failure_flags) ||
03578                     (TestCaseFailureReason::CouldHaveFailedAndDid & failure_flags) ||
03579                     (TestCaseFailureReason::FailedExactlyNumTimes & failure_flags);
03580
03581     // if any subcase has failed - the whole test case has failed
03582     testCaseSuccess = !(failure_flags && !ok_to_fail);
03583     if(!testCaseSuccess)
03584         numTestCasesFailed++;
03585 }
03586 };
03587
03588 ContextState* g_cs = nullptr;
03589
03590 // used to avoid locks for the debug output
03591 // TODO: figure out if this is indeed necessary/correct - seems like either there still
03592 // could be a race or that there wouldn't be a race even if using the context directly
03593 DOCTEST_THREAD_LOCAL bool g_no_colors;
03594
03595 #endif // DOCTEST_CONFIG_DISABLE
03596 } // namespace detail
03597
03598 char* String::allocate(size_type sz) {
03599     if (sz <= last) {
03600         buf[sz] = '\0';
03601         setLast(last - sz);
03602         return buf;
03603     } else {
03604         setOnHeap();
03605         data.size = sz;
03606         data.capacity = data.size + 1;
03607         data.ptr = new char[data.capacity];
03608         data.ptr[sz] = '\0';
03609         return data.ptr;
03610     }
03611 }
03612
03613 void String::setOnHeap() noexcept { *reinterpret_cast<unsigned char*>(&buf[last]) = 128; }
03614 void String::setLast(size_type in) noexcept { buf[last] = char(in); }
03615 void String::setSize(size_type sz) noexcept {
03616     if (isOnStack()) { buf[sz] = '\0'; setLast(last - sz); }
03617     else { data.ptr[sz] = '\0'; data.size = sz; }
03618 }
03619
03620 void String::copy(const String& other) {
03621     if(other.isOnStack()) {
03622         memcpy(buf, other.buf, len);
03623     } else {
03624         memcpy(allocate(other.data.size), other.data.ptr, other.data.size);
03625     }
03626 }
03627
03628 String::String() noexcept {
03629     buf[0] = '\0';
03630     setLast();
03631 }
03632
03633 String::~String() {
03634     if(!isOnStack())
03635         delete[] data.ptr;
03636 } // NOLINT(clang-analyzer-cplusplus.NewDeleteLeaks)
03637
03638 String::String(const char* in)
03639     : String(in, strlen(in)) {}
03640
03641 String::String(const char* in, size_type in_size) {
03642     memcpy(allocate(in_size), in, in_size);
03643 }
03644
03645 String::String(std::istream& in, size_type in_size) {
03646     in.read(allocate(in_size), in_size);
03647 }
03648
03649 String::String(const String& other) { copy(other); }
03650
03651 String& String::operator=(const String& other) {

```

```

03652     if(this != &other) {
03653         if(!isOnStack())
03654             delete[] data.ptr;
03655         copy(other);
03656     }
03657 }
03658
03659 return *this;
03660 }
03661
03662 String& String::operator+=(const String& other) {
03663     const size_type my_old_size = size();
03664     const size_type other_size = other.size();
03665     const size_type total_size = my_old_size + other_size;
03666     if(isOnStack()) {
03667         if(total_size < len) {
03668             // append to the current stack space
03669             memcpy(buf + my_old_size, other.c_str(), other_size + 1);
03670             // NOLINTNEXTLINE(clang-analyzer-cplusplus.NewDeleteLeaks)
03671             setLast(last - total_size);
03672         } else {
03673             // alloc new chunk
03674             char* temp = new char[total_size + 1];
03675             // copy current data to new location before writing in the union
03676             memcpy(temp, buf, my_old_size); // skip the +1 ('\0') for speed
03677             // update data in union
03678             setOnHeap();
03679             data.size = total_size;
03680             data.capacity = data.size + 1;
03681             data.ptr = temp;
03682             // transfer the rest of the data
03683             memcpy(data.ptr + my_old_size, other.c_str(), other_size + 1);
03684         }
03685     } else {
03686         if(data.capacity > total_size) {
03687             // append to the current heap block
03688             data.size = total_size;
03689             memcpy(data.ptr + my_old_size, other.c_str(), other_size + 1);
03690         } else {
03691             // resize
03692             data.capacity *= 2;
03693             if(data.capacity <= total_size)
03694                 data.capacity = total_size + 1;
03695             // alloc new chunk
03696             char* temp = new char[data.capacity];
03697             // copy current data to new location before releasing it
03698             memcpy(temp, data.ptr, my_old_size); // skip the +1 ('\0') for speed
03699             // release old chunk
03700             delete[] data.ptr;
03701             // update the rest of the union members
03702             data.size = total_size;
03703             data.ptr = temp;
03704             // transfer the rest of the data
03705             memcpy(data.ptr + my_old_size, other.c_str(), other_size + 1);
03706         }
03707     }
03708 }
03709 return *this;
03710 }
03711
03712 String::String(String&& other) noexcept {
03713     memcpy(buf, other.buf, len);
03714     other.buf[0] = '\0';
03715     other.setLast();
03716 }
03717
03718 String& String::operator=(String&& other) noexcept {
03719     if(this != &other) {
03720         if(!isOnStack())
03721             delete[] data.ptr;
03722         memcpy(buf, other.buf, len);
03723         other.buf[0] = '\0';
03724         other.setLast();
03725     }
03726     return *this;
03727 }
03728
03729 char String::operator[](size_type i) const {
03730     return const_cast<String*>(this)->operator[](i);
03731 }
03732
03733 char& String::operator[](size_type i) {
03734     if(isOnStack())
03735         return reinterpret_cast<char*>(buf)[i];
03736     return data.ptr[i];
03737 }
03738

```

```

03739 DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wmaybe-uninitialized")
03740 String::size_type String::size() const {
03741     if(isOnStack())
03742         return last - (size_type(buf[last]) & 31); // using "last" would work only if "len" is 32
03743     return data.size;
03744 }
03745 DOCTEST_GCC_SUPPRESS_WARNING_POP
03746
03747 String::size_type String::capacity() const {
03748     if(isOnStack())
03749         return len;
03750     return data.capacity;
03751 }
03752
03753 String String::substr(size_type pos, size_type cnt) && {
03754     cnt = std::min(cnt, size() - 1 - pos);
03755     char* cptr = c_str();
03756     memmove(cptr, cptr + pos, cnt);
03757     setSize(cnt);
03758     return std::move(*this);
03759 }
03760
03761 String String::substr(size_type pos, size_type cnt) const & {
03762     cnt = std::min(cnt, size() - 1 - pos);
03763     return String{ c_str() + pos, cnt };
03764 }
03765
03766 String::size_type String::find(char ch, size_type pos) const {
03767     const char* begin = c_str();
03768     const char* end = begin + size();
03769     const char* it = begin + pos;
03770     for (; it < end && *it != ch; it++);
03771     if (it < end) { return static_cast<size_type>(it - begin); }
03772     else { return npos; }
03773 }
03774
03775 String::size_type String::rfind(char ch, size_type pos) const {
03776     const char* begin = c_str();
03777     const char* it = begin + std::min(pos, size() - 1);
03778     for (; it >= begin && *it != ch; it--);
03779     if (it >= begin) { return static_cast<size_type>(it - begin); }
03780     else { return npos; }
03781 }
03782
03783 int String::compare(const char* other, bool no_case) const {
03784     if(no_case)
03785         return doctest::stricmp(c_str(), other);
03786     return std::strcmp(c_str(), other);
03787 }
03788
03789 int String::compare(const String& other, bool no_case) const {
03790     return compare(other.c_str(), no_case);
03791 }
03792
03793 String operator+(const String& lhs, const String& rhs) { return String(lhs) += rhs; }
03794
03795 bool operator==(const String& lhs, const String& rhs) { return lhs.compare(rhs) == 0; }
03796 bool operator!=(const String& lhs, const String& rhs) { return lhs.compare(rhs) != 0; }
03797 bool operator<(const String& lhs, const String& rhs) { return lhs.compare(rhs) < 0; }
03798 bool operator>(const String& lhs, const String& rhs) { return lhs.compare(rhs) > 0; }
03799 bool operator<=(const String& lhs, const String& rhs) { return (lhs != rhs) ? lhs.compare(rhs) < 0 :
true; }
03800 bool operator>=(const String& lhs, const String& rhs) { return (lhs != rhs) ? lhs.compare(rhs) > 0 :
true; }
03801
03802 std::ostream& operator<<(std::ostream& s, const String& in) { return s << in.c_str(); }
03803
03804 Contains::Contains(const String& str) : string(str) { }
03805
03806 bool Contains::checkWith(const String& other) const {
03807     return strstr(other.c_str(), string.c_str()) != nullptr;
03808 }
03809
03810 String toString(const Contains& in) {
03811     return "Contains( " + in.string + " )";
03812 }
03813
03814 bool operator==(const String& lhs, const Contains& rhs) { return rhs.checkWith(lhs); }
03815 bool operator==(const Contains& lhs, const String& rhs) { return lhs.checkWith(rhs); }
03816 bool operator!=(const String& lhs, const Contains& rhs) { return !rhs.checkWith(lhs); }
03817 bool operator!=(const Contains& lhs, const String& rhs) { return !lhs.checkWith(rhs); }
03818
03819 namespace {
03820     void color_to_stream(std::ostream&, Color::Enum) DOCTEST_BRANCH_ON_DISABLED({}, );
03821 } // namespace
03822
03823 namespace Color {

```

```

03824     std::ostream& operator<<(std::ostream& s, Color::Enum code) {
03825         color_to_stream(s, code);
03826         return s;
03827     }
03828 } // namespace Color
03829
03830 // clang-format off
03831 const char* assertString(assertType::Enum at) {
03832     DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4061) // enum 'x' in switch of enum 'y' is not explicitly
handled
03833     #define DOCTEST_GENERATE_ASSERT_TYPE_CASE(assert_type) case assertType::DT_ ## assert_type: return
assert_type
03834     #define DOCTEST_GENERATE_ASSERT_TYPE_CASES(assert_type) \
03835         DOCTEST_GENERATE_ASSERT_TYPE_CASE(WARN_ ## assert_type); \
03836         DOCTEST_GENERATE_ASSERT_TYPE_CASE(CHECK_ ## assert_type); \
03837         DOCTEST_GENERATE_ASSERT_TYPE_CASE(REQUIRE_ ## assert_type)
03838     switch (at) {
03839         DOCTEST_GENERATE_ASSERT_TYPE_CASE(WARN);
03840         DOCTEST_GENERATE_ASSERT_TYPE_CASE(CHECK);
03841         DOCTEST_GENERATE_ASSERT_TYPE_CASE(REQUIRE);
03842
03843         DOCTEST_GENERATE_ASSERT_TYPE_CASES(FALSE);
03844
03845         DOCTEST_GENERATE_ASSERT_TYPE_CASES(THROWS);
03846
03847         DOCTEST_GENERATE_ASSERT_TYPE_CASES(THROWS_AS);
03848
03849         DOCTEST_GENERATE_ASSERT_TYPE_CASES(THROWS_WITH);
03850
03851         DOCTEST_GENERATE_ASSERT_TYPE_CASES(THROWS_WITH_AS);
03852
03853         DOCTEST_GENERATE_ASSERT_TYPE_CASES(NOTHROW);
03854
03855         DOCTEST_GENERATE_ASSERT_TYPE_CASES(EQ);
03856         DOCTEST_GENERATE_ASSERT_TYPE_CASES(NE);
03857         DOCTEST_GENERATE_ASSERT_TYPE_CASES(GT);
03858         DOCTEST_GENERATE_ASSERT_TYPE_CASES(LT);
03859         DOCTEST_GENERATE_ASSERT_TYPE_CASES(GE);
03860         DOCTEST_GENERATE_ASSERT_TYPE_CASES(LE);
03861
03862         DOCTEST_GENERATE_ASSERT_TYPE_CASES(UNARY);
03863         DOCTEST_GENERATE_ASSERT_TYPE_CASES(UNARY_FALSE);
03864
03865         default: DOCTEST_INTERNAL_ERROR("Tried stringifying invalid assert type!");
03866     }
03867     DOCTEST_MSVC_SUPPRESS_WARNING_POP
03868 }
03869 // clang-format on
03870
03871 const char* failureString(assertType::Enum at) {
03872     if (at & assertType::is_warn)
03873         return "WARNING";
03874     if (at & assertType::is_check)
03875         return "ERROR";
03876     if (at & assertType::is_require)
03877         return "FATAL ERROR";
03878     return "";
03879 }
03880
03881 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wnull-dereference")
03882 DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wnull-dereference")
03883 // depending on the current options this will remove the path of filenames
03884 const char* skipPathFromFilename(const char* file) {
03885     #ifndef DOCTEST_CONFIG_DISABLE
03886         if (getContextOptions()->no_path_in_filenames) {
03887             auto back = std::strrchr(file, '\\');
03888             auto forward = std::strrchr(file, '/');
03889             if (back || forward) {
03890                 if (back > forward)
03891                     forward = back;
03892                 return forward + 1;
03893             }
03894         }
03895     #endif // DOCTEST_CONFIG_DISABLE
03896     return file;
03897 }
03898 DOCTEST_CLANG_SUPPRESS_WARNING_POP
03899 DOCTEST_GCC_SUPPRESS_WARNING_POP
03900
03901 bool SubcaseSignature::operator==(const SubcaseSignature& other) const {
03902     return m_line == other.m_line
03903         && std::strcmp(m_file, other.m_file) == 0
03904         && m_name == other.m_name;
03905 }
03906
03907 bool SubcaseSignature::operator<(const SubcaseSignature& other) const {
03908     if (m_line != other.m_line)

```

```

03909         return m_line < other.m_line;
03910     if(std::strcmp(m_file, other.m_file) != 0)
03911         return std::strcmp(m_file, other.m_file) < 0;
03912     return m_name.compare(other.m_name) < 0;
03913 }
03914
03915 DOCTEST_DEFINE_INTERFACE(IContextScope)
03916
03917 namespace detail {
03918     void filldata<const void*>::fill(std::ostream* stream, const void* in) {
03919         if (in) { *stream << in; }
03920         else { *stream << "nullptr"; }
03921     }
03922
03923     template <typename T>
03924     String toStreamLit(T t) {
03925         std::ostream* os = tlssPush();
03926         os->operator<<(t);
03927         return tlssPop();
03928     }
03929 }
03930
03931 #ifdef DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
03932 String toString(const char* in) { return String("\0" + (in ? in : "{null string}") + "\0"; }
03933 #endif // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
03934
03935 #if DOCTEST_MSVC >= DOCTEST_COMPILER(19, 20, 0)
03936 // see this issue on why this is needed: https://github.com/doctest/doctest/issues/183
03937 String toString(const std::string& in) { return in.c_str(); }
03938 #endif // VS 2019
03939
03940 String toString(String in) { return in; }
03941
03942 String toString(std::nullptr_t) { return "nullptr"; }
03943
03944 String toString(bool in) { return in ? "true" : "false"; }
03945
03946 String toString(float in) { return toStreamLit(in); }
03947 String toString(double in) { return toStreamLit(in); }
03948 String toString(double long in) { return toStreamLit(in); }
03949
03950 String toString(char in) { return toStreamLit(static_cast<signed>(in)); }
03951 String toString(char signed in) { return toStreamLit(static_cast<signed>(in)); }
03952 String toString(char unsigned in) { return toStreamLit(static_cast<unsigned>(in)); }
03953 String toString(short in) { return toStreamLit(in); }
03954 String toString(short unsigned in) { return toStreamLit(in); }
03955 String toString(signed in) { return toStreamLit(in); }
03956 String toString(unsigned in) { return toStreamLit(in); }
03957 String toString(long in) { return toStreamLit(in); }
03958 String toString(long unsigned in) { return toStreamLit(in); }
03959 String toString(long long in) { return toStreamLit(in); }
03960 String toString(long long unsigned in) { return toStreamLit(in); }
03961
03962 Approx::Approx(double value)
03963     : m_epsilon(static_cast<double>(std::numeric_limits<float>::epsilon()) * 100)
03964     , m_scale(1.0)
03965     , m_value(value) {}
03966
03967 Approx Approx::operator()(double value) const {
03968     Approx approx(value);
03969     approx.epsilon(m_epsilon);
03970     approx.scale(m_scale);
03971     return approx;
03972 }
03973
03974 Approx& Approx::epsilon(double newEpsilon) {
03975     m_epsilon = newEpsilon;
03976     return *this;
03977 }
03978 Approx& Approx::scale(double newScale) {
03979     m_scale = newScale;
03980     return *this;
03981 }
03982
03983 bool operator==(double lhs, const Approx& rhs) {
03984     // Thanks to Richard Harris for his help refining this formula
03985     return std::fabs(lhs - rhs.m_value) <
03986         rhs.m_epsilon * (rhs.m_scale + std::max<double>(std::fabs(lhs), std::fabs(rhs.m_value)));
03987 }
03988 bool operator==(const Approx& lhs, double rhs) { return operator==(rhs, lhs); }
03989 bool operator!=(double lhs, const Approx& rhs) { return !operator==(lhs, rhs); }
03990 bool operator!=(const Approx& lhs, double rhs) { return !operator==(rhs, lhs); }
03991 bool operator<=(double lhs, const Approx& rhs) { return lhs < rhs.m_value || lhs == rhs; }
03992 bool operator<=(const Approx& lhs, double rhs) { return lhs.m_value < rhs || lhs == rhs; }
03993 bool operator>=(double lhs, const Approx& rhs) { return lhs > rhs.m_value || lhs == rhs; }
03994 bool operator>=(const Approx& lhs, double rhs) { return lhs.m_value > rhs || lhs == rhs; }
03995 bool operator<(double lhs, const Approx& rhs) { return lhs < rhs.m_value && lhs != rhs; }

```

```

03996 bool operator<(const Approx& lhs, double rhs) { return lhs.m_value < rhs && lhs != rhs; }
03997 bool operator>(double lhs, const Approx& rhs) { return lhs > rhs.m_value && lhs != rhs; }
03998 bool operator>(const Approx& lhs, double rhs) { return lhs.m_value > rhs && lhs != rhs; }
03999
04000 String toString(const Approx& in) {
04001     return "Approx( " + doctest::toString(in.m_value) + " )";
04002 }
04003 const ContextOptions* getContextOptions() { return DOCTEST_BRANCH_ON_DISABLED(nullptr, g_cs); }
04004
04005 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4738)
04006 template <typename F>
04007 IsNaN<F>::operator bool() const {
04008     return std::isnan(value) ^ flipped;
04009 }
04010 DOCTEST_MSVC_SUPPRESS_WARNING_POP
04011 template struct DOCTEST_INTERFACE_DEF IsNaN<float>;
04012 template struct DOCTEST_INTERFACE_DEF IsNaN<double>;
04013 template struct DOCTEST_INTERFACE_DEF IsNaN<long double>;
04014 template <typename F>
04015 String toString(IsNaN<F> in) { return String(in.flipped ? "! " : "") + "IsNaN( " +
    doctest::toString(in.value) + " )"; }
04016 String toString(IsNaN<float> in) { return toString<float>(in); }
04017 String toString(IsNaN<double> in) { return toString<double>(in); }
04018 String toString(IsNaN<double long> in) { return toString<double long>(in); }
04019
04020 } // namespace doctest
04021
04022 #ifdef DOCTEST_CONFIG_DISABLE
04023 namespace doctest {
04024     Context::Context(int, const char* const*) {}
04025     Context::~Context() = default;
04026     void Context::applyCommandLine(int, const char* const*) {}
04027     void Context::addFilter(const char*, const char*) {}
04028     void Context::clearFilters() {}
04029     void Context::setOption(const char*, bool) {}
04030     void Context::setOption(const char*, int) {}
04031     void Context::setOption(const char*, const char*) {}
04032     bool Context::shouldExit() { return false; }
04033     void Context::setAsDefaultForAssertsOutOfTestCases() {}
04034     void Context::setAssertHandler(detail::assert_handler) {}
04035     void Context::setCout(std::ostream*) {}
04036     int Context::run() { return 0; }
04037
04038     int IReporter::get_num_active_contexts() { return 0; }
04039     const IContextScope* const* IReporter::get_active_contexts() { return nullptr; }
04040     int IReporter::get_num_stringified_contexts() { return 0; }
04041     const String* IReporter::get_stringified_contexts() { return nullptr; }
04042
04043     int registerReporter(const char*, int, IReporter*) { return 0; }
04044 } // namespace doctest
04045 #else // DOCTEST_CONFIG_DISABLE
04046
04047 #if !defined(DOCTEST_CONFIG_COLORS_NONE)
04048 #if !defined(DOCTEST_CONFIG_COLORS_WINDOWS) && !defined(DOCTEST_CONFIG_COLORS_ANSI)
04049 #ifdef DOCTEST_PLATFORM_WINDOWS
04050 #define DOCTEST_CONFIG_COLORS_WINDOWS
04051 #else // linux
04052 #define DOCTEST_CONFIG_COLORS_ANSI
04053 #endif // platform
04054 #endif // DOCTEST_CONFIG_COLORS_WINDOWS && DOCTEST_CONFIG_COLORS_ANSI
04055 #endif // DOCTEST_CONFIG_COLORS_NONE
04056
04057 namespace doctest_detail_test_suite_ns {
04058     // holds the current test suite
04059     doctest::detail::TestSuite& getCurrentTestSuite() {
04060         static doctest::detail::TestSuite data{};
04061         return data;
04062     }
04063 } // namespace doctest_detail_test_suite_ns
04064
04065 namespace doctest {
04066     namespace {
04067         // the int (priority) is part of the key for automatic sorting - sadly one can register a
04068         // reporter with a duplicate name and a different priority but hopefully that won't happen often
04069         :|
04070         using reporterMap = std::map<std::pair<int, String>, reporterCreatorFunc>;
04071         reporterMap& getReporters() {
04072             static reporterMap data;
04073             return data;
04074         }
04075         reporterMap& getListeners() {
04076             static reporterMap data;
04077             return data;
04078         }
04079     }
04080 } // namespace

```

```

04081 namespace detail {
04082 #define DOCTEST_ITERATE_THROUGH_REPORTERS(function, ...) \
04083     for(auto& curr_rep : g_cs->reporters_currently_used) \
04084         curr_rep->function(__VA_ARGS__)
04085
04086     bool checkIfShouldThrow(assertType::Enum at) {
04087         if(at & assertType::is_require)
04088             return true;
04089
04090         if((at & assertType::is_check)
04091             && getContextOptions()->abort_after > 0 &&
04092             (g_cs->numAssertsFailed + g_cs->numAssertsFailedCurrentTest_atomic) >=
04093             getContextOptions()->abort_after)
04094             return true;
04095
04096         return false;
04097     }
04098
04099 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
04100     DOCTEST_NORETURN void throwException() {
04101         g_cs->shouldLogCurrentException = false;
04102         throw TestFailureException(); // NOLINT(hicpp-exception-baseclass)
04103     }
04104 #else // DOCTEST_CONFIG_NO_EXCEPTIONS
04105     void throwException() {}
04106 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
04107 } // namespace detail
04108
04109 namespace {
04110     using namespace detail;
04111     // matching of a string against a wildcard mask (case sensitivity configurable) taken from
04112     // https://www.codeproject.com/Articles/1088/Wildcard-string-compare-globbing
04113     int wilddcmp(const char* str, const char* wild, bool caseSensitive) {
04114         const char* cp = str;
04115         const char* mp = wild;
04116
04117         while((*str) && (*wild != '*')) {
04118             if((caseSensitive ? (*wild != *str) : (tolower(*wild) != tolower(*str))) &&
04119                 (*wild != '?')) {
04120                 return 0;
04121             }
04122             wild++;
04123             str++;
04124         }
04125
04126         while(*str) {
04127             if(*wild == '*') {
04128                 if(!+++wild) {
04129                     return 1;
04130                 }
04131                 mp = wild;
04132                 cp = str + 1;
04133             } else if((caseSensitive ? (*wild == *str) : (tolower(*wild) == tolower(*str))) ||
04134                 (*wild == '?')) {
04135                 wild++;
04136                 str++;
04137             } else {
04138                 wild = mp;
04139                 str = cp++;
04140             }
04141         }
04142
04143         while(*wild == '*') {
04144             wild++;
04145         }
04146         return !*wild;
04147     }
04148
04149     // checks if the name matches any of the filters (and can be configured what to do when empty)
04150     bool matchesAny(const char* name, const std::vector<String>& filters, bool matchEmpty,
04151         bool caseSensitive) {
04152         if (filters.empty() && matchEmpty)
04153             return true;
04154         for (auto& curr : filters)
04155             if (wilddcmp(name, curr.c_str(), caseSensitive))
04156                 return true;
04157         return false;
04158     }
04159
04160     DOCTEST_NO_SANITIZE_INTEGER
04161     unsigned long long hash(unsigned long long a, unsigned long long b) {
04162         return (a << 5) + b;
04163     }
04164
04165     // C string hash function (djb2) - taken from http://www.cse.yorku.ca/~oz/hash.html
04166     DOCTEST_NO_SANITIZE_INTEGER
04167     unsigned long long hash(const char* str) {

```



```

04168         unsigned long long hash = 5381;
04169         char c;
04170         while ((c = *str++))
04171             hash = ((hash << 5) + hash) + c; // hash * 33 + c
04172         return hash;
04173     }
04174
04175     unsigned long long hash(const SubcaseSignature& sig) {
04176         return hash(hash(hash(sig.m_file), hash(sig.m_name.c_str())), sig.m_line);
04177     }
04178
04179     unsigned long long hash(const std::vector<SubcaseSignature>& sigs, size_t count) {
04180         unsigned long long running = 0;
04181         auto end = sigs.begin() + count;
04182         for (auto it = sigs.begin(); it != end; it++) {
04183             running = hash(running, hash(*it));
04184         }
04185         return running;
04186     }
04187
04188     unsigned long long hash(const std::vector<SubcaseSignature>& sigs) {
04189         unsigned long long running = 0;
04190         for (const SubcaseSignature& sig : sigs) {
04191             running = hash(running, hash(sig));
04192         }
04193         return running;
04194     }
04195 } // namespace
04196 namespace detail {
04197     bool Subcase::checkFilters() {
04198         if (g_cs->subcaseStack.size() < size_t(g_cs->subcase_filter_levels)) {
04199             if (!matchesAny(m_signature.m_name.c_str(), g_cs->filters[6], true, g_cs->case_sensitive))
04200                 return true;
04201             if (matchesAny(m_signature.m_name.c_str(), g_cs->filters[7], false, g_cs->case_sensitive))
04202                 return true;
04203         }
04204         return false;
04205     }
04206
04207     Subcase::Subcase(const String& name, const char* file, int line)
04208         : m_signature({name, file, line}) {
04209         if (!g_cs->reachedLeaf) {
04210             if (g_cs->nextSubcaseStack.size() <= g_cs->subcaseStack.size()
04211                 || g_cs->nextSubcaseStack[g_cs->subcaseStack.size()] == m_signature) {
04212                 // Going down.
04213                 if (checkFilters()) { return; }
04214
04215                 g_cs->subcaseStack.push_back(m_signature);
04216                 g_cs->currentSubcaseDepth++;
04217                 m_entered = true;
04218                 DOCTEST_ITERATE_THROUGH_REPORTERS(subcase_start, m_signature);
04219             }
04220         } else {
04221             if (g_cs->subcaseStack[g_cs->currentSubcaseDepth] == m_signature) {
04222                 // This subcase is reentered via control flow.
04223                 g_cs->currentSubcaseDepth++;
04224                 m_entered = true;
04225                 DOCTEST_ITERATE_THROUGH_REPORTERS(subcase_start, m_signature);
04226             } else if (g_cs->nextSubcaseStack.size() <= g_cs->currentSubcaseDepth
04227                 && g_cs->fullyTraversedSubcases.find(hash(hash(g_cs->subcaseStack,
04228 g_cs->currentSubcaseDepth), hash(m_signature)))
04229                 == g_cs->fullyTraversedSubcases.end()) {
04230                 if (checkFilters()) { return; }
04231                 // This subcase is part of the one to be executed next.
04232                 g_cs->nextSubcaseStack.clear();
04233                 g_cs->nextSubcaseStack.insert(g_cs->nextSubcaseStack.end(),
04234 g_cs->subcaseStack.begin(), g_cs->subcaseStack.begin() +
04235 g_cs->currentSubcaseDepth);
04236                 g_cs->nextSubcaseStack.push_back(m_signature);
04237             }
04238         }
04239
04240         DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4996) // std::uncaught_exception is deprecated in C++17
04241         DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wdeprecated-declarations")
04242         DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wdeprecated-declarations")
04243
04244         Subcase::~~Subcase() {
04245             if (m_entered) {
04246                 g_cs->currentSubcaseDepth--;
04247
04248                 if (!g_cs->reachedLeaf) {
04249                     // Leaf.
04250                     g_cs->fullyTraversedSubcases.insert(hash(g_cs->subcaseStack));
04251                     g_cs->nextSubcaseStack.clear();
04252                     g_cs->reachedLeaf = true;
04253                 } else if (g_cs->nextSubcaseStack.empty()) {

```

```

04253             // All children are finished.
04254             g_cs->fullyTraversedSubcases.insert(hash(g_cs->subcaseStack));
04255         }
04256
04257 #if defined(__cpp_lib_uncaught_exceptions) && __cpp_lib_uncaught_exceptions >= 201411L &&
04258 (!defined(__MAC_OS_X_VERSION_MIN_REQUIRED) || __MAC_OS_X_VERSION_MIN_REQUIRED >= 101200)
04259     if(std::uncaught_exceptions() > 0
04260 #else
04261     if(std::uncaught_exception()
04262 #endif
04263         && g_cs->shouldLogCurrentException() {
04264         DOCTEST_ITERATE_THROUGH_REPORTERS(
04265             test_case_exception, {"exception thrown in subcase - will translate later "
04266                                 "when the whole test case has been exited (cannot "
04267                                 "translate while there is an active exception)",
04268                                 false});
04269         g_cs->shouldLogCurrentException = false;
04270     }
04271     DOCTEST_ITERATE_THROUGH_REPORTERS(subcase_end, DOCTEST_EMPTY);
04272 }
04273 }
04274
04275 DOCTEST_CLANG_SUPPRESS_WARNING_POP
04276 DOCTEST_GCC_SUPPRESS_WARNING_POP
04277 DOCTEST_MSVC_SUPPRESS_WARNING_POP
04278
04279 Subcase::operator bool() const { return m_entered; }
04280
04281 Result::Result(bool passed, const String& decomposition)
04282     : m_passed(passed)
04283     , m_decomp(decomposition) {}
04284
04285 ExpressionDecomposer::ExpressionDecomposer(assertType::Enum at)
04286     : m_at(at) {}
04287
04288 TestSuite& TestSuite::operator*(const char* in) {
04289     m_test_suite = in;
04290     return *this;
04291 }
04292
04293 TestCase::TestCase(funcType test, const char* file, unsigned line, const TestSuite& test_suite,
04294     const String& type, int template_id) {
04295     m_file = file;
04296     m_line = line;
04297     m_name = nullptr; // will be later overridden in operator*
04298     m_test_suite = test_suite.m_test_suite;
04299     m_description = test_suite.m_description;
04300     m_skip = test_suite.m_skip;
04301     m_no_breaks = test_suite.m_no_breaks;
04302     m_no_output = test_suite.m_no_output;
04303     m_may_fail = test_suite.m_may_fail;
04304     m_should_fail = test_suite.m_should_fail;
04305     m_expected_failures = test_suite.m_expected_failures;
04306     m_timeout = test_suite.m_timeout;
04307
04308     m_test = test;
04309     m_type = type;
04310     m_template_id = template_id;
04311 }
04312
04313 TestCase::TestCase(const TestCase& other)
04314     : TestCaseData() {
04315     *this = other;
04316 }
04317
04318 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(26434) // hides a non-virtual function
04319 TestCase& TestCase::operator=(const TestCase& other) {
04320     TestCaseData::operator=(other);
04321     m_test = other.m_test;
04322     m_type = other.m_type;
04323     m_template_id = other.m_template_id;
04324     m_full_name = other.m_full_name;
04325
04326     if(m_template_id != -1)
04327         m_name = m_full_name.c_str();
04328     return *this;
04329 }
04330 DOCTEST_MSVC_SUPPRESS_WARNING_POP
04331
04332 TestCase& TestCase::operator*(const char* in) {
04333     m_name = in;
04334     // make a new name with an appended type for templated test case
04335     if(m_template_id != -1) {
04336         m_full_name = String(m_name) + "<" + m_type + ">";
04337         // redirect the name to point to the newly constructed full name
04338         m_name = m_full_name.c_str();
04339     }
04340 }

```

```

04339     }
04340     return *this;
04341 }
04342
04343 bool TestCase::operator<(const TestCase& other) const {
04344     // this will be used only to differentiate between test cases - not relevant for sorting
04345     if(m_line != other.m_line)
04346         return m_line < other.m_line;
04347     const int name_cmp = strcmp(m_name, other.m_name);
04348     if(name_cmp != 0)
04349         return name_cmp < 0;
04350     const int file_cmp = m_file.compare(other.m_file);
04351     if(file_cmp != 0)
04352         return file_cmp < 0;
04353     return m_template_id < other.m_template_id;
04354 }
04355
04356 // all the registered tests
04357 std::set<TestCase>& getRegisteredTests() {
04358     static std::set<TestCase> data;
04359     return data;
04360 }
04361 } // namespace detail
04362 namespace {
04363     using namespace detail;
04364     // for sorting tests by file/line
04365     bool fileOrderComparator(const TestCase* lhs, const TestCase* rhs) {
04366         // this is needed because MSVC gives different case for drive letters
04367         // for __FILE__ when evaluated in a header and a source file
04368         const int res = lhs->m_file.compare(rhs->m_file, bool(DOCTEST_MSVC));
04369         if(res != 0)
04370             return res < 0;
04371         if(lhs->m_line != rhs->m_line)
04372             return lhs->m_line < rhs->m_line;
04373         return lhs->m_template_id < rhs->m_template_id;
04374     }
04375
04376     // for sorting tests by suite/file/line
04377     bool suiteOrderComparator(const TestCase* lhs, const TestCase* rhs) {
04378         const int res = std::strcmp(lhs->m_test_suite, rhs->m_test_suite);
04379         if(res != 0)
04380             return res < 0;
04381         return fileOrderComparator(lhs, rhs);
04382     }
04383
04384     // for sorting tests by name/suite/file/line
04385     bool nameOrderComparator(const TestCase* lhs, const TestCase* rhs) {
04386         const int res = std::strcmp(lhs->m_name, rhs->m_name);
04387         if(res != 0)
04388             return res < 0;
04389         return suiteOrderComparator(lhs, rhs);
04390     }
04391
04392     DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wdeprecated-declarations")
04393     void color_to_stream(std::ostream& s, Color::Enum code) {
04394         static_cast<void>(s); // for DOCTEST_CONFIG_COLORS_NONE or DOCTEST_CONFIG_COLORS_WINDOWS
04395         static_cast<void>(code); // for DOCTEST_CONFIG_COLORS_NONE
04396 #ifndef DOCTEST_CONFIG_COLORS_ANSI
04397         if(g_no_colors ||
04398             (isatty(STDOUT_FILENO) == false && getContextOptions()->force_colors == false))
04399             return;
04400
04401         auto col = "";
04402         // clang-format off
04403         switch(code) {
04404             case Color::Red: col = "[0;31m"; break;
04405             case Color::Green: col = "[0;32m"; break;
04406             case Color::Blue: col = "[0;34m"; break;
04407             case Color::Cyan: col = "[0;36m"; break;
04408             case Color::Yellow: col = "[0;33m"; break;
04409             case Color::Grey: col = "[1;30m"; break;
04410             case Color::LightGrey: col = "[0;37m"; break;
04411             case Color::BrightRed: col = "[1;31m"; break;
04412             case Color::BrightGreen: col = "[1;32m"; break;
04413             case Color::BrightWhite: col = "[1;37m"; break;
04414             case Color::Bright: // invalid
04415             case Color::None:
04416             case Color::White:
04417             default: col = "[0m";
04418         }
04419         // clang-format on
04420         s << "\033" << col;
04421 #endif // DOCTEST_CONFIG_COLORS_ANSI
04422
04423 #ifndef DOCTEST_CONFIG_COLORS_WINDOWS
04424         if(g_no_colors ||
04425             (_isatty(_fileno(stdout)) == false && getContextOptions()->force_colors == false))

```

```

04426         return;
04427
04428     static struct ConsoleHelper {
04429         HANDLE stdoutHandle;
04430         WORD    origFgAttrs;
04431         WORD    origBgAttrs;
04432
04433         ConsoleHelper() {
04434             stdoutHandle = GetStdHandle(STD_OUTPUT_HANDLE);
04435             CONSOLE_SCREEN_BUFFER_INFO csbiInfo;
04436             GetConsoleScreenBufferInfo(stdoutHandle, &csbiInfo);
04437             origFgAttrs = csbiInfo.wAttributes & ~(BACKGROUND_GREEN | BACKGROUND_RED |
04438                 BACKGROUND_BLUE | BACKGROUND_INTENSITY);
04439             origBgAttrs = csbiInfo.wAttributes & ~(FOREGROUND_GREEN | FOREGROUND_RED |
04440                 FOREGROUND_BLUE | FOREGROUND_INTENSITY);
04441         }
04442     } ch;
04443
04444     #define DOCTEST_SET_ATTR(x) SetConsoleTextAttribute(ch.stdoutHandle, x | ch.origBgAttrs)
04445
04446     // clang-format off
04447     switch (code) {
04448         case Color::White:      DOCTEST_SET_ATTR(FOREGROUND_GREEN | FOREGROUND_RED |
FOREGROUND_BLUE); break;
04449         case Color::Red:        DOCTEST_SET_ATTR(FOREGROUND_RED);
break;
04450         case Color::Green:      DOCTEST_SET_ATTR(FOREGROUND_GREEN);
break;
04451         case Color::Blue:       DOCTEST_SET_ATTR(FOREGROUND_BLUE);
break;
04452         case Color::Cyan:       DOCTEST_SET_ATTR(FOREGROUND_BLUE | FOREGROUND_GREEN);
break;
04453         case Color::Yellow:     DOCTEST_SET_ATTR(FOREGROUND_RED | FOREGROUND_GREEN);
break;
04454         case Color::Grey:       DOCTEST_SET_ATTR(0);
break;
04455         case Color::LightGrey:  DOCTEST_SET_ATTR(FOREGROUND_INTENSITY);
break;
04456         case Color::BrightRed:  DOCTEST_SET_ATTR(FOREGROUND_INTENSITY | FOREGROUND_RED);
break;
04457         case Color::BrightGreen: DOCTEST_SET_ATTR(FOREGROUND_INTENSITY | FOREGROUND_GREEN);
break;
04458         case Color::BrightWhite: DOCTEST_SET_ATTR(FOREGROUND_INTENSITY | FOREGROUND_GREEN |
FOREGROUND_RED | FOREGROUND_BLUE); break;
04459         case Color::None:
04460         case Color::Bright: // invalid
04461         default:                DOCTEST_SET_ATTR(ch.origFgAttrs);
04462     }
04463     // clang-format on
04464     #endif // DOCTEST_CONFIG_COLORS_WINDOWS
04465 }
04466 DOCTEST_CLANG_SUPPRESS_WARNING_POP
04467
04468 std::vector<const IExceptionTranslator*> getExceptionTranslators() {
04469     static std::vector<const IExceptionTranslator*> data;
04470     return data;
04471 }
04472
04473 String translateActiveException() {
04474     #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
04475     String res;
04476     auto& translators = getExceptionTranslators();
04477     for(auto& curr : translators)
04478         if(curr->translate(res))
04479             return res;
04480     // clang-format off
04481     DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wcatch-value")
04482     try {
04483         throw;
04484     } catch(std::exception& ex) {
04485         return ex.what();
04486     } catch(std::string& msg) {
04487         return msg.c_str();
04488     } catch(const char* msg) {
04489         return msg;
04490     } catch(...) {
04491         return "unknown exception";
04492     }
04493     DOCTEST_GCC_SUPPRESS_WARNING_POP
04494     // clang-format on
04495     #else // DOCTEST_CONFIG_NO_EXCEPTIONS
04496     return "";
04497     #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
04498 }
04499 } // namespace
04500
04501 namespace detail {

```

```

04502 // used by the macros for registering tests
04503 int regTest(const TestCase& tc) {
04504     getRegisteredTests().insert(tc);
04505     return 0;
04506 }
04507
04508 // sets the current test suite
04509 int setTestSuite(const TestSuite& ts) {
04510     doctest_detail_test_suite_ns::getCurrentTestSuite() = ts;
04511     return 0;
04512 }
04513
04514 #ifdef DOCTEST_IS_DEBUGGER_ACTIVE
04515     bool isDebuggerActive() { return DOCTEST_IS_DEBUGGER_ACTIVE(); }
04516 #else // DOCTEST_IS_DEBUGGER_ACTIVE
04517 #ifdef DOCTEST_PLATFORM_LINUX
04518     class ErrnoGuard {
04519     public:
04520         ErrnoGuard() : m_oldErrno(errno) {}
04521         ~ErrnoGuard() { errno = m_oldErrno; }
04522     private:
04523         int m_oldErrno;
04524     };
04525     // See the comments in Catch2 for the reasoning behind this implementation:
04526     // https://github.com/catchorg/Catch2/blob/v2.13.1/include/internal/catch_debugger.cpp#L79-L102
04527     bool isDebuggerActive() {
04528         ErrnoGuard guard;
04529         std::ifstream in("/proc/self/status");
04530         for(std::string line; std::getline(in, line);) {
04531             static const int PREFIX_LEN = 11;
04532             if(line.compare(0, PREFIX_LEN, "TracerPid:") == 0) {
04533                 return line.length() > PREFIX_LEN && line[PREFIX_LEN] != '0';
04534             }
04535         }
04536         return false;
04537     }
04538 #elif defined(DOCTEST_PLATFORM_MAC)
04539     // The following function is taken directly from the following technical note:
04540     // https://developer.apple.com/library/archive/qa/qa1361/_index.html
04541     // Returns true if the current process is being debugged (either
04542     // running under the debugger or has a debugger attached post facto).
04543     bool isDebuggerActive() {
04544         int mib[4];
04545         kinfo_proc info;
04546         size_t size;
04547         // Initialize the flags so that, if sysctl fails for some bizarre
04548         // reason, we get a predictable result.
04549         info.kp_proc.p_flag = 0;
04550         // Initialize mib, which tells sysctl the info we want, in this case
04551         // we're looking for information about a specific process ID.
04552         mib[0] = CTL_KERN;
04553         mib[1] = KERN_PROC;
04554         mib[2] = KERN_PROC_PID;
04555         mib[3] = getpid();
04556         // Call sysctl.
04557         size = sizeof(info);
04558         if(sysctl(mib, DOCTEST_COUNTOF(mib), &info, &size, 0, 0) != 0) {
04559             std::cerr << "\nCall to sysctl failed - unable to determine if debugger is active **\n";
04560             return false;
04561         }
04562         // We're being debugged if the P_TRACED flag is set.
04563         return ((info.kp_proc.p_flag & P_TRACED) != 0);
04564     }
04565 #elif DOCTEST_MSVC || defined(__MINGW32__) || defined(__MINGW64__)
04566     bool isDebuggerActive() { return ::IsDebuggerPresent() != 0; }
04567 #else
04568     bool isDebuggerActive() { return false; }
04569 #endif // Platform
04570 #endif // DOCTEST_IS_DEBUGGER_ACTIVE
04571
04572 void registerExceptionTranslatorImpl(const IExceptionTranslator* et) {
04573     if(std::find(getExceptionTranslators().begin(), getExceptionTranslators().end(), et) ==
04574        getExceptionTranslators().end())
04575         getExceptionTranslators().push_back(et);
04576 }
04577
04578 DOCTEST_THREAD_LOCAL std::vector<IContextScope*> g_infoContexts; // for logging with INFO()
04579
04580 ContextScopeBase::ContextScopeBase() {
04581     g_infoContexts.push_back(this);
04582 }
04583
04584 ContextScopeBase::ContextScopeBase(ContextScopeBase&& other) noexcept {
04585     if (other.need_to_destroy) {
04586         other.destroy();
04587     }
04588     other.need_to_destroy = false;

```

```

04589         g_infoContexts.push_back(this);
04590     }
04591
04592     DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4996) // std::uncaught_exception is deprecated in C++17
04593     DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wdeprecated-declarations")
04594     DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wdeprecated-declarations")
04595
04596     // destroy cannot be inlined into the destructor because that would mean calling stringify after
04597     // ContextScope has been destroyed (base class destructors run after derived class destructors).
04598     // Instead, ContextScope calls this method directly from its destructor.
04599     void ContextScopeBase::destroy() {
04600 #if defined(__cpp_lib_uncaught_exceptions) && __cpp_lib_uncaught_exceptions >= 201411L &&
    (!defined(__MAC_OS_X_VERSION_MIN_REQUIRED) || __MAC_OS_X_VERSION_MIN_REQUIRED >= 101200)
        if(std::uncaught_exceptions() > 0) {
04601
04602 #else
04603         if(std::uncaught_exception()) {
04604 #endif
04605             std::ostringstream s;
04606             this->stringify(&s);
04607             g_cs->stringifiedContexts.push_back(s.str().c_str());
04608         }
04609         g_infoContexts.pop_back();
04610     }
04611
04612     DOCTEST_CLANG_SUPPRESS_WARNING_POP
04613     DOCTEST_GCC_SUPPRESS_WARNING_POP
04614     DOCTEST_MSVC_SUPPRESS_WARNING_POP
04615 } // namespace detail
04616 namespace {
04617     using namespace detail;
04618
04619 #if !defined(DOCTEST_CONFIG_POSIX_SIGNALS) && !defined(DOCTEST_CONFIG_WINDOWS_SEH)
04620     struct FatalConditionHandler
04621     {
04622         static void reset() {}
04623         static void allocateAltStackMem() {}
04624         static void freeAltStackMem() {}
04625     };
04626 #else // DOCTEST_CONFIG_POSIX_SIGNALS || DOCTEST_CONFIG_WINDOWS_SEH
04627
04628     void reportFatal(const std::string&);
04629
04630 #ifdef DOCTEST_PLATFORM_WINDOWS
04631     struct SignalDefs
04632     {
04633     {
04634         DWORD id;
04635         const char* name;
04636     };
04637     // There is no 1-1 mapping between signals and windows exceptions.
04638     // Windows can easily distinguish between SO and SigSegV,
04639     // but SigInt, SigTerm, etc are handled differently.
04640     SignalDefs signalDefs[] = {
04641         {static_cast<DWORD>(EXCEPTION_ILLEGAL_INSTRUCTION),
04642          "SIGILL - Illegal instruction signal"},
04643         {static_cast<DWORD>(EXCEPTION_STACK_OVERFLOW), "SIGSEGV - Stack overflow"},
04644         {static_cast<DWORD>(EXCEPTION_ACCESS_VIOLATION),
04645          "SIGSEGV - Segmentation violation signal"},
04646         {static_cast<DWORD>(EXCEPTION_INT_DIVIDE_BY_ZERO), "Divide by zero error"},
04647     };
04648
04649     struct FatalConditionHandler
04650     {
04651         static LONG CALLBACK handleException(PEXCEPTION_POINTERS ExceptionInfo) {
04652             // Multiple threads may enter this filter/handler at once. We want the error message to be
04653             // printed on the
04654             // console just once no matter how many threads have crashed.
04655             DOCTEST_DECLARE_STATIC_MUTEX(mutex)
04656             static bool execute = true;
04657             {
04658                 DOCTEST_LOCK_MUTEX(mutex)
04659                 if(execute) {
04660                     bool reported = false;
04661                     for(size_t i = 0; i < DOCTEST_COUNTOF(signalDefs); ++i) {
04662                         if(ExceptionInfo->ExceptionRecord->ExceptionCode == signalDefs[i].id) {
04663                             reportFatal(signalDefs[i].name);
04664                             reported = true;
04665                             break;
04666                         }
04667                     }
04668                     if(reported == false)
04669                         reportFatal("Unhandled SEH exception caught");
04670                     if(isDebuggerActive() && !g_cs->no_breaks)
04671                         DOCTEST_BREAK_INTO_DEBUGGER();
04672                 }
04673                 execute = false;
04674             }
04675         }
04676     };

```

```

04674         std::exit(EXIT_FAILURE);
04675     }
04676
04677     static void allocateAltStackMem() {}
04678     static void freeAltStackMem() {}
04679
04680     FatalConditionHandler() {
04681         isSet = true;
04682         // 32k seems enough for doctest to handle stack overflow,
04683         // but the value was found experimentally, so there is no strong guarantee
04684         guaranteeSize = 32 * 1024;
04685         // Register an unhandled exception filter
04686         previousTop = SetUnhandledExceptionFilter(handleException);
04687         // Pass in guarantee size to be filled
04688         SetThreadStackGuarantee(&guaranteeSize);
04689
04690         // On Windows uncaught exceptions from another thread, exceptions from
04691         // destructors, or calls to std::terminate are not a SEH exception
04692
04693         // The terminal handler gets called when:
04694         // - std::terminate is called FROM THE TEST RUNNER THREAD
04695         // - an exception is thrown from a destructor FROM THE TEST RUNNER THREAD
04696         original_terminate_handler = std::get_terminate();
04697         std::set_terminate([]() DOCTEST_NOEXCEPT {
04698             reportFatal("Terminate handler called");
04699             if(isDebuggerActive() && !g_cs->no_breaks)
04700                 DOCTEST_BREAK_INTO_DEBUGGER();
04701             std::exit(EXIT_FAILURE); // explicitly exit - otherwise the SIGABRT handler may be
called as well
04702         });
04703
04704         // SIGABRT is raised when:
04705         // - std::terminate is called FROM A DIFFERENT THREAD
04706         // - an exception is thrown from a destructor FROM A DIFFERENT THREAD
04707         // - an uncaught exception is thrown FROM A DIFFERENT THREAD
04708         prev_sigabrt_handler = std::signal(SIGABRT, [](int signal) DOCTEST_NOEXCEPT {
04709             if(signal == SIGABRT) {
04710                 reportFatal("SIGABRT - Abort (abnormal termination) signal");
04711                 if(isDebuggerActive() && !g_cs->no_breaks)
04712                     DOCTEST_BREAK_INTO_DEBUGGER();
04713                 std::exit(EXIT_FAILURE);
04714             }
04715         });
04716
04717         // The following settings are taken from google test, and more
04718         // specifically from UnitTest::Run() inside of gtest.cc
04719
04720         // the user does not want to see pop-up dialogs about crashes
04721         prev_error_mode_1 = SetErrorMode(SEM_FAILCRITICALERRORS | SEM_NOALIGNMENTFAULTEXCEPT |
SEM_NOGPFAULTERRORBOX | SEM_NOOPENFILEERRORBOX);
04722
04723         // This forces the abort message to go to stderr in all circumstances.
04724         prev_error_mode_2 = _set_error_mode(_OUT_TO_STDERR);
04725         // In the debug version, Visual Studio pops up a separate dialog
04726         // offering a choice to debug the aborted program - we want to disable that.
04727         prev_abort_behavior = _set_abort_behavior(0x0, _WRITE_ABORT_MSG | _CALL_REPORTFAULT);
04728         // In debug mode, the Windows CRT can crash with an assertion over invalid
04729         // input (e.g. passing an invalid file descriptor). The default handling
04730         // for these assertions is to pop up a dialog and wait for user input.
04731         // Instead ask the CRT to dump such assertions to stderr non-interactively.
04732         prev_report_mode = _CrtSetReportMode(_CRT_ASSERT, _CRTDBG_MODE_FILE | _CRTDBG_MODE_DEBUG);
04733         prev_report_file = _CrtSetReportFile(_CRT_ASSERT, _CRTDBG_FILE_STDERR);
04734     }
04735
04736     static void reset() {
04737         if(isSet) {
04738             // Unregister handler and restore the old guarantee
04739             SetUnhandledExceptionFilter(previousTop);
04740             SetThreadStackGuarantee(&guaranteeSize);
04741             std::set_terminate(original_terminate_handler);
04742             std::signal(SIGABRT, prev_sigabrt_handler);
04743             SetErrorMode(prev_error_mode_1);
04744             _set_error_mode(prev_error_mode_2);
04745             _set_abort_behavior(prev_abort_behavior, _WRITE_ABORT_MSG | _CALL_REPORTFAULT);
04746             static_cast<void>(_CrtSetReportMode(_CRT_ASSERT, prev_report_mode));
04747             static_cast<void>(_CrtSetReportFile(_CRT_ASSERT, prev_report_file));
04748             isSet = false;
04749         }
04750     }
04751
04752     ~FatalConditionHandler() { reset(); }
04753
04754 private:
04755     static UINT          prev_error_mode_1;
04756     static int           prev_error_mode_2;
04757     static unsigned int  prev_abort_behavior;
04758     static int           prev_report_mode;
04759     static _HFILE        prev_report_file;

```

```

04760     static void (DOCTEST_CDECL *prev_sigabrt_handler)(int);
04761     static std::terminate_handler original_terminate_handler;
04762     static bool isSet;
04763     static ULONG guaranteeSize;
04764     static LPTOP_LEVEL_EXCEPTION_FILTER previousTop;
04765 };
04766
04767 UINT      FatalConditionHandler::prev_error_mode_1;
04768 int       FatalConditionHandler::prev_error_mode_2;
04769 unsigned int FatalConditionHandler::prev_abort_behavior;
04770 int       FatalConditionHandler::prev_report_mode;
04771 _HFILE    FatalConditionHandler::prev_report_file;
04772 void (DOCTEST_CDECL *FatalConditionHandler::prev_sigabrt_handler)(int);
04773 std::terminate_handler FatalConditionHandler::original_terminate_handler;
04774 bool FatalConditionHandler::isSet = false;
04775 ULONG FatalConditionHandler::guaranteeSize = 0;
04776 LPTOP_LEVEL_EXCEPTION_FILTER FatalConditionHandler::previousTop = nullptr;
04777
04778 #else // DOCTEST_PLATFORM_WINDOWS
04779
04780 struct SignalDefs
04781 {
04782     int      id;
04783     const char* name;
04784 };
04785 SignalDefs signalDefs[] = {{SIGINT, "SIGINT - Terminal interrupt signal"},
04786                             {SIGILL, "SIGILL - Illegal instruction signal"},
04787                             {SIGFPE, "SIGFPE - Floating point error signal"},
04788                             {SIGSEGV, "SIGSEGV - Segmentation violation signal"},
04789                             {SIGTERM, "SIGTERM - Termination request signal"},
04790                             {SIGABRT, "SIGABRT - Abort (abnormal termination) signal"}};
04791
04792 struct FatalConditionHandler
04793 {
04794     static bool      isSet;
04795     static struct sigaction oldSigActions[DOCTEST_COUNTOF(signalDefs)];
04796     static stack_t  oldSigStack;
04797     static size_t   altStackSize;
04798     static char*    altStackMem;
04799
04800     static void handleSignal(int sig) {
04801         const char* name = "<unknown signal>";
04802         for(std::size_t i = 0; i < DOCTEST_COUNTOF(signalDefs); ++i) {
04803             SignalDefs& def = signalDefs[i];
04804             if(sig == def.id) {
04805                 name = def.name;
04806                 break;
04807             }
04808         }
04809         reset();
04810         reportFatal(name);
04811         raise(sig);
04812     }
04813
04814     static void allocateAltStackMem() {
04815         altStackMem = new char[altStackSize];
04816     }
04817
04818     static void freeAltStackMem() {
04819         delete[] altStackMem;
04820     }
04821
04822     FatalConditionHandler() {
04823         isSet = true;
04824         stack_t sigStack;
04825         sigStack.ss_sp = altStackMem;
04826         sigStack.ss_size = altStackSize;
04827         sigStack.ss_flags = 0;
04828         sigaltstack(&sigStack, &oldSigStack);
04829         struct sigaction sa = {};
04830         sa.sa_handler = handleSignal;
04831         sa.sa_flags = SA_ONSTACK;
04832         for(std::size_t i = 0; i < DOCTEST_COUNTOF(signalDefs); ++i) {
04833             sigaction(signalDefs[i].id, &sa, &oldSigActions[i]);
04834         }
04835     }
04836
04837     ~FatalConditionHandler() { reset(); }
04838     static void reset() {
04839         if(isSet) {
04840             // Set signals back to previous values -- hopefully nobody overwrote them in the
04841             meantime
04842             for(std::size_t i = 0; i < DOCTEST_COUNTOF(signalDefs); ++i) {
04843                 sigaction(signalDefs[i].id, &oldSigActions[i], nullptr);
04844             }
04845             // Return the old stack
04846             sigaltstack(&oldSigStack, nullptr);

```



```

04846         isSet = false;
04847     }
04848 }
04849 };
04850
04851 bool FatalConditionHandler::isSet = false;
04852 struct sigaction FatalConditionHandler::oldSigActions[DOCTEST_COUNTOF(signalDefs)] = {};
04853 stack_t FatalConditionHandler::oldSigStack = {};
04854 size_t FatalConditionHandler::altStackSize = 4 * SIGSTKSZ;
04855 char* FatalConditionHandler::altStackMem = nullptr;
04856
04857 #endif // DOCTEST_PLATFORM_WINDOWS
04858 #endif // DOCTEST_CONFIG_POSIX_SIGNALS || DOCTEST_CONFIG_WINDOWS_SEH
04859
04860 } // namespace
04861
04862 namespace {
04863     using namespace detail;
04864
04865     #ifndef DOCTEST_PLATFORM_WINDOWS
04866     #define DOCTEST_OUTPUT_DEBUG_STRING(text) ::OutputDebugStringA(text)
04867     #else
04868     // TODO: integration with XCode and other IDEs
04869     #define DOCTEST_OUTPUT_DEBUG_STRING(text)
04870     #endif // Platform
04871
04872     void addAssert(assertType::Enum at) {
04873         if((at & assertType::is_warn) == 0)
04874             g_cs->numAssertsCurrentTest_atomic++;
04875     }
04876
04877     void addFailedAssert(assertType::Enum at) {
04878         if((at & assertType::is_warn) == 0)
04879             g_cs->numAssertsFailedCurrentTest_atomic++;
04880     }
04881
04882     #if defined(DOCTEST_CONFIG_POSIX_SIGNALS) || defined(DOCTEST_CONFIG_WINDOWS_SEH)
04883     void reportFatal(const std::string& message) {
04884         g_cs->failure_flags |= TestCaseFailureReason::Crash;
04885
04886         DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_exception, {message.c_str(), true});
04887
04888         while (g_cs->subcaseStack.size()) {
04889             g_cs->subcaseStack.pop_back();
04890             DOCTEST_ITERATE_THROUGH_REPORTERS(subcase_end, DOCTEST_EMPTY);
04891         }
04892
04893         g_cs->finalizeTestCaseData();
04894
04895         DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_end, *g_cs);
04896
04897         DOCTEST_ITERATE_THROUGH_REPORTERS(test_run_end, *g_cs);
04898     }
04899     #endif // DOCTEST_CONFIG_POSIX_SIGNALS || DOCTEST_CONFIG_WINDOWS_SEH
04900 } // namespace
04901
04902 AssertData::AssertData(assertType::Enum at, const char* file, int line, const char* expr,
04903     const char* exception_type, const StringContains& exception_string)
04904     : m_test_case(g_cs->currentTest), m_at(at), m_file(file), m_line(line), m_expr(expr),
04905     m_failed(true), m_threw(false), m_threw_as(false), m_exception_type(exception_type),
04906     m_exception_string(exception_string) {
04907     #if DOCTEST_MSVC
04908     if (m_expr[0] == ' ') // this happens when variadic macros are disabled under MSVC
04909         ++m_expr;
04910     #endif // MSVC
04911 }
04912
04913 namespace detail {
04914     ResultBuilder::ResultBuilder(assertType::Enum at, const char* file, int line, const char* expr,
04915         const char* exception_type, const String& exception_string)
04916         : AssertData(at, file, line, expr, exception_type, exception_string) {}
04917
04918     ResultBuilder::ResultBuilder(assertType::Enum at, const char* file, int line, const char* expr,
04919         const char* exception_type, const Contains& exception_string)
04920         : AssertData(at, file, line, expr, exception_type, exception_string) {}
04921
04922     void ResultBuilder::setResult(const Result& res) {
04923         m_decomp = res.m_decomp;
04924         m_failed = !res.m_passed;
04925     }
04926
04927     void ResultBuilder::translateException() {
04928         m_threw = true;
04929         m_exception = translateActiveException();
04930     }
04931
04932     bool ResultBuilder::log() {

```

```

04933         if(m_at & assertType::is_throws) {
04934             m_failed = !m_threw;
04935         } else if((m_at & assertType::is_throws_as) && (m_at & assertType::is_throws_with)) {
04936             m_failed = !m_threw_as || !m_exception_string.check(m_exception);
04937         } else if(m_at & assertType::is_throws_as) {
04938             m_failed = !m_threw_as;
04939         } else if(m_at & assertType::is_throws_with) {
04940             m_failed = !m_exception_string.check(m_exception);
04941         } else if(m_at & assertType::is_nothrow) {
04942             m_failed = m_threw;
04943         }
04944
04945         if(m_exception.size())
04946             m_exception = "\"" + m_exception + "\"";
04947
04948         if(is_running_in_test) {
04949             addAssert(m_at);
04950             DOCTEST_ITERATE_THROUGH_REPORTERS(log_assert, *this);
04951
04952             if(m_failed)
04953                 addFailedAssert(m_at);
04954         } else if(m_failed) {
04955             failed_out_of_a_testing_context(*this);
04956         }
04957
04958         return m_failed && isDebuggerActive() && !getContextOptions()->no_breaks &&
04959             (g_cs->currentTest == nullptr || !g_cs->currentTest->m_no_breaks); // break into debugger
04960     }
04961
04962     void ResultBuilder::react() const {
04963         if(m_failed && checkIfShouldThrow(m_at))
04964             throwException();
04965     }
04966
04967     void failed_out_of_a_testing_context(const AssertData& ad) {
04968         if(g_cs->ah)
04969             g_cs->ah(ad);
04970         else
04971             std::abort();
04972     }
04973
04974     bool decomp_assert(assertType::Enum at, const char* file, int line, const char* expr,
04975                       const Result& result) {
04976         bool failed = !result.m_passed;
04977
04978         // #####
04979         // IF THE DEBUGGER BREAKS HERE - GO 1 LEVEL UP IN THE CALLSTACK FOR THE FAILING ASSERT
04980         // THIS IS THE EFFECT OF HAVING 'DOCTEST_CONFIG_SUPER_FAST_ASSERTS' DEFINED
04981         // #####
04982         DOCTEST_ASSERT_OUT_OF_TESTS(result.m_decomp);
04983         DOCTEST_ASSERT_IN_TESTS(result.m_decomp);
04984         return !failed;
04985     }
04986
04987     MessageBuilder::MessageBuilder(const char* file, int line, assertType::Enum severity) {
04988         m_stream = tlssPush();
04989         m_file = file;
04990         m_line = line;
04991         m_severity = severity;
04992     }
04993
04994     MessageBuilder::~MessageBuilder() {
04995         if (!logged)
04996             tlssPop();
04997     }
04998
04999     DOCTEST_DEFINE_INTERFACE(IEExceptionTranslator)
05000
05001     bool MessageBuilder::log() {
05002         if (!logged) {
05003             m_string = tlssPop();
05004             logged = true;
05005         }
05006
05007         DOCTEST_ITERATE_THROUGH_REPORTERS(log_message, *this);
05008
05009         const bool isWarn = m_severity & assertType::is_warn;
05010
05011         // warn is just a message in this context so we don't treat it as an assert
05012         if(!isWarn) {
05013             addAssert(m_severity);
05014             addFailedAssert(m_severity);
05015         }
05016
05017         return isDebuggerActive() && !getContextOptions()->no_breaks && !isWarn &&
05018             (g_cs->currentTest == nullptr || !g_cs->currentTest->m_no_breaks); // break into debugger
05019     }

```

```

05020
05021     void MessageBuilder::react() {
05022         if(m_severity & assertType::is_require)
05023             throwException();
05024     }
05025 } // namespace detail
05026 namespace {
05027     using namespace detail;
05028
05029     // clang-format off
05030
05031     // =====
05032     // The following code has been taken verbatim from Catch2/include/internal/catch_xmlwriter.h/cpp
05033     // This is done so cherry-picking bug fixes is trivial - even the style/formatting is untouched.
05034     // =====
05035
05036     class XmlEncode {
05037     public:
05038         enum ForWhat { ForTextNodes, ForAttributes };
05039
05040         XmlEncode( std::string const& str, ForWhat forWhat = ForTextNodes );
05041
05042         void encodeTo( std::ostream& os ) const;
05043
05044         friend std::ostream& operator < ( std::ostream& os, XmlEncode const& xmlEncode );
05045
05046     private:
05047         std::string m_str;
05048         ForWhat m_forWhat;
05049     };
05050
05051     class XmlWriter {
05052     public:
05053
05054         class ScopedElement {
05055         public:
05056             ScopedElement( XmlWriter* writer );
05057
05058             ScopedElement( ScopedElement&& other ) DOCTEST_NOEXCEPT;
05059             ScopedElement& operator=( ScopedElement&& other ) DOCTEST_NOEXCEPT;
05060
05061             ~ScopedElement();
05062
05063             ScopedElement& writeText( std::string const& text, bool indent = true );
05064
05065             template<typename T>
05066             ScopedElement& writeAttribute( std::string const& name, T const& attribute ) {
05067                 m_writer->writeAttribute( name, attribute );
05068                 return *this;
05069             }
05070
05071         private:
05072             mutable XmlWriter* m_writer = nullptr;
05073         };
05074
05075     #ifndef DOCTEST_CONFIG_NO_INCLUDE_Iostream
05076         XmlWriter( std::ostream& os = std::cout );
05077     #else // DOCTEST_CONFIG_NO_INCLUDE_Iostream
05078         XmlWriter( std::ostream& os );
05079     #endif // DOCTEST_CONFIG_NO_INCLUDE_Iostream
05080         ~XmlWriter();
05081
05082         XmlWriter( XmlWriter const& ) = delete;
05083         XmlWriter& operator=( XmlWriter const& ) = delete;
05084
05085         XmlWriter& startElement( std::string const& name );
05086
05087         ScopedElement scopedElement( std::string const& name );
05088
05089         XmlWriter& endElement();
05090
05091         XmlWriter& writeAttribute( std::string const& name, std::string const& attribute );
05092
05093         XmlWriter& writeAttribute( std::string const& name, const char* attribute );
05094
05095         XmlWriter& writeAttribute( std::string const& name, bool attribute );
05096
05097         template<typename T>
05098         XmlWriter& writeAttribute( std::string const& name, T const& attribute ) {
05099             std::stringstream rss;
05100             rss << attribute;
05101             return writeAttribute( name, rss.str() );
05102         }
05103
05104         XmlWriter& writeText( std::string const& text, bool indent = true );
05105
05106         //XmlWriter& writeComment( std::string const& text );

```

```

05107
05108     //void writeStylesheetRef( std::string const& url );
05109
05110     //XmlWriter& writeBlankLine();
05111
05112     void ensureTagClosed();
05113
05114     void writeDeclaration();
05115
05116     private:
05117
05118         void newlineIfNecessary();
05119
05120         bool m_tagIsOpen = false;
05121         bool m_needsNewline = false;
05122         std::vector<std::string> m_tags;
05123         std::string m_indent;
05124         std::ostream& m_os;
05125     };
05126
05127 // =====
05128 // The following code has been taken verbatim from Catch2/include/internal/catch_xmlwriter.h/cpp
05129 // This is done so cherry-picking bug fixes is trivial - even the style/formatting is untouched.
05130 // =====
05131
05132 using uchar = unsigned char;
05133
05134 namespace {
05135
05136     size_t trailingBytes(unsigned char c) {
05137         if ((c & 0xE0) == 0xC0) {
05138             return 2;
05139         }
05140         if ((c & 0xF0) == 0xE0) {
05141             return 3;
05142         }
05143         if ((c & 0xF8) == 0xF0) {
05144             return 4;
05145         }
05146         DOCTEST_INTERNAL_ERROR("Invalid multibyte utf-8 start byte encountered");
05147     }
05148
05149     uint32_t headerValue(unsigned char c) {
05150         if ((c & 0xE0) == 0xC0) {
05151             return c & 0x1F;
05152         }
05153         if ((c & 0xF0) == 0xE0) {
05154             return c & 0x0F;
05155         }
05156         if ((c & 0xF8) == 0xF0) {
05157             return c & 0x07;
05158         }
05159         DOCTEST_INTERNAL_ERROR("Invalid multibyte utf-8 start byte encountered");
05160     }
05161
05162     void hexEscapeChar(std::ostream& os, unsigned char c) {
05163         std::ios_base::fmtflags f(os.flags());
05164         os << "\\x"
05165             << std::uppercase << std::hex << std::setfill('0') << std::setw(2)
05166             << static_cast<int>(c);
05167         os.flags(f);
05168     }
05169 } // anonymous namespace
05170
05171 XmlEncode::XmlEncode( std::string const& str, ForWhat forWhat )
05172 :   m_str( str ),
05173   m_forWhat( forWhat )
05174 {}
05175
05176 void XmlEncode::encodeTo( std::ostream& os ) const {
05177     // Apostrophe escaping not necessary if we always use " to write attributes
05178     // (see: https://www.w3.org/TR/xml/#syntax)
05179
05180     for( std::size_t idx = 0; idx < m_str.size(); ++ idx ) {
05181         uchar c = m_str[idx];
05182         switch (c) {
05183             case '<': os << "&lt;"; break;
05184             case '&': os << "&amp;"; break;
05185
05186             case '>':
05187                 // See: https://www.w3.org/TR/xml/#syntax
05188                 if (idx > 2 && m_str[idx - 1] == '[' && m_str[idx - 2] == ']')
05189                     os << "&gt;";
05190                 else
05191                     os << c;
05192             break;
05193         }
05194     }

```

```

05194
05195         case '\\':
05196             if (m_forWhat == ForAttributes)
05197                 os << "&quot;";
05198             else
05199                 os << c;
05200             break;
05201
05202         default:
05203             // Check for control characters and invalid utf-8
05204
05205             // Escape control characters in standard ascii
05206             // see
https://stackoverflow.com/questions/404107/why-are-control-characters-illegal-in-xml-1-0
05207             if (c < 0x09 || (c > 0x0D && c < 0x20) || c == 0x7F) {
05208                 hexEscapeChar(os, c);
05209                 break;
05210             }
05211
05212             // Plain ASCII: Write it to stream
05213             if (c < 0x7F) {
05214                 os << c;
05215                 break;
05216             }
05217
05218             // UTF-8 territory
05219             // Check if the encoding is valid and if it is not, hex escape bytes.
05220             // Important: We do not check the exact decoded values for validity, only the encoding
05221
05222             format
05223             // First check that this bytes is a valid lead byte:
05224             // This means that it is not encoded as 1111 1XXX
05225             // Or as 10XX XXXX
05226             if (c < 0xC0 ||
05227                 c >= 0xF8) {
05228                 hexEscapeChar(os, c);
05229                 break;
05230             }
05231
05232             auto encBytes = trailingBytes(c);
05233             // Are there enough bytes left to avoid accessing out-of-bounds memory?
05234             if (idx + encBytes - 1 >= m_str.size()) {
05235                 hexEscapeChar(os, c);
05236                 break;
05237             }
05238             // The header is valid, check data
05239             // The next encBytes bytes must together be a valid utf-8
05240             // This means: bitpattern 10XX XXXX and the extracted value is sane (ish)
05241             bool valid = true;
05242             uint32_t value = headerValue(c);
05243             for (std::size_t n = 1; n < encBytes; ++n) {
05244                 uchar nc = m_str[idx + n];
05245                 valid &= ((nc & 0xC0) == 0x80);
05246                 value = (value << 6) | (nc & 0x3F);
05247             }
05248             if (
05249                 // Wrong bit pattern of following bytes
05250                 (!valid) ||
05251                 // Overlong encodings
05252                 (value < 0x80) ||
05253                 (
05254                     value < 0x800 && encBytes > 2) || // removed "0x80 <= value
05255                     &&" because redundant
05256                     (0x800 < value && value < 0x10000 && encBytes > 3) ||
05257                     // Encoded value out of range
05258                     (value >= 0x110000)
05259                 ) {
05260                 hexEscapeChar(os, c);
05261                 break;
05262             }
05263
05264             // If we got here, this is in fact a valid(ish) utf-8 sequence
05265             for (std::size_t n = 0; n < encBytes; ++n) {
05266                 os << m_str[idx + n];
05267             }
05268             idx += encBytes - 1;
05269             break;
05270         }
05271     }
05272
05273     std::ostream& operator << ( std::ostream& os, XmlEncode const& xmlEncode ) {
05274         xmlEncode.encodeTo( os );
05275         return os;
05276     }
05277
05278     XmlWriter::ScopedElement::ScopedElement( XmlWriter* writer )
05279     : m_writer( writer )

```

```

05278     {}
05279
05280     XmlWriter::ScopedElement::ScopedElement( ScopedElement&& other ) DOCTEST_NOEXCEPT
05281     :   m_writer( other.m_writer ){
05282         other.m_writer = nullptr;
05283     }
05284     XmlWriter& XmlWriter::ScopedElement::operator=( ScopedElement&& other )
DOCTEST_NOEXCEPT {
05285         if ( m_writer ) {
05286             m_writer->endElement();
05287         }
05288         m_writer = other.m_writer;
05289         other.m_writer = nullptr;
05290         return *this;
05291     }
05292
05293
05294     XmlWriter::ScopedElement::~ScopedElement() {
05295         if( m_writer )
05296             m_writer->endElement();
05297     }
05298
05299     XmlWriter& XmlWriter::ScopedElement::writeText( std::string const& text, bool
indent ) {
05300         m_writer->writeText( text, indent );
05301         return *this;
05302     }
05303
05304     XmlWriter::XmlWriter( std::ostream& os ) : m_os( os )
05305     {
05306         // writeDeclaration(); // called explicitly by the reporters that use the writer class - see
issue #627
05307     }
05308
05309     XmlWriter::~XmlWriter() {
05310         while( !m_tags.empty() )
05311             endElement();
05312     }
05313
05314     XmlWriter& XmlWriter::startElement( std::string const& name ) {
05315         ensureTagClosed();
05316         newlineIfNecessary();
05317         m_os << m_indent << '<' << name;
05318         m_tags.push_back( name );
05319         m_indent += " ";
05320         m_tagIsOpen = true;
05321         return *this;
05322     }
05323
05324     XmlWriter::ScopedElement XmlWriter::scopedElement( std::string const& name ) {
05325         ScopedElement scoped( this );
05326         startElement( name );
05327         return scoped;
05328     }
05329
05330     XmlWriter& XmlWriter::endElement() {
05331         newlineIfNecessary();
05332         m_indent = m_indent.substr( 0, m_indent.size()-2 );
05333         if( m_tagIsOpen ) {
05334             m_os << ">";
05335             m_tagIsOpen = false;
05336         }
05337         else {
05338             m_os << m_indent << "</" << m_tags.back() << ">";
05339         }
05340         m_os << std::endl;
05341         m_tags.pop_back();
05342         return *this;
05343     }
05344
05345     XmlWriter& XmlWriter::writeAttribute( std::string const& name, std::string const& attribute ) {
05346         if( !name.empty() && !attribute.empty() )
05347             m_os << ' ' << name << "=" << XmlEncode( attribute, XmlEncode::ForAttributes ) << ' ';
05348         return *this;
05349     }
05350
05351     XmlWriter& XmlWriter::writeAttribute( std::string const& name, const char* attribute ) {
05352         if( !name.empty() && attribute && attribute[0] != '\0' )
05353             m_os << ' ' << name << "=" << XmlEncode( attribute, XmlEncode::ForAttributes ) << ' ';
05354         return *this;
05355     }
05356
05357     XmlWriter& XmlWriter::writeAttribute( std::string const& name, bool attribute ) {
05358         m_os << ' ' << name << "=" << ( attribute ? "true" : "false" ) << ' ';
05359         return *this;
05360     }
05361

```

```

05362     XmlWriter& XmlWriter::writeText( std::string const& text, bool indent ) {
05363         if( !text.empty() ){
05364             bool tagWasOpen = m_tagIsOpen;
05365             ensureTagClosed();
05366             if( tagWasOpen && indent )
05367                 m_os << m_indent;
05368             m_os << XmlEncode( text );
05369             m_needsNewline = true;
05370         }
05371         return *this;
05372     }
05373
05374     //XmlWriter& XmlWriter::writeComment( std::string const& text ) {
05375     //     ensureTagClosed();
05376     //     m_os << m_indent << "<!--" << text << "-->";
05377     //     m_needsNewline = true;
05378     //     return *this;
05379     //}
05380
05381     //void XmlWriter::writeStylesheetRef( std::string const& url ) {
05382     //     m_os << "<?xml-stylesheet type='text/xsl' href='" << url << "\"?>\n";
05383     //}
05384
05385     //XmlWriter& XmlWriter::writeBlankLine() {
05386     //     ensureTagClosed();
05387     //     m_os << '\n';
05388     //     return *this;
05389     //}
05390
05391     void XmlWriter::ensureTagClosed() {
05392         if( m_tagIsOpen ) {
05393             m_os << ">" << std::endl;
05394             m_tagIsOpen = false;
05395         }
05396     }
05397
05398     void XmlWriter::writeDeclaration() {
05399         m_os << "<?xml version='1.0' encoding='UTF-8'>\n";
05400     }
05401
05402     void XmlWriter::newlineIfNecessary() {
05403         if( m_needsNewline ) {
05404             m_os << std::endl;
05405             m_needsNewline = false;
05406         }
05407     }
05408
05409     // =====
05410     // End of copy-pasted code from Catch
05411     // =====
05412
05413     // clang-format on
05414
05415     struct XmlReporter : public IReporter
05416     {
05417         XmlWriter xml;
05418         DOCTEST_DECLARE_MUTEX(mutex)
05419
05420         // caching pointers/references to objects of these types - safe to do
05421         const ContextOptions& opt;
05422         const TestCaseData* tc = nullptr;
05423
05424         XmlReporter(const ContextOptions& co)
05425             : xml(*co.cout)
05426             , opt(co) {}
05427
05428         void log_contexts() {
05429             int num_contexts = get_num_active_contexts();
05430             if(num_contexts) {
05431                 auto contexts = get_active_contexts();
05432                 std::stringstream ss;
05433                 for(int i = 0; i < num_contexts; ++i) {
05434                     contexts[i]->stringify(&ss);
05435                     xml.scopedElement("Info").writeText(ss.str());
05436                     ss.str("");
05437                 }
05438             }
05439         }
05440
05441         unsigned line(unsigned l) const { return opt.no_line_numbers ? 0 : l; }
05442
05443         void test_case_start_impl(const TestCaseData& in) {
05444             bool open_ts_tag = false;
05445             if(tc != nullptr) { // we have already opened a test suite
05446                 if(std::strcmp(tc->m_test_suite, in.m_test_suite) != 0) {
05447                     xml.endElement();
05448                     open_ts_tag = true;

```

```

05449     }
05450     }
05451     else {
05452         open_ts_tag = true; // first test case ==> first test suite
05453     }
05454
05455     if(open_ts_tag) {
05456         xml.startElement("TestSuite");
05457         xml.writeAttribute("name", in.m_test_suite);
05458     }
05459
05460     tc = &in;
05461     xml.startElement("TestCase")
05462         .writeAttribute("name", in.m_name)
05463         .writeAttribute("filename", skipPathFromFilename(in.m_file.c_str()))
05464         .writeAttribute("line", line(in.m_line))
05465         .writeAttribute("description", in.m_description);
05466
05467     if(Approx(in.m_timeout) != 0)
05468         xml.writeAttribute("timeout", in.m_timeout);
05469     if(in.m_may_fail)
05470         xml.writeAttribute("may_fail", true);
05471     if(in.m_should_fail)
05472         xml.writeAttribute("should_fail", true);
05473 }
05474
05475 // =====
05476 // WHAT FOLLOWS ARE OVERRIDES OF THE VIRTUAL METHODS OF THE REPORTER INTERFACE
05477 // =====
05478
05479 void report_query(const QueryData& in) override {
05480     test_run_start();
05481     if(opt.list_reporters) {
05482         for(auto& curr : getListeners())
05483             xml.scopedElement("Listener")
05484                 .writeAttribute("priority", curr.first.first)
05485                 .writeAttribute("name", curr.first.second);
05486         for(auto& curr : getReporters())
05487             xml.scopedElement("Reporter")
05488                 .writeAttribute("priority", curr.first.first)
05489                 .writeAttribute("name", curr.first.second);
05490     } else if(opt.count || opt.list_test_cases) {
05491         for(unsigned i = 0; i < in.num_data; ++i) {
05492             xml.scopedElement("TestCase").writeAttribute("name", in.data[i]->m_name)
05493                 .writeAttribute("testsuite", in.data[i]->m_test_suite)
05494                 .writeAttribute("filename", skipPathFromFilename(in.data[i]->m_file.c_str()))
05495                 .writeAttribute("line", line(in.data[i]->m_line))
05496                 .writeAttribute("skipped", in.data[i]->m_skip);
05497         }
05498         xml.scopedElement("OverallResultsTestCases")
05499             .writeAttribute("unskipped", in.run_stats->numTestCasesPassingFilters);
05500     } else if(opt.list_test_suites) {
05501         for(unsigned i = 0; i < in.num_data; ++i)
05502             xml.scopedElement("TestSuite").writeAttribute("name", in.data[i]->m_test_suite);
05503         xml.scopedElement("OverallResultsTestCases")
05504             .writeAttribute("unskipped", in.run_stats->numTestCasesPassingFilters);
05505         xml.scopedElement("OverallResultsTestSuites")
05506             .writeAttribute("unskipped", in.run_stats->numTestSuitesPassingFilters);
05507     }
05508     xml.endElement();
05509 }
05510
05511 void test_run_start() override {
05512     xml.writeDeclaration();
05513
05514     // remove .exe extension - mainly to have the same output on UNIX and Windows
05515     std::string binary_name = skipPathFromFilename(opt.binary_name.c_str());
05516 #ifdef DOCTEST_PLATFORM_WINDOWS
05517     if(binary_name.rfind(".exe") != std::string::npos)
05518         binary_name = binary_name.substr(0, binary_name.length() - 4);
05519 #endif // DOCTEST_PLATFORM_WINDOWS
05520
05521     xml.startElement("doctest").writeAttribute("binary", binary_name);
05522     if(opt.no_version == false)
05523         xml.writeAttribute("version", DOCTEST_VERSION_STR);
05524
05525     // only the consequential ones (TODO: filters)
05526     xml.scopedElement("Options")
05527         .writeAttribute("order_by", opt.order_by.c_str())
05528         .writeAttribute("rand_seed", opt.rand_seed)
05529         .writeAttribute("first", opt.first)
05530         .writeAttribute("last", opt.last)
05531         .writeAttribute("abort_after", opt.abort_after)
05532         .writeAttribute("subcase_filter_levels", opt.subcase_filter_levels)
05533         .writeAttribute("case_sensitive", opt.case_sensitive)
05534         .writeAttribute("no_throw", opt.no_throw)
05535         .writeAttribute("no_skip", opt.no_skip);

```



```

05536     }
05537
05538     void test_run_end(const TestRunStats& p) override {
05539         if(tc) // the TestSuite tag - only if there has been at least 1 test case
05540             xml.endElement();
05541
05542         xml.scopedElement("OverallResultsAsserts")
05543             .writeAttribute("successes", p.numAsserts - p.numAssertsFailed)
05544             .writeAttribute("failures", p.numAssertsFailed);
05545
05546         xml.startElement("OverallResultsTestCases")
05547             .writeAttribute("successes",
05548                 p.numTestCasesPassingFilters - p.numTestCasesFailed)
05549             .writeAttribute("failures", p.numTestCasesFailed);
05550         if(opt.no_skipped_summary == false)
05551             xml.writeAttribute("skipped", p.numTestCases - p.numTestCasesPassingFilters);
05552         xml.endElement();
05553
05554         xml.endElement();
05555     }
05556
05557     void test_case_start(const TestCaseData& in) override {
05558         test_case_start_impl(in);
05559         xml.ensureTagClosed();
05560     }
05561
05562     void test_case_reenter(const TestCaseData&) override {}
05563
05564     void test_case_end(const CurrentTestCaseStats& st) override {
05565         xml.startElement("OverallResultsAsserts")
05566             .writeAttribute("successes",
05567                 st.numAssertsCurrentTest - st.numAssertsFailedCurrentTest)
05568             .writeAttribute("failures", st.numAssertsFailedCurrentTest)
05569             .writeAttribute("test_case_success", st.testCaseSuccess);
05570         if(opt.duration)
05571             xml.writeAttribute("duration", st.seconds);
05572         if(tc->m_expected_failures)
05573             xml.writeAttribute("expected_failures", tc->m_expected_failures);
05574         xml.endElement();
05575
05576         xml.endElement();
05577     }
05578
05579     void test_case_exception(const TestCaseException& e) override {
05580         DOCTEST_LOCK_MUTEX(mutex)
05581
05582         xml.scopedElement("Exception")
05583             .writeAttribute("crash", e.is_crash)
05584             .writeText(e.error_string.c_str());
05585     }
05586
05587     void subcase_start(const SubcaseSignature& in) override {
05588         xml.startElement("SubCase")
05589             .writeAttribute("name", in.m_name)
05590             .writeAttribute("filename", skipPathFromFilename(in.m_file))
05591             .writeAttribute("line", line(in.m_line));
05592         xml.ensureTagClosed();
05593     }
05594
05595     void subcase_end() override { xml.endElement(); }
05596
05597     void log_assert(const AssertData& rb) override {
05598         if(!rb.m_failed && !opt.success)
05599             return;
05600
05601         DOCTEST_LOCK_MUTEX(mutex)
05602
05603         xml.startElement("Expression")
05604             .writeAttribute("success", !rb.m_failed)
05605             .writeAttribute("type", assertString(rb.m_at))
05606             .writeAttribute("filename", skipPathFromFilename(rb.m_file))
05607             .writeAttribute("line", line(rb.m_line));
05608
05609         xml.scopedElement("Original").writeText(rb.m_expr);
05610
05611         if(rb.m_threw)
05612             xml.scopedElement("Exception").writeText(rb.m_exception.c_str());
05613
05614         if(rb.m_at & assertType::is_throws_as)
05615             xml.scopedElement("ExpectedException").writeText(rb.m_exception_type);
05616         if(rb.m_at & assertType::is_throws_with)
05617             xml.scopedElement("ExpectedExceptionString").writeText(rb.m_exception_string.c_str());
05618         if((rb.m_at & assertType::is_normal) && !rb.m_threw)
05619             xml.scopedElement("Expanded").writeText(rb.m_decomp.c_str());
05620
05621         log_contexts();
05622     }

```

```

05623         xml.endElement();
05624     }
05625
05626     void log_message(const MessageData& mb) override {
05627         DOCTEST_LOCK_MUTEX(mutex)
05628
05629         xml.startElement("Message")
05630             .writeAttribute("type", failureString(mb.m_severity))
05631             .writeAttribute("filename", skipPathFromFilename(mb.m_file))
05632             .writeAttribute("line", line(mb.m_line));
05633
05634         xml.scopedElement("Text").writeText(mb.m_string.c_str());
05635
05636         log_contexts();
05637
05638         xml.endElement();
05639     }
05640
05641     void test_case_skipped(const TestCaseData& in) override {
05642         if(opt.no_skipped_summary == false) {
05643             test_case_start_impl(in);
05644             xml.writeAttribute("skipped", "true");
05645             xml.endElement();
05646         }
05647     }
05648 };
05649
05650 DOCTEST_REGISTER_REPORTER("xml", 0, XmlReporter);
05651
05652 void fulltext_log_assert_to_stream(std::ostream& s, const AssertData& rb) {
05653     if((rb.m_at & (assertType::is_throws_as | assertType::is_throws_with)) ==
05654         0)
05655         s << Color::Cyan << assertString(rb.m_at) << "( " << rb.m_expr << " ) "
05656         << Color::None;
05657
05658     if(rb.m_at & assertType::is_throws) {
05659         s << (rb.m_threw ? "threw as expected!" : "did NOT throw at all!") << "\n";
05660     } else if((rb.m_at & assertType::is_throws_as) &&
05661         (rb.m_at & assertType::is_throws_with)) {
05662         s << Color::Cyan << assertString(rb.m_at) << "( " << rb.m_expr << ", \"
05663         << rb.m_exception_string.c_str()
05664         << "\", " << rb.m_exception_type << " ) " << Color::None;
05665         if(rb.m_threw) {
05666             if(!rb.m_failed) {
05667                 s << "threw as expected!\n";
05668             } else {
05669                 s << "threw a DIFFERENT exception! (contents: " << rb.m_exception << ")\n";
05670             }
05671         } else {
05672             s << "did NOT throw at all!\n";
05673         }
05674     } else if(rb.m_at &
05675         assertType::is_throws_as) {
05676         s << Color::Cyan << assertString(rb.m_at) << "( " << rb.m_expr << ", "
05677         << rb.m_exception_type << " ) " << Color::None
05678         << (rb.m_threw ? (rb.m_threw_as ? "threw as expected!" :
05679             "threw a DIFFERENT exception: ") :
05680             "did NOT throw at all!")
05681         << Color::Cyan << rb.m_exception << "\n";
05682     } else if(rb.m_at &
05683         assertType::is_throws_with) {
05684         s << Color::Cyan << assertString(rb.m_at) << "( " << rb.m_expr << ", \"
05685         << rb.m_exception_string.c_str()
05686         << "\" ) " << Color::None
05687         << (rb.m_threw ? (!rb.m_failed ? "threw as expected!" :
05688             "threw a DIFFERENT exception: ") :
05689             "did NOT throw at all!")
05690         << Color::Cyan << rb.m_exception << "\n";
05691     } else if(rb.m_at & assertType::is_nothrow) {
05692         s << (rb.m_threw ? "THREW exception: " : "didn't throw!") << Color::Cyan
05693         << rb.m_exception << "\n";
05694     } else {
05695         s << (rb.m_threw ? "THREW exception: " :
05696             (!rb.m_failed ? "is correct!\n" : "is NOT correct!\n"));
05697         if(rb.m_threw)
05698             s << rb.m_exception << "\n";
05699         else
05700             s << " values: " << assertString(rb.m_at) << "( " << rb.m_decomp << " )\n";
05701     }
05702 }
05703
05704 // TODO:
05705 // - log_message()
05706 // - respond to queries
05707 // - honor remaining options
05708 // - more attributes in tags
05709 struct JUnitReporter : public IReporter

```

```

05710     {
05711         XmlWriter xml;
05712         DOCTEST_DECLARE_MUTEX(mutex)
05713         Timer timer;
05714         std::vector<String> deepestSubcaseStackNames;
05715
05716         struct JUnitTestCaseData
05717         {
05718             static std::string getCurrentTimestamp() {
05719                 // Beware, this is not reentrant because of backward compatibility issues
05720                 // Also, UTC only, again because of backward compatibility (%z is C++11)
05721                 time_t rawtime;
05722                 std::time(&rawtime);
05723                 auto const timeStampSize = sizeof("2017-01-16T17:06:45Z");
05724
05725                 std::tm timeInfo;
05726 #ifdef DOCTEST_PLATFORM_WINDOWS
05727                 gmtime_s(&timeInfo, &rawtime);
05728 #else // DOCTEST_PLATFORM_WINDOWS
05729                 gmtime_r(&rawtime, &timeInfo);
05730 #endif // DOCTEST_PLATFORM_WINDOWS
05731
05732                 char timeStamp[timeStampSize];
05733                 const char* const fmt = "%Y-%m-%dT%H:%M:%SZ";
05734
05735                 std::strftime(timeStamp, timeStampSize, fmt, &timeInfo);
05736                 return std::string(timeStamp);
05737             }
05738
05739             struct JUnitTestMessage
05740             {
05741                 JUnitTestMessage(const std::string& _message, const std::string& _type, const
std::string& _details)
05742                     : message(_message), type(_type), details(_details) {}
05743
05744                 JUnitTestMessage(const std::string& _message, const std::string& _details)
05745                     : message(_message), type(), details(_details) {}
05746
05747                 std::string message, type, details;
05748             };
05749
05750             struct JUnitTestCase
05751             {
05752                 JUnitTestCase(const std::string& _classname, const std::string& _name)
05753                     : classname(_classname), name(_name), time(0), failures() {}
05754
05755                 std::string classname, name;
05756                 double time;
05757                 std::vector<JUnitTestMessage> failures, errors;
05758             };
05759
05760             void add(const std::string& classname, const std::string& name) {
05761                 testcases.emplace_back(classname, name);
05762             }
05763
05764             void appendSubcaseNamesToLastTestcase(std::vector<String> nameStack) {
05765                 for(auto& curr: nameStack)
05766                     if(curr.size())
05767                         testcases.back().name += std::string("/") + curr.c_str();
05768             }
05769
05770             void addTime(double time) {
05771                 if(time < 1e-4)
05772                     time = 0;
05773                 testcases.back().time = time;
05774                 totalSeconds += time;
05775             }
05776
05777             void addFailure(const std::string& message, const std::string& type, const std::string&
details) {
05778                 testcases.back().failures.emplace_back(message, type, details);
05779                 ++totalFailures;
05780             }
05781
05782             void addError(const std::string& message, const std::string& details) {
05783                 testcases.back().errors.emplace_back(message, details);
05784                 ++totalErrors;
05785             }
05786
05787             std::vector<JUnitTestCase> testcases;
05788             double totalSeconds = 0;
05789             int totalErrors = 0, totalFailures = 0;
05790         };
05791
05792         JUnitTestCaseData testCaseData;
05793
05794         // caching pointers/references to objects of these types - safe to do

```

```

05795     const ContextOptions& opt;
05796     const TestCaseData*   tc = nullptr;
05797
05798     JUnitReporter(const ContextOptions& co)
05799         : xml(*co.cout)
05800         , opt(co) {}
05801
05802     unsigned line(unsigned l) const { return opt.no_line_numbers ? 0 : l; }
05803
05804     // =====
05805     // WHAT FOLLOWS ARE OVERRIDES OF THE VIRTUAL METHODS OF THE REPORTER INTERFACE
05806     // =====
05807
05808     void report_query(const QueryData&) override {
05809         xml.writeDeclaration();
05810     }
05811
05812     void test_run_start() override {
05813         xml.writeDeclaration();
05814     }
05815
05816     void test_run_end(const TestRunStats& p) override {
05817         // remove .exe extension - mainly to have the same output on UNIX and Windows
05818         std::string binary_name = skipPathFromFilename(opt.binary_name.c_str());
05819 #ifndef DOCTEST_PLATFORM_WINDOWS
05820         if(binary_name.rfind(".exe") != std::string::npos)
05821             binary_name = binary_name.substr(0, binary_name.length() - 4);
05822 #endif // DOCTEST_PLATFORM_WINDOWS
05823         xml.startElement("testsuites");
05824         xml.startElement("testsuite").writeAttribute("name", binary_name)
05825             .writeAttribute("errors", testCaseData.totalErrors)
05826             .writeAttribute("failures", testCaseData.totalFailures)
05827             .writeAttribute("tests", p.numAsserts);
05828         if(opt.no_time_in_output == false) {
05829             xml.writeAttribute("time", testCaseData.totalSeconds);
05830             xml.writeAttribute("timestamp", JUnitTestCaseData::getCurrentTimestamp());
05831         }
05832         if(opt.no_version == false)
05833             xml.writeAttribute("doctest_version", DOCTEST_VERSION_STR);
05834
05835         for(const auto& testCase : testCaseData.testcases) {
05836             xml.startElement("testcase")
05837                 .writeAttribute("classname", testCase.classname)
05838                 .writeAttribute("name", testCase.name);
05839             if(opt.no_time_in_output == false)
05840                 xml.writeAttribute("time", testCase.time);
05841             // This is not ideal, but it should be enough to mimic gtest's junit output.
05842             xml.writeAttribute("status", "run");
05843
05844             for(const auto& failure : testCase.failures) {
05845                 xml.scopedElement("failure")
05846                     .writeAttribute("message", failure.message)
05847                     .writeAttribute("type", failure.type)
05848                     .writeText(failure.details, false);
05849             }
05850
05851             for(const auto& error : testCase.errors) {
05852                 xml.scopedElement("error")
05853                     .writeAttribute("message", error.message)
05854                     .writeText(error.details);
05855             }
05856
05857             xml.endElement();
05858         }
05859         xml.endElement();
05860     }
05861
05862     void test_case_start(const TestCaseData& in) override {
05863         testCaseData.add(skipPathFromFilename(in.m_file.c_str()), in.m_name);
05864         timer.start();
05865     }
05866
05867     void test_case_reenter(const TestCaseData& in) override {
05868         testCaseData.addTime(timer.getElapsedSeconds());
05869         testCaseData.appendSubcaseNamesToLastTestcase(deepestSubcaseStackNames);
05870         deepestSubcaseStackNames.clear();
05871
05872         timer.start();
05873         testCaseData.add(skipPathFromFilename(in.m_file.c_str()), in.m_name);
05874     }
05875
05876     void test_case_end(const CurrentTestCaseStats&) override {
05877         testCaseData.addTime(timer.getElapsedSeconds());
05878         testCaseData.appendSubcaseNamesToLastTestcase(deepestSubcaseStackNames);
05879         deepestSubcaseStackNames.clear();
05880     }
05881

```

```

05882
05883     void test_case_exception(const TestCaseException& e) override {
05884         DOCTEST_LOCK_MUTEX(mutex)
05885         testCaseData.addError("exception", e.error_string.c_str());
05886     }
05887
05888     void subcase_start(const SubcaseSignature& in) override {
05889         deepestSubcaseStackNames.push_back(in.m_name);
05890     }
05891
05892     void subcase_end() override {}
05893
05894     void log_assert(const AssertData& rb) override {
05895         if(!rb.m_failed) // report only failures & ignore the `success` option
05896             return;
05897
05898         DOCTEST_LOCK_MUTEX(mutex)
05899
05900         std::ostringstream os;
05901         os << skipPathFromFilename(rb.m_file) << (opt.gnu_file_line ? ":" : "(")
05902         << line(rb.m_line) << (opt.gnu_file_line ? ":" : ")") << std::endl;
05903
05904         fulltext_log_assert_to_stream(os, rb);
05905         log_contexts(os);
05906         testCaseData.addFailure(rb.m_decomp.c_str(), assertString(rb.m_at), os.str());
05907     }
05908
05909     void log_message(const MessageData& mb) override {
05910         if(mb.m_severity & assertType::is_warn) // report only failures
05911             return;
05912
05913         DOCTEST_LOCK_MUTEX(mutex)
05914
05915         std::ostringstream os;
05916         os << skipPathFromFilename(mb.m_file) << (opt.gnu_file_line ? ":" : "(")
05917         << line(mb.m_line) << (opt.gnu_file_line ? ":" : ")") << std::endl;
05918
05919         os << mb.m_string.c_str() << "\n";
05920         log_contexts(os);
05921
05922         testCaseData.addFailure(mb.m_string.c_str(),
05923             mb.m_severity & assertType::is_check ? "FAIL_CHECK" : "FAIL", os.str());
05924     }
05925
05926     void test_case_skipped(const TestCaseData&) override {}
05927
05928     void log_contexts(std::ostream& s) {
05929         int num_contexts = get_num_active_contexts();
05930         if(num_contexts) {
05931             auto contexts = get_active_contexts();
05932
05933             s << "  logged: ";
05934             for(int i = 0; i < num_contexts; ++i) {
05935                 s << (i == 0 ? "" : " ");
05936                 contexts[i]->stringify(&s);
05937                 s << std::endl;
05938             }
05939         }
05940     }
05941 };
05942
05943 DOCTEST_REGISTER_REPORTER("junit", 0, JUnitReporter);
05944
05945 struct Whitespace
05946 {
05947     int nrSpaces;
05948     explicit Whitespace(int nr)
05949         : nrSpaces(nr) {}
05950 };
05951
05952 std::ostream& operator<<(std::ostream& out, const Whitespace& ws) {
05953     if(ws.nrSpaces != 0)
05954         out << std::setw(ws.nrSpaces) << ' ';
05955     return out;
05956 }
05957
05958 struct ConsoleReporter : public IReporter
05959 {
05960     std::ostream& s;
05961     bool hasLoggedCurrentTestStart;
05962     std::vector<SubcaseSignature> subcasesStack;
05963     size_t currentSubcaseLevel;
05964     DOCTEST_DECLARE_MUTEX(mutex)
05965
05966     // caching pointers/references to objects of these types - safe to do
05967     const ContextOptions& opt;
05968     const TestCaseData* tc;

```

```

05969
05970 ConsoleReporter(const ContextOptions& co)
05971     : s(*co.cout)
05972     , opt(co) {}
05973
05974 ConsoleReporter(const ContextOptions& co, std::ostream& ostr)
05975     : s(ostr)
05976     , opt(co) {}
05977
05978 // =====
05979 // WHAT FOLLOWS ARE HELPERS USED BY THE OVERRIDES OF THE VIRTUAL METHODS OF THE INTERFACE
05980 // =====
05981
05982 void separator_to_stream() {
05983     s << Color::Yellow
05984     << "=====
05985     << "\n";
05986 }
05987
05988 const char* getSuccessOrFailString(bool success, assertType::Enum at,
05989                                     const char* success_str) {
05990     if(success)
05991         return success_str;
05992     return failureString(at);
05993 }
05994
05995 Color::Enum getSuccessOrFailColor(bool success, assertType::Enum at) {
05996     return success ? Color::BrightGreen :
05997         (at & assertType::is_warn) ? Color::Yellow : Color::Red;
05998 }
05999
06000 void successOrFailColoredStringToStream(bool success, assertType::Enum at,
06001                                         const char* success_str = "SUCCESS") {
06002     s << getSuccessOrFailColor(success, at)
06003     << getSuccessOrFailString(success, at, success_str) << ": ";
06004 }
06005
06006 void log_contexts() {
06007     int num_contexts = get_num_active_contexts();
06008     if(num_contexts) {
06009         auto contexts = get_active_contexts();
06010
06011         s << Color::None << " logged: ";
06012         for(int i = 0; i < num_contexts; ++i) {
06013             s << (i == 0 ? "" : " ");
06014             contexts[i]->stringify(&s);
06015             s << "\n";
06016         }
06017     }
06018
06019     s << "\n";
06020 }
06021
06022 // this was requested to be made virtual so users could override it
06023 virtual void file_line_to_stream(const char* file, int line,
06024                                 const char* tail = "") {
06025     s << Color::LightGrey << skipPathFromFilename(file) << (opt.gnu_file_line ? ":" : "(")
06026     << (opt.no_line_numbers ? 0 : line) // 0 or the real num depending on the option
06027     << (opt.gnu_file_line ? ":" : ")") << tail;
06028 }
06029
06030 void logTestStart() {
06031     if(hasLoggedCurrentTestStart)
06032         return;
06033
06034     separator_to_stream();
06035     file_line_to_stream(tc->m_file.c_str(), tc->m_line, "\n");
06036     if(tc->m_description)
06037         s << Color::Yellow << "DESCRIPTION: " << Color::None << tc->m_description << "\n";
06038     if(tc->m_test_suite && tc->m_test_suite[0] != '\0')
06039         s << Color::Yellow << "TEST SUITE: " << Color::None << tc->m_test_suite << "\n";
06040     if(strncmp(tc->m_name, " Scenario:", 11) != 0)
06041         s << Color::Yellow << "TEST CASE: ";
06042     s << Color::None << tc->m_name << "\n";
06043
06044     for(size_t i = 0; i < currentSubcaseLevel; ++i) {
06045         if(subcasesStack[i].m_name[0] != '\0')
06046             s << " " << subcasesStack[i].m_name << "\n";
06047     }
06048
06049     if(currentSubcaseLevel != subcasesStack.size()) {
06050         s << Color::Yellow << "\nDEEPEST SUBCASE STACK REACHED (DIFFERENT FROM THE CURRENT
06051 ONE):\n" << Color::None;
06052         for(size_t i = 0; i < subcasesStack.size(); ++i) {
06053             if(subcasesStack[i].m_name[0] != '\0')
06054                 s << " " << subcasesStack[i].m_name << "\n";
06055         }
06056     }

```

```

06055 }
06056
06057     s << "\n";
06058
06059     hasLoggedCurrentTestStart = true;
06060 }
06061
06062 void printVersion() {
06063     if(opt.no_version == false)
06064         s << Color::Cyan << "[doctest] " << Color::None << "doctest version is \"
06065             << DOCTEST_VERSION_STR << "\"\n";
06066 }
06067
06068 void printIntro() {
06069     if(opt.no_intro == false) {
06070         printVersion();
06071         s << Color::Cyan << "[doctest] " << Color::None
06072             << "run with \"--\" DOCTEST_OPTIONS_PREFIX_DISPLAY \"help\" for options\n";
06073     }
06074 }
06075
06076 void printHelp() {
06077     int sizePrefixDisplay = static_cast<int>(strlen(DOCTEST_OPTIONS_PREFIX_DISPLAY));
06078     printVersion();
06079     // clang-format off
06080     s << Color::Cyan << "[doctest]\n" << Color::None;
06081     s << Color::Cyan << "[doctest] " << Color::None;
06082     s << "boolean values: \"1/on/yes/true\" or \"0/off/no/false\"\n";
06083     s << Color::Cyan << "[doctest] " << Color::None;
06084     s << "filter values: \"str1,str2,str3\" (comma separated strings)\n";
06085     s << Color::Cyan << "[doctest]\n" << Color::None;
06086     s << Color::Cyan << "[doctest] " << Color::None;
06087     s << "filters use wildcards for matching strings\n";
06088     s << Color::Cyan << "[doctest] " << Color::None;
06089     s << "something passes a filter if any of the strings in a filter matches\n";
06090 #ifndef DOCTEST_CONFIG_NO_UNPREFIXED_OPTIONS
06091     s << Color::Cyan << "[doctest]\n" << Color::None;
06092     s << Color::Cyan << "[doctest] " << Color::None;
06093     s << "ALL FLAGS, OPTIONS AND FILTERS ALSO AVAILABLE WITH A \"
DOCTEST_CONFIG_OPTIONS_PREFIX "\" PREFIX!!!\n";
06094 #endif
06095     s << Color::Cyan << "[doctest]\n" << Color::None;
06096     s << Color::Cyan << "[doctest] " << Color::None;
06097     s << "Query flags - the program quits after them. Available:\n\n";
06098     s << "  -- DOCTEST_OPTIONS_PREFIX_DISPLAY \"help, -"
DOCTEST_OPTIONS_PREFIX_DISPLAY "h"
06099         << Whitespace(sizePrefixDisplay*0) << "prints this message\n";
06100     s << "  -- DOCTEST_OPTIONS_PREFIX_DISPLAY \"v, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "version"
"
06101         << Whitespace(sizePrefixDisplay*1) << "prints the version\n";
06102     s << "  -- DOCTEST_OPTIONS_PREFIX_DISPLAY \"c, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "count"
"
06103         << Whitespace(sizePrefixDisplay*1) << "prints the number of matching tests\n";
06104     s << "  -- DOCTEST_OPTIONS_PREFIX_DISPLAY \"ltc, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"list-test-cases"
06105         << Whitespace(sizePrefixDisplay*1) << "lists all matching tests by name\n";
06106     s << "  -- DOCTEST_OPTIONS_PREFIX_DISPLAY \"lts, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"list-test-suites"
06107         << Whitespace(sizePrefixDisplay*1) << "lists all matching test suites\n";
06108     s << "  -- DOCTEST_OPTIONS_PREFIX_DISPLAY \"lr, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"list-reporters"
06109         << Whitespace(sizePrefixDisplay*1) << "lists all registered reporters\n\n";
06110     // ===== << 79
06111     s << Color::Cyan << "[doctest] " << Color::None;
06112     s << "The available <int>/<string> options/filters are:\n\n";
06113     s << "  -- DOCTEST_OPTIONS_PREFIX_DISPLAY \"tc, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"test-case=<filters>"
06114         << Whitespace(sizePrefixDisplay*1) << "filters tests by their name\n";
06115     s << "  -- DOCTEST_OPTIONS_PREFIX_DISPLAY \"tce, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"test-case-exclude=<filters>"
06116         << Whitespace(sizePrefixDisplay*1) << "filters OUT tests by their name\n";
06117     s << "  -- DOCTEST_OPTIONS_PREFIX_DISPLAY \"sf, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"source-file=<filters>"
06118         << Whitespace(sizePrefixDisplay*1) << "filters tests by their file\n";
06119     s << "  -- DOCTEST_OPTIONS_PREFIX_DISPLAY \"sfe, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"source-file-exclude=<filters>"
06120         << Whitespace(sizePrefixDisplay*1) << "filters OUT tests by their file\n";
06121     s << "  -- DOCTEST_OPTIONS_PREFIX_DISPLAY \"ts, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"test-suite=<filters>"
06122         << Whitespace(sizePrefixDisplay*1) << "filters tests by their test suite\n";
06123     s << "  -- DOCTEST_OPTIONS_PREFIX_DISPLAY \"tse, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"test-suite-exclude=<filters>"
06124         << Whitespace(sizePrefixDisplay*1) << "filters OUT tests by their test suite\n";
06125     s << "  -- DOCTEST_OPTIONS_PREFIX_DISPLAY \"sc, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"subcase=<filters>"
06126         << Whitespace(sizePrefixDisplay*1) << "filters subcases by their name\n";
06127     s << "  -- DOCTEST_OPTIONS_PREFIX_DISPLAY \"sce, --" DOCTEST_OPTIONS_PREFIX_DISPLAY

```

```

"subcase-exclude=<filters>      "
06128     « Whitespace(sizePrefixDisplay*1) « "filters OUT subcases by their name\n";
06129     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "r, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"reporters=<filters>            "
06130     « Whitespace(sizePrefixDisplay*1) « "reporters to use (console is default)\n";
06131     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "o, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"out=<string>                    "
06132     « Whitespace(sizePrefixDisplay*1) « "output filename\n";
06133     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "ob, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"order-by=<string>              "
06134     « Whitespace(sizePrefixDisplay*1) « "how the tests should be ordered\n";
06135     s « Whitespace(sizePrefixDisplay*3) « "                                <string> -
[file/suite/name/rand/none]\n";
06136     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "rs, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"rand-seed=<int>                "
06137     « Whitespace(sizePrefixDisplay*1) « "seed for random ordering\n";
06138     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "f, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"first=<int>                     "
06139     « Whitespace(sizePrefixDisplay*1) « "the first test passing the filters to\n";
06140     s « Whitespace(sizePrefixDisplay*3) « "                                execute -
for range-based execution\n";
06141     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "l, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"last=<int>                     "
06142     « Whitespace(sizePrefixDisplay*1) « "the last test passing the filters to\n";
06143     s « Whitespace(sizePrefixDisplay*3) « "                                execute -
for range-based execution\n";
06144     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "aa, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"abort-after=<int>              "
06145     « Whitespace(sizePrefixDisplay*1) « "stop after <int> failed assertions\n";
06146     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "scfl,--" DOCTEST_OPTIONS_PREFIX_DISPLAY
"subcase-filter-levels=<int>    "
06147     « Whitespace(sizePrefixDisplay*1) « "apply filters for the first <int> levels\n";
06148     s « Color::Cyan « "\n[doctest] " « Color::None;
06149     s « "Bool options - can be used like flags and true is assumed. Available:\n\n";
06150     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "s, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"success=<bool>                 "
06151     « Whitespace(sizePrefixDisplay*1) « "include successful assertions in output\n";
06152     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "cs, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"case-sensitive=<bool>          "
06153     « Whitespace(sizePrefixDisplay*1) « "filters being treated as case sensitive\n";
06154     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "e, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"exit=<bool>                    "
06155     « Whitespace(sizePrefixDisplay*1) « "exits after the tests finish\n";
06156     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "d, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"duration=<bool>                "
06157     « Whitespace(sizePrefixDisplay*1) « "prints the time duration of each test\n";
06158     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "m, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"minimal=<bool>                 "
06159     « Whitespace(sizePrefixDisplay*1) « "minimal console output (only failures)\n";
06160     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "q, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"quiet=<bool>                   "
06161     « Whitespace(sizePrefixDisplay*1) « "no console output\n";
06162     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nt, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-throw=<bool>                "
06163     « Whitespace(sizePrefixDisplay*1) « "skips exceptions-related assert checks\n";
06164     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "ne, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-exitcode=<bool>             "
06165     « Whitespace(sizePrefixDisplay*1) « "returns (or exits) always with success\n";
06166     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nr, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-run=<bool>                   "
06167     « Whitespace(sizePrefixDisplay*1) « "skips all runtime doctest operations\n";
06168     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "ni, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-intro=<bool>                 "
06169     « Whitespace(sizePrefixDisplay*1) « "omit the framework intro in the output\n";
06170     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nv, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-version=<bool>              "
06171     « Whitespace(sizePrefixDisplay*1) « "omit the framework version in the output\n";
06172     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nc, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-colors=<bool>               "
06173     « Whitespace(sizePrefixDisplay*1) « "disables colors in output\n";
06174     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "fc, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"force-colors=<bool>            "
06175     « Whitespace(sizePrefixDisplay*1) « "use colors even when not in a tty\n";
06176     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nb, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-breaks=<bool>               "
06177     « Whitespace(sizePrefixDisplay*1) « "disables breakpoints in debuggers\n";
06178     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "ns, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-skip=<bool>                 "
06179     « Whitespace(sizePrefixDisplay*1) « "don't skip test cases marked as skip\n";
06180     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "gfl, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"gnu-file-line=<bool>           "
06181     « Whitespace(sizePrefixDisplay*1) « ":n: vs (n): for line numbers in output\n";
06182     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "npf, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-path-filenames=<bool>       "
06183     « Whitespace(sizePrefixDisplay*1) « "only filenames and no paths in output\n";
06184     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nln, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-line-numbers=<bool>         "

```



```

06185         « Whitespace(sizePrefixDisplay*1) « "0 instead of real line numbers in output\n";
06186         // ===== « 79
06187         // clang-format on
06188
06189         s « Color::Cyan « "\n[doctest] " « Color::None;
06190         s « "for more information visit the project documentation\n\n";
06191     }
06192
06193     void printRegisteredReporters() {
06194         printVersion();
06195         auto printReporters = [this] (const reporterMap& reporters, const char* type) {
06196             if(reporters.size()) {
06197                 s « Color::Cyan « "[doctest] " « Color::None « "listing all registered " « type «
06198                 "\n";
06199                 for(auto& curr : reporters)
06200                     s « "priority: " « std::setw(5) « curr.first.first
06201                     « " name: " « curr.first.second « "\n";
06202             }
06203         };
06204         printReporters(getListeners(), "listeners");
06205         printReporters(getReporters(), "reporters");
06206     }
06207
06208     // =====
06209     // WHAT FOLLOWS ARE OVERRIDES OF THE VIRTUAL METHODS OF THE REPORTER INTERFACE
06210     // =====
06211     void report_query(const QueryData& in) override {
06212         if(opt.version) {
06213             printVersion();
06214         } else if(opt.help) {
06215             printHelp();
06216         } else if(opt.list_reporters) {
06217             printRegisteredReporters();
06218         } else if(opt.count || opt.list_test_cases) {
06219             if(opt.list_test_cases) {
06220                 s « Color::Cyan « "[doctest] " « Color::None
06221                 « "listing all test case names\n";
06222                 separator_to_stream();
06223             }
06224
06225             for(unsigned i = 0; i < in.num_data; ++i)
06226                 s « Color::None « in.data[i]->m_name « "\n";
06227
06228             separator_to_stream();
06229
06230             s « Color::Cyan « "[doctest] " « Color::None
06231             « "unskipped test cases passing the current filters: "
06232             « g_cs->numTestCasesPassingFilters « "\n";
06233
06234         } else if(opt.list_test_suites) {
06235             s « Color::Cyan « "[doctest] " « Color::None « "listing all test suites\n";
06236             separator_to_stream();
06237
06238             for(unsigned i = 0; i < in.num_data; ++i)
06239                 s « Color::None « in.data[i]->m_test_suite « "\n";
06240
06241             separator_to_stream();
06242
06243             s « Color::Cyan « "[doctest] " « Color::None
06244             « "unskipped test cases passing the current filters: "
06245             « g_cs->numTestCasesPassingFilters « "\n";
06246             s « Color::Cyan « "[doctest] " « Color::None
06247             « "test suites with unskipped test cases passing the current filters: "
06248             « g_cs->numTestSuitesPassingFilters « "\n";
06249         }
06250     }
06251
06252     void test_run_start() override {
06253         if(!opt.minimal)
06254             printIntro();
06255     }
06256
06257     void test_run_end(const TestRunStats& p) override {
06258         if(opt.minimal && p.numTestCasesFailed == 0)
06259             return;
06260
06261         separator_to_stream();
06262         s « std::dec;
06263
06264         auto totwidth =
06265             int(std::ceil(log10(static_cast<double>(std::max(p.numTestCasesPassingFilters,
06266                 static_cast<unsigned>(p.numAsserts))) + 1)));
06267         auto passwidth =
06268             int(std::ceil(log10(static_cast<double>(std::max(p.numTestCasesPassingFilters - p.numTestCasesFailed,
06269                 static_cast<unsigned>(p.numAsserts - p.numAssertsFailed))) + 1)));
06270         auto failwidth = int(std::ceil(log10(static_cast<double>(std::max(p.numTestCasesFailed,

```

```

static_cast<unsigned>(p.numAssertsFailed))) + 1));
06267     const bool anythingFailed = p.numTestCasesFailed > 0 || p.numAssertsFailed > 0;
06268     s << Color::Cyan << "[doctest] " << Color::None << "test cases: " << std::setw(totwidth)
06269     << p.numTestCasesPassingFilters << " | "
06270     << ((p.numTestCasesPassingFilters == 0 || anythingFailed) ? Color::None :
06271         Color::Green)
06272     << std::setw(passwidth) << p.numTestCasesPassingFilters - p.numTestCasesFailed << " passed"
06273     << Color::None << " | " << (p.numTestCasesFailed > 0 ? Color::Red : Color::None)
06274     << std::setw(failwidth) << p.numTestCasesFailed << " failed" << Color::None << " |";
06275     if(opt.no_skipped_summary == false) {
06276         const int numSkipped = p.numTestCases - p.numTestCasesPassingFilters;
06277         s << " " << (numSkipped == 0 ? Color::None : Color::Yellow) << numSkipped
06278         << " skipped" << Color::None;
06279     }
06280     s << "\n";
06281     s << Color::Cyan << "[doctest] " << Color::None << "assertions: " << std::setw(totwidth)
06282     << p.numAsserts << " | "
06283     << ((p.numAsserts == 0 || anythingFailed) ? Color::None : Color::Green)
06284     << std::setw(passwidth) << (p.numAsserts - p.numAssertsFailed) << " passed" << Color::None
06285     << " | " << (p.numAssertsFailed > 0 ? Color::Red : Color::None) << std::setw(failwidth)
06286     << p.numAssertsFailed << " failed" << Color::None << " | \n";
06287     s << Color::Cyan << "[doctest] " << Color::None
06288     << "Status: " << (p.numTestCasesFailed > 0 ? Color::Red : Color::Green)
06289     << ((p.numTestCasesFailed > 0) ? "FAILURE!" : "SUCCESS!") << Color::None << std::endl;
06290 }
06291
06292 void test_case_start(const TestCaseData& in) override {
06293     hasLoggedCurrentTestStart = false;
06294     tc                        = &in;
06295     subcasesStack.clear();
06296     currentSubcaseLevel = 0;
06297 }
06298
06299 void test_case_reenter(const TestCaseData&) override {
06300     subcasesStack.clear();
06301 }
06302
06303 void test_case_end(const CurrentTestCaseStats& st) override {
06304     if(tc->m_no_output)
06305         return;
06306
06307     // log the preamble of the test case only if there is something
06308     // else to print - something other than that an assert has failed
06309     if(opt.duration ||
06310        (st.failure_flags && st.failure_flags !=
06311         static_cast<int>(TestCaseFailureReason::AssertFailure)))
06312         logTestStart();
06313
06314     if(opt.duration)
06315         s << Color::None << std::setprecision(6) << std::fixed << st.seconds
06316         << " s: " << tc->m_name << "\n";
06317
06318     if(st.failure_flags & TestCaseFailureReason::Timeout)
06319         s << Color::Red << "Test case exceeded time limit of " << std::setprecision(6)
06320         << std::fixed << tc->m_timeout << "! \n";
06321
06322     if(st.failure_flags & TestCaseFailureReason::ShouldHaveFailedButDidnt) {
06323         s << Color::Red << "Should have failed but didn't! Marking it as failed!\n";
06324     } else if(st.failure_flags & TestCaseFailureReason::ShouldHaveFailedAndDid) {
06325         s << Color::Yellow << "Failed as expected so marking it as not failed\n";
06326     } else if(st.failure_flags & TestCaseFailureReason::CouldHaveFailedAndDid) {
06327         s << Color::Yellow << "Allowed to fail so marking it as not failed\n";
06328     } else if(st.failure_flags & TestCaseFailureReason::DidntFailExactlyNumTimes) {
06329         s << Color::Red << "Didn't fail exactly " << tc->m_expected_failures
06330         << " times so marking it as failed!\n";
06331     } else if(st.failure_flags & TestCaseFailureReason::FailedExactlyNumTimes) {
06332         s << Color::Yellow << "Failed exactly " << tc->m_expected_failures
06333         << " times as expected so marking it as not failed!\n";
06334     }
06335     if(st.failure_flags & TestCaseFailureReason::TooManyFailedAsserts) {
06336         s << Color::Red << "Aborting - too many failed asserts!\n";
06337     }
06338     s << Color::None; // lgTM [cpp/useless-expression]
06339 }
06340
06341 void test_case_exception(const TestCaseException& e) override {
06342     DOCTEST_LOCK_MUTEX(mutex)
06343     if(tc->m_no_output)
06344         return;
06345
06346     logTestStart();
06347
06348     file_line_to_stream(tc->m_file.c_str(), tc->m_line, " ");
06349     successOrFailColoredStringToStream(false, e.is_crash ? assertType::is_require :
06350                                         assertType::is_check);
06351     s << Color::Red << (e.is_crash ? "test case CRASHED: " : "test case THREW exception: ")
06352     << Color::Cyan << e.error_string << "\n";

```

```

06352
06353     int num_stringified_contexts = get_num_stringified_contexts();
06354     if(num_stringified_contexts) {
06355         auto stringified_contexts = get_stringified_contexts();
06356         s << Color::None << "   logged: ";
06357         for(int i = num_stringified_contexts; i > 0; --i) {
06358             s << (i == num_stringified_contexts ? " " : " ")
06359                << stringified_contexts[i - 1] << "\n";
06360         }
06361     }
06362     s << "\n" << Color::None;
06363 }
06364
06365 void subcase_start(const SubcaseSignature& subc) override {
06366     subcasesStack.push_back(subc);
06367     ++currentSubcaseLevel;
06368     hasLoggedCurrentTestStart = false;
06369 }
06370
06371 void subcase_end() override {
06372     --currentSubcaseLevel;
06373     hasLoggedCurrentTestStart = false;
06374 }
06375
06376 void log_assert(const AssertData& rb) override {
06377     if(!rb.m_failed && !opt.success) || tc->m_no_output)
06378         return;
06379
06380     DOCTEST_LOCK_MUTEX(mutex)
06381
06382     logTestStart();
06383
06384     file_line_to_stream(rb.m_file, rb.m_line, " ");
06385     successOrFailColoredStringToStream(!rb.m_failed, rb.m_at);
06386
06387     fulltext_log_assert_to_stream(s, rb);
06388
06389     log_contexts();
06390 }
06391
06392 void log_message(const MessageData& mb) override {
06393     if(tc->m_no_output)
06394         return;
06395
06396     DOCTEST_LOCK_MUTEX(mutex)
06397
06398     logTestStart();
06399
06400     file_line_to_stream(mb.m_file, mb.m_line, " ");
06401     s << getSuccessOrFailColor(false, mb.m_severity)
06402        << getSuccessOrFailString(mb.m_severity & assertType::is_warn, mb.m_severity,
06403                                "MESSAGE") << ": ";
06404     s << Color::None << mb.m_string << "\n";
06405     log_contexts();
06406 }
06407
06408 void test_case_skipped(const TestCaseData&) override {}
06409 };
06410
06411 DOCTEST_REGISTER_REPORTER("console", 0, ConsoleReporter);
06412
06413 #ifndef DOCTEST_PLATFORM_WINDOWS
06414 struct DebugOutputWindowReporter : public ConsoleReporter
06415 {
06416     DOCTEST_THREAD_LOCAL static std::ostringstream oss;
06417
06418     DebugOutputWindowReporter(const ContextOptions& co)
06419         : ConsoleReporter(co, oss) {}
06420
06421 #define DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(func, type, arg)
06422     void func(type arg) override {
06423         bool with_col = g_no_colors;
06424         g_no_colors = false;
06425         ConsoleReporter::func(arg);
06426         if(oss.tellp() != std::streampos{}) {
06427             DOCTEST_OUTPUT_DEBUG_STRING(oss.str().c_str());
06428             oss.str("");
06429         }
06430         g_no_colors = with_col;
06431     }
06432
06433     DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_run_start, DOCTEST_EMPTY, DOCTEST_EMPTY)
06434     DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_run_end, const TestRunStats&, in)
06435     DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_case_start, const TestCaseData&, in)
06436     DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_case_reenter, const TestCaseData&, in)
06437     DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_case_end, const CurrentTestStats&, in)
06438     DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_case_exception, const TestCaseException&, in)

```

```

06439     DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(subcase_start, const SubcaseSignature&, in)
06440     DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(subcase_end, DOCTEST_EMPTY, DOCTEST_EMPTY)
06441     DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(log_assert, const AssertData&, in)
06442     DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(log_message, const MessageData&, in)
06443     DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_case_skipped, const TestCaseData&, in)
06444 };
06445
06446     DOCTEST_THREAD_LOCAL std::ostringstream DebugOutputWindowReporter::oss;
06447 #endif // DOCTEST_PLATFORM_WINDOWS
06448
06449     // the implementation of parseOption()
06450     bool parseOptionImpl(int argc, const char* const* argv, const char* pattern, String* value) {
06451         // going from the end to the beginning and stopping on the first occurrence from the end
06452         for(int i = argc; i > 0; --i) {
06453             auto index = i - 1;
06454             auto temp = std::strstr(argv[index], pattern);
06455             if(temp && (value || strlen(temp) == strlen(pattern))) {
06456                 // eliminate matches in which the chars before the option are not '-'
06457                 bool noBadCharsFound = true;
06458                 auto curr = argv[index];
06459                 while(curr != temp) {
06460                     if(*curr++ != '-') {
06461                         noBadCharsFound = false;
06462                         break;
06463                     }
06464                 }
06465                 if(noBadCharsFound && argv[index][0] == '-') {
06466                     if(value) {
06467                         // parsing the value of an option
06468                         temp += strlen(pattern);
06469                         const unsigned len = strlen(temp);
06470                         if(len) {
06471                             *value = temp;
06472                             return true;
06473                         }
06474                     } else {
06475                         // just a flag - no value
06476                         return true;
06477                     }
06478                 }
06479             }
06480         }
06481         return false;
06482     }
06483
06484     // parses an option and returns the string after the '=' character
06485     bool parseOption(int argc, const char* const* argv, const char* pattern, String* value = nullptr,
06486                     const String& defaultVal = String()) {
06487         if(value)
06488             *value = defaultVal;
06489 #ifndef DOCTEST_CONFIG_NO_UNPREFIXED_OPTIONS
06490         // offset (normally 3 for "dt-") to skip prefix
06491         if(parseOptionImpl(argc, argv, pattern + strlen(DOCTEST_CONFIG_OPTIONS_PREFIX), value))
06492             return true;
06493 #endif // DOCTEST_CONFIG_NO_UNPREFIXED_OPTIONS
06494         return parseOptionImpl(argc, argv, pattern, value);
06495     }
06496
06497     // locates a flag on the command line
06498     bool parseFlag(int argc, const char* const* argv, const char* pattern) {
06499         return parseOption(argc, argv, pattern);
06500     }
06501
06502     // parses a comma separated list of words after a pattern in one of the arguments in argv
06503     bool parseCommaSepArgs(int argc, const char* const* argv, const char* pattern,
06504                           std::vector<String>& res) {
06505         String filtersString;
06506         if(parseOption(argc, argv, pattern, &filtersString)) {
06507             // tokenize with "," as a separator, unless escaped with backslash
06508             std::ostringstream s;
06509             auto flush = [&s, &res]() {
06510                 auto string = s.str();
06511                 if(string.size() > 0) {
06512                     res.push_back(string.c_str());
06513                 }
06514                 s.str("");
06515             };
06516
06517             bool seenBackslash = false;
06518             const char* current = filtersString.c_str();
06519             const char* end = current + strlen(current);
06520             while(current != end) {
06521                 char character = *current++;
06522                 if(seenBackslash) {
06523                     seenBackslash = false;
06524                     if(character == ',' || character == '\\') {
06525                         s.put(character);

```

```

06526         continue;
06527     }
06528     s.put('\\');
06529 }
06530 if(character == '\\') {
06531     seenBackslash = true;
06532 } else if(character == ',') {
06533     flush();
06534 } else {
06535     s.put(character);
06536 }
06537 }
06538 if(seenBackslash) {
06539     s.put('\\');
06540 }
06541 flush();
06542 return true;
06543 }
06544 return false;
06545 }
06546 }
06547
06548 enum optionType
06549 {
06550     option_bool,
06551     option_int
06552 };
06553
06554 // parses an int/bool option from the command line
06555 bool parseIntOption(int argc, const char* const* argv, const char* pattern, optionType type,
06556                    int& res) {
06557     String parsedValue;
06558     if(!parseOption(argc, argv, pattern, &parsedValue))
06559         return false;
06560
06561     if(type) {
06562         // integer
06563         // TODO: change this to use std::stoi or something else! currently it uses undefined
06564         behavior - assumes '0' on failed parse...
06565         int theInt = std::atoi(parsedValue.c_str());
06566         if (theInt != 0) {
06567             res = theInt;
06568             return true;
06569         }
06570     } else {
06571         // boolean
06572         const char positive[] = { "1", "true", "on", "yes" }; // 5 - strlen("true") + 1
06573         const char negative[] = { "0", "false", "off", "no" }; // 6 - strlen("false") + 1
06574
06575         // if the value matches any of the positive/negative possibilities
06576         for (unsigned i = 0; i < 4; i++) {
06577             if (parsedValue.compare(positive[i], true) == 0) {
06578                 res = 1;
06579                 return true;
06580             }
06581             if (parsedValue.compare(negative[i], true) == 0) {
06582                 res = 0;
06583                 return true;
06584             }
06585         }
06586         return false;
06587     }
06588 } // namespace
06589
06590 Context::Context(int argc, const char* const* argv)
06591 : p(new detail::ContextState) {
06592     parseArgs(argc, argv, true);
06593     if(argc)
06594         p->binary_name = argv[0];
06595 }
06596
06597 Context::~Context() {
06598     if(g_cs == p)
06599         g_cs = nullptr;
06600     delete p;
06601 }
06602
06603 void Context::applyCommandLine(int argc, const char* const* argv) {
06604     parseArgs(argc, argv);
06605     if(argc)
06606         p->binary_name = argv[0];
06607 }
06608
06609 // parses args
06610 void Context::parseArgs(int argc, const char* const* argv, bool withDefaults) {
06611     using namespace detail;

```

```

06612
06613 // clang-format off
06614 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "source-file=", p->filters[0]);
06615 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "sf=", p->filters[0]);
06616 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "source-file-exclude=", p->filters[1]);
06617 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "sfe=", p->filters[1]);
06618 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "test-suite=", p->filters[2]);
06619 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "ts=", p->filters[2]);
06620 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "test-suite-exclude=", p->filters[3]);
06621 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "tse=", p->filters[3]);
06622 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "test-case=", p->filters[4]);
06623 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "tc=", p->filters[4]);
06624 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "test-case-exclude=", p->filters[5]);
06625 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "tce=", p->filters[5]);
06626 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "subcase=", p->filters[6]);
06627 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "sc=", p->filters[6]);
06628 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "subcase-exclude=", p->filters[7]);
06629 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "sce=", p->filters[7]);
06630 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "reporters=", p->filters[8]);
06631 parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "r=", p->filters[8]);
06632 // clang-format on
06633
06634 int intRes = 0;
06635 String strRes;
06636
06637 #define DOCTEST_PARSE_AS_BOOL_OR_FLAG(name, sname, var, default) \
06638     if(parseIntOption(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX name "=", option_bool, intRes) || \
06639         parseIntOption(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX sname "=", option_bool, intRes)) \
06640         p->var = static_cast<bool>(intRes); \
06641     else if(parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX name) || \
06642         parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX sname)) \
06643         p->var = true; \
06644     else if(withDefaults) \
06645         p->var = default \
06646
06647 #define DOCTEST_PARSE_INT_OPTION(name, sname, var, default) \
06648     if(parseIntOption(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX name "=", option_int, intRes) || \
06649         parseIntOption(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX sname "=", option_int, intRes)) \
06650         p->var = intRes; \
06651     else if(withDefaults) \
06652         p->var = default \
06653
06654 #define DOCTEST_PARSE_STR_OPTION(name, sname, var, default) \
06655     if(parseOption(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX name "=", &strRes, default) || \
06656         parseOption(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX sname "=", &strRes, default) || \
06657         withDefaults) \
06658         p->var = strRes \
06659
06660 // clang-format off
06661 DOCTEST_PARSE_STR_OPTION("out", "o", out, "");
06662 DOCTEST_PARSE_STR_OPTION("order-by", "ob", order_by, "file");
06663 DOCTEST_PARSE_INT_OPTION("rand-seed", "rs", rand_seed, 0);
06664
06665 DOCTEST_PARSE_INT_OPTION("first", "f", first, 0);
06666 DOCTEST_PARSE_INT_OPTION("last", "l", last, UINT_MAX);
06667
06668 DOCTEST_PARSE_INT_OPTION("abort-after", "aa", abort_after, 0);
06669 DOCTEST_PARSE_INT_OPTION("subcase-filter-levels", "scfl", subcase_filter_levels, INT_MAX);
06670
06671 DOCTEST_PARSE_AS_BOOL_OR_FLAG("success", "s", success, false);
06672 DOCTEST_PARSE_AS_BOOL_OR_FLAG("case-sensitive", "cs", case_sensitive, false);
06673 DOCTEST_PARSE_AS_BOOL_OR_FLAG("exit", "e", exit, false);
06674 DOCTEST_PARSE_AS_BOOL_OR_FLAG("duration", "d", duration, false);
06675 DOCTEST_PARSE_AS_BOOL_OR_FLAG("minimal", "m", minimal, false);
06676 DOCTEST_PARSE_AS_BOOL_OR_FLAG("quiet", "q", quiet, false);
06677 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-throw", "nt", no_throw, false);
06678 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-exitcode", "ne", no_exitcode, false);
06679 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-run", "nr", no_run, false);
06680 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-intro", "ni", no_intro, false);
06681 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-version", "nv", no_version, false);
06682 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-colors", "nc", no_colors, false);
06683 DOCTEST_PARSE_AS_BOOL_OR_FLAG("force-colors", "fc", force_colors, false);
06684 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-breaks", "nb", no_breaks, false);
06685 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-skip", "ns", no_skip, false);
06686 DOCTEST_PARSE_AS_BOOL_OR_FLAG("gnu-file-line", "gfl", gnu_file_line, !bool(DOCTEST_MSVC));
06687 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-path-filenames", "npf", no_path_in_filenames, false);
06688 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-line-numbers", "nln", no_line_numbers, false);
06689 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-debug-output", "ndo", no_debug_output, false);
06690 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-skipped-summary", "nss", no_skipped_summary, false);
06691 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-time-in-output", "ntio", no_time_in_output, false);
06692 // clang-format on
06693
06694 if(withDefaults) {
06695     p->help = false;
06696     p->version = false;
06697     p->count = false;
06698     p->list_test_cases = false;

```

```

06699     p->list_test_suites = false;
06700     p->list_reporters   = false;
06701 }
06702 if(parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "help") ||
06703    parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "h") ||
06704    parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "?")) {
06705     p->help = true;
06706     p->exit = true;
06707 }
06708 if(parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "version") ||
06709    parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "v")) {
06710     p->version = true;
06711     p->exit    = true;
06712 }
06713 if(parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "count") ||
06714    parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "c")) {
06715     p->count = true;
06716     p->exit  = true;
06717 }
06718 if(parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "list-test-cases") ||
06719    parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "ltc")) {
06720     p->list_test_cases = true;
06721     p->exit            = true;
06722 }
06723 if(parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "list-test-suites") ||
06724    parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "lts")) {
06725     p->list_test_suites = true;
06726     p->exit            = true;
06727 }
06728 if(parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "list-reporters") ||
06729    parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "lr")) {
06730     p->list_reporters = true;
06731     p->exit          = true;
06732 }
06733 }
06734
06735 // allows the user to add procedurally to the filters from the command line
06736 void Context::addFilter(const char* filter, const char* value) { setOption(filter, value); }
06737
06738 // allows the user to clear all filters from the command line
06739 void Context::clearFilters() {
06740     for(auto& curr : p->filters)
06741         curr.clear();
06742 }
06743
06744 // allows the user to override procedurally the bool options from the command line
06745 void Context::setOption(const char* option, bool value) {
06746     setOption(option, value ? "true" : "false");
06747 }
06748
06749 // allows the user to override procedurally the int options from the command line
06750 void Context::setOption(const char* option, int value) {
06751     setOption(option, toString(value).c_str());
06752 }
06753
06754 // allows the user to override procedurally the string options from the command line
06755 void Context::setOption(const char* option, const char* value) {
06756     auto argv = String("-") + option + "=" + value;
06757     auto lvalue = argv.c_str();
06758     parseArgs(1, &lvalue);
06759 }
06760
06761 // users should query this in their main() and exit the program if true
06762 bool Context::shouldExit() { return p->exit; }
06763
06764 void Context::setAsDefaultForAssertsOutOfTestCases() { g_cs = p; }
06765
06766 void Context::setAssertHandler(detail::assert_handler ah) { p->ah = ah; }
06767
06768 void Context::setCout(std::ostream* out) { p->cout = out; }
06769
06770 static class DiscardOStream : public std::ostream
06771 {
06772 private:
06773     class : public std::streambuf
06774     {
06775     private:
06776         // allowing some buffering decreases the amount of calls to overflow
06777         char buf[1024];
06778
06779     protected:
06780         std::streamsize xsputn(const char_type*, std::streamsize count) override { return count; }
06781
06782         int_type overflow(int_type ch) override {
06783             setp(std::begin(buf), std::end(buf));
06784             return traits_type::not_eof(ch);
06785         }
06786     };

```

```

06786     } discardBuf;
06787
06788 public:
06789     DiscardOStream()
06790         : std::ostream(&discardBuf) {}
06791 } discardOut;
06792
06793 // the main function that does all the filtering and test running
06794 int Context::run() {
06795     using namespace detail;
06796
06797     // save the old context state in case such was setup - for using asserts out of a testing context
06798     auto old_cs = g_cs;
06799     // this is the current contest
06800     g_cs = p;
06801     is_running_in_test = true;
06802
06803     g_no_colors = p->no_colors;
06804     p->resetRunData();
06805
06806     std::fstream fstr;
06807     if(p->cout == nullptr) {
06808         if(p->quiet) {
06809             p->cout = &discardOut;
06810         } else if(p->out.size()) {
06811             // to a file if specified
06812             fstr.open(p->out.c_str(), std::fstream::out);
06813             p->cout = &fstr;
06814         } else {
06815             #ifndef DOCTEST_CONFIG_NO_INCLUDE_Iostream
06816                 // stdout by default
06817                 p->cout = &std::cout;
06818             #else // DOCTEST_CONFIG_NO_INCLUDE_Iostream
06819                 return EXIT_FAILURE;
06820             #endif // DOCTEST_CONFIG_NO_INCLUDE_Iostream
06821         }
06822     }
06823
06824     FatalConditionHandler::allocateAltStackMem();
06825
06826     auto cleanup_and_return = [&]() {
06827         FatalConditionHandler::freeAltStackMem();
06828
06829         if(fstr.is_open())
06830             fstr.close();
06831
06832         // restore context
06833         g_cs = old_cs;
06834         is_running_in_test = false;
06835
06836         // we have to free the reporters which were allocated when the run started
06837         for(auto& curr : p->reporters_currently_used)
06838             delete curr;
06839         p->reporters_currently_used.clear();
06840
06841         if(p->numTestCasesFailed && !p->no_exitcode)
06842             return EXIT_FAILURE;
06843         return EXIT_SUCCESS;
06844     };
06845
06846     // setup default reporter if none is given through the command line
06847     if(p->filters[8].empty())
06848         p->filters[8].push_back("console");
06849
06850     // check to see if any of the registered reporters has been selected
06851     for(auto& curr : getReporters()) {
06852         if(matchesAny(curr.first.second.c_str(), p->filters[8], false, p->case_sensitive))
06853             p->reporters_currently_used.push_back(curr.second(*g_cs));
06854     }
06855
06856     // TODO: check if there is nothing in reporters_currently_used
06857
06858     // prepend all listeners
06859     for(auto& curr : getListeners())
06860         p->reporters_currently_used.insert(p->reporters_currently_used.begin(), curr.second(*g_cs));
06861
06862     #ifndef DOCTEST_PLATFORM_WINDOWS
06863         if(isDebuggerActive() && p->no_debug_output == false)
06864             p->reporters_currently_used.push_back(new DebugOutputWindowReporter(*g_cs));
06865     #endif // DOCTEST_PLATFORM_WINDOWS
06866
06867     // handle version, help and no_run
06868     if(p->no_run || p->version || p->help || p->list_reporters) {
06869         DOCTEST_ITERATE_THROUGH_REPORTERS(report_query, QueryData());
06870
06871         return cleanup_and_return();
06872     }

```



```

06873
06874     std::vector<const TestCase*> testArray;
06875     for(auto& curr : getRegisteredTests())
06876         testArray.push_back(&curr);
06877     p->numTestCases = testArray.size();
06878
06879     // sort the collected records
06880     if(!testArray.empty()) {
06881         if(p->order_by.compare("file", true) == 0) {
06882             std::sort(testArray.begin(), testArray.end(), fileOrderComparator);
06883         } else if(p->order_by.compare("suite", true) == 0) {
06884             std::sort(testArray.begin(), testArray.end(), suiteOrderComparator);
06885         } else if(p->order_by.compare("name", true) == 0) {
06886             std::sort(testArray.begin(), testArray.end(), nameOrderComparator);
06887         } else if(p->order_by.compare("rand", true) == 0) {
06888             std::srand(p->rand_seed);
06889
06890             // random_shuffle implementation
06891             const auto first = &testArray[0];
06892             for(size_t i = testArray.size() - 1; i > 0; --i) {
06893                 int idxToSwap = std::rand() % (i + 1);
06894
06895                 const auto temp = first[i];
06896
06897                 first[i] = first[idxToSwap];
06898                 first[idxToSwap] = temp;
06899             }
06900         } else if(p->order_by.compare("none", true) == 0) {
06901             // means no sorting - beneficial for death tests which call into the executable
06902             // with a specific test case in mind - we don't want to slow down the startup times
06903         }
06904     }
06905
06906     std::set<String> testSuitesPassingFilter;
06907
06908     bool query_mode = p->count || p->list_test_cases ||
p->list_test_suites;
06909     std::vector<const TestCaseData*> queryResults;
06910
06911     if(!query_mode)
06912         DOCTEST_ITERATE_THROUGH_REPORTERS(test_run_start, DOCTEST_EMPTY);
06913
06914     // invoke the registered functions if they match the filter criteria (or just count them)
06915     for(auto& curr : testArray) {
06916         const auto& tc = *curr;
06917
06918         bool skip_me = false;
06919         if(tc.m_skip && !p->no_skip)
06920             skip_me = true;
06921
06922         if(!matchesAny(tc.m_file.c_str(), p->filters[0], true, p->case_sensitive))
06923             skip_me = true;
06924         if(matchesAny(tc.m_file.c_str(), p->filters[1], false, p->case_sensitive))
06925             skip_me = true;
06926         if(!matchesAny(tc.m_test_suite, p->filters[2], true, p->case_sensitive))
06927             skip_me = true;
06928         if(matchesAny(tc.m_test_suite, p->filters[3], false, p->case_sensitive))
06929             skip_me = true;
06930         if(!matchesAny(tc.m_name, p->filters[4], true, p->case_sensitive))
06931             skip_me = true;
06932         if(matchesAny(tc.m_name, p->filters[5], false, p->case_sensitive))
06933             skip_me = true;
06934
06935         if(!skip_me)
06936             p->numTestCasesPassingFilters++;
06937
06938         // skip the test if it is not in the execution range
06939         if((p->last < p->numTestCasesPassingFilters && p->first <= p->last) ||
06940            (p->first > p->numTestCasesPassingFilters))
06941             skip_me = true;
06942
06943         if(skip_me) {
06944             if(!query_mode)
06945                 DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_skipped, tc);
06946             continue;
06947         }
06948
06949         // do not execute the test if we are to only count the number of filter passing tests
06950         if(p->count)
06951             continue;
06952
06953         // print the name of the test and don't execute it
06954         if(p->list_test_cases) {
06955             queryResults.push_back(&tc);
06956             continue;
06957         }
06958

```

```

06959         // print the name of the test suite if not done already and don't execute it
06960         if(p->list_test_suites) {
06961             if((testSuitesPassingFilt.count(tc.m_test_suite) == 0) && tc.m_test_suite[0] != '\0') {
06962                 queryResults.push_back(&tc);
06963                 testSuitesPassingFilt.insert(tc.m_test_suite);
06964                 p->numTestSuitesPassingFilters++;
06965             }
06966             continue;
06967         }
06968
06969         // execute the test if it passes all the filtering
06970         {
06971             p->currentTest = &tc;
06972
06973             p->failure_flags = TestCaseFailureReason::None;
06974             p->seconds = 0;
06975
06976             // reset atomic counters
06977             p->numAssertsFailedCurrentTest_atomic = 0;
06978             p->numAssertsCurrentTest_atomic = 0;
06979
06980             p->fullyTraversedSubcases.clear();
06981
06982             DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_start, tc);
06983
06984             p->timer.start();
06985
06986             bool run_test = true;
06987
06988             do {
06989                 // reset some of the fields for subcases (except for the set of fully passed ones)
06990                 p->reachedLeaf = false;
06991                 // May not be empty if previous subcase exited via exception.
06992                 p->subcaseStack.clear();
06993                 p->currentSubcaseDepth = 0;
06994
06995                 p->shouldLogCurrentException = true;
06996
06997                 // reset stuff for logging with INFO()
06998                 p->stringifiedContexts.clear();
06999
07000 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
07001                 try {
07002 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
07003                 // MSVC 2015 diagnoses fatalConditionHandler as unused (because reset() is a static method)
07004                 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4101) // unreferenced local variable
07005                     FatalConditionHandler fatalConditionHandler; // Handle signals
07006                     // execute the test
07007                     tc.m_test();
07008                     fatalConditionHandler.reset();
07009                 DOCTEST_MSVC_SUPPRESS_WARNING_POP
07010 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
07011                 } catch(const TestFailureException&) {
07012                     p->failure_flags |= TestCaseFailureReason::AssertFailure;
07013                 } catch(...) {
07014                     DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_exception,
07015                                                         {translateActiveException(), false});
07016                     p->failure_flags |= TestCaseFailureReason::Exception;
07017                 }
07018 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
07019
07020                 // exit this loop if enough assertions have failed - even if there are more subcases
07021                 if(p->abort_after > 0 &&
07022                    p->numAssertsFailed + p->numAssertsFailedCurrentTest_atomic >= p->abort_after) {
07023                     run_test = false;
07024                     p->failure_flags |= TestCaseFailureReason::TooManyFailedAsserts;
07025                 }
07026
07027                 if(!p->nextSubcaseStack.empty() && run_test)
07028                     DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_reenter, tc);
07029                 if(p->nextSubcaseStack.empty())
07030                     run_test = false;
07031             } while(run_test);
07032
07033             p->finalizeTestCaseData();
07034
07035             DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_end, *g_cs);
07036
07037             p->currentTest = nullptr;
07038
07039             // stop executing tests if enough assertions have failed
07040             if(p->abort_after > 0 && p->numAssertsFailed >= p->abort_after)
07041                 break;
07042         }
07043     }
07044
07045     if(!query_mode) {

```

```

07046     DOCTEST_ITERATE_THROUGH_REPORTERS(test_run_end, *g_cs);
07047 } else {
07048     QueryData qdata;
07049     qdata.run_stats = g_cs;
07050     qdata.data = queryResults.data();
07051     qdata.num_data = unsigned(queryResults.size());
07052     DOCTEST_ITERATE_THROUGH_REPORTERS(report_query, qdata);
07053 }
07054
07055 return cleanup_and_return();
07056 }
07057
07058 DOCTEST_DEFINE_INTERFACE(IReporter)
07059
07060 int IReporter::get_num_active_contexts() { return detail::g_infoContexts.size(); }
07061 const IContextScope* const* IReporter::get_active_contexts() {
07062     return get_num_active_contexts() ? &detail::g_infoContexts[0] : nullptr;
07063 }
07064
07065 int IReporter::get_num_stringified_contexts() { return detail::g_cs->stringifiedContexts.size(); }
07066 const String* IReporter::get_stringified_contexts() {
07067     return get_num_stringified_contexts() ? &detail::g_cs->stringifiedContexts[0] : nullptr;
07068 }
07069
07070 namespace detail {
07071     void registerReporterImpl(const char* name, int priority, reporterCreatorFunc c, bool isReporter)
07072     {
07073         if(isReporter)
07074             getReporters().insert(reporterMap::value_type(reporterMap::key_type(priority, name), c));
07075         else
07076             getListeners().insert(reporterMap::value_type(reporterMap::key_type(priority, name), c));
07077     } // namespace detail
07078 } // namespace doctest
07079
07080 #ifndef DOCTEST_CONFIG_DISABLE
07081 #ifdef DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
07082 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4007) // 'function' : must be 'attribute' - see issue #182
07083 int main(int argc, char** argv) { return doctest::Context(argc, argv).run(); }
07084 #endif // DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
07085
07086 DOCTEST_CLANG_SUPPRESS_WARNING_POP
07087 DOCTEST_MSVC_SUPPRESS_WARNING_POP
07088 DOCTEST_GCC_SUPPRESS_WARNING_POP
07089 DOCTEST_SUPPRESS_COMMON_WARNINGS_POP
07090
07091 #endif // DOCTEST_LIBRARY_IMPLEMENTATION
07092 #endif // DOCTEST_CONFIG_IMPLEMENT
07093
07094 #ifdef DOCTEST_UNDEF_WIN32_LEAN_AND_MEAN
07095 #undef WIN32_LEAN_AND_MEAN
07096 #undef DOCTEST_UNDEF_WIN32_LEAN_AND_MEAN
07097 #endif // DOCTEST_UNDEF_WIN32_LEAN_AND_MEAN
07098
07099 #ifdef DOCTEST_UNDEF_NOMINMAX
07100 #undef NOMINMAX
07101 #undef DOCTEST_UNDEF_NOMINMAX
07102 #endif // DOCTEST_UNDEF_NOMINMAX

```

6.3 failo_oper.h

```

00001
00002 #ifndef FAILO_OPER_H
00003 #define FAILO_OPER_H
00004
00005 #include "studentas.h"
00006 #include <string>
00007 #include <vector>
00008
00009 // Funkcijos failų skaitymui
00010 std::vector<Student> nuskaitytiStudentus(const std::string& failoPavadinimas, double& skaitymoLaikas);
00011
00012 // Funkcijos rezultatų išvedimui
00013 void spausdintiRezultatus(const std::vector<Student>& studentai);
00014 void rasytiRezultatus(const std::string& failoPavadinimas, const std::vector<Student>& studentai);
00015 void pasirinktiIsvestiesBuda(const std::vector<Student>& studentai);
00016
00017 #endif

```

6.4 generavimasf.h

```
00001 #ifndef FILE_GENERATOR_H
00002 #define FILE_GENERATOR_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include "studentas.h"
00007
00008 void generuotiStudentuFailus(const std::vector<int>& kiekiai, std::vector<std::string>& failugen);
00009 void rusiotiStudentusISFailus(const std::string& failoPavadinimas, std::vector<std::string>&
    failugen);
00010 void rikiuotiStudentusPriesRusiavima(std::vector<Student>& studentai);
00011
00012 #endif
```

6.5 ListO.h

```
00001 #ifndef LISTO_H
00002 #define LISTO_H
00003
00004 #include <list>
00005 #include "studentas.h"
00006
00007 void rusiotiStudentusISFailusList(const std::string& failas, std::list<std::string>& failugenList);
00008 void rikiuotiStudentusPriesSkirstymaList(std::list<Student>& studentai);
00009
00010 #endif
```

6.6 papild.h

```
00001 // skirtas kad tikrinti ar string'e yra tik raides
00002 #ifndef PAPILD_H
00003 #define PAPILD_H
00004
00005 #include <string>
00006
00007 // Pagalbinės funkcijos kad tikrinti ar gerai parasytas string
00008 bool tikrintiRaides(const std::string& str);
00009
00010 #endif
```

6.7 studentas.h

```
00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include <string>
00005 #include <vector>
00006 #include "zmogus.h"
00007
00008 class Student : public Zmogus {
00009 private:
00010     std::vector<int> nd_balai;
00011     int egzaminas;
00012
00013 public:
00014
00015     friend std::istream& operator>>(std::istream& ivedimas, Student& s); // ivedimo operatorius
00016     friend std::ostream& operator<<(std::ostream& isvedimas, const Student& s); // isvedimo operatorius
00017
00018
00019     // Konstruktoriai
00020     Student();
00021
00022     // Rule of five
00023     Student(const Student& kitas); // copy konstruktorius
00024     Student& operator=(const Student& kitas); // copy priskirimo operatorius
00025
00026     //Destruktorius
00027     ~Student();
00028
00029     Student(Student&& kitas); // move konstruktorius
```

```

00030     Student& operator=(Student&& kitas); // move priskirimo operatorius
00031     // Rule of five pabaiga
00032
00033     // Getteriai
00034     std::string getVardas() const;
00035     std::string getPavarde() const;
00036     std::vector<int> getNdBalai() const;
00037     int getEgzaminas() const;
00038
00039     // Setteriai
00040     void setVardas(const std::string& vardas);
00041     void setPavarde(const std::string& pavarde);
00042     void setNdBalai(const std::vector<int>& nd);
00043     void setEgzaminas(int egzaminas);
00044
00045     // Kitos funkcijos
00046     double skaiciuotiVidurki() const;
00047     double skaiciuotiMediana() const;
00048     double galutinisPazymys(bool naudotiVidurki = true) const;
00049 };
00050
00051 // Funkcijos generavimui ir ivedimui (naudos viešus metodus)
00052 Student generuotiStudenta();
00053 void generuotiPazymius(Student& studentas);
00054 std::vector<Student> generuotiStudentus(int kiekis);
00055 std::vector<Student> ivestiStudentus();
00056
00057 // Rikiavimo funkcijos (naudojasi getteriais)
00058 bool palygintiPagalVarda(const Student& a, const Student& b);
00059 bool palygintiPagalPavarde(const Student& a, const Student& b);
00060 bool palygintiPagalVidurkiAsc(const Student& a, const Student& b);
00061 bool palygintiPagalVidurkiDesc(const Student& a, const Student& b);
00062 bool palygintiPagalMedianaAsc(const Student& a, const Student& b);
00063 bool palygintiPagalMedianaDesc(const Student& a, const Student& b);
00064 void rikiuotiStudentus(std::vector<Student>& studentai, char pasirinkimas, char tvarka = 'a');
00065 void rikiuotiStudentusPagalPasirinkima(std::vector<Student>& studentai);
00066
00067 #endif

```

6.8 testai.h

```

00001 #ifndef TESTAI_H
00002 #define TESTAI_H
00003
00004 #include <vector>
00005 #include <string>
00006
00007 void atliktiLaikoTestusSuFailais(std::vector<std::string>& failugen);
00008
00009 #endif

```

6.9 zmogus.h

```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005
00006 class Zmogus {
00007 protected:
00008     std::string vardas;
00009     std::string pavarde;
00010
00011 public:
00012     virtual ~Zmogus() = default;
00013
00014     virtual std::string getVardas() const = 0;
00015     virtual std::string getPavarde() const = 0;
00016
00017     virtual void setVardas(const std::string& v) = 0;
00018     virtual void setPavarde(const std::string& p) = 0;
00019 };
00020
00021 #endif

```


Index

doctest::Approx, 11
doctest::AssertData, 12
doctest::AssertData::StringContains, 33
doctest::Contains, 13
doctest::Context, 13
doctest::ContextOptions, 14
doctest::CurrentTestCaseStats, 16
doctest::detail::ContextScope< L >, 15
 stringify, 15
doctest::detail::ContextScopeBase, 16
doctest::detail::deferred_false< T >, 17
doctest::detail::ExceptionTranslator< T >, 18
 translate, 18
doctest::detail::Expression_lhs< L >, 18
 operator Result, 19
doctest::detail::ExpressionDecomposer, 19
doctest::detail::filldata< const char[N]>, 20
doctest::detail::filldata< const void * >, 20
doctest::detail::filldata< T >, 20
doctest::detail::filldata< T * >, 21
doctest::detail::filldata< T[N]>, 21
doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream &>()), declval< const T &>()), void()>, 22
doctest::detail::has_insertion_operator< T, typename >, 21
doctest::detail::IExceptionTranslator, 23
doctest::detail::MessageBuilder, 27
doctest::detail::RelationalComparator< int, L, R >, 29
doctest::detail::Result, 30
doctest::detail::ResultBuilder, 31
 unary_assert, 32
doctest::detail::should_stringify_as_underlying_type< T >, 32
doctest::detail::StringMakerBase< C >, 34
doctest::detail::StringMakerBase< true >, 34
doctest::detail::Subcase, 36
doctest::detail::TestCase, 36
doctest::detail::TestFailureException, 38
doctest::detail::TestSuite, 39
doctest::detail::types::enable_if< COND, T >, 17
doctest::detail::types::enable_if< true, T >, 17
doctest::detail::types::false_type, 19
doctest::detail::types::is_array< T >, 24
doctest::detail::types::is_array< T[SIZE]>, 24
doctest::detail::types::is_enum< T >, 25
doctest::detail::types::is_pointer< T >, 25
doctest::detail::types::is_pointer< T * >, 25
doctest::detail::types::is_rvalue_reference< T >, 26
doctest::detail::types::is_rvalue_reference< T && >, 26
doctest::detail::types::remove_const< const T >, 29
doctest::detail::types::remove_const< T >, 29
doctest::detail::types::remove_reference< T >, 29
doctest::detail::types::remove_reference< T & >, 30
doctest::detail::types::remove_reference< T && >, 30
doctest::detail::types::true_type, 39
doctest::detail::types::underlying_type< T >, 40
doctest::IContextScope, 22
doctest::IReporter, 23
doctest::IsNaN< F >, 27
doctest::MessageData, 28
doctest::QueryData, 28
doctest::String, 32
doctest::StringMaker< T >, 33
doctest::SubcaseSignature, 36
doctest::TestCaseData, 37
doctest::TestCaseException, 38
doctest::TestRunStats, 38

getPavarde
 Student, 35
getVardas
 Student, 35

OOP, 1
operator Result
 doctest::detail::Expression_lhs< L >, 19

setPavarde
 Student, 35
setVardas
 Student, 35

std::basic_istream< charT, traits >, 12
std::basic_ostream< charT, traits >, 13
std::char_traits< charT >, 13
std::tuple< Types >, 39
stringify
 doctest::detail::ContextScope< L >, 15
Student, 34
 getPavarde, 35
 getVardas, 35
 setPavarde, 35
 setVardas, 35

translate
 doctest::detail::ExceptionTranslator< T >, 18

unary_assert
 doctest::detail::ResultBuilder, 32

Zmogus, 40